The Architecture of AI Agents and LLM Systems

Patterns, Approaches, and Engineering for Enterprise Automation

Aleksandr Drobiazkin

This edition: July 2026. Sources reflect late 2025–mid-2026 (OWASP Agentic 2026, MCP under the Linux Foundation, the EU AI Act). Facts that can date are tagged with a year; where the industry is still in motion, the text hedges explicitly.

Preface

Who this book is for

This book is addressed to the engineer who already knows how to design distributed systems—sagas, orchestration and choreography, DDD, event-driven architecture, security—and wants to add LLMs and agents to that toolbox **as architectural components**, not as magic. It does not rehash the fundamentals of programming or distributed systems; instead it explains what in the LLM world maps onto the distributed patterns you already know, what is genuinely new—and where the analogies break down.

How this book is organized

The book fleshes out a structured learning path in depth: the path's stages become the book's parts.

The material is organized as a **pattern language**. A pattern is a named solution to a recurring problem in a particular context (the term goes back to Christopher Alexander; in software it was cemented by the Gang of Four and, for distributed systems, by Chris Richardson). Every significant pattern follows a single template:

> **Pattern template**
>
> - **Context**—the situation in which the problem arises.
> - **Problem**—the question the pattern answers.
> - **Forces**—competing concerns (latency vs. accuracy, cost vs. quality, autonomy vs. control).
> - **Solution**—the pattern itself.
> - **Mechanics**—how it works under the hood.
> - **Example**—a concrete implementation on a mainstream stack (e.g., Java/Spring, PostgreSQL, Kubernetes).
> - **Resulting Context**—what the pattern buys you and what you pay for it.
> - **When to Use / When Not To.**
> - **Related Patterns.**
> - **Sources**—the books and papers it draws on, for going deeper.

Terms are set in **bold on first appearance** and collected in the Glossary. Some terms carry so much weight that they get an article-length section of their own inside the relevant chapter (for example, "Attention," "Retrieval-Augmented Generation," "Prompt Injection").

The third tier of terminology support is the **In the Margins** block at the end of each chapter: short explanations of terms and mechanisms the chapter *uses* but that are unpacked elsewhere in the book—or outside it entirely. The in-text marker is an ordinary footnote; each explanation ends with correlation tags: "→ §N.M"—where the topic is covered in full; "⇐ Glossary"—there is a glossary entry; "≠"—a false friend (the term does not mean what intuition

suggests). A footnote without tags is a self-contained definition with nowhere else in the book to point. Only prose counts as a use of a term; diagram labels do not get markers. The block's rule: no more than a dozen footnotes per chapter—anything that needs more deserves the Glossary or an article-length section.

A convention on sources. Wherever the text presents someone else's result, the end of the section cites the primary source (paper or book) where it can be verified and pursued further. Each topic is covered deeply enough in the text that the reference reads as a logical *continuation*, not a stub.

Diagrams are in Mermaid (they render in GitHub, VS Code, Obsidian, and mermaid.live).

Structure. The material unfolds across eight parts (twenty-six chapters) with five appendices: A—the linear-algebra substrate; B—a pattern index; C—the anatomy of a single trajectory (an end-to-end walkthrough of one request); D—the architect's preflight checklist; E—a consolidated bibliography.

Scope. The book concentrates on text-based LLM systems: multimodality (images, audio, video) comes up only where it intersects the core material (multimodal RAG, Chapter 7); voice agents and computer use stay outside the frame. The reason is not that these areas are immature (by mid-2026, browser and computer-use agents are a working replacement for RPA automation) but a separation of layers: both voice and UI control are *I/O modalities* on top of the same agent architecture this book dissects. Their specifics—the VAD/ASR/TTS pipeline and real-time latency in voice, coordinate grounding and resilience to layout changes in computer use—deserve dedicated sources; the foundation—the loop, tools, memory, security—is here. For a maturity yardstick on computer use, look to WebArena-class benchmarks (§17.5).

On the format. The pattern template above is the discipline's common inheritance, not this book's invention: Christopher Alexander gave it Context → Problem → Forces → Solution (*A Pattern Language*, 1977), the Gang of Four fixed it for software (*Design Patterns*, 1994), and Chris Richardson carried it into distributed systems (*Microservices Patterns*, 2018) — which is where this book's exposition takes its cue, so that every concept arrives not as a bare claim but with context, mechanics, an example, trade-offs, and connections to other patterns. Mechanics, Example, When to Use / When Not To, and Sources are this book's additions. Those names appear here for reference only: neither Chris Richardson nor Manning Publications is affiliated with this book, has reviewed it, or endorses it.

A note on tools. This book was written in Russian and translated into English by its author. Software assisted: an assistant (Claude, Anthropic) reviewed drafts, flagged errors and inconsistencies, and proposed alternative wordings. Every claim, every decision about structure, every source, and every sentence as published was chosen, written or rewritten, and verified by the author, who takes responsibility for all of it. The model is a tool, not a party: it holds no authorship and no rights, and Anthropic is not affiliated with this book and has not reviewed it. The full statement is in the NOTICE file at https://github.com/Drobiazkin/ai-agent-architecture.

Demonstrating the format: the Orchestrator-Workers pattern

To show the catalog's depth right away, here is one pattern in full. It belongs in Part IV (Agents) but appears here as a specimen. Its structure is a close relative of the orchestrated saga, with the coordinator replaced by a language model.

Pattern: Orchestrator-Workers

Context. The automation task does not decompose into a fixed set of steps up front: the number and nature of the subtasks depend on the input. “Handle an incoming customer request,” for example, may take one action (answer from the knowledge base) or five (check the order, consult policy, process a refund, notify the warehouse, reply to the customer)—and which ones is unknown until the request arrives.

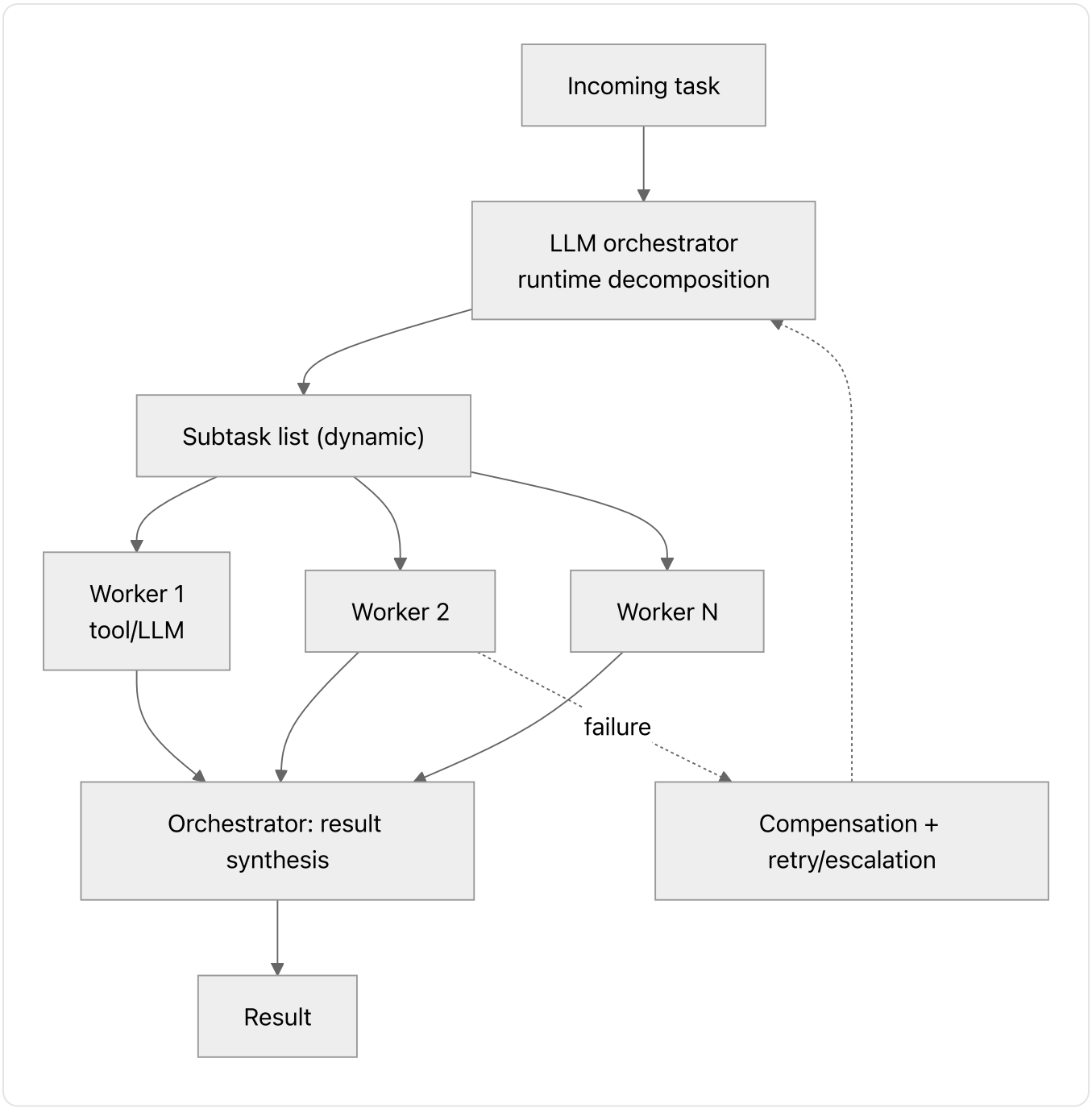
Problem. How do you execute a task whose decomposition is known only at runtime, while preserving reliability, observability, and the ability to roll back—the properties saga orchestration routinely provides?

Forces.

- A static graph (a workflow) is predictable and cheap, but it cannot handle dynamic decomposition.
- A fully autonomous single agent handles the dynamics, but it is harder to control and debug.
- Organizations demand auditing, idempotency, and compensations—regardless of who decided on a step: a human, a state machine, or an LLM.

Solution. Introduce an **LLM orchestrator** that analyzes the task at runtime, dynamically generates a list of subtasks, and hands them out to **workers** (each worker is a separate LLM or tool call with a narrow responsibility). The orchestrator gathers the workers’ results and synthesizes them into the final output. Architecturally this is **Saga orchestration** (in Richardson’s terms), with two differences: (1) the coordinator is nondeterministic, and (2) the set and order of steps are determined dynamically rather than by a predefined state machine.

Mechanics.


Incoming task
LLM orchestrator
runtime decomposition
Subtask list (dynamic)
Worker 1
tool/LLM
Worker 2
Worker N
failure
Orchestrator: result
synthesis
Compensation +
retry/escalation
Result

The orchestrator keeps a **step log** (much like an orchestrator-based saga log). Every worker step is made **idempotent** (keyed by task), given a **timeout**, and paired with a defined **compensation** in case of failure. An **iteration limit/budget** is mandatory; otherwise the orchestrator can loop forever (the coordinator's nondeterminism is a source of nontermination risk for the control loop—a failure mode that deterministic orchestration simply does not have).

Example. Handling a customer request: at execution time the orchestrator identifies the necessary subtasks (check order status, consult policy, process a refund, notify the warehouse) and distributes them across workers. The step log, per-task idempotency keys, and compensating operations are implemented with standard saga-orchestration machinery. Irreversible subtasks (charging money, sending a notification) go through human confirmation (human-in-the-loop).

Resulting Context. The pattern delivers the flexibility of dynamic decomposition while preserving auditing, idempotency, and rollback. The price: a nondeterministic plan (two runs on the same input can produce different decompositions, which complicates testing—see the Trajectory Evaluation pattern), higher cost (every worker is a model call), and latency.

When to Use. The task genuinely requires dynamic decomposition, the number of subtasks varies, and there are reliability/audit requirements. **When Not To.** If the decomposition is fixed, take a static workflow (the Prompt Chaining pattern / a deterministic graph): it is cheaper, more predictable, and easier to debug. Do not add complexity before the task proves you need it.

Related Patterns. Prompt Chaining (the static alternative); Routing (a special case—the orchestrator only picks a branch); Evaluator-Optimizer (for checking step quality); Human-in-the-Loop Gate (for irreversible actions, §10.6); Trajectory Evaluation (how to test a nondeterministic plan).

Sources. Anthropic, "Building Effective Agents" (2024)—the orchestrator-workers pattern. Chris Richardson, *Microservices Patterns* (2018), Chapter 4—saga orchestration, compensating transactions, idempotency. V. Dibia, *Designing Multi-Agent Systems* (2025)—supervisor/handoff orchestration.

That is what every pattern in this book looks like. Now for the full plan—and the foundations.

Table of Contents

Part I. The Language Model as a Component

- Chapter 1. The Language Model as an Architectural Component
  - 1.1 Why you need to understand the model's internals
  - 1.2 Tokenization
  - 1.3 Embeddings and vector space
  - 1.4 Attention and the transformer *(article-length section)*
  - 1.5 Dense vs. Mixture-of-Experts
  - 1.6 The generation loop and inference
  - 1.7 Decoding strategies
  - 1.8 The context window and how it fails
  - 1.9 Why the model hallucinates and what it means to "know"
  - 1.10 Summary: the component's contract
- Chapter 2. Economics and the Nonfunctional Properties of an LLM

Part II. The Interface to the Model

- Chapter 3. Prompt Engineering as Contract Design
  - 3.1 The framing: a prompt as an interface specification · 3.2 Anatomy of a prompt · 3.3 A catalog of prompting patterns · 3.4 Chain-of-Thought, in depth · 3.5 The system prompt as an invariant · 3.6 Prompts as code · 3.7 The limits of prompt engineering
- Chapter 4. Context Engineering and Context-Management Patterns *(article-length section)*
  - 4.1 Shifting the unit of design · 4.2 Anatomy of the context · 4.3 Forces: why this is nontrivial · 4.4 A catalog of context-management patterns · 4.5 Memory as long-term context · 4.6 The limits of context, and trust · 4.7 Summary
- Chapter 5. Structured Output and Reliable Parsing
  - 5.1 The boundary between probabilistic and deterministic · 5.2 The spectrum of techniques · 5.3 Constrained decoding (in depth) · 5.4 Schema design as contract design · 5.5 Reliability patterns · 5.6 Structure ≠ correctness · 5.7 Implementation on the JVM · 5.8 Summary

Part III. Grounding Knowledge: RAG

- Chapter 6. Retrieval-Augmented Generation: Framing the Problem *(article-length section)*
  - 6.1 The problem · 6.2 What RAG is · 6.3 RAG vs. the alternatives · 6.4 Anatomy of a RAG system · 6.5 Retrieval is the bottleneck · 6.6 A taxonomy of failures · 6.7 Grounding and faithfulness · 6.8 Implementing the retrieval layer · 6.9 Summary
- Chapter 7. A Catalog of Retrieval Patterns (Naive, Hybrid, Rerank, GraphRAG, Agentic, Adaptive)
  - 7.1 The pattern ladder · 7.2 Naive RAG · 7.3 Hybrid Search · 7.4 Reranking · 7.5 Query transformation · 7.6 Chunking (+ multimodal RAG, choosing an embedder) · 7.7 GraphRAG · 7.8 Agentic RAG · 7.9 Adaptive RAG · 7.10 Summary and a decision framework
- Chapter 8. Vector Indexes and Retrieval Infrastructure (ANN/HNSW, pgvector, Hybrid Search)

    - 8.1 Nearest-neighbor search · 8.2 The curse of dimensionality · 8.3 Families of ANN indexes · 8.4 The universal trade-off · 8.5 pgvector · 8.6 Hybrid search and filtering · 8.7 RBAC/ABAC at retrieval · 8.8 Operational concerns · 8.9 Summary
- Chapter 9. Evaluating RAG Quality
    - 9.1 Eval-driven RAG · 9.2 A two-axis metric framework · 9.3 The golden set · 9.4 LLM-as-judge for RAG · 9.5 The eval-driven RAG loop · 9.6 Online evaluation · 9.7 Summary

Part IV. Action: Agents and Orchestration

- Chapter 10. The Agent as a Distributed System
    - 10.1 What an agent is · 10.2 The Augmented LLM · 10.3 Workflow vs agent · 10.4 The agent as a distributed system · 10.5 The control loop and nontermination · 10.6 Reliability, reversibility, and the Human-in-the-Loop Gate · 10.7 Levels of agency · 10.8 Summary
- Chapter 11. A Catalog of Agent Patterns (Chaining, Routing, Parallelization, Orchestrator-Workers, Evaluator-Optimizer)
    - 11.1 The ladder and composition · 11.2 Prompt Chaining · 11.3 Routing · 11.4 Parallelization · 11.5 Orchestrator-Workers · 11.6 Evaluator-Optimizer · 11.7 Composing patterns · 11.8 What is a workflow and what is an agent · 11.9 Summary and a decision framework
- Chapter 12. Tools and Tool Calling as a Contract
    - 12.1 What tool calling is · 12.2 The mechanics · 12.3 The ACI as contract design · 12.4 Tool errors · 12.5 Tool security · 12.6 Code execution and Skills · 12.7 Implementation on the JVM · 12.8 Summary
- Chapter 13. Memory and Reasoning Strategies (ReAct, Plan-and-Execute, Reflection)
    - 13.1 Memory in the agent loop · 13.2 The family of strategies · 13.3 ReAct · 13.4 Plan-and-Execute · 13.5 Reflection · 13.6 Tree-of-Thought · 13.7 Choosing a strategy · 13.8 Reasoning models change the calculus · 13.9 Summary
- Chapter 14. Multi-Agent Systems and a Taxonomy of Failures
    - 14.1 When you need multi-agent · 14.2 Coordination patterns · 14.3 The cost of multi-agent · 14.4 A taxonomy of failures (MAST) · 14.5 Treating them with distributed-systems discipline · 14.6 Summary

Part V. Integration: Embedding into the Architecture

- Chapter 15. Model Context Protocol (MCP) *(article-length section)*
    - 15.1 The N×M problem · 15.2 Host, client, server · 15.3 Primitives · 15.4 The session lifecycle · 15.5 Transports · 15.6 MCP and tool calling (+ A2A) · 15.7 MCP security · 15.8 Implementation on the JVM · 15.9 Summary
- Chapter 16. Assembling the Stack on the JVM (Spring AI, LangChain4j)
    - 16.1 The two options · 16.2 Mapping concerns onto constructs · 16.3 Interceptors vs. declarative services · 16.4 Agents and workflows on the JVM · 16.5 Production concerns · 16.6 Choosing and combining · 16.7 Summary

Part VI. Production: Operating the System

- Chapter 17. Evaluation: Building an Eval System
    - 17.1 Evaluation is the central discipline · 17.2 Levels and types · 17.3 LLM-as-judge · 17.4 Judge biases · 17.5 Evaluating agents and trajectories · 17.6 Offline vs. online · 17.7 Statistical rigor · 17.8 The architecture of an eval system · 17.9 Summary
- Chapter 18. Observability

    - 18.1 Why it is harder · 18.2 The three signals · 18.3 Tracing an agent · 18.4 The OpenTelemetry GenAI conventions · 18.5 Metrics and cost attribution · 18.6 Content capture and privacy · 18.7 Debugging nondeterministic trajectories · 18.8 Observability and evaluation together · 18.9 Summary
- Chapter 19. Security: Prompt Injection and Defense in Depth *(article-length section)*
    - 19.1 Instructions and data in one channel · 19.2 Injection vs. jailbreaking · 19.3 The OWASP threat map · 19.4 The lethal trifecta and excessive agency · 19.5 Guardrails and their limits · 19.6 Architectural defense · 19.7 Defense in depth · 19.8 Summary
- Chapter 20. Cost and Latency
    - 20.1 Cost as a product constraint · 20.2 The economics of caching · 20.3 Routing and cascades · 20.4 Cutting tokens · 20.5 Latency · 20.6 Budgets, the gateway, and attribution · 20.7 The combined effect · 20.8 Summary

Part VII. Your Own Models

- Chapter 21. Fine-Tuning: Adapting the Model (LoRA, QLoRA, DPO, Distillation)
    - 21.1 When to fine-tune · 21.2 Full fine-tuning vs. PEFT · 21.3 LoRA · 21.4 QLoRA and memory · 21.5 DoRA and the Kronecker product · 21.6 RLHF and DPO · 21.7 Knowledge distillation · 21.8 Practice · 21.9 Summary
- Chapter 22. Serving Open Models
    - 22.1 Why · 22.2 The serving stack · 22.3 PagedAttention · 22.4 Continuous batching · 22.5 Prefill vs. decode and disaggregation · 22.6 Quantization for inference · 22.7 Speculative decoding · 22.8 Throughput vs. latency · 22.9 Summary
- Chapter 23. Pre-Training
    - 23.1 When it is justified · 23.2 The pipeline · 23.3 Scaling laws · 23.4 The C ≈ 6ND budget · 23.5 Compute-optimal vs. inference-optimal (+ test-time compute) · 23.6 Data and infrastructure · 23.7 Summary

Part VIII. Synthesis

- Chapter 24. The Reference Architecture
    - 24.1 The layers · 24.2 The full component schematic · 24.3 The request flow · 24.4 Boundaries and invariants (+ physical topology) · 24.5 Cross-cutting concerns · 24.6 Decisions by layer · 24.7 Summary
- Chapter 25. Organizational Adoption
    - 25.1 Why architecture is not enough · 25.2 Ownership and the lifecycle of AI artifacts · 25.3 The AI Gateway as a governance point · 25.4 The PII containment loop · 25.5 The regulatory landscape · 25.6 TCO and ROI · 25.7 Teams and process · 25.8 Summary
- Chapter 26. A Worked Example and a Package of Architectural Decisions
    - 26.1 The scenario and the functional requirements · 26.2 NFRs and budgets · 26.3 The ADR format · 26.4 The decision package (12 ADRs) · 26.5 The instantiated architecture · 26.6 How the system evolves · 26.7 Summary of the book

Appendices

- Appendix A. The Linear-Algebra Substrate
- Appendix B. Pattern Index
- Appendix C. Anatomy of a Single Trajectory
- Appendix D. The Architect’s Pre-Flight Checklist
- Appendix E. Consolidated Bibliography

Glossary

Terms marked (→ Chapter N) get a section of their own in the chapter named.

The model and its internals

- **Token**—the smallest unit the model operates on: a subword from the BPE vocabulary, not a character and not a word. The unit of billing and of limits. (→ Chapter 1)
- **Embedding**—a dense vector of fixed dimensionality that encodes meaning; closeness in meaning ≈ closeness of vectors (cosine). One word, two different things: the internal vector of a token inside the model itself, and the vector of a whole fragment produced by a separate embedding model for search; they live on different hardware and are not interchangeable (§1.3). (→ Chapter 1)
- **Attention**—the mechanism that lets each token "look" at the others in a weighted way while computing its own representation; the core of the transformer. (→ Chapter 1)
- **Transformer**—the attention-based neural-network architecture that modern LLMs are built from.
- **Dense model**—a model in which every parameter is active for every token.
- **MoE (Mixture of Experts)**—a sparse model in which a router activates only a subset of experts per token; large total size at small active compute. (→ Chapter 1)
- **KV-cache**—the cache of attention keys/values, kept so they need not be recomputed during autoregression; the biggest consumer of memory after the weights. (→ Chapter 1)
- **Prefill / Decode**—the two phases of inference: processing the prompt in parallel and generating sequentially. (→ Chapter 1)
- **Logits**—the model's raw (pre-softmax) scores for every token in the vocabulary.
- **Temperature / Top-p / Top-k**—decoding parameters that control how random the token choice is; the "determinism knob." (→ Chapter 1)
- **Context window**—the maximum number of tokens the model processes at once; a finite budget. (→ Chapter 1)
- **Hallucination**—a confidently generated but false statement; a consequence of predicting plausibility rather than truth. (→ Chapter 1)
- **RoPE (Rotary Position Embedding)**—encoding position by rotating the Q/K vectors: attention sees relative positions; the basis of context-extension techniques. (→ Chapter 1)
- **TTFT / TPOT**—time-to-first-token and time-per-output-token: the two components of generation latency, corresponding to the prefill and decode phases. (→ Chapter 2)
- **Reasoning model**—a model trained (with RLVR) to spend compute on a long internal chain of reasoning before it answers; the reasoning is billed as output tokens. (→ Chapter 13)

Interface

- **Prompt**—the model's input; in production, not the text of the query but the whole assembled context.
- **System prompt**—the part of the prompt that sets the invariant of behavior; a soft contract. (→ Chapter 3)
- **Few-shot**—setting format and behavior by examples placed right in the prompt.
- **Context engineering**—the discipline of assembling the right content into the window at every step. (→ Chapter 4)

- **Structured output**—forcing the model onto an output schema (JSON/grammar). (→ Chapter 5)
- **Prompt caching**—caching the static prefix of the prompt for the sake of cost and latency.
- **Constrained (grammar) decoding**—masking the tokens the grammar disallows at each step of generation; guarantees the structure of the output (not its correctness). (→ Chapter 5)

Grounding

- **RAG (Retrieval-Augmented Generation)**—feeding retrieved data into the context instead of, or in addition to, parametric knowledge. (→ Chapter 6)
- **Bi-encoder**—a model that encodes query and document independently (fast retrieval).
- **Cross-encoder / Reranker**—a model that scores the [query, document] pair jointly (precise rescoring). (→ Chapter 7)
- **Chunking**—splitting documents into fragments for indexing. (→ Chapter 7)
- **Hybrid search**—combining dense (vector) and sparse (BM25) search. (→ Chapter 7)
- **ANN / HNSW**—approximate nearest-neighbor search and its popular index structure. (→ Chapter 8)
- **BM25**—the classic sparse (lexical) ranking function: TF-IDF with frequency saturation and length normalization; the lexical half of hybrid search. (→ Chapter 7)
- **RRF (Reciprocal Rank Fusion)**—merging several ranked lists by reciprocal ranks; the standard way to combine dense and sparse results. (→ Chapter 7)
- **GraphRAG**—retrieval over a knowledge graph (entities, relations, communities) rather than over independent fragments. (→ Chapter 7)
- **Faithfulness**—the answer's fidelity to the retrieved context (whether anything was made up); an axis of quality orthogonal to the quality of retrieval. (→ Chapters 6, 9)

Agents

- **Agent**—a system with an LLM as the decision-making core, acting in an observe→decide→act loop; always a pair, "LLM + harness." (→ Chapter 10)
- **Harness (agent harness)**—the deterministic application code around the model: it assembles the context, executes tool calls, carries the state of the loop, and enforces budgets and gates. (→ §10.2)
- **Workflow**—a predefined graph of steps with LLM calls (as opposed to an agent).
- **Tool calling / Function calling**—the mechanism in which the model produces a structured request to call a function and the harness executes it. (→ Chapter 12)
- **Code execution (code mode)**—presenting tools as a code API: the model writes code that composes the calls in a sandbox, instead of a sequence of JSON calls. (→ §12.6)
- **Skill (Agent Skills)**—a versioned artifact of procedural knowledge (SKILL.md: instructions + resources) with progressive disclosure; the model reads it, unlike a tool, which the harness executes. (→ §12.6)
- **ReAct**—the pattern that interleaves reasoning and action (Thought→Action→Observation). (→ Chapter 13)
- **Saga orchestration**—coordinating steps from a center, with compensations; the basis of the Orchestrator-Workers pattern.
- **Human-in-the-Loop Gate (human-gate)**—a deterministic gate requiring human confirmation before an irreversible or consequential agent action. (→ Chapter 10)

Integration

- **MCP (Model Context Protocol)**—an open standard (JSON-RPC) for connecting agents to tools and data. (→ Chapter 15)

- **A2A (Agent2Agent)**—an open protocol for agents to interact with one another (Agent Card, tasks with a lifecycle); complementary to MCP. (→ Chapter 15)

Production

- **Eval**—a measurable check on the quality of a probabilistic system; it replaces a boolean assert with a distribution. (→ Chapter 17)
- **LLM-as-judge**—using an LLM to score another LLM's output against a rubric. (→ Chapter 17)
- **Trajectory evaluation**—scoring every step of an agent's trajectory, not just the final answer. (→ Chapter 17)
- **Prompt Injection**—an attack that exploits the failure to separate instructions from data in a single channel. (→ Chapter 19)
- **Excessive Agency**—an agent holding an excess of permissions/functions/autonomy (agency—in this book these are synonyms, §10.7); an OWASP class of risks. (→ Chapter 19)
- **Guardrails**—deterministic checks on the input and the output around a probabilistic core. (→ Chapter 19)
- **Non-human identity (NHI)**—a machine account (service, bot, agent); the basis of an agent's identity and of delegating its authority. (→ §15.7)
- **Shadow AI**—using LLMs outside the organization's registry and perimeters: personal keys, extensions, "gray" automations. (→ §25.2)

Serving and inference

- **Quantization**—representing weights/activations/cache at reduced precision (int8, 4-bit, FP8/FP4) for the sake of memory and throughput. (→ Chapter 22)
- **PagedAttention**—organizing the KV-cache in pages, on the model of OS virtual memory: it removes fragmentation and lets common prefixes be shared. (→ Chapter 22)
- **Continuous batching**—dynamically slotting requests into the batch as slots free up; the main lever on accelerator utilization. (→ Chapter 22)
- **Speculative decoding**—a draft model proposes several tokens, the target model verifies them in one parallel pass; speeds up decode without changing the distribution. (→ Chapter 22)

Training

- **Fine-tuning**—further training of a model for form/style/schema (not for facts that change). (→ Chapter 21)
- **LoRA / QLoRA**—parameter-efficient fine-tuning with low-rank adapters (+ a 4-bit base). (→ Chapter 21)
- **DPO**—direct preference optimization; a replacement for classic RLHF. (→ Chapter 21)
- **Scaling laws**—empirical relationships tying quality to model size and data volume (Chinchilla). (→ Chapter 23)
- **RLHF**—reinforcement-learning fine-tuning against a reward model trained on human preferences. (→ Chapter 21)
- **RLVR**—RL on verifiable rewards (running tests, checking against a reference); the mechanism behind training reasoning models. (→ Chapter 13)
- **PPO / GRPO**—the RL algorithms for the optimization step in RLHF/RLVR: PPO with a separate value model to estimate advantage, GRPO with a group baseline in its place. (→ Chapters 13, 21)
- **Knowledge distillation**—training a compact "student" to reproduce the output distribution or the reasoning of a "teacher." (→ Chapter 21)

Part I. The Language Model as a Component

Chapter 1. The Language Model as an Architectural Component

1.1 Why you need to understand the model's internals

A language model can be treated as an external service that takes a request and returns a response. For a prototype that is enough; for designing a system it is not. The model has nonfunctional properties that shape the architecture built around it, and they do not reduce to the input and output format:

- cost and latency depend nonlinearly on input length—a consequence of the quadratic complexity of attention (§1.4);
- answer quality degrades not only when the context window overflows but also when it fills up with irrelevant material (§1.8);
- reproducibility of the output is not guaranteed and is governed by decoding parameters (§1.7);
- the choice between a dense and a sparse (Mixture-of-Experts) architecture sets the memory and compute requirements for serving (§1.5, §1.6).

Each of these properties drives design decisions: the context budget, the choice of model, the serving strategy, latency requirements. So the goal of this chapter is not to train a model from scratch (for that, see the sources) but to build a minimally sufficient model of its internals from which these properties follow. The organizing idea of the chapter: a language model is a component with a defined set of properties (collected in §1.10), and much of what follows in this book (prompt design, RAG, agents, operations) is about managing the probabilistic nature of that component.

Definition. A large language model (LLM) is a parameterized function that, given a sequence of tokens, computes a probability distribution over the next token, and is applied autoregressively: the chosen token is appended to the input and the computation repeats. All observable model behavior grows out of this single operation, performed by a model with billions of parameters trained on trillions of tokens. What matters is that the model computes a *distribution* (not a single answer)—from which follow both the nondeterminism of the output (§1.7) and the absence of any built-in signal of truthfulness (§1.9).

Sources for the chapter as a whole: S. Raschka, *Build a Large Language Model (From Scratch)* (2024)—an implementation from scratch; J. Alammar & M. Grootendorst, *Hands-On Large Language Models* (2024)—intuition; Chip Huyen, *AI Engineering* (2025)—the engineering frame; Stanford CS336—the academic course.

1.2 Tokenization

Token—the smallest unit the model operates on: not a character and not a word, but a **subword**, an entry in a vocabulary built by an algorithm of the **BPE (Byte-Pair Encoding)** family. The algorithm has two stages. Training the tokenizer (once, before the model itself is trained) starts from a base alphabet and iteratively merges the most frequent pair of adjacent units into a new one until the vocabulary reaches a target size (usually 30,000–130,000); the result is an ordered table of merges. Applying it segments text against that table deterministically: frequent words and morphemes encode to a single token, rare ones get split. That is how a fixed vocabulary covers an unbounded language without a special "unknown" token.

Byte-level BPE. Modern models (starting with GPT-2) apply BPE not over Unicode characters but over UTF-8 **bytes**. This guarantees that *any* string is representable: an unseen character, an emoji, or corrupted input decomposes into a sequence of byte tokens. The consequence—true out-of-vocabulary does not exist; the "unknown" is simply encoded more expensively (more tokens). For Cyrillic this also explains the fragmentation: a character outside the basic Latin range takes several bytes, and therefore potentially several tokens.

Alternative tokenization algorithms. Although BPE dominates modern LLMs (the GPT and LLaMA families), other powerful approaches exist, both historically and architecturally. What separates them is the mathematical criterion by which characters are merged into subwords or split apart.

1. WordPiece (used in BERT, DistilBERT) If BPE is a "greedy" algorithm that merges pairs of characters purely by their **absolute frequency** in the text, WordPiece (developed at Google) takes a probabilistic approach. It scores adjacent token pairs A and B with a metric close to pointwise mutual information (PMI):

$$Score = \frac{P(AB)}{P(A) \times P(B)}$$

Symbol	What it is	How it is computed	Live value
N	corpus size in tokens	—	100 in the walkthrough below; for BERT, on the order of 10^9
$P(A)$, $P(B)$	the share of token A (or B) among all tokens of the corpus	$count(A)/N$	40/100 = 0.4
$P(AB)$	the share of positions where A stands immediately before B	$count(AB)/N$	10/100 = 0.1
$Score$	dimensionless rank of a pair; the pair with the highest one gets merged	$P(AB)/(P(A)P(B))$	0.1/0.16 = 0.625

In plain terms: the metric asks not "how often does this pair occur" but "how many times more often than if A and B became neighbors by chance." The denominator $P(A) \times P(B)$ is exactly the expected share of the pair under independence: absent any pull toward each other, the tokens would land side by side at precisely that rate. Hence the scale: $Score = 1$—no relationship, coincidence; $Score \gg 1$—these two hold on to each other; $Score < 1$—they avoid each other's company. (A sequence of N tokens has exactly $N - 1$ pairs, not N, but at N in the thousands and above this does not affect the ranking.)

A footnote on implementations. In the teaching implementation from the HuggingFace LLM Course (ch. 6.6) the score is computed on raw counts, $count(AB)/(count(A) \times count(B))$, without dividing by N. That is the same metric up to a constant factor of N: on a fixed corpus N is a constant, the ranking of pairs does not change, and

neither does the choice of which pair to merge.

Looking for this formula in production code, though, is pointless: there are no probabilities there, and no score either. Google never opened the original WordPiece training algorithm, and the course honestly flags its walkthrough as a guess based on published papers. HuggingFace's `tokenizers` library sidesteps the problem radically: its `WordPieceTrainer` builds an ordinary BPE inside, trains it, and only then converts the result into a WordPiece vocabulary (`WordPiece::from_bpe`)—the pair to merge is chosen by raw `count`, and by nothing else. A vocabulary trained by that trainer came out of the BPE criterion, and the PMI metric from the formula above plays no part in it. `min_frequency`, which cuts off rare pairs (more on it below), lives there too and defaults to zero—the filter is off until someone turns it on explicitly. The metric describes WordPiece as an algorithm; the popular implementation of its training does not reproduce it.

How it works under the hood (the iterative loop):

```
vocab = get_base_alphabet() # [a, b, c, ..., z, а, б, в...]
text = load_gigantic_corpus()

# Spin the loop until we have the vocabulary size we want
while len(vocab) < 30000:
# 1. Count the frequencies of all tokens and pairs in the current text

stats = compute_statistics(text)

best_score = -1
best_pair = None

# 2. Find the ONE best pair on this iteration
for pair in stats.pairs:
    if pair.count < min_frequency:
        continue # Throw out the rare junk

    score = calculate_wordpiece_score(pair, stats)

    if score > best_score:
        best_score = score
        best_pair = pair

# 3. Add the best pair to the vocabulary
new_token = merge(best_pair[0], best_pair[1])
vocab.add(new_token)

# 4. UPDATE THE TEXT: replace the pair with the new token
text = replace_in_text(text, best_pair, new_token)
```

The algorithm does not stop at one merge. It runs in a loop, gradually growing the vocabulary from the base alphabet up to the target size (for example, `vocab_size = 30,000`).

1. **Gathering statistics:** The whole training corpus is scanned, and probabilities P (the share of the total token count) are computed for all existing tokens and their pairs.
2. **Ranking:** A Score is computed for every pair.

3. **Merging:** Exactly **one pair** with the maximum Score is selected. It is merged into a new token *AB* and added to the vocabulary.
4. **Re-segmentation (the critical step):** The algorithm walks the entire text and mechanically replaces every adjacent *A* and *B* with the new single token *AB*.
5. **Repeat:** The loop starts over on the updated text. Thanks to the re-segmentation a "snowball effect" appears: on the early iterations `про` and `грам` merge (into `програм`), and on later iterations the new token `програм` can merge with `мист`, forming `программист`. The loop stops when the vocabulary reaches the limit.

Engineering nuances of the WordPiece metric:

- **The popularity penalty:** The denominator $P(A) \times P(B)$ keeps the algorithm from merging frequent but independent letters (for example, "a" and "o"). The Score comes out high only if the tokens occur *together* far more often than *separately*. That forces the algorithm to isolate meaningful morphemes.
- **Protection against rare events (rare event bias):** The same denominator also works against the algorithm: it inflates the score of pairs whose parts almost never occur apart, the limiting case being a pair that occurred exactly once. To keep the algorithm from stuffing the vocabulary with junk, real code applies the `min_frequency` hyperparameter: pairs occurring fewer than 2–5 times are excluded from the computation.

The popularity penalty in numbers. A hypothetical corpus, $N = 100$ tokens. The junk pair `а` + `о` stands adjacent 10 times, the morpheme pair `про` + `грам` 5: the junk is **twice as frequent**, and greedy BPE would have merged it without a second thought. But "а" and "о" do just fine on their own—40 occurrences each—whereas "про" and "грам" occur 5 times each and only next to each other:

$$Score(\text{а}, \text{о}) = \frac{10/100}{(40/100) \times (40/100)} = \frac{0.1}{0.16} = 0.625$$

$$Score(\text{про}, \text{грам}) = \frac{5/100}{(5/100) \times (5/100)} = \frac{0.05}{0.0025} = 20$$

The denominator $0.4 \times 0.4 = 0.16$ collapses the junk pair: the twice-rarer `про` + `грам` wins by a factor of 32. And $0.625 < 1$—"а" and "о" turn up as neighbors even *less* often than chance (16 pairs expected, 10 observed), so the metric does not merely withhold encouragement from the merge, it positively records the absence of a relationship. The winning `про` + `грам` is what opens the "snowball effect" from step 5 above: once it is the token `програм`, later iterations will grow it into `программист`.

Why the metric needs a lower bound. Take a pair whose parts occur only together (like `про` + `грам`): $count(A) = count(B) = count(AB) = c$. The metric collapses:

$$Score = \frac{c/N}{(c/N) \times (c/N)} = \frac{N}{c}$$

For a perfectly welded pair the score is **inversely proportional to its frequency**: the rarer, the higher. One-off junk ($c = 1$) gives exactly $Score = N$, the corpus size: on a toy $N = 100$ that is 100 against 20 for a genuine morpheme. And the gap between them equals $N/(N/c) = c$—the frequency of the morpheme itself: **the more useful and frequent the morpheme, the more badly it loses to one-off junk.** BERT's corpus is BooksCorpus (800M words) plus English Wikipedia (2.5B words), so N is on the order of 10^9 tokens: a pair whose parts occur nowhere else in the corpus would score on the order of N—a billion, that is.

That is an upper bound, not a fact about how BERT was trained. The condition "the parts occur only together" does not hold for a real typo on a corpus that size: its pieces are workaday Latin subwords, they turn up everywhere, $count(A)$ and $count(B)$ are enormous, and its score comes out negligible instead—the very "popularity penalty" from the point above. The threshold protects against perfectly welded junk: a rare name, a fragment of a URL, a random sequence that surfaces nowhere else.

Hence a correction to the common formulation “the denominator tends to zero, so the score tends to infinity”: logically it does not close, because the numerator $P(AB)$ goes to zero along with the denominator, and “the denominator is small” implies nothing on its own. The score of a rare pair does not run off to infinity—it takes the finite value N, which grows with the corpus. The practical conclusion is the same, but now it is proved: the lower bound is not an empirical hack but a forced correction to a metric that by construction ranks the rarest of the welded pairs first.

2. Unigram Language Model (used in ALBERT, T5) Unlike BPE and WordPiece, which build the vocabulary bottom-up (from letters to words), Unigram works **top-down**.

- **Initialization:** The algorithm starts from a giant vocabulary that includes every possible subword from the training corpus.
- **Optimization:** A probability is computed for each subword. The algorithm then iteratively prunes the tokens whose removal costs the least in overall corpus likelihood.
- **Decoding:** At inference time the same word can be split in different ways. Unigram uses the Viterbi algorithm to find the mathematically most probable segmentation path for a given word.

3. SentencePiece (the engineering standard for multilinguality) An important point: SentencePiece is **not an algorithm but a framework** (a Google library) inside which you can run either BPE or Unigram. Its main architectural innovation is dropping pre-tokenization. Standard tokenizers first split text on whitespace and punctuation, which breaks on languages without spaces (Chinese, Japanese) or languages with rich morphology. SentencePiece works on the raw byte/character stream and treats the space as an ordinary character (usually replacing it with the meta-character `_` or `U+2581`). This lets the model learn for itself whether the space is a meaningful separator in a given context.

An engineer’s iron rule: The tokenizer and the network weights are **a single, unbreakable binary contract**. The model does not understand text; it understands only the IDs from its own particular vocabulary. Feeding a LLaMA model (which uses BPE) tokens cut by BERT’s tokenizer (WordPiece) will destroy the semantics entirely (garbage in, garbage out). To count tokens, trim context, or assemble prompts you must instantiate the tokenizer strictly from the target model (for example, via `AutoTokenizer.from_pretrained(model_name)`). Where the resulting array of IDs goes next, and what happens at the CPU/accelerator boundary—§1.6.

Special and control tokens. Beyond subwords, the vocabulary holds service tokens: beginning and end of sequence (BOS/EOS), padding (pad), and, in chat models, **role and turn markup** (separator tokens of the form `<|im_start|>`, `<|im_end|>`, and analogues). Out of these a **chat template** is built—the exact structure in which the roles (system/user/assistant) are presented to the model. This is a practically important and often underrated skill: the model was trained on a *specific* markup format, and hand-assembling a prompt with the wrong separators degrades quality, sometimes into incoherence. The rule: use the tokenizer’s stock chat template rather than constructing the markup by concatenating strings.

Multimodal models (VLMs) integrate non-text data through **special anchor tokens** (for example, `<image>`). The images themselves are processed by a separate vision encoder (for example, ViT) and split into **visual patches**—fragments that are projected into vectors and, at the tensor level, embedded into the shared sequence.

Assembling a multimodal sequence (under the hood) When a VLM takes an image and text as input, it forms a single array of embeddings. Take a concrete model—LLaVA-1.5, whose numbers are used throughout the rest of the section. At the tensor level the assembly looks like a concatenation of vectors:

$$E_{seq} = [E_{BOS}] \oplus \Phi(V_{patches}) \oplus E_{text} \oplus [E_{EOS}]$$

Two stages, not one. The confusion around `<image>` dissolves once you separate in time what the developer writes from what the model receives.

Stage 1 — the prompt string (`<image>` is present here). The developer submits text with a single placeholder token:

```
USER: <image>
What is in the picture? ASSISTANT:
```

`<image>` (id 32000) is the one visual marker. There is no image yet at this stage—only a note that says "the picture goes here."

Stage 2 — the model input (`<image>` is already gone). During embedding assembly this single token is **replaced** by the 576 patch vectors $\Phi(V_{patches})$: the marker is removed and the image block takes its place. That is why E_{seq} has no separate `<image>` term—$\Phi(V_{patches})$ sits exactly where it was: not a neighbor of the anchor but its replacement.

Where:

- **[BOS] (Begin of Sequence):** Always stands at the start. In the Transformer architecture it does not "initialize the hidden state" (as in the old RNNs) but exhibits the **attention sink** property: softmax's excess attention emergently accumulates on the initial positions (Xiao et al., StreamingLLM, 2023), which stabilizes the attention mechanism. This is not a role assigned to the token but a consequence of the architecture; the [BOS] token's K/V are simply cached first during prefill, as for any token.
- **$\Phi(V_{patches})$:** Visual patches passed through a projection adapter (an MLP or a Q-Former). The adapter is what brings the dimensionality of the visual features to the dimensionality of the LLM's text embeddings (D_{model}). The patches themselves arise geometrically: a ViT cuts the image with a regular grid of squares of fixed size $P \times P$ pixels and linearly projects each square into a single vector. Typical P is 14, 16, or 32; in a model's name it is the second number (`ViT-L/14`—"Large, patch 14"). Hence the main consequence, which we will come back to below: the number of patches is not a constant of the model but a function of the input resolution and P.
- **[EOS] (End of Sequence):** Placed at the end of training examples. At inference (generation) time the appearance of this token is a strict, deterministic trigger to stop the autoregressive loop (autoregressive decoding).

Inside the model's hidden space (embedding space) there is no difference between text and image. The identifiers `[BOS]` and `[EOS]` are converted into standard trainable vectors of the same dimensionality D_{model} as the projected visual patches. Thanks to that mathematical alignment, self-attention ($Softmax(QK^T/\sqrt{d})V$) can compute attention weights among service tokens, printed words, and image fragments seamlessly, handling them as one continuous context.

⊕ is gluing, not addition. The symbol ⊕ denotes **concatenation**—assembling a sequence end to end. The rejected alternative is element-wise addition of vectors. *Adding* → means writing the sum into the same array cells: the shape does not change, but text and image get mixed into the same numbers, and the addends cannot be recovered from the sum—self-attention further up the stack would then be fundamentally unable to separate the image from the words. *Gluing* → means lengthening the sequence: each modality takes its own positions and stays separately addressable. That makes the D_{model} requirement from the paragraph above read exactly: the vectors' width has to match so the rows line up in one table (element-wise `+` also requires matching shape, §A.3)—but matching width does not mean the vectors get added.

At the PyTorch level the assembly is one operation, a glue along the token axis:

```
# The numbers are LLaVA-1.5 (ViT-L/14 @ 336px + Vicuna-7B).
e_bos      # [  1, 4096]
v_patches  # [576, 4096]  <- (336/14)^2 = 24 x 24 = 576 patches after the adapter; <image> lands here
e_text     # [ 20, 4096]  <- a prompt of a nominal 20 tokens
e_eos      # [  1, 4096]

E_seq = torch.cat([e_bos, v_patches, e_text, e_eos], dim=0)
# E_seq.shape == [598, 4096]
```

How to read the shapes. The first number is `seq`, the count of token rows; the second is D_{model}, the width of one token's vector (4096 in Vicuna-7B—the base LLM in LLaVA-1.5; the same numbers as in the trace in §1.4). `dim=0` is the index of the axis along which the growth happens; here it is the token axis. The arithmetic: 1 + 576 + 20 + 1 = **598** rows, and 4096 stays 4096. *In plain terms:* gluing extends the table downward but does not widen it sideways—there are more tokens, and each token's vector is unchanged. And unlike `+`, concatenation does not require matching size along the axis it grows on (576 and 20 coexist perfectly well)—only along all the others.

The cost of an image in tokens. Of those 598 rows, 576 are the image: 96% of the sequence against 3% for the prompt text. An image costs not "one embedding" but on the order of 576 tokens, and for attention that is quadratic in the full length (§1.4). That is what gives meaning to the choice of adapter, named $\Phi(V_{patches})$ in the walkthrough above. The **MLP** (a small fully connected network) in LLaVA-1.5 works per patch: 576 patches in yield 576 vectors out—the count does not change, the adapter only fits the width. A **resampler**, rather than passing patches through one-to-one, runs them through a fixed set of trainable **query vectors**: cross-attention, in which each query "reads out" from all the patches what it needs—and returns exactly as many rows as it has queries, regardless of the input resolution. The original **Q-Former** (BLIP-2) has 32 queries; the **Qwen-VL** adapter (officially, a Position-aware Vision-Language Adapter; single-layer cross-attention) has 256. The spread of 32 against 256 under identical mechanics is precisely the resampler invariant: with a fixed-query adapter the count of visual tokens is set by the adapter, not by the input resolution; with a per-patch MLP it is the reverse—the count is inherited wholesale from the resolution. Choosing an adapter is choosing the cost of an image in tokens, not an implementation detail.

Where 576 comes from: grid arithmetic. The invariant above says what the adapter does with the patch count, but not where that count came from. And it comes from pure geometry: the encoder cuts the image with a grid of $P \times P$ squares and returns one vector per square. How many sequence positions that costs takes two steps to work out:

$$N_{patches} = \frac{H \times W}{P^2}, \qquad N_{slots} = \frac{N_{patches}}{m^2} + k$$

Symbol	What it is	Live value
H, W	height and width in pixels **after** the resize for the encoder, not those of the original file	336 × 336 in LLaVA-1.5: the input is hard-fixed (`crop_size: 336×336`)
P	the side of a square patch in pixels	14 in CLIP ViT-L/14-336 and in Qwen2-VL; 16 in ViT-B/16; 32 in ViT-B/32
$N_{patches}$	how many vectors the encoder returns	336 × 336/14² = 24 × 24 = 576
m	the side of the block of patches the adapter merges into one token	m = 1—no merging (LLaVA-1.5, the MLP works per patch); m = 2—four neighboring patches collapse into one (Qwen2-VL)
k	how many service positions go to the framing	k = 2 in Qwen2-VL (`<|vision_start|>` and `<|vision_end|>`—two real positions); k = 0 in LLaVA-1.5: there is no closing token, and `<image>` is replaced by the visual vectors rather than added to them
N_{slots}	the resulting cost of an image in positions	576 in LLaVA-1.5; 66 in Qwen2-VL on a 224×224 input

Live numbers. LLaVA-1.5: 336 × 336/14² = 576 patches, no merging (m = 1), no framing (k = 0)—exactly **576 positions** per image. Qwen2-VL on a 224×224 input: 224 × 224/14² = 16 × 16 = 256 patches, 2×2 merging gives 256/4 = 64 tokens, plus the pair of delimiters—**66**. The Qwen2-VL authors quote that last figure themselves, and it includes both service tokens: even the model's developers count the delimiters into the budget rather than setting them aside.

In plain terms: the first fraction is "how many squares fit into the image," and nothing more; the second formula is a correction for what the particular architecture does with those squares (glues them four at a time or not, spends two positions on boundary markup or not).

Consequence: resolution is a budget line item. In $N_{patches} = HW/P^2$ the resolution sits in the numerator as a **product**, so doubling the *side* of the image quadruples the patch count. And attention is quadratic in sequence length (§1.4)—so that factor of four becomes ×16 the work wherever the visual part dominates. Even without a doubling the count is sobering: 576 visual positions on top of a 1000-token prompt give seq = 1576—the length grew by 58%, the attention work by a factor of about (1576/1000)² ≈ 2.5. **Invariant:** resolution is a parameter of the context budget, not only of image quality; resizing down to the lowest resolution at which the task still works cuts seq and the quadratic price behind it. A caveat: this knob does not always exist, and the reason is the input, not the adapter. LLaVA-1.5's input is fixed, and an image costs 576 patches whether it holds a schematic or an empty background; in models with dynamic resolution (Qwen2-VL) N grows with the resolution up to a set ceiling—and there resolution really is a knob you can turn.

Fertility and multilinguality. *Fertility*—the average number of tokens per word—is a measurable property of the pair "tokenizer × language": close to one for English, substantially higher for Cyrillic. It directly sets cost and the *effective* context length: at an equal window size in tokens, Russian text exhausts it faster, so less meaning fits into the window. Which is why cost and context-budget estimates for a Russian-language system are made by counting tokens with the specific model over a representative sample of documents, not by counting characters: the phrase "автоматизация процессов" (23 characters) encodes to 6–9 tokens against 2–3 for "process automation,"

and across hundreds of thousands of documents that difference determines both the bill and how many fragments will fit into the window during retrieval (Part III). Made in advance, such an estimate makes cost and capacity predictable before launch.

Pitfalls that lead to bugs. Several practical traps follow directly from how tokenization works:

- *Numbers*. Multi-digit numbers are split into tokens inconsistently (one digit at a time, or in groups), which makes arithmetic and exact digit manipulation unreliable; for tasks involving numbers this gets designed for (check the result, move the computation out to an external tool).
- *Whitespace and boundaries*. In most tokenizers a leading space is part of the token (`the` and `the` are different tokens). Hence the sensitivity to boundaries: breaking in the middle of a "natural" token keeps the model from continuing optimally—a problem solved at the decoding level (token healing, §5.3).
- *Code*. Source code tokenizes inefficiently unless the tokenizer was adapted for it (indentation, long identifiers), which inflates both length and cost.
- *Glitch tokens*. Rare strings that made it into the vocabulary but barely occurred during training (the `SolidGoldMagikarp` example, Rumbelow & Watkins, 2023) trigger anomalous behavior—a demonstration that the model's picture of text is mediated by the vocabulary.

Vocabulary size as a trade-off. A larger vocabulary shortens sequences (cheaper for attention, §1.4) but grows the embedding and output-projection matrices—memory for parameters (§1.6). This is a design decision made when the model is built; when you use it, what matters is the consequence: vocabulary size is fixed together with the model and determines its fertility. Tokenization also interacts with **structured output**: because the model emits tokens, constraining the output by a grammar or a schema (constrained decoding, §5.3) operates at the token level and accounts for which tokens are admissible at each position.

Comparing segmentation approaches.

Approach	Advantage	Drawback
By characters	Tiny vocabulary, no unknown words	Very long sequences, weak signal per unit
By words	Short sequences	Huge vocabulary, unknown words on rare vocabulary
Subwords (BPE / WordPiece / Unigram)	Balance of coverage and length, no true OOV (byte level)	Splitting artifacts, unevenness across languages

Tools. For counting and inspecting tokens, use `tiktoken` (OpenAI models) and the specific model's tokenizer via the `transformers` library; the key requirement is to use the same tokenizer the target model uses.

Sources. R. Sennrich, B. Haddow, A. Birch, "Neural Machine Translation of Rare Words with Subword Units" (2016)—BPE. M. Schuster, K. Nakajima, "Japanese and Korean Voice Search" (2012)—WordPiece. Y. Wu et al., "Google's Neural Machine Translation System" (2016)—the likelihood criterion of WordPiece. J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers" (2018)—BERT's corpus and vocabulary. HuggingFace LLM Course, ch. 6.6, "WordPiece tokenization"—the teaching implementation of the metric on raw counts. T. Kudo, "Subword Regularization" and SentencePiece (2018)—Unigram LM. A. Karpathy, "Let's build the GPT Tokenizer" (2024). Rumbelow & Watkins, "SolidGoldMagikarp" (2023). H. Liu, C. Li, Y. Li, Y. J. Lee, "Improved Baselines with Visual Instruction Tuning" (arXiv:2310.03744, 2023)—LLaVA-1.5: CLIP-ViT-L-336px + a two-layer MLP connector. J. Bai et al., "Qwen-VL" (arXiv:2308.12966, 2023)—Position-aware Vision-Language Adapter: single-layer cross-attention, compression of the visual run to 256, delimiters `<img>` / `</img>`. J. Li, D. Li, S.

Savarese, S. Hoi, “BLIP-2” (arXiv:2301.12597, 2023), §3.1—Q-Former: 32 queries of dimensionality 768. S. Zhang, Q. Fang, Z. Yang, Y. Feng, “LLaVA-Mini” (2025)—24×24 = 576 visual tokens in LLaVA-1.5. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale” (arXiv:2010.11929, 2021)—ViT and the patch grid $N = HW/P^2$. P. Wang et al., “Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution” (arXiv:2409.12191, 2024)—dynamic resolution, 2×2 patch merging, delimiters `<|vision_start|>` / `<|vision_end|>`, the authors’ own estimate “224×224 at patch_size=14 → 66 tokens.” The vocabulary and geometry were checked against vendor artifacts: `llava-hf/llava-1.5-7b-hf` (`added_tokens.json`—only `<image>` 32000 and `<pad>` 32001; `config.json`—`image_size: 336`, `patch_size: 14`), `liuhaotian/llava-v1.5-7b` (`config.json`—`mm_use_im_start_end: false`), `Qwen/Qwen2-VL-7B-Instruct` (`config.json`—`vision_start_token_id: 151652`, `vision_end_token_id: 151653`; `preprocessor_config.json`—`patch_size: 14`, `merge_size: 2`).

1.3 Embeddings and the vector space

Embedding—a mapping of a token or a fragment of text into a **dense vector** of fixed dimensionality `d` (typically 384, 768, 1024, 1536, 4096) under which geometric proximity of the vectors corresponds to semantic proximity of the objects. The basis is the **distributional hypothesis** (Firth, 1957): the meaning of a unit comes from the contexts it is used in, so units from similar contexts get similar vectors.

Two different notions worth keeping apart. *The model’s internal embeddings*—the transformer’s first layer, mapping a token identifier to a vector, which attention then enriches with context; part of the language model itself. *Embedding models*—separate models that encode a *whole fragment* into a single vector for search; RAG is built on them (Part III).

Where these vectors physically live. The distinction is not only structural but hardware-level—and that is the most reliable way to keep from confusing them. An embedding model’s vectors are *data*: they lie in a vector store (pgvector, Qdrant, Milvus), that is, in the RAM or on the SSD of an ordinary database server, next to the ANN index graph (§8.3). The model’s internal embeddings are *computation*: they arise in accelerator memory (HBM°, §1.6) inside the inference process and never leave it. Different hardware, different processes, different ops teams.

What is permanent here and what is instantaneous. Inside the model itself, “embedding” names two different things, and they need pulling apart. Permanent is the **embedding matrix**—a lookup table of size `V × d_model`, one row per vocabulary token; these are trainable weights, they sit in accelerator memory for the whole life of the process and count toward the “model size” (§1.6). Its volume:

```
Embedding matrix (bytes) = V · d_model · bytes
```

- `V`—vocabulary size; in Llama 3, 128,256 tokens.
- `d_model`—the dimensionality of the model’s internal vector; in Llama 3 8B, 4096.
- `bytes`—bytes per number; at bf16, 2.

Substituting: 128,256 · 4096 · 2 = 1,050,673,152 bytes ≈ 0.98 GB—almost a gigabyte of weights for the lookup table alone (and that is without the output projection: in Llama 3 8B it is a separate matrix of the same size, the weights are not tied, §1.2). Instantaneous is that table’s **output**: one token’s vector is 4096 · 2 = 8 KB of activations, which live within a single forward pass and are freed after it. The ratio of permanent to instantaneous here is exactly `V`:

one row against the table's 128,256. *In plain terms:* what sits in accelerator memory forever is a **vocabulary of vectors**, while a particular token's vector is a consumable: computed, passed up the stack, thrown away. What survives across decode steps is not it but the KV-cache (§1.6).

A vector in a RAG index is built exactly the other way around: in pgvector it takes `4 · d + 8` bytes, that is ≈4 KB at `d` = 1024 (bge-m3), and it is written to disk once—to lie there for months, until the embedder changes and everything is reindexed (§8.8). Per vector, both cases come to the same order of magnitude (kilobytes), but the lifetimes differ by orders: milliseconds against months.

≠ Do not confuse: RAG vectors never reach the model at all. The commonest misconception about RAG is that the retrieved vectors get "fed" to the model. They do not. A fragment's vector is needed for exactly one operation—comparison against the query vector during nearest-neighbor search (§8.1); once the neighbors are found, the vector has done its job. What goes into the context is the **text** of the fragment, pulled from storage by identifier. That text then travels the whole path over again: the model's tokenizer (§1.2) → token identifiers → the model's embedding matrix → its internal vectors. So the fragment makes a loop, "text → embedder vector → (search) → text → model vector," and the two vector spaces never touch once: the only thing they exchange is text. Hence a practical consequence that disposes of a needless question: the embedder's dimensionality (1024) and the model's `d_model` (4096) need not match and usually do not—there is nothing to reconcile, they never meet.

Below, the discussion is mostly about the second kind.

How tokens turn into a single vector: pooling. An embedding model emits one vector per token; to get a single vector for the fragment, the vectors are aggregated. The main strategies: *mean pooling* (averaging over the tokens—a common choice), *CLS pooling* (the vector of the special token in BERT-like models), *last-token pooling* (the last token's vector in decoder embedders). Pooling is part of a given model's architecture; what matters practically is that it affects quality and cannot be swapped out at will.

Why vector proximity reflects meaning: contrastive training. This is not a self-evident property but the result of training. An embedder is trained with a contrastive objective on pairs: the vectors of semantically close texts are pulled together, distant ones pushed apart under a chosen measure. Technically, negative examples are often taken *within the batch* (in-batch negatives), and quality depends heavily on how "hard" the negatives are. After training, the proximity of directions becomes an estimate of semantic proximity—and that is what justifies using cosine similarity rather than an arbitrary function.

InfoNCE—the loss function that creates this geometry. The standard contrastive objective is called **InfoNCE** (Oord et al., 2018; noise-contrastive estimation—telling a real signal from admixed noise). For a query q, its relevant document d^+, and a set of negatives d_j^-:

$$L = -\log \frac{\exp(\mathrm{sim}(q, d^+)/\tau)}{\exp(\mathrm{sim}(q, d^+)/\tau) + \sum_{j=1}^{N} \exp(\mathrm{sim}(q, d_j^-)/\tau)}$$

- q, d^+, d_j^-—the vectors of the query, of the relevant document (the positive, exactly one per query), and of the irrelevant ones (negatives, $j = 1..N$); with in-batch negatives $N = B - 1$, where B is the batch size.
- sim—the similarity measure; in practice the cosine between L2-normalized vectors, dimensionless, range $[-1, 1]$.
- τ—the **temperature**, dimensionless, typically 0.01–0.05: in E5 it is 0.01 (L. Wang et al., 2022, §4.1), in SimCSE 0.05 (T. Gao et al., 2021, table D.1). The mechanics are exactly those of decoding temperature (§1.7)—divide by τ before softmax—but the job and the scale differ: there, T on the order of unity governs the diversity of generation at inference; here, τ is one to two orders of magnitude smaller and decides which negatives make it into the gradient during training at all.
- L—the loss in nats; zero means "the positive was identified without error."

In plain terms. This is the ordinary cross-entropy of the task "guess which of the $N + 1$ candidates is the real one," and the penalty is charged for exactly one thing: the model failing to separate the positive from the negatives **by cosine**. Two things follow. First, about the measure: the cosine sits *inside* the loss, so training optimizes not "meaning in general" but precisely the quantity written into sim; the requirement below (see "Normalization and similarity measures")—that the search measure must match the training measure—is not a convention of convenience but a condition of correctness. Second, about the negatives: what puts meaning into the vectors is not the positive but the crowd it had to be told apart from.

Live numbers: what τ actually does. Suppose the cosine to the positive is 0.80, to one hard negative 0.75, and the batch holds another 30 easy negatives with a cosine of about 0.10.

- $\tau = 0.1 \rightarrow$ logits 8.0, 7.5, 1.0. Denominator: $2981 + 1808 + 30 \times 2.72 = 4871$. The easy negatives supply 1.7% of the denominator—little, but not zero. $L = -\ln(2981/4871) \approx 0.49$.
- $\tau = 0.01 \rightarrow$ logits 80, 75, 10. The positive leads the hard negative by a factor of $e^5 \approx 148$, while all 30 easy ones together contribute $30 \cdot e^{-70} \approx 10^{-29}$ of the denominator—for the gradient they simply are not there. $L = \ln(1 + e^{-5}) \approx 0.0067$, and that loss is generated entirely by the single hard negative.

Shifting τ by an order of magnitude is not "slightly different sharpness" but a decision about how many negatives take part in the gradient at all. The property has been proved formally: contrastive losses are **hardness-aware**—the closer the negative, the harder it gets pushed away. The gradient with respect to a negative is proportional to $\exp(\text{sim}/\tau)$: the penalty grows with proximity not linearly but **exponentially**, and τ sets the steepness of that growth (F. Wang, H. Liu, 2021, §3.1). Which is exactly why a cosine gap of a mere 0.05 turned into a factor of 148 above, and why the easy negatives fell through to 10^{-29}: what enters the gradient is not proximity itself but its exponential. In the limit $\tau \rightarrow 0$ InfoNCE degenerates into a triplet loss with zero margin and looks **only at the nearest negative** (ibid., §3.2). The flip side is the "uniformity-tolerance dilemma" (ibid., §4): an excessively small τ crushes the nearest neighbors, and those are precisely the ones most often semantically akin to the query.

Batch size is a quality hyperparameter, not a performance one. This follows straight from the denominator: it holds $B - 1$ negatives, so a bigger batch = more candidates the positive must be told apart from = a harder task = a stronger signal. Measurably: in E5 pretraining the average score across six sets rises 45.8 → 50.2 → 51.6 as the batch goes 1K → 8K → 32K (L. Wang et al., 2022, table 5)—almost 6 points from batch size alone; the final configuration uses $B = 32\,768$. The engineering consequence is counterintuitive for a backend developer: you cannot trim the batch here "to save memory"—what gets trimmed is the model's quality. Working around that is exactly what **GradCache** (Gao et al., 2021) exists for—a technique that breaks the encoder's backward-pass dependency along the batch axis, which makes it possible to compute a 32K batch in memory it does not wholly fit into.

A failure mode: false negatives. Hard negatives are obtained by mining: run the query over the corpus with the current model and take the top of the results. The trap sits exactly where the payoff is: the top of the results is the very place the *genuinely* relevant documents lie, merely unlabeled. In numbers: the RocketQA authors hand-labeled the top results that carried no positive label in MS MARCO and found that **70% of them are in fact positives** (Qu et al., 2021, §1). Plug such a document into the formula as d^- and it will honestly push a relevant document away from the query—and at a small τ it will pour almost the entire gradient of the step into doing so. Unfiltered mining poisons retrieval the more precisely the sharper τ is.

The fix is the same in spirit as WordPiece's `min_frequency` (§1.2)—do not trust raw statistics:

- *Filtering by a teacher*. RocketQA rescores candidates with a cross-encoder (§7.4) and keeps as negatives only those the teacher confidently predicted to be negative (in their implementation, with a score below 0.1).

- *A rank cutoff*. In `sentence-transformers` this is what the `range_min` parameter of `mine_hard_negatives` is for—the minimum rank from which a candidate is considered a negative at all; its purpose is documented outright as "skip the most similar texts, so as not to label as a negative what is in fact a positive." There is no canonical value—the default is 0, meaning the filter is off—so the rank window gets tuned on your own data.

A practical invariant: in a dataset for contrastive fine-tuning (see "Domain adaptation" below) the negatives are not filler but half the signal. Query-positive pairs without thought-through, filtered negatives produce a model that never learned to *tell anything apart*.

Asymmetry and instruction prefixes—a practical pitfall. Many modern embedders are *instruction-tuned* and asymmetric: query and document are encoded with different prefixes (for example, `query:` and `passage:`, or an instruction like "represent this sentence for search"). If the prescribed prefix is not supplied, search quality degrades *silently*—no errors, no warnings. This is one of the commonest practical mistakes: using a model without the input format it was built for. The rule—apply exactly the query and document format the model card specifies.

Normalization and similarity measures. With L2-normalized vectors (brought to unit length) the dot product equals cosine similarity—which is why search normally normalizes and uses the cosine or an equivalent dot. The measures: *cosine* (the angle, invariant to length—the standard for semantics); *dot product* (accounts for length—when length carries meaning); *Euclidean distance* (on normalized vectors, monotonically related to the cosine). The chosen measure is fixed when the index is configured (Chapter 8) and must match the one the model was trained on.

Anisotropy. Raw embeddings from language models suffer from *anisotropy*—they occupy a narrow cone in the space, so arbitrary pairs look similar and proximity separates poorly. Specialized embedding models correct this through training (contrastive training among others), yielding a more evenly distributed space. The practical conclusion: for search, take a dedicated embedding model rather than pulling vectors out of a generative one.

Dimensionality as a trade-off. A larger `d` holds more distinctions but linearly increases memory and the cost of searching the index (§8). *Matryoshka Representation Learning* trains an embedding so that it can be **truncated** to a smaller dimensionality with a controlled loss of quality—a flexible trade of "memory and speed ↔ accuracy" without retraining (truncating 1024→256 for a cheap first pass, for example).

Evaluation and model choice. The standard reference point is the **MTEB**[°] benchmark (retrieval, semantic textual similarity, clustering, reranking, and other tasks); pick a model by *the* task that matches your target one (for RAG, retrieval), and always check it on *your own* data rather than by the leaderboard alone, since domain and language specifics decide. For Russian-language systems, strong Russian support is critical: many top English-oriented models have worse fertility and quality on Russian, which you verify on monolingual and cross-lingual pairs.

Domain adaptation. The most powerful lever on retrieval quality in a narrow domain is *fine-tuning the embedder* on query-relevant-document pairs from that domain (the pairs come from click and feedback logs, or are generated synthetically and then checked). This often buys more than switching the retrieval pattern (Chapter 7).

Application. The practical point of all the above is semantic search: each fragment of the corpus is encoded into a vector and stored in a vector store (pgvector, for example); the query "how do I return a defective item" is encoded into a vector and matched against the nearest fragments even if the document used different wording—"the procedure for filing a complaint about substandard goods." The value is that search runs on meaning rather than on word overlap, so the material gets found when the query's vocabulary and the document's diverge. The limitation is of the same nature: literal terms, codes, and part numbers, where an exact match matters, can be missed by semantic search—which is what motivates hybrid search (§7.3). The operational consequence: vectors from different models are incomparable, so changing the embedder means fully reindexing the corpus (§8.8), and a

model is chosen with long use in mind; to save memory, vectors are additionally quantized (§8; this is quantization of index vectors—not to be confused with quantization of model weights, Chapter 22) with the quality loss kept under control.

Sources. T. Mikolov et al. (2013)—word2vec. N. Reimers, I. Gurevych, "Sentence-BERT" (2019). N. Muennighoff et al., "MTEB" (2022). A. Kusupati et al., "Matryoshka Representation Learning" (2022). A. van den Oord, Y. Li, O. Vinyals, "Representation Learning with Contrastive Predictive Coding" (2018)—InfoNCE. L. Wang et al., "Text Embeddings by Weakly-Supervised Contrastive Pre-training" (2022)—E5. F. Wang, H. Liu, "Understanding the Behaviour of Contrastive Loss" (2021)—temperature and hard negatives. T. Gao, X. Yao, D. Chen, "SimCSE: Simple Contrastive Learning of Sentence Embeddings" (2021)—temperature 0.05. Y. Qu et al., "RocketQA" (2021)—false negatives. L. Gao et al., "Scaling Deep Contrastive Learning Batch Size under Memory Limited Setup" (2021)—GradCache.

1.4 Attention and the transformer *(a long-form section)*

This is the core of the architecture: the attention mechanism sets both the model's capabilities and its computational cost.

The problem attention solves. Earlier architectures processed text recurrently (RNN, LSTM): token by token, carrying a hidden state along. That imposed two limits: the sequence cannot be parallelized, since each step depends on the previous one (slow training), and information about long-range dependencies decays as it is carried through the state (the beginning of a long text "washes out" by the end). Attention removes both limits: it lets every position address any other directly in one step, and it is computed in parallel for the whole sequence.

Scaled dot-product attention. Every position is projected by three trainable matrices into three vectors: **Query (Q)**, **Key (K)**, and **Value (V)**. The computation is given by the formula:

```
Attention(Q, K, V) = softmax( Q·Kᵀ / √d_k ) · V
```

Step by step:

1. `Q·Kᵀ`—the matrix of pairwise dot products between queries and keys. A dot product measures how well two vectors' directions agree, that is, how well one position's query matches another's key; the result is a score of how much each position should take each other position into account.
2. Dividing by `√d_k`—scaling, and the justification is rigorous: with independent components of unit variance, the variance of a dot product grows linearly with dimensionality `d_k`, so its magnitude is on the order of `√d_k`. Without the normalization, a large `d_k` makes softmax's arguments large, the distribution degenerates (one weight close to 1, the rest close to 0), the softmax gradient goes to nearly zero, and training stalls. Dividing by `√d_k` brings the variance back to one and keeps softmax in its working range.
3. `softmax`—turns the scores into weights summing to one: a position's attention distribution over all positions.
4. Multiplying by `V`—a weighted sum of values: the new representation of a position is a blend of the values of the positions its attention is spread across.

Multi-head attention. One set of Q/K/V captures one kind of relationship. So the computation is run in `h` **heads** in parallel: `d_model` is split into `h` subspaces of dimensionality `d_head = d_model / h`, attention is computed independently in each, the results are concatenated and passed through an output linear projection.

Different heads specialize in different relationships—syntactic links, coreference, long-range topical dependencies—which raises expressiveness at the same total dimensionality.

The causal mask. In a generative (decoder-only) model, position `i` must not take future positions `> i` into account: otherwise predicting the next token would become trivial, and generation incorrect. Before softmax, the matrix elements corresponding to future positions are filled with `−∞`, which gives zero weight after softmax. The causal mask is what makes autoregression (§1.6) correct.

The KV-cache as a consequence. During autoregressive generation the keys and values of already-processed positions do not change, so they are cached and reused at every step rather than recomputed. This cache (KV-cache) is the biggest consumer of memory at inference after the weights themselves (§1.6), and its size is exactly what motivates the attention variants below.

Attention variants and what motivates them. What follows are ways to economize on KV-cache memory while holding quality:

- **MHA (multi-head attention)**—the base variant: every head has its own K and V. Maximum expressiveness and maximum KV-cache.
- **MQA (multi-query attention)**—all heads share a single set of K, V. Minimum KV-cache at the price of a noticeable loss of quality.
- **GQA (grouped-query attention)**—the compromise: heads are split into groups, and within a group they share a set of K, V. Substantially less KV-cache at quality close to MHA; used in Llama 2 (the larger 34B/70B variants), in all sizes of Llama 3, in Mistral, and others.
- **MLA (multi-head latent attention)**—compressing K, V into a low-dimensional latent representation from which they are reconstructed; shrinks the KV-cache further. Used in DeepSeek-V2/V3.

Positional encoding and RoPE. Attention itself is order-invariant (an operation over a set), so positional information is supplied separately. The modern standard is **RoPE (Rotary Position Embedding)**: the Q and K vectors are rotated by an angle proportional to the position, so that the dot product between positions `i` and `j` depends on their difference `i − j`—that is, the *relative* position is what gets encoded. The mechanics in two lines: the vector's components are grouped into pairs, and each pair is rotated in its own plane by an angle θ_i·pos with its own frequency θ_i (fast frequencies encode local order, slow ones long-range context); and since rotating Q by α and K by β changes their dot product as a function of the difference (α − β), attention sees only the relative offset. RoPE's practically significant property is its ability to extrapolate to lengths not seen during training; context is extended by interpolating the frequencies (NTK-aware scaling, YaRN°), but that too has a limit—beyond the training lengths quality degrades (related to §1.8).

Complexity and its practical consequences. The matrix `Q·Kᵀ` has size `n×n` at length `n`, so attention's computational complexity and intermediate memory are **O(n²·d)**: doubling the context length quadruples attention's work. The direct practical consequences:

- feeding all available text into the context does not scale—cost and latency grow quadratically, so retrieving what is relevant (RAG, Part III) beats "load everything" not only on quality (§1.8) but on compute as well;
- quadratic growth is a target for optimization. **Sliding-window attention** limits attention to a window of the last `w` positions: a row of the attention matrix shrinks from `n` to `w`, so the table holds `n` rows × `w` columns —**O(n·w)** cells instead of `n²`. The factor `d` (the dimensionality of one position's vector) enters both estimates identically and therefore cancels in the comparison: written out in full it is `O(n·w·d)` against `O(n²·d)` above. The linearity arises precisely because `w` is a constant of the architecture rather than a property of the request (Longformer, Beltagy et al., 2020); in Mistral 7B, `w = 4096`. Hence the live count: the saving equals `n/w` and appears only once the context is longer than the window. At an arbitrary length `n = 32,768` (the number here is for illustration only) one head's table compresses from `32,768² ≈ 1.07 billion` cells to

`32,768 × 4096 ≈ 134 million`—exactly 8×; at `n ≤ 4096` the window cuts nothing off and saves nothing. The price is usually stated as losing *direct* access to distant positions—and the word "direct" carries the whole meaning here (see the next paragraph); **sparse attention** limits the matrix to a prescribed pattern of connections.

The window does not cut off distant context: the receptive field grows with depth. The window limits not what a token *knows* but where it looks directly—and that limit is applied afresh at every layer. In Mistral 7B, the state of position `i` at layer `k` sees the previous layer's states in the range `i − w … i` (Jiang et al., 2023). But the previous layer's states have already absorbed their own `w` neighbors: at layer 1 position `i` gathered `i − 4096 … i`; at layer 2 it looks at those same nearest neighbors, and each of them carries another 4096 positions back—the total view doubles to 8192. So over `L` layers a **receptive field** accumulates—the maximum distance from which information is capable of reaching a position at all:

$$R = w \cdot L$$

Symbol	What it is	In Mistral 7B
`w`	Window width: how many positions back a token looks directly at one layer	4096
`L`	The number of layers; the window is applied afresh at each	32
`R`	The top layer's receptive field, in tokens	4096 × 32 = 131,072 ≈ 131k

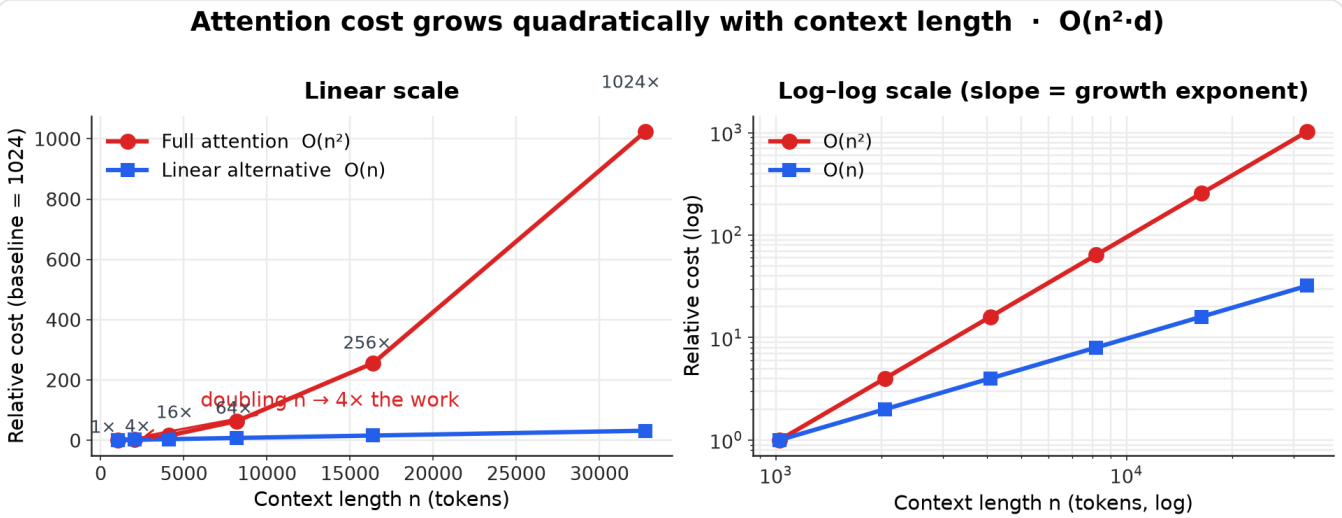
In plain terms: a token at the top layer carries traces of 131 thousand preceding tokens, although at no layer did it look further back than 4096. The information travels not in one jump but as a relay—layer by layer, through the neighbors' intermediate representations. The relay analogy is convenient but breaks down on the main point: in a relay an *object* is handed over, whereas here a token's vector at each layer is a weighted average of `w` neighbors (step 4 of the trace below)—on each handoff the content is not copied but blended into a mixture.

Hence the limit of applicability. The paper itself calls the 131k a *theoretical attention span*, and the word is chosen precisely: `w·L` sets where information is *capable* of reaching, not the length over which it arrives still distinguishable. A distant ancestor's contribution washes out the more blendings it has been through—which is why a sliding-window model is not equivalent to a full-attention model at the same length. The gap here is of the same nature as the one between a window's stated and effective size (§1.8): there is no published norm for exactly how much quality sags beyond `w`—the amount depends on the model and the task. The engineering conclusion: `w·L` is an upper bound on the view, not a working length; if a system leans on connections beyond the window, measure the view on your own task (§4.3).

FlashAttention—an important clarification. FlashAttention (Dao et al., 2022) is often put in the same row as the variants above, but it is *not* an approximation and not a change of architecture: it computes **the same** result exactly. It is an IO-aware algorithm: the computation is broken into blocks (tiling), and the full `n×n` matrix is never materialized in the GPU's slow memory (HBM)—what is needed gets recomputed in fast memory. As a result, HBM traffic and memory consumption drop from quadratic to linear in `n`, which speeds up both training and inference on long sequences dramatically. The FlashAttention-2/3 versions develop the same principle.

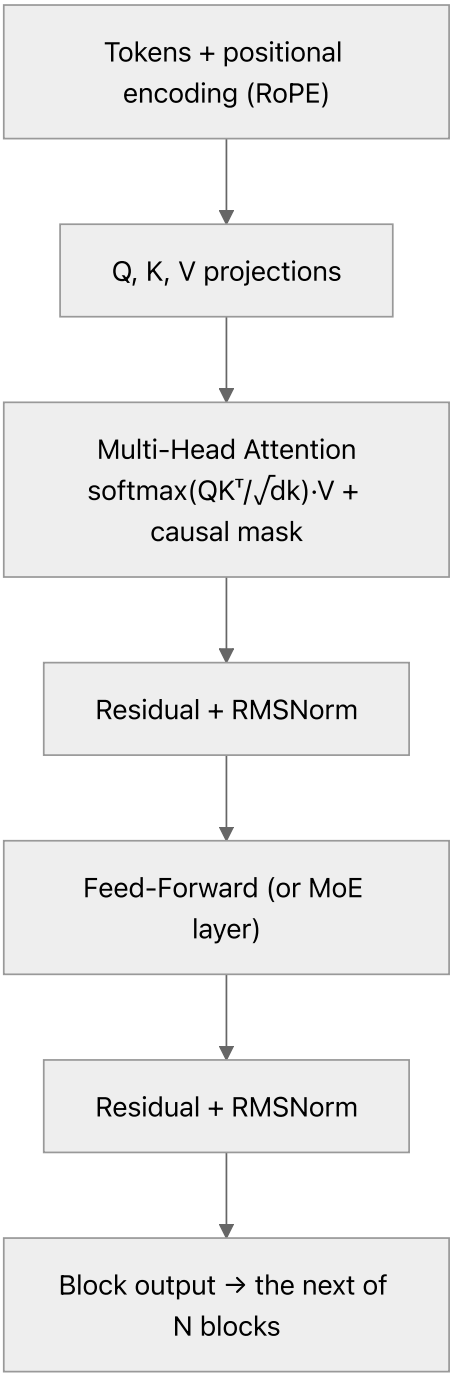
Comparing the attention variants.

Mechanism	Idea	Effect
MHA (multi-head)	Each head has its own K/V	Baseline; maximum KV-cache
MQA (multi-query)	One set of K/V for all heads	Minimum KV-cache, loss of quality
GQA (grouped-query)	Groups of heads share K/V	Less KV-cache at quality close to MHA
MLA (multi-head latent)	K/V compressed into a latent space	Less KV-cache still
Sliding-window	Attention within a window of the last `w` positions	`O(n·w·d)` instead of `O(n²·d)`; long-range links not directly but by accumulation across layers (`w·L`)
FlashAttention	Exact computation without materializing `n×n`	Dramatically faster and leaner (a systems optimization, not an architectural one)



At 32k context, full attention takes ~1000× the work relative to 1k — a direct consequence of the n×n matrix (§1.4). The values are exact (this is the complexity itself).

A diagram of the transformer block.



A full transformer is a stack of `N` such blocks (in large models, tens to hundreds), closed off by a linear projection into the vocabulary (which gives the logits, §1.7). **Residual connections** (adding a block's input to its output) and **normalization** (RMSNorm°) are what make depth like that trainable, preventing the gradient from vanishing as the signal passes through many layers.

Deep Dive: linear layers and the block's computational flow

Attention above was given conceptually; here—*which operations* the block actually consists of, because the architectural consequences (cost, memory, what the GPU does) are visible only at this level. The full vocabulary of operations is Appendix A; here—how the transformer reduces to them.

The linear layer is the basic element. Everything a transformer calls a "projection" is a **linear layer**: `y = xW (+ b)`, that is, a **matrix multiplication** of the input by a trainable weight matrix `W`. The key realization: **the model's parameters are exactly these weight matrices.** A "7B-parameter model" = the total size of all the `W` in its linear layers (plus the embeddings). A linear layer does not "understand" anything—it multiplies by a matrix.

Where the linear layers are in the block (almost the entire mass of weights sits in them). The notation `a → b` reads "the layer takes a vector of `a` numbers and returns a vector of `b` numbers"; its weight matrix has shape `a × b`, that is, `a·b` parameters:

- **The Q, K, V projections**—three linear layers `d_model → d_model` (the result is cut into `h` heads of dimensionality `d_head = d_model/h`).
- **The output projection** of attention—a linear layer `d_model → d_model`.
- **FFN**—in the classic variant, *two* linear layers: `d_model → d_ff` (usually `d_ff ≈ 4·d_model`) and back, `d_ff → d_model`, with a nonlinearity between them. Most modern open-weight models (Llama, Mistral, Qwen, Gemma) use the **gated variant (SwiGLU)** with *three* matrices: gate and up projections `d_model → d_ff`, whose results are multiplied element-wise (with an activation on the gate branch), then a down projection `d_ff → d_model`; to hold the parameter budget across three matrices, theoretical parity gives `d_ff ≈ 8/3·d_model`; in practice models take more—for the Llama-3-8B configuration used below, `d_ff/d_model = 14336/4096 = 3.5`. The FFN holds the greater part of the block's parameters—and it is exactly what MoE replaces (§1.5).

A legend for the notation. Before reading the formulas—what each symbol means. Every number in the third column is real, taken from the open Llama-3-8B, so that concrete quantities stand behind the letters:

Symbol	What it is	In Llama-3-8B
`seq`	Sequence length in tokens. The only quantity that depends on the request—everything else is fixed by the architecture	from 1 to 8192
`d_model`	The model's "width": how many numbers are in the vector representing one token inside the block	4096
`h`	The number of attention heads	32
`d_head`	The piece of the vector that goes to one head: `d_model / h`	4096/32 = 128
`d_ff`	The FFN's inner width (wider than `d_model`)	14336
`x`	The block's input—not a "variable" but a whole table `seq × d_model`: one *row* per token, `d_model` numbers in the row	e.g., 1000 × 4096
`Wq, Wk, Wv, Wo, W1, W2`	Trainable weight matrices—the very "model parameters" from the paragraph above. Fixed after training	each ~4096 × 4096 or 4096 × 14336

How to read the shapes. The notation `seq × d_model` is a table's shape: rows × columns. There is one rule, and it explains every step below: multiplying tables `(a × b) · (b × c) → a × c` is allowed when the "inner" sizes match—they collapse, and the "outer" ones remain. An example: `(1000 × 4096) · (4096 × 4096) → 1000 × 4096`—a thousand tokens went in, a thousand came out, but every vector was recomputed. The `ᵀ` symbol (transpose) swaps rows and columns; it is used to line shapes up. Nothing else from linear algebra is needed to read the trace (the full vocabulary is Appendix A).

A trace of the flow (the input is `x` of shape `seq × d_model`); after each step's formula comes its reading in words:

1. `Q,K,V = xWq, xWk, xWv` → three tables `seq × d_model`, each cut into heads → `h × seq × d_head`. *In plain terms:* each token's vector is multiplied by three different weight matrices—producing three "readings" of one token: Q—"what I am looking for," K—"what I can be found by," V—"what I hand over if I am picked" (the roles from the conceptual description above). Cutting into heads is literally slicing a vector of length 4096 into 32 pieces of 128 numbers: from there on, every head works with its own piece independently (and therefore in parallel).
2. `scores = Q·Kᵀ / √d_head` → a table `h × seq × seq`. *In plain terms:* for every pair of tokens, the dot product of one's "query" and the other's "key" is computed—one number, a measure of their compatibility. The pairs are "everyone with everyone," hence the `seq × seq` table in each head: at 1000 tokens that is 1,000,000 numbers per head—**this is the physical origin of the quadratic price of long context** (§1.4). The transpose `Kᵀ` here is purely technical: `(seq × d_head) · (d_head × seq) → seq × seq`—the shapes were lined up. Dividing by `√d_head` is a normalization of scale: dot products of long vectors are big numbers, and the next step's softmax "saturates" on big numbers (emitting nearly pure 0/1 and losing the ability to tell shades apart); the division brings the numbers back into the working range.
3. `weights = softmax(scores)`—leaves the shapes unchanged. *In plain terms:* every row of the compatibility table turns into a set of positive weights summing to 1—the token's "attention budget," spread over all the tokens visible to it. What was a "raw compatibility number" has become a "share of attention."
4. `attn = weights·V` → `h × seq × d_head`, the heads are concatenated back → `seq × d_model`. *In plain terms:* the token's new vector is a *weighted average* of the V vectors of all visible tokens with the weights just computed: the token "collects" information from those it looks at, in proportion to its attention. Assembling the heads is the inverse of the cut in step 1: 32 pieces of 128 numbers are glued back into one vector of length 4096.
5. `out = attn·Wo` → `seq × d_model`. *In plain terms:* until this point the heads worked in isolation, each on its own piece; multiplying by the output matrix `Wo` mixes their results together—one head's result can influence any coordinate of the final vector.
6. `x = x + out`; RMSNorm. *In plain terms:* the residual connection—attention's output does not *replace* the token's vector but is *added* to it: the block edits the draft rather than rewriting it (which is why the signal is not destroyed across dozens of blocks). Normalization levels the scale of the numbers before the next stage.
7. FFN: `x·W1 → activation → ·W2` → `seq × d_model` (in SwiGLU, three matrices: `(activation(x·W_gate) ⊙ x·W_up)·W_down`, where `⊙` is element-wise multiplication of two tables of the same shape). *In plain terms:* every token, *independently of the rest*, is run through an expansion (4096 → 14336), a nonlinearity, and a compression back (14336 → 4096). The division of labor inside the block is exactly this: attention (steps 1–5) is the only place where tokens exchange information; the FFN "digests" what was collected inside each token separately. The activation (the nonlinearity) is what makes a stack of layers more expressive than one big multiplication: without it, two linear layers would collapse mathematically into one.
8. `x = x + ffn`; norm—the same residual edit as in step 6.

The takeaway. The whole block reduces to a small number of matrix multiplications (the projections, attention's two, and the FFN's two or three) combined with softmax, normalization, and element-wise operations. From this the key properties follow: computational cost (FLOPs) is dominated by matrix multiplication, which is why

accelerators (GPU/TPU) are optimized for exactly that; the `seq²` factor in step 2 is the source of the quadratic price of long context (§1.4); the sizes of the `W` matrices set the parameter count and the memory for the weights (§1.6). Picturing the model as a stack of matrix multiplications wrapped in an attention mechanism and nonlinearities is the base level from which the architectural consequences are derived. The individual operations are covered in Appendix A.

Sources. A. Vaswani et al., "Attention Is All You Need" (2017)—the primary source. J. Alammar, "The Illustrated Transformer"—the best visual intuition. J. Su et al., "RoFormer" (2021)—RoPE. T. Dao et al., "FlashAttention" (2022). I. Beltagy, M. E. Peters, A. Cohan, "Longformer: The Long-Document Transformer" (2020)—sliding-window attention, the `O(n·w)` complexity and the `ℓ·w` receptive field. A. Q. Jiang et al., "Mistral 7B" (2023)—SWA in practice: `w = 4096`, 32 layers, a theoretical attention span of ≈131k. Stanford CS336, lectures 3–4. S. Raschka (2024)—attention implemented by hand.

1.5 Dense versus Mixture-of-Experts

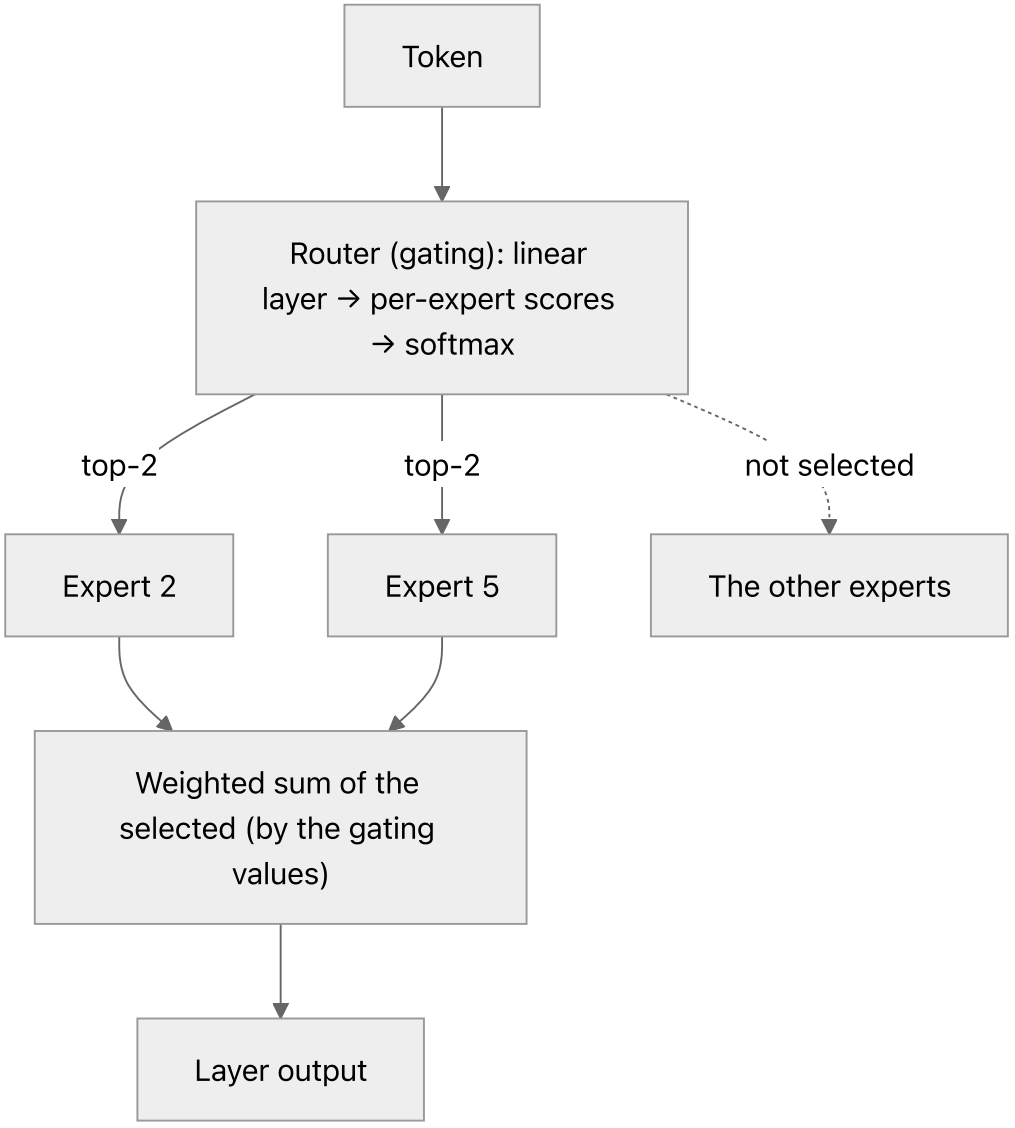
Where a block's parameters are concentrated. The bulk of a transformer block's weights is held not by attention but by the **feed-forward network (FFN)**—the fully connected layers after attention (§1.4): two in the classic variant, three in the gated SwiGLU. The Mixture-of-Experts (MoE) idea is to replace one large FFN with a set of **experts** (several smaller FFNs) plus a **router**, which for each token selects only `k` experts out of the total (typically top-1 or top-2). ≠ Do not confuse this with **Model Routing** (§2.2, §11.3): the MoE router is a trainable layer *inside* the forward pass, choosing experts for each token at each layer; Model Routing is an *application* component, choosing a whole model for a whole request. One word, two non-adjacent levels of the stack.

A dense model—all parameters are active when processing every token. **MoE (Mixture of Experts)**—for each token only a small share of the parameters is active (the selected experts), while the rest take no part in the computation. The difference is captured by two quantities:

- **Total parameters (total)**—how many weights are stored; this sets knowledge capacity and memory requirements.
- **Active parameters (active)**—how many take part in computing one token; these set the computational cost and the speed of inference.

In a dense model these quantities are equal. In an MoE the active parameters are an order of magnitude below the total. That is MoE's key idea: **separate the model's capacity (parameter count) from the cost of processing a token (FLOPs)**—get more parameters without a proportional growth in compute. An example: DeepSeek-V3 stores 671 billion parameters but activates ≈37 billion per token.

The mechanics of routing.



The router is a linear layer (gating) that computes a score for each expert from the token; after softmax the `k` experts with the highest scores are selected, and their outputs are combined with weights taken from the gating values. The choice of `k` is a trade-off: top-1 (Switch Transformers) minimizes compute, top-2 raises quality at the price of a bigger bill.

The accompanying mechanisms and characteristic failures:

- **Load balancing.** Left to itself, the router tends to concentrate tokens on a few experts, overloading them and leaving the rest undertrained. The classic remedy is an **auxiliary loss** (a penalty for uneven expert usage); a newer approach is **balancing without an auxiliary loss** (DeepSeek-V3 adjusts routing biases instead of penalizing), which removes the interference the auxiliary term injects into the main training.

- **Expert capacity and token dropping.** Each expert is assigned a limited capacity; once it is exceeded, the tokens over the limit are not processed by the selected expert (they are skipped, or pass through the residual connection). This follows directly from unbalanced routing and lowers quality.
- **Shared experts.** Some experts are made permanently active: they carry general knowledge, while the routed ones specialize in the narrow (the DeepSeekMoE scheme, often combined with a larger number of smaller—fine-grained—experts).
- **Training instability.** Training the router and the experts jointly is prone to divergence; stabilizing techniques are applied (regularizing the router's logits, for instance).

Expert parallelism and its price. The experts are spread across different devices (**expert parallelism**), which makes it possible to train and serve very large models. The price is twofold: first, **all** experts must be in memory even though only `k` are active per token—so MoE's memory requirements are high despite the small number of active parameters; second, routing generates heavy inter-device traffic (all-to-all communication), since the experts selected for a token may sit on other devices.

The key practical paradox. MoE is **cheaper on compute** (few active parameters) but **hungrier for memory** (all the experts are stored) and harder to serve (routing, inter-device communication, balancing at inference—the tokens of one batch scatter across different experts, producing imbalance). So "cheaper" as applied to MoE refers to FLOPs, not to the total cost of deployment.

The architectural choice.

Criterion	Dense	MoE
Latency (small batch)	More predictable, often lower	Higher overhead (routing, memory)
Throughput (large batch)	More expensive for the quality delivered	Wins (capacity at low FLOPs)
Memory for weights	By model size	Large (all experts are stored)
Reasoning (math, logic)	Often stronger at equal total	—
Knowledge-heavy tasks	—	Often stronger (more total parameters)
Fine-tuning and serving	Simpler	Harder (instability, expert parallelism)

The practical design consequence: a system sensitive to latency at small batch (an online agent) usually gains from a dense model of moderate size—predictable latency and simple serving; a high-throughput load on knowledge-heavy tasks (batch processing) justifies MoE—large capacity at low FLOPs. The choice directly sets the accelerator memory requirements and the serving model (§1.6).

Sources. N. Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated MoE Layer" (2017)—the origin. W. Fedus et al., "Switch Transformers" (2021)—the simplification to top-1 and the scaling. "DeepSeek-V3 Technical Report" (2024)—MLA, DeepSeekMoE, balancing without an auxiliary loss. Meta, "Llama 4" (2025)—the first MoE generation of the Llama family. S. Raschka, "The Big LLM Architecture Comparison" (2025)—dense and MoE on current models. Stanford CS336, lecture 4.

1.6 The generation loop and inference

The setup. A request's path: from JSON to the accelerator. Everything described below starts from an array of identifiers already sitting in accelerator memory. How it got there is an architecturally significant question, because the answer draws a boundary: **before the model, ordinary deterministic code runs on the CPU, and only after it does probabilistic computation on the accelerator begin**. The path of a single call: an HTTP request with a JSON body → assembling the turns with the stock chat template (§1.2) → the tokenizer turns the string into an array of integers, indices into the vocabulary (§1.2) → counting the length and checking it against the window (§1.8) → copying the array into accelerator memory. The first four steps are executed by the host process: not a single matrix multiplication, not a gram of nondeterminism. This is the same **harness** (§10.2)—the deterministic wrapper around a probabilistic core—only seen at the scale of one call rather than an agent loop: there it parses tool calls and imposes budgets, here it lays text out into numbers and decides whether to let the request onto the accelerator at all. The boundary artifact between the two worlds is the tokenizer's output: an array of integers, and nothing else.

What the move costs. The array of identifiers crosses the "host → accelerator" boundary over a bus—in a typical server, PCIe (in tight couplings like Grace Hopper, the faster NVLink-C2C, but the conclusion does not change). The copy time:

$$t_{copy} = \frac{n \times b}{BW_{bus}}$$

Where:

- n—the prompt length in tokens. Take a deliberately long context: n = 100,000.
- b—the size of one identifier in bytes. The `nn.Embedding` layer takes indices of type Int/Long, and the stock HF tokenizer with `return_tensors="pt"` returns int64, so b = 8 B.
- BW_{bus}—the bus's one-way bandwidth, B/s. For PCIe 5.0 x16 that is ≈64 GB/s (16 lanes at 32 GT/s—gigatransfers per second per lane).

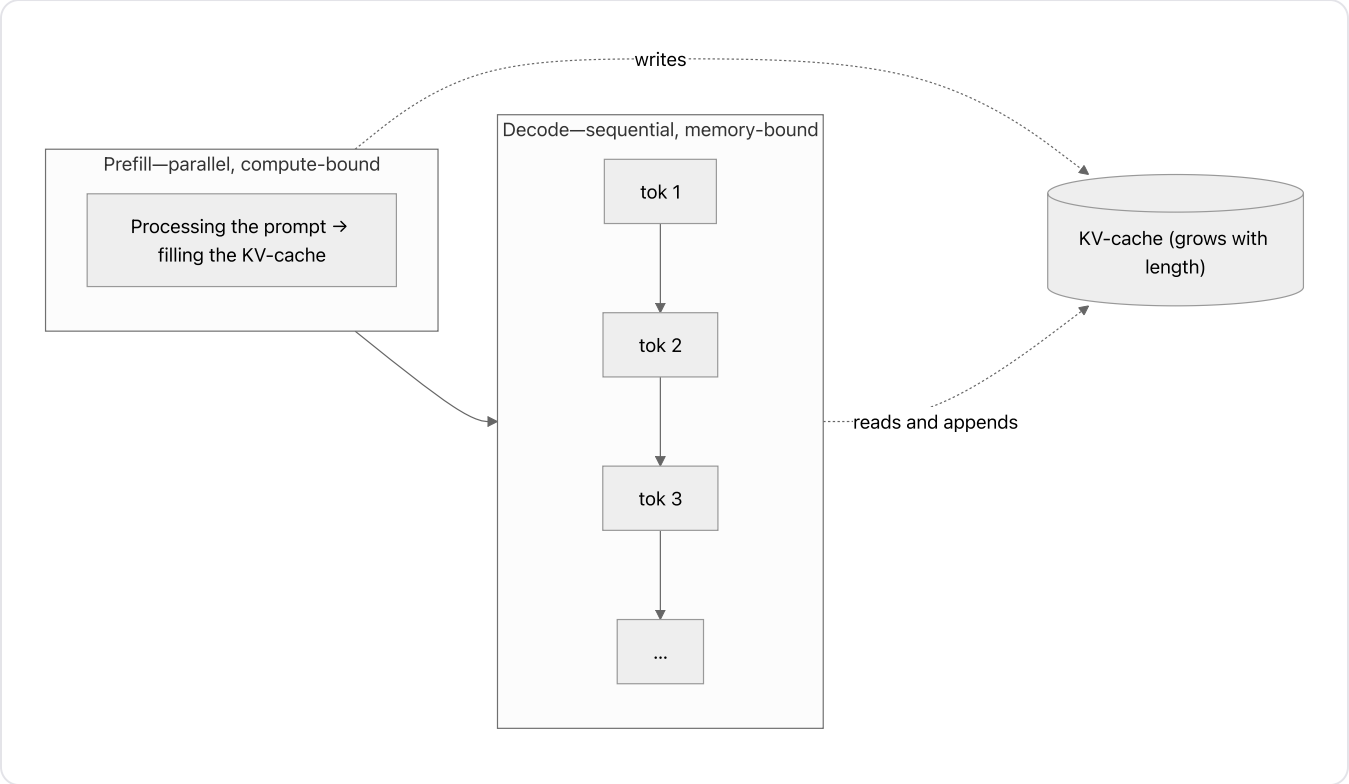
Live numbers: 100,000 × 8 B = **800 KB**; 800 KB ÷ 64 GB/s ≈ **12.5 µs**. In plain terms: *the entire hundred-thousand-token prompt moves into the accelerator in about ten microseconds—the move itself costs nothing*. The reason is that at this boundary the prompt is no longer text but numbers, and there are few numbers: even a million-token window gives 8 MB against the tens of gigabytes of weights permanently resident in the accelerator. The design consequence: there is nothing to economize on in delivering the prompt—TTFT (§2.3) grows with input length not because of the bus but because of prefill, that is, because of the arithmetic.

≠ Do not confuse the "host↔accelerator" bus with the accelerator's own memory (**HBM**, §22.5): these are different channels, and the bottleneck is the second. PCIe connects the CPU to the accelerator, fires **once per request**, and hauls hundreds of kilobytes. HBM connects the accelerator's memory to its own compute units, works **at every decode step**, and re-reads all the weights and the whole KV-cache—gigabytes. Hence the gap in width: on an H100 SXM, HBM3 bandwidth is 3.35 TB/s against ≈64 GB/s for PCIe 5.0 x16, roughly **50 times** more—and even that is not enough (why, below). So from here on the section is about batching and the KV-cache, not about the bus.

Autoregression. The model generates text one token at a time: it computes the next token's distribution → picks a token (§1.7) → appends it to the input → repeats. From the sequential nature of the process follow two inference phases with fundamentally different computational profiles:

- **Prefill**—processing the whole input prompt. All the prompt's tokens are available at once, so the phase runs in a single parallel pass and is **compute-bound**: the arithmetic load is large and the accelerator's compute units are busy.

- **Decode**—generating the answer token by token. Each step depends on the previous one, so the phase is **sequential** and **memory-bandwidth-bound**.

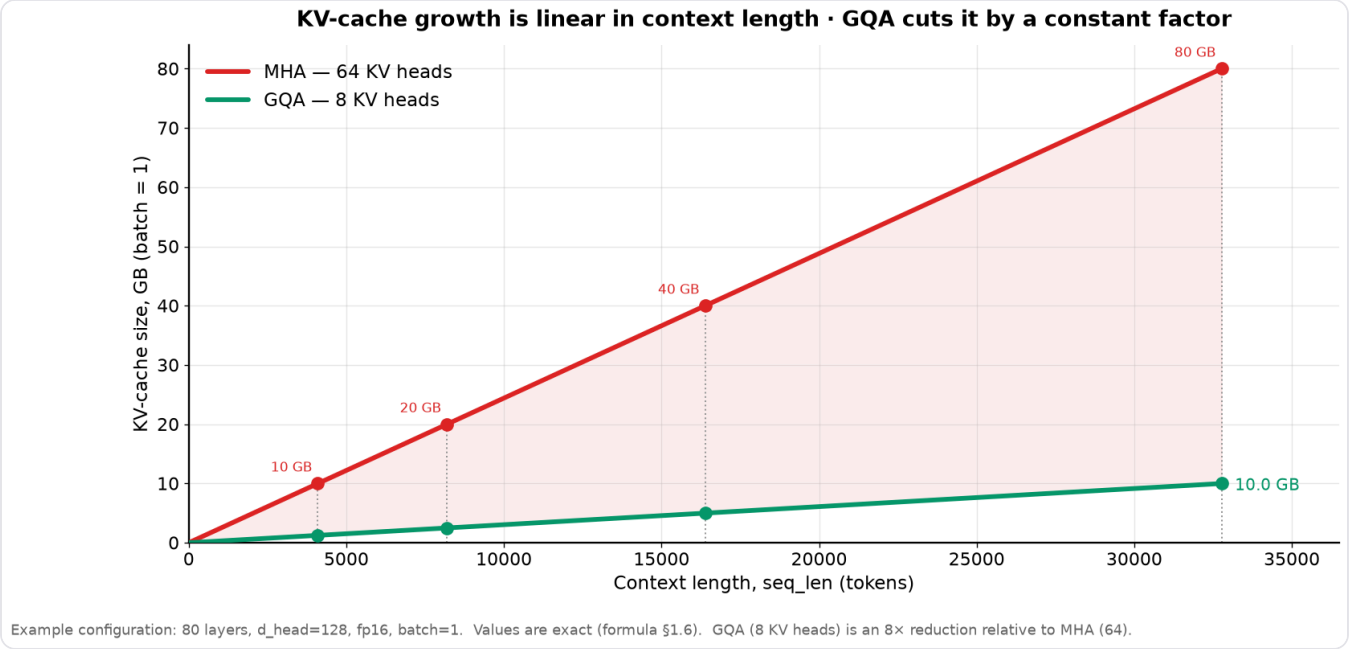


Why decode is bound by memory rather than compute. For every generated token, all the model's weights and the whole KV-cache have to be read out of accelerator memory (HBM), while comparatively little arithmetic is performed (processing one token). The ratio of computation to data read (*arithmetic intensity*) is low, so memory bandwidth becomes the bottleneck and the compute units idle. This is the key difference between decode and prefill, and the explanation of why batching (below) is so effective: it lets the weights be read once and applied to many sequences at once, raising arithmetic intensity. The link to latency: prefill's duration sets the time to first token (TTFT, which grows with prompt length), and decode sets the time per subsequent token (TPOT); more in §2.3.

KV-cache. So that attention over all previous positions need not be recomputed at every decode step, the Keys and Values already computed are cached and reused (§1.4). This speeds up generation radically, but the cache **grows linearly with sequence length** and is **the biggest consumer of accelerator memory after the model's weights**. Its size:

```
KV-cache (bytes) ≈ 2 · n_layers · n_kv_heads · d_head · seq_len · batch · bytes
```

The factor 2 is for Key and Value. A concrete example (a configuration at the level of a 70B model with GQA: 80 layers, 8 KV heads, `d_head` = 128, fp16): one token takes 2·80·8·128·2 ≈ 320 KB, so a sequence of 8192 tokens occupies ≈2.5 GB, and a hundred parallel sessions of that length—hundreds of gigabytes for the cache alone. Hence a direct consequence for capacity planning: accelerator memory is split between the weights and the KV-cache, and the cache's volume grows as the product of the number of parallel sessions and the context length—that, and not the size of the weights alone, is what limits how many sessions can be served at once. Shrinking the KV-cache is the goal of the GQA/MQA/MLA attention variants (§1.4) and of cache quantization.



Serving optimizations (in detail—Chapter 22):

- **Continuous batching**—dynamically adding new requests to the batch as slots free up, without waiting for all the sequences of the current batch to finish. Because sequences finish at different times, a static batch idles on the longest one; continuous batching removes that idle and is the main lever on accelerator utilization. Implemented in vLLM[°] (following Orca's ideas).
- **PagedAttention**—managing the KV-cache in fixed-size blocks (pages) through an indirection table instead of contiguous allocation. This eliminates memory fragmentation (internal and external) and lets different sequences share common pages (a shared prompt prefix, for example, with copy-on-write). The technique carries the principle of paged memory over from operating systems; it underlies vLLM.
- **Speculative decoding**—a small "draft" model proposes several tokens ahead, and the main model checks them in one parallel pass, accepting the longest matching prefix. The speedup comes from the check putting to work the compute units that idle during decode (the check is compute-parallel, whereas ordinary decode is memory-bound); the size of the speedup is set by the share of accepted tokens, that is, by how closely the draft and main models agree.
- **Prefill/decode disaggregation**—separating the two inference phases across different accelerator pools (or interleaving them via chunked prefill): compute-bound prefill and memory-bound decode get in each other's way inside a shared batch, and splitting them removes the interference, improving both latency and utilization (§22.5).
- **Quantization**—storing the weights and/or the cache at reduced precision (Chapter 22) to save memory.

In sum. LLM inference has a "throughput—latency—memory" profile rather than the profile of computational business logic: prefill is compute-bound, decode is memory-bound, and accelerator memory is split between the weights and a growing KV-cache. These properties set cost, latency, and the ceiling on parallel sessions (§2).

Sources. W. Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention" (2023)—vLLM. Chip Huyen, *AI Engineering* (2025), the chapters on inference. Y. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding" (2023).

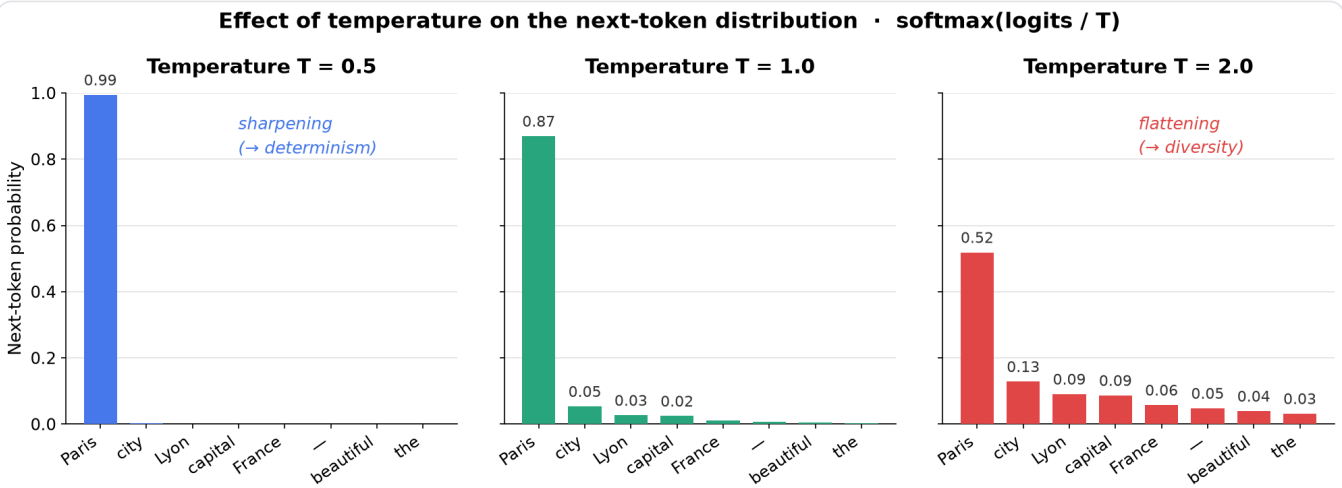
1.7 Decoding strategies

The setup. The transformer's last layer emits **logits**—one number per vocabulary token; `softmax` turns them into a probability distribution over the next token (§1.4, §1.6). The way a particular token gets chosen from that distribution is what is called a *decoding strategy*; it is what sets how deterministic the output is.

Greedy selection and why it is not enough. The simplest strategy is **greedy**: take the most probable token (argmax). It is deterministic in its choice, but on open-ended generation it leads to degeneration—repetition, looping, monotony. The cause was established empirically (Holtzman et al., 2019): maximum-likelihood sequences do not coincide with natural text—human speech regularly picks a token that is not the most probable one, and greedy likelihood maximization drifts systematically into monotony. **Beam search** (keeping the `b` most probable sequences) raises likelihood and suits tasks with a single correct answer (machine translation, for example), but on open-ended generation it amplifies the same problem—producing smooth yet monotonous text. So open-ended generation is run by *sampling* from the distribution rather than by maximization.

Shaping the distribution before sampling:

- **Temperature** `T`—before softmax the logits are divided by `T`: `softmax(logits / T)`. The mechanics: softmax raises e to the power of each logit, so its output is determined by the *gaps* between logits; dividing by `T < 1` stretches those gaps (the leader pulls away even further—the distribution sharpens), dividing by `T > 1` compresses them (the candidates converge—the distribution flattens, and the probability of less likely tokens rises). Hence the limits: at `T → 0` the gaps are infinite and only the argmax remains (the greedy limit); at `T = 1` the distribution is unchanged. The main regulator of the "predictability ↔ diversity" balance (see the figure).
- **Top-k**—sampling only from the `k` most probable tokens, the rest zeroed out. The drawback is that `k` is fixed: when the model is confident, the surplus options let noise in; when it is not, the cutoff clips off apt continuations.
- **Top-p (nucleus sampling)**—sampling from the smallest set of tokens whose total probability is ≥ `p` (Holtzman et al., 2019). The set is adaptive: it narrows where the model is confident and widens where it is not—which is how top-p removes top-k's rigidity.
- **Min-p**—a newer threshold, set relative to the probability of the most probable token (everything below a fixed fraction of the peak is cut off); more robust than top-p at high temperatures.



Example prompt: "The capital of France is —". The softmax computation is exact; the logit values are a didactic example.

Repetition penalties. A separate family of parameters suppresses looping: *presence penalty* (a penalty for a token appearing at all, regardless of frequency), *frequency penalty* (a penalty proportional to the number of occurrences), *repetition penalty* (a multiplier on the logits of tokens already seen), *no-repeat-ngram* (an outright ban on emitting an n-gram a second time).

The order of operations. The parameters are applied in a definite sequence: first the repetition penalties on the logits, then temperature scaling, then the top-k / top-p truncation, then sampling. The order affects the result—a practically significant detail when reproducing behavior across different implementations.

Structured / constrained decoding. At the token-selection stage you can **forbid** every token that violates a given grammar or JSON schema by masking its logits (setting them to `−∞` before sampling). This guarantees syntactically valid output—critical when the result is handed to a parser or to a tool call—and is more reliable than a request to "return JSON" with no guarantee. The mechanism is covered in detail in §5.3; an important caveat: it guarantees the *structure*, not the *correctness* of the content (§5.6).

Practical choice. For field extraction, classification, and tool calls, use a low temperature (≈0–0.3), often together with constrained decoding—this returns predictability to a probabilistic core where the result has to be machine-processable. For generating text and ideas, temperature and top-p go up for the sake of diversity. That said, even for "deterministic" tasks a small nonzero temperature is sometimes preferred over pure greedy, to avoid degenerate repetition. Finally, choosing `T = 0` is deterministic only in the sense of token selection and **does not guarantee bit-identical output** across runs (§2.4).

A comparison.

Strategy	Determinism	Diversity	Typical use
Greedy	Maximum	Minimum	Extraction, classification
Low-T + top-p	High	Low	Tool-calling, structured output
High-T + top-p	Low	High	Creative work, brainstorming
Constrained	Schema guaranteed	Bounded by the grammar	Output to a parser/tool

Sources. A. Holtzman et al., "The Curious Case of Neural Text Degeneration" (2019)—nucleus sampling and why greedy/beam degrade. Anthropic/OpenAI documentation on sampling parameters and structured outputs.

1.8 The context window and its failure modes

The context window—the maximum number of tokens the model processes in one call (the prompt together with the generation). It is a finite budget: its size was fixed during training (positional encoding is trained up to some maximum length, §1.4), and going beyond it requires extrapolating positions (RoPE scaling) at a cost in quality.

One budget for input and output. Why the window covers both the prompt and the answer follows directly from autoregression (§1.6): every generated token **is appended to the input** and takes the next position in the sequence. To the model, its own token is indistinguishable from one that arrived in the prompt—it spends the same position out of the same trained range (§1.4). So input and output are not two quotas but one:

$$T_{in} + T_{out} \le W$$

Where:

- T_{in}—the input, in tokens: everything that goes into the request—the system instruction, the tool definitions, the history, the retrieved chunks, the query itself. A cached prefix takes up room too: prompt caching (§2.2) changes a token's price, not its presence in the window. In the API response this is the `input_tokens` field.
- T_{out}—the output, in tokens: everything the model produced during the call, including the internal reasoning of reasoning models—models that emit a long chain of reasoning before the answer and bill it as output tokens (§13.8). The `output_tokens` field.
- W—the window, in tokens: a constant of the model, fixed during training. For Claude Sonnet 4.5 and Claude Haiku 4.5 it is 200,000; for the Sonnet 4.6 generation and newer, 1,000,000.
- `max_tokens`—a request parameter, in tokens: the ceiling you yourself put on T_{out}. For Claude Sonnet 4.5, no more than 64,000.

Live arithmetic on Claude Sonnet 4.5: 200,000 − 64,000 = **136,000**. That is what remains for the input if you asked for the largest answer this model allows. In plain terms: the room for the answer is not handed out on top of the window—it is subtracted from it, and every token added to the input is one token fewer the model will get to generate.

The reserve is planning, not validation. An essential detail: `max_tokens` is not booked in advance. On Claude 4.5 models and newer, a request in which T_{in} + `max_tokens` exceeds the window is **accepted**—the limit fires not at validation but as generation actually happens (on earlier models, by contrast, a validation error came back). So subtracting W − `max_tokens` gives not a refusal threshold but the boundary up to which the requested answer volume **is guaranteed** to fit. Step over it and you get not an error on the input but a cutoff on the output—the case below.

More context does not mean better. Even when all the material fits into the window, answer quality can fall, for three reasons:

1. **Positional bias (lost in the middle).** Models use information at the beginning and the end of the context more fully and the middle less well; a fact placed in the middle of a long context may go unaccounted for (Liu et al., 2023). The empirics of this effect (the U-shaped curve, the measurement methodology) are covered in detail in §4.3.
2. **Attention dilution (context rot).** As the token count grows, attention is spread thinner (the cause is softmax normalization: the attention budget is fixed by summing to 1 and gets divided among ever more tokens; the derivation—§4.3), while irrelevant material adds noise; reasoning quality degrades as the window fills, even at a formally sufficient length.
3. **The quadratic cost** of processing with length (§1.4).

Effective length versus stated length. A stated window size (128k, say) is an upper bound on processing, not a length at which quality holds: models often degrade on hard tasks long before the limit. So the *effective* context length is measured on the specific task (§4.3) rather than assumed equal to the stated one.

Behavior on overflow: two different failures. From the *input* side, behavior depends on the implementation: an explicit error (at Anthropic, a 400 "prompt is too long" if the input alone will not fit the window) or silent truncation of the input (trimming the start or the end). Silent truncation is dangerous because it removes part of the prompt unnoticed, so overflow handling gets built into the system explicitly.

The symmetric and more insidious failure is on the *output* side: the prompt fit, the call was accepted, generation started—and broke off mid-word. There is no error: the answer arrives with a success code and looks genuine right up to the point of the break. The only sign is the stop-reason field (`stop_reason` at Anthropic, `finish_reason` at OpenAI), and it distinguishes two cases whose cures are opposite:

- **Hit the requested ceiling**— `stop_reason: "max_tokens"`, at OpenAI `finish_reason: "length"`: generation used up the `max_tokens` it was given. Cured by raising the limit, or by continuing the answer in a follow-up call.
- **Hit the window itself**— `stop_reason: "model_context_window_exceeded"` (Anthropic, Claude 4.5 models and newer): there is physically no room left. Raising `max_tokens` is pointless—what needs cutting is the input.

An engineer's iron rule: check the stop reason on every call, *before* reading the response body. A handler that parses `content` straight away will not notice a budget cutoff—it will see a plausible stump. For structured output this is exactly the truncated JSON from §5.1: an invalid tail comes not from a network failure but from an exhausted budget, and the only thing telling them apart is the stop reason.

Behind both failures lies one physical cause. An error on the input and a cutoff on the output look like two quirks of implementation, but what they protect is the same thing: **room for the KV-cache in accelerator memory**. The window is bounded twice over. From above—by the trained length of the positional encoding (§1.4); that is a quality limit, and crossing it gives degradation, not a crash. From below—by the physical volume of memory for the cache (§1.6); that limit is hard, since the cache grows linearly with sequence length, and accelerator memory running out is not degradation but a process dying. Hence the right reading of both failures: **the length check is a software fuse placed ahead of a hardware failure**. Cheap arithmetic on the host (§1.6: length is counted before the copy into the accelerator) stands ahead of an expensive crash, which is why it is done there.

What the engine actually computes. The KV-cache formula from §1.6 is not a back-of-the-envelope estimate: the inference engine computes it literally, right at startup. Take Qwen3-8B with its stock window of 40,960 tokens:

$$2 \times n_{layers} \times n_{kv_heads} \times d_{head} \times seq_len \times batch \times bytes = 2 \times 36 \times 8 \times 128 \times 40\,960 \times 1 \times 2 = 6.04 \cdot 10^9 \text{ bytes} \approx 5.62 \text{ GiB}$$

The legend (notation from §1.6, values from the model's `config.json`): the factor **2**—for Key and Value; n_{layers} = 36—the number of layers; n_{kv_heads} = 8—the number of KV heads, four times fewer than the attention heads (32 against 8)—that is GQA, §1.4; d_{head} = 128—the dimensionality of one head; seq_len = 40,960—the full window in tokens; $batch$ = 1—one session, exactly the minimum the engine checks at startup; $bytes$ = 2—bytes per number in bf16 (`torch_dtype: bfloat16`). In plain terms: to serve **one single** request across the full window takes 5.62 GiB of accelerator memory for the cache alone—on top of the model's weights, which live in the same place and are not going anywhere.

That is precisely the number vLLM (§22.2) prints when it **refuses to start**: "*5.62 GiB KV cache is needed, which is larger than the available KV cache memory (4.89 GiB)*"—and it offers either to give the cache a larger share of memory (`gpu_memory_utilization`) or to cut the window down (`max_model_len`). The engine would rather not come up at all than come up and die on memory at the first long request.

An incidental lesson lies in which model that log was taken from: not the 8B but Qwen3-**4B**, on a 16 GiB card. The number is the same, and that is no coincidence: the 4B has the same attention geometry (36 layers, 8 KV heads, d_{head} = 128), and the formula holds no "parameter count" factor at all—the model is half the size, and it needs **the same** 5.62 GiB for the cache. KV-cache volume is set by the depth and width of attention, not by the number of weights; a smaller model frees memory under the weights, but not under the cache.

Why refuse rather than try. The KV-cache pool is **shared** across all sessions on the accelerator (§1.6), so running out of memory would hit not the one culprit but every neighbor sharing the accelerator at that moment. Hence three echelons of checks, and not one of them is left to "we'll catch the exception":

- **at startup**—does even one session at the full window fit; if not, the engine does not come up (the case above);
- **on accepting a request**—does this input fit; if not, refusal on the host side, before the accelerator (at Anthropic that is the 400 "prompt is too long," the case above). What exactly goes into that check—only the input, or the input together with the requested `max_tokens`—is a matter of implementation policy, not of

arithmetic: as shown above, Claude 4.5 models and newer do not book the output in advance;

- **at runtime**—when blocks run short, the engine **evicts** a sequence: its KV blocks go to the neighbors, and it is later recomputed from scratch (in vLLM V1 this is the `RECOMPUTE` mode, taken as the default because it is cheaper than swapping the cache out to RAM). The request is delayed but does not fail.

The engineering consequence for design: silent truncation and an explicit error are not two vendor quirks but two ways of not letting accelerator memory run out. The choice between them is your system's to make: if you do not make it, the provider will—silently, and not in your favor.

Hence the consequences that lay the foundation of Parts II–III: context is **curated**, not filled with arbitrary material (context engineering, Chapter 4); critical information goes at the edges of the window, not in the middle; replacing retrieval (RAG) by feeding all available text into a long window is a false economy—retrieving a few relevant fragments (Part III) is usually both more accurate and cheaper than supplying a whole document, on cost (quadratic growth) and on quality (dilution) alike. For example, rather than supplying an 80-page regulation in full, you index it, retrieve 3–5 relevant fragments, and put them at the start of the context with an explicit instruction—cheaper, faster, more accurate.

Sources. N. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts" (2023); Chroma Research, "Context Rot" (2025). Anthropic, Claude API documentation: "Context windows" (the window as the sum of input and output; overflow behavior, `model_context_window_exceeded`), "Handling stop reasons", "Models overview" (windows and output ceilings by model). OpenAI, API reference—`finish_reason`. The KV-cache check at startup, eviction with recomputation (`RECOMPUTE`), and the `gpu_memory_utilization` / `max_model_len` parameters—the vLLM documentation; the startup refusal text is quoted verbatim from vLLM, issue #38516 (Qwen3-4B on an RTX 5080, 16 GiB). The Qwen3-8B and Qwen3-4B configurations (layer count, KV heads, `head_dim`, `max_position_embeddings`, `torch_dtype`)—the models' `config.json` on Hugging Face. The empirics and the measurement methodology—§4.3.

1.9 Why the model hallucinates, and what "knowing" means

Hallucination—a confidently generated statement that is factually wrong or invented. This is not a defect of implementation but a consequence of how the model works: it was trained to predict a *plausible continuation*, not to establish truth. The training task (predicting the next token over a corpus) holds no notion of factual correctness; plausibility and truth correlate but do not coincide, and where they part ways the model confidently produces plausible falsehood.

The sources of hallucination differ, and separating them is useful:

- *A gap in knowledge*—the fact was absent from the training data or never took hold; the model fills in something plausible.
- *Staleness*—parametric knowledge is frozen as of the cutoff date and does not reflect later changes.
- *Faulty interpolation*—a plausible but wrong combination of disconnected facts.
- *Prompt-induced*—following a false premise in the input, and **a tendency to agree with the user's framing** (sycophancy): the model gravitates toward confirming the view it was handed.

Calibration. As a rule the model is *poorly calibrated* with respect to factual correctness: an answer's confident tone does not match the probability of its being right, and there is no built-in "I don't know" signal. This is the core of the trust problem in LLM systems.

A measurable definition. Calibration is the agreement between stated confidence and the actual share of correct answers: a perfectly calibrated model that answered "80% confident" is right in exactly 80% of such cases. The standard measure of the discrepancy is **ECE (Expected Calibration Error)**, the average calibration error: answers are sorted into confidence bins, in each bin the actual accuracy is compared against the stated confidence, and the gap is averaged across bins, weighted by the number of examples.

$$ECE = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|$$

The symbols: `M` —the number of confidence bins, in practice 10–15 (Guo et al. use 15; 10 is taken below, that is, intervals 0.0–0.1, …, 0.9–1.0); `B_m` —the set of answers whose confidence landed in bin `m`; `N` —the total answers in the measurement; `|B_m|/N` —the bin's weight, that is, the share of traffic in it; `acc(B_m)` —the share of actually correct answers in the bin, 0–1; `conf(B_m)` —the average stated confidence in the bin, 0–1. ECE is dimensionless and reads in percentage points.

Live numbers—a notional example: the figures were picked to illustrate the mechanics of the calculation, not measured. Suppose there are `N` = 1000 answers in all, the bins below 0.7 are empty, and the remainder is distributed like this:

Bin	`conf`	`acc`	Gap	Bin weight	Contribution
0.7–0.8	0.75	0.65	0.10	200/1000 = 0.2	0.020
0.8–0.9	0.85	0.75	0.10	300/1000 = 0.3	0.030
0.9–1.0	0.95	0.80	0.15	500/1000 = 0.5	0.075

ECE = 0.020 + 0.030 + 0.075 = **0.125**. In plain terms: averaged over traffic, the model promises roughly 12.5 percentage points more confidence than it delivers. This is a notional quantity from a teaching example, not anyone's measurement—the real numbers come below, with a citation. The structure of the measurement matters too: half the mass sits in the 0.9–1.0 bin, and the gap there is the largest—a shift of mass into the upper bins is the typical pathology of an aligned model.

The breaking mechanism: calibration is destroyed by alignment. Poor calibration is not innate but acquired, and that is a matter of principle. A base (pretrained) model is calibrated almost perfectly: its only task is to predict the next token's distribution, and the probability in that distribution honestly reflects the frequency observed in the corpus. Alignment for dialogue (RLHF, §21.6) destroys that calibration: measured on a subset of MMLU, ECE is ≈0.007 for the pretrained model against ≈0.074 for the same model after post-training—an order of magnitude worse (GPT-4 Technical Report, 2023, fig. 8). The cause is direct: the reward model was trained on human preferences, and people prefer answers that sound confident and helpful; maximizing such a reward presses the output into the upper confidence bins regardless of whether the model knows the answer or not. From which it is clear that sycophancy (above) and the loss of calibration are not two independent defects but one root: both are side effects of optimizing for human approval. The consequence for an architect: a chat model from any provider is a model *after* alignment, its confidence is inflated systematically and by construction; a prompt will not cure it, since the instruction "report your uncertainty honestly" is fighting what was wired in by gradient.

Calibration is not the same as discrimination. These are different properties, and a cascade leans on the second, not the first. ECE answers whether confidence matches correctness *on average*. Discrimination is a signal's ability to *separate* correct answers from incorrect ones; it is measured by AUROC°. A model that assigns every

answer a confidence of 0.85 and is right in exactly 85% of cases is perfectly calibrated (ECE = 0) but useless for escalation: AUROC = 0.5, there is nowhere to draw a threshold—the signal is identical on the correct and the incorrect. So the signals below are judged by AUROC, not by ECE.

Three confidence signals, in descending order of reliability.

- *Sample agreement*—the most reliable. The same request is run `k` times, and the measure of confidence is how well the answers agree (the entropy across them). This is self-consistency from §3.4, applied as a *gauge* rather than an improver. The price—×k calls. The size of the win: on GSM8K arithmetic, moving from a single verbalized estimate to five samples aggregated by agreement lifts AUROC from 54.8% (the level of a coin flip) to 92.7% (Xiong et al., 2023; GPT-3.5 with CoT). A development of the technique is *semantic entropy*: the samples are first clustered by meaning and only then is entropy computed across the clusters, so that paraphrases of one answer do not count as disagreement (Farquhar et al., Nature, 2024).
- *Logprob*—the logarithm of the probability the model assigned to the answer's own tokens: that same post-softmax distribution from §1.7, only on a logarithmic scale (where a sequence's probabilities add rather than multiply). Cheap (one call), but it measures the wrong thing: this is confidence in the *next token*, not in a statement's truth. The textbook example from Xiong et al.—"Chocolate milk comes from brown cows": every token is highly probable in its place, and the statement is false. An engineering caveat: what the returned logprob actually is depends on the engine—vLLM by default hands back "raw" log probabilities computed at `T = 1` regardless of the sampling temperature, but that is a configurable mode (`logprobs_mode`), not a universal law; if you lean on logprob as a signal, check this against your own engine's documentation.
- *Verbalized confidence* ("90% confident")—the worst estimator: it is style, not probability. The values cluster in the 80–100% range and are multiples of five—the model reproduces the human manner of *talking* about confidence, not its own distribution (Xiong et al., 2023).

A caveat, for honesty: there is no unconditional leader among the three—on hard tasks all of them are weak, and in the same measurement the gap between the verbalized estimate and the methods with access to the logits came out narrow (AUROC 0.522 against 0.605). A confidence signal is an instrument with a known error, not an oracle.

An engineer's iron rule: a model's raw confidence number is not a probability. A cascade threshold (§2.2, §20.3) or a guardrail (§19.5) that compares that number against a constant is escalating on an uncalibrated instrument. Set the threshold from *your own* labeled set: run a representative sample of traffic, record whether each answer was right, build the ECE and AUROC of the chosen signal—and only then pick the cutoff point, by the matrix of error costs (§2.2), not by the beauty of the number. This is the same logic as "validating the judge against humans" (§9.4): before building a decision on an instrument's reading, tie the instrument to reality.

Two kinds of knowledge (the distinction drives the choice between RAG and fine-tuning, Parts III and VII):

- **Parametric knowledge** is fixed in the weights during training: static as of the cutoff, not updated, with no traceable source.
- **Non-parametric knowledge** is supplied into the context at runtime (RAG): current, checkable, and sourced.

Hence the two themes running through the book: grounding the answer on retrieved facts with a source instead of leaning on memory (**RAG**, Part III), and measuring when and how far the model departs from the facts (**evaluation**, Part VI), since a confident wrong answer is visually indistinguishable from a right one. A separate case is a *knowledge conflict*, where the retrieved context contradicts the model's parametric prior (covered in §6.7).

A practical principle: do not design a system on the assumption that parametric knowledge is factually accurate—for facts, use grounding (RAG) and checking (evaluation), and lean on the model for what concerns form, style, and reasoning.

Sources. Surveys of LLM calibration and hallucination (2023–2025); the "parametric and non-parametric knowledge" frame—Chip Huyen, *AI Engineering* (2025). ECE—M. P. Naeini et al. (2015); the standard form and its popularization—C. Guo et al., "On Calibration of Modern Neural Networks" (2017). Calibration before and after post-training—OpenAI, "GPT-4 Technical Report" (2023), fig. 8. The confidence signals and their comparison—M. Xiong et al., "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs" (2023). Semantic entropy—S. Farquhar, J. Kossen, L. Kuhn, Y. Gal, "Detecting hallucinations in large language models using semantic entropy", Nature (2024).

1.10 In sum: the component's contract

The chapter's material reduces to the thing it was laid out for—**a set of properties of the language model as an architectural component**. When designing a system on top of a model, the following properties, and the places they are managed, matter:

Property	What it means	Where it is managed
Input and output are text	Not strictly typed; the output needs validation	Structured output (Chapter 5)
Nondeterminism	One input → different outputs	Decoding (§1.7); evaluation (Chapter 17)
Cost ∝ tokens, nonlinear in length	Long context grows quadratically more expensive	Context engineering (Chapter 4); RAG (Part III)
Latency: prefill + decode	A "throughput—latency—memory" profile	Serving (Chapter 22)
A finite window and degradation in the middle	More does not mean better	Curating the context (§1.8, §4.3)
Parametric knowledge is static and not guaranteed	Facts go stale and get invented	RAG (Part III); evaluation (Chapter 17)
Weak calibration	Confidence ≠ correctness	Guardrails (Chapter 19); evaluation
Dense or MoE changes the resource requirements	total ≠ active parameters	Model choice; serving (§1.6, Chapter 22)

The rest of the book is methods for managing the rows of this table: designing prompts and context (Part II) orders the input; RAG (Part III) grounds knowledge; agents and orchestration (Part IV) organize the actions of a nondeterministic core; integration (Part V) embeds the system into existing infrastructure; production (Part VI) restores predictability at the boundaries—evaluation, observability, guardrails, cost; your own models (Part VII) change the component itself. Command of this set of properties replaces treating the model as an unpredictable "black box" with working with it as a component that has known nonfunctional characteristics.

In the Margins: Chapter 1

↩ **MTEB**—Massive Text Embedding Benchmark: a composite benchmark of embedding models across dozens of datasets by task type (Muennighoff et al., 2022). → §7.6.

↩ **NTK-aware scaling, YaRN**—techniques for interpolating RoPE frequencies that stretch the working context length without full retraining; YaRN (Peng et al., 2023) develops NTK interpolation. → §1.8.

↩ **HBM**—High Bandwidth Memory: stacked memory on the accelerator's substrate. Its bandwidth is the bottleneck of the decode phase: every generation step re-reads the weights and the KV-cache. → §1.6, §22.5.

↩ **RMSNorm** (Zhang & Sennrich, 2019)—a simplified LayerNorm: normalization by the root-mean-square value with a trainable scale, without subtracting the mean—cheaper for the same stabilizing effect. → Appendix A (§A.3).

↩ **vLLM**—an open-source LLM inference server (PagedAttention, continuous batching); the industry reference point for self-hosting. → Chapter 22.

↩ **AUROC**—the area under the ROC curve: the probability that a randomly chosen correct answer receives a higher score from the confidence signal than a randomly chosen incorrect one. 0.5—the signal does not tell correct from incorrect (a coin flip), 1.0—it separates them perfectly. It measures *discrimination*, not calibration: a perfectly calibrated signal can have AUROC = 0.5.

Chapter 2. Economics and Nonfunctional Properties of LLMs

Chapter 1 gave the component's *contract*. This chapter turns its nonfunctional properties—cost, latency, determinism, reliability—into **budgets**: the currency design actually gets done in. The framing mirrors the latency and error budgets that SRE apportions across SLOs: scarce resources get allocated to each feature. The difference is that one of the resources is trust in a probabilistic component.

2.1 Three budgets

Every design decision in an LLM system trades off three scarce budgets:

- **Cost** (tokens → money),
- **Latency** (time to an answer),
- **Determinism/quality** (how far the answer can be trusted without a human checking it).

These resources conflict. Cutting cost usually means a smaller model and, with it, lower quality; raising quality through extended reasoning, a larger context, or orchestration raises cost and latency; raising reliability through checks and guardrails adds latency. Design work, then, is not an abstract "make it good" but a deliberate allocation of these resources to a specific feature against its requirements (SLO): an online feature in a conversational interface needs a hard latency budget, while for background batch data processing cost is the dominant constraint. This chapter is about measuring and allocating those three budgets.

2.2 The cost model

Billing by token, with an asymmetry. API cost is counted per token, and **output tokens usually cost several times more than input tokens**. The reason is technical (§1.6): input is processed in the prefill phase (in parallel, cheap per token), while output is generated in decode (sequentially, more expensive per token). The practical takeaway: long *answers* hit the bill harder than long *prompts*.

Input caching. A static prefix resubmitted on every call (system prompt + constant context) is billed far below ordinary input through **prompt caching**. That makes it economically worthwhile to move the stable part of the context into a cacheable prefix.

The cost of context. The per-token rate is linear, but a long context means more input tokens on *every* call—and in an agent loop the context also grows from step to step. Attention's quadratic complexity (§1.4) sits on the provider's side (where it drives latency); in token billing, context gets more expensive linearly, but on every single call.

The agent multiplier is the main cost trap. An agent loop of K steps is K model calls, each with a growing context. Cost grows superlinearly. This is exactly why the book's running principle ("don't add complexity before the task proves it necessary") is also about money: a multi-agent swarm on a task that one call would have handled

can cost tens of times more with no gain in quality.

Model tiers. Providers offer a lineup: small and cheap, mid-range, frontier—with a 10–50× spread in price. Sending every request to the frontier model is wasteful. Hence the pattern:

Pattern: Model Routing / Cascade

Context. The request stream is heterogeneous in difficulty: most are simple, a minority are hard. **Problem.** How do you hold cost and latency down without sacrificing quality on the hard requests? **Forces.** The frontier model is accurate but expensive and slow; the cheap one is the reverse. Sending everything to the frontier is wasteful; sending everything to the cheap model is risky. **Solution.** A (cheap) classifier scores difficulty and **routes**: simple → cheap model, hard → frontier. The **cascade** variant: the cheap model first, then a check (confidence/validator/judge); on failure, escalate to the frontier. **Mechanics.** Routing implements the general pattern of picking a branch by request content; the cascade implements escalation after a check. The key risk: classifying or checking must not cost more than the savings it buys. **Example.** Classifying support inquiries: the cheap model closes 80% of the routine ones, the 20% that are hard escalate. The savings are often multiples. **Resulting Context.** Cost and latency down by a multiple; the price is pipeline complexity and the risk of routing errors (a simple request went to the cheap model and failed there). **When to Use.** Clearly heterogeneous load and sensitivity to cost. **When Not.** A uniformly hard stream—there is nothing to route. **Related Patterns.** Adaptive RAG (the same idea applied to retrieval, Chapter 7); Evaluator-Optimizer (check before escalating). **Sources.** Chip Huyen, *AI Engineering* (2025)—model routing/cascades; S. Jeong et al., "Adaptive-RAG" (2024)—the same difficulty-routing idea applied to retrieval (Chapter 7).

Deep Dive: the routing classifier—anatomy of a component

The word "classifier" in the card above is easy to skim past, yet it is a distinct artifact that carries the whole construction—here, in Adaptive RAG (§7.9), in the Routing pattern (§11.3), in gateway cascades (§20.3), and in ADR-4 of the capstone (§26.4). Let's take it apart as a component: what it can be built from, where its "knowledge" comes from, and how it fails.

Three implementations, in ascending order of cost. (1) **A heuristic over observable features**—deterministic code: request length, presence of code or attachments, language, client type, channel. Free and instant, but it sees only the request's form, not its meaning. (2) **A trained classifier over embeddings**—the request is run through an embedding model (Chapter 7—the same one used in retrieval), and a lightweight head (logistic regression, a small MLP) emits a class. The cost is one embedding call (on the order of hundredths of a cent and tens of milliseconds), but it requires labeled data. (3) **A prompted cheap LLM call**—a junior model instructed to "score the difficulty/category," with structured output (Chapter 5). The most flexible and the most expensive option: a full LLM call on every request, hundreds of milliseconds of latency—the very "cost of classification" the card named as the key risk.

Where the labels come from. The classifier needs to know "what counts as simple," and there is nowhere to get that except your own traffic. The primary source is historical traces (Chapter 18), labeled after the fact: run a sample of requests through both models and label where the cheap one succeeded (the judgment comes from an LLM judge or a human, Chapter 17). The cascade has the more convenient arrangement: it *generates the labels itself*—every request that passes the check on the cheap model is a confirmed example of "simple," every escalation an example of "hard." So a mature router often starts life as a cascade and only later, once it has accumulated labels, acquires an upfront classifier that removes the double call.

Calibration is a matrix of error costs, not accuracy. The router's two errors are asymmetric: "hard → cheap" costs *quality* (the user got a bad answer), "simple → frontier" costs only *money*. So the decision threshold is set not at maximum accuracy but by the ratio of those costs: when a quality error is expensive, the threshold shifts toward the frontier and you accept overpaying for false positives. This is the ordinary threshold logic of a binary classifier (precision/recall), applied to money.

The failure mode is silent. The fundamental difference from tool errors (§12.4): a bad route has *no feedback loop*. A tool that fails returns an error—the model sees it and adapts. A misrouted request does not fail: the cheap model confidently produces a mediocre answer, and no component in the pipeline learns that it was the router that erred. Detection is external only: offline eval on a labeled sample (Chapter 17), monitoring route shares and per-route quality in telemetry (Chapter 18), canaries. A separate risk is **traffic drift**: the classifier was trained on yesterday's request distribution; a new category of inquiries quietly goes the wrong way, and you can spot it only in the metrics, not in the exceptions. The cascade is partly protected (its check *is* the built-in signal); the upfront router is not.

A worked cost estimate. For an auto-reply feature for support inquiries (≈1,500 input and 400 output tokens per request, 5,000 requests per day), the monthly cost is estimated as the number of tokens times the rate times the volume; the estimate then shows that cost coming down through (a) caching the stable system prompt and (b) routing 80% of requests to the cheap model. An estimate like this accompanies any feature proposal, turning it into a measurable decision.

Two corrections move the bill materially. First, in **reasoning models** the internal reasoning is billed as output tokens, so a call to one costs more than an ordinary call for the same visible answer (§13.8). Second, many providers offer a **batch tier** at a significant discount for asynchronous processing with no latency guarantees—which suits background loads, not online ones.

The crossover to self-host. At high *steady* volume, serving an open-weight model yourself can come out cheaper than paying per API call; at low or spiky volume, hosted wins. Where the crossover sits turns on hardware utilization. Details in §2.6 and Chapter 22.

Sources. Chip Huyen, *AI Engineering* (2025)—cost/serving; provider pricing pages; Google SRE Book (Beyer et al., 2016)—the budget and SLO framing.

2.3 The latency model

Two metrics instead of one. "LLM latency" breaks into:

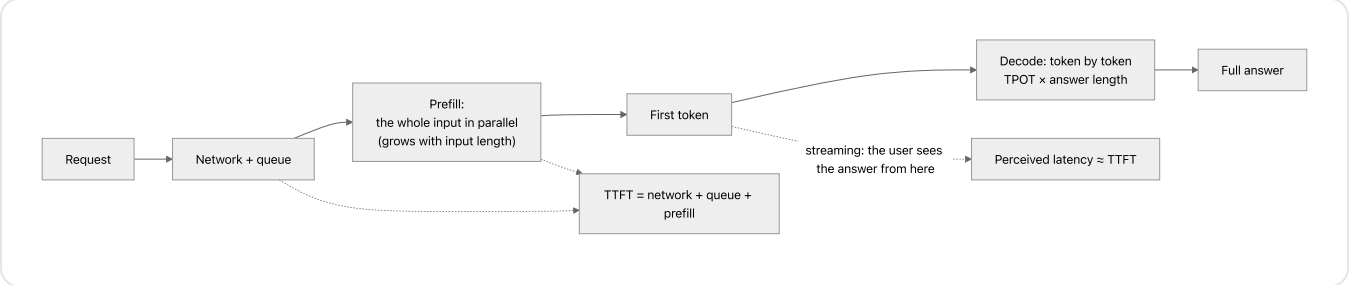
- **TTFT (time to first token)**—time to the first token of the answer. Dominated by the prefill phase, grows with the length of the *input*.
- **TPOT (time per output token)** / generation speed (tokens/s)—dominated by decode.

Total time ≈ `TTFT + TPOT × answer_length`. The direct levers follow: shorter input → lower TTFT; shorter answer → lower overall tail. Caching a stable prefix (§2.2) lowers TTFT further by shrinking the volume of prefill. Reasoning models run higher latency, because a significant share of the generation goes to internal reasoning before any useful answer appears (§13.8).

Live numbers. The legend: **TTFT**—time to the first token, set by prefill and growing with input length; **TPOT**—time per output token; **answer_length**—how many tokens are to be generated. Databricks' reference points: TPOT = 100 ms/token is 10 tokens/s per user, ≈450 words per minute—that is, faster than a person reads; TTFT in the same source was measured at 114 ms for Llama2-70B on 4×A100-40GB with a 512-token input and a batch of 1.

The TTFT figure is tied to its own configuration and will be higher on a long input—but the order of magnitude holds: fractions of a second, single digits in heavy cases. Substitute the upper bound—let the model generate across the entire standard window of Qwen3-8B (40,960 tokens, §1.8): 0.114 s + 40,960 × 0.1 s ≈ 4,096 s ≈ **68 minutes**. TTFT drowns in the rounding here—thousandths of a percent of the second term—so the total comes down entirely to generation; that is a general property of long answers, not a quirk of this example. In plain terms: at a generation speed comfortable for reading, the ceiling on output is measured not in seconds but in an hour.

`max_tokens` **bounds occupancy, not just the bill.** This shows why output gets capped hard. §20.4 works through `max_tokens` as a lever on the bill and as space taken away from the input in the shared window (§1.8); when you serve the model yourself it has a third role—the parameter sets **how long a request holds its slot and its KV-cache blocks**. A correction to the common phrasing "a long generation monopolizes the GPU": under continuous batching (§1.6) the accelerator serves the neighbors the entire time, so there is no monopoly on a GPU cycle—memory is the scarce thing. An hour of an occupied session is an hour during which its cache blocks are unavailable to anyone else, and the number of concurrent sessions runs up against exactly that cache capacity (§1.6). An unset `max_tokens` is an unset upper bound on holding shared memory: in multi-tenant serving you set it not only for the bill but for the neighbors.



Streaming hides latency. Stream tokens to the user as they are generated and *perceived* latency drops to TTFT—the user sees that it "started" without waiting for the end. This is a UX pattern, mandatory for chat interfaces.

The latency of an agent loop. `N` sequential model calls add up to `N` network round trips, and their latencies sum, as in any chain of synchronous calls. Hence the standard way to cut response time: run independent steps in parallel (the Parallelization pattern, Chapter 11). A six-step sequential orchestration on the user path is six latencies stacked, some of which can usually be parallelized.

Batching: throughput versus latency. Continuous batching (§1.6) raises the accelerator's throughput but adds queueing time—the standard trade of "throughput ↔ single-request latency." The online and batch serving paths get separated, because their priorities differ.

A worked latency budget. For a user-facing agent with an SLO of p95 = 3 s, the budget decomposes by stage: retrieval (say, ≤300 ms), model calls (≤2 s in total, some in parallel), tool calls (≤500 ms), guardrails (≤200 ms). Go over it and you cut the number of steps, parallelize the independent ones, and apply caching. This is the standard latency-budget decomposition, applied to the stages of an LLM pipeline.

Sources. Chip Huyen, *AI Engineering* (2025)—latency/serving metrics; W. Kwon et al. (2023)—vLLM/PagedAttention. Databricks, "LLM Inference Performance Engineering: Best Practices" (2023)—the TTFT/TPOT definitions, the total-time formula, the 100 ms/token reference point (10 tokens/s, ≈450 words per minute), and the 114 ms TTFT measurement for Llama2-70B on 4×A100-40GB.

2.4 Determinism and reproducibility

Even T=0 is not bit-for-bit deterministic. Setting temperature=0 makes token selection greedy but **does not guarantee identical output across runs**: floating-point operations are non-associative, GPU kernels depend on batch size, hardware differs, and the provider's backend changes imperceptibly. You cannot rely on bitwise reproducibility of the output.

The source of the noise is the shape of the batch, not "randomness." The stock explanation ("the GPU computes in parallel, atomic additions arrive in random order") is wrong for the forward pass: forward-pass kernels are deterministic—the same input on the same hardware gives bitwise the same result, and as a rule there is no atomic addition in them at all (Thinking Machines Lab, 2025). The real chain has four links.

1. **Floating-point addition is non-associative:** $(a + b) + c \neq a + (b + c)$. With a finite mantissa the result depends on the order of summation, and the divergence settles into the low-order bits.
2. **The kernel picks the summation order to fit the shape of the batch.** The reduction (for matrix multiplication, the contraction over the shared dimension k; for attention, over the KV-cache length) is cut into tiles, and the split is chosen to fit the current batch size for the sake of accelerator utilization. A different batch → a different split → a different addition order → different low-order bits in the logits. Standard kernels lack the property needed here, **batch invariance**—"the result for my row does not depend on who else is riding in the same batch." Exactly three operations are affected, the ones containing a reduction: RMSNorm (activation normalization, §A.3), matrix multiplication, and attention.
3. **Batch size is set by continuous batching (§1.6)**—that is, by the traffic of your batch-mates in that millisecond. Hence a formulation worth saying in full: **at T=0 your output depends on how many other people's requests arrived at the server at the same moment as yours.** That is the main cause of nondeterminism in public APIs, not the model's "creative temperament."
4. **The greedy choice flips only on a near-tie.** argmax is stable as long as the gap between the first and second logit is appreciably larger than the noise; at the overwhelming majority of positions it is, and the noise is not visible at all. Occasionally the top-2 converge to within a hair—and then those very low-order bits decide the outcome. There is no numeric threshold here: it differs at every position, and the primary source does not give one.

A seed does not cure this mechanism. A seed pins the sampler's pseudorandom number generator, but at T=0 sampling degenerates into argmax—there is nothing there to pin, and the noise enters higher up the pipeline, in the forward pass itself. So a seed reproduces the choice at T>0 on an unchanged backend, and cancels neither the batch dependence nor the divergence across versions and backends.

Why the divergence is "rare, but total." A run matches the reference only if not a single position flipped:

$$P_{\text{match}} = (1 - p)^N$$

- P_{match}—the probability that a run matched the reference bit for bit; dimensionless, 0.078 in the measurement below.
- p—the probability of an argmax flip at a single position (the share of positions where the top-2 converged closer than the noise); dimensionless, ≈ 0.0025 here.
- N—the generation length in tokens; 1,000 here.

Live numbers. Thinking Machines Lab ran Qwen3-235B-A22B-Instruct-2507 a thousand times at T=0 on a single prompt, 1,000 tokens each: the result was **80 distinct answers**, the most frequent one appearing 78 times out of 1,000. Substitute $N = 1000$ and $P_{\text{match}} = 78/1000 = 0.078$: $(1 - p)^{1000} = 0.078$, from which $p = 1 - 0.078^{1/1000} = 1 - e^{-2.551/1000} \approx 0.0025$—a quarter of a percent, that is, one near-tie roughly every 390 tokens.

In plain terms. At an individual token a flip is nearly impossible, but length raises that near-impossibility to a power: at a thousand tokens a bitwise match becomes the exception rather than the rule. The caveat without which the formula lies: p here is an average over positions, and the real distribution is extremely uneven. At most positions the top-2 gap is large and $p \approx 0$; the whole mass sits at the rare branch points. Averaging is fine for an estimate, but it does not describe "smooth noise."

The branching itself is visible in the same data. All 1,000 runs went identically up to the 102nd token; at the **103rd**, 992 runs continued with "Queens, New York" and 8 with "New York City." From there autoregression takes over (§1.6): the chosen token returns to the input and changes the conditioning of the entire remainder—one flipped position diverts not the neighboring word but **the whole rest of the sequence**. Out of a handful of such points grow 80 different answers.

Nondeterminism correlates with load. Since batch size is a function of other people's traffic, the probability of divergence is not a constant of the model but a function of the provider's load profile. What that looks like in practice: an eval that is green on quiet traffic (night, a dev stand, a batch of one request) and flaky at peak—without a single line of code changed. "Doesn't reproduce for me" here is not an excuse but a literal description of the environment.

The engineer's iron rule: test reproducibility under the batch pressure the system actually runs under in production—or accept the flap as a property of the environment rather than a bug. Bitwise reproducibility is a property you buy, not a given: batch-invariant kernels exist (the Thinking Machines Lab library, integrated into SGLang[°]) and deliver 1,000 identical runs out of 1,000, but they cost throughput: on the authors' Qwen3-8B, 26 s → 42 s after optimizing the attention kernel (the unoptimized variant: 55 s); on SGLang, from +24% to +55% latency depending on the attention backend and the context length. That price is justified where bit-for-bit matters substantively (on-policy RL, for example, where sampling and training must compute bit-for-bit identically or training diverges), and unjustified in an application agent, for which checking properties is enough.

The consequence for testing. Because output is nondeterministic even at zero temperature, a pointwise `assertEquals(expected, actual)` does not apply. What is needed is a check of the output's *properties*—evaluation (Chapter 17): schema validity, relevance, groundedness, not an exact string match. This is the fundamental reason LLM systems need a testing discipline of their own.

Model version drift is an operational risk. The provider updates the model under the same name—behavior shifts silently, and prompts tuned against the previous version can degrade. The countermeasure is the one applied to any external dependency: pin the model version (where the provider allows it) and run the eval set on every version change (regression testing). Changing the model is a deployment, not a spontaneous update.

Sources. H. He et al. (Thinking Machines Lab), "Defeating Nondeterminism in LLM Inference" (2025)—batch invariance, the Qwen3-235B experiment, batch-invariant kernels and their price. LMSYS/SGLang, "Towards Deterministic Inference in SGLang and Reproducible RL Training" (2025)—the integration and the overhead measurements. Google SRE Book (2016)—regressions and change management.

2.5 Reliability: the LLM as an unreliable external dependency

A language model called over the network is an **unreliable external dependency**, and the standard resilience patterns developed for any external service apply to it (the canonical source: M. Nygard, *Release It!*).

Failure modes: rate limit (429), timeouts, provider unavailability (5xx/outage), refusal by the content filter, a "context length exceeded" error, invalid or truncated output, latency degradation under load. The insidious combination: the connection drops in the middle of a *streaming* generation with structured output—what is left in

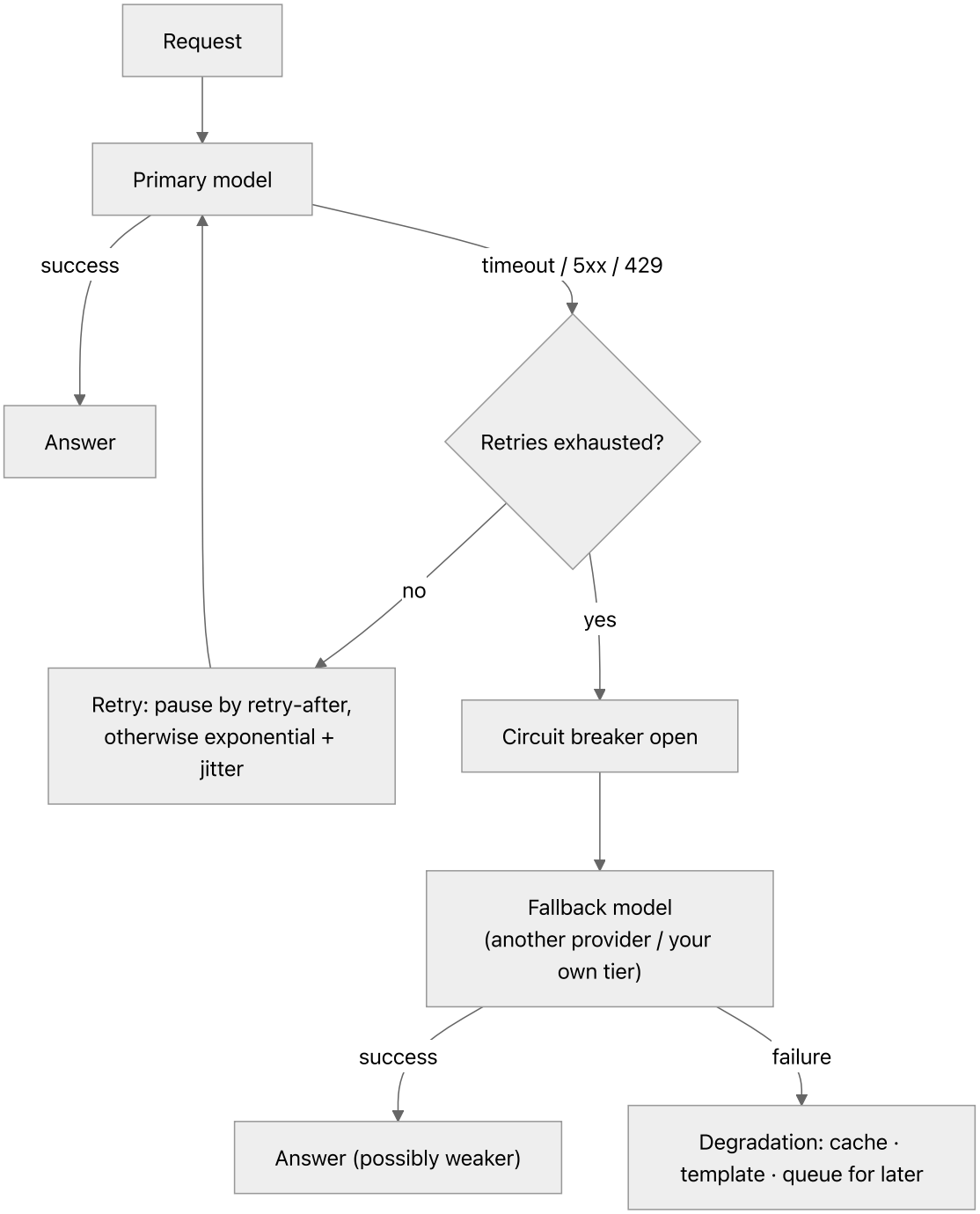
your hands is syntactically invalid partial JSON, and catching it is the job of a post-hoc validator with a retry (§5.5), not of the parser on the happy path.

Applicable resilience patterns:

- **Retry with exponential backoff and jitter**—for transient errors (429/5xx; on a 429, time the pause by the `retry-after` header, see below).
- **Circuit breaker**—fail fast during a provider outage instead of piling up a queue.
- **Timeout + graceful degradation**—on a timeout, return a cached, partial, or deterministic fallback instead of hanging.
- **Bulkhead and client-side rate limiting**—stay inside the provider's quotas, isolate workloads from one another.
- **Idempotent agent actions**—retries are the norm here, so operations with side effects must be idempotent (keyed by operation, as in the saga pattern).

Pattern: Fallback Model

Context. A feature's availability depends on an external LLM provider that can degrade or go down. **Problem.** How do you keep working when the primary model/provider fails? **Forces.** One provider = a single point of failure and vendor lock-in; several providers = complexity (different APIs, different behavior, different prompts). **Solution.** Keep a secondary model (from another provider or self-served) and switch to it when the primary fails or times out. Abstract the call behind a single provider-agnostic interface (the JVM frameworks Spring AI and LangChain4j, for example, provide such abstractions). **Mechanics.** A wrapper with a circuit breaker: primary → on the open state → secondary. Bear in mind that models behave differently, so the fallback needs prompts and tests of its own.



Resulting Context. Resilience to a provider outage and less lock-in; the price is maintaining prompts/evals twice over and the risk of degraded quality on the fallback. **When to Use.** The feature is availability-critical. **When Not.** Internal or non-critical scenarios where waiting out the recovery is simpler. **Related Patterns.** Circuit Breaker; Model Routing (the fallback as the “cheap tier”); Graceful Degradation. **Sources.** M. Nygard, *Release It!* (2nd ed., 2018)—circuit breaker, bulkhead, timeout; the Spring AI/LangChain4j docs—provider-agnostic abstractions.

What matters is that none of these patterns are specific to a language model—they apply to any external dependency. What is distinctive about the model as a dependency is that it is less reliable, more expensive, and nondeterministic, which makes resilience discipline here mandatory rather than optional.

How a retry to a model differs from a retry to a service. The caveat above is true at the level of mechanics and misleading at the level of economics. Retrying an ordinary REST endpoint costs latency and someone else's CPU; retrying a model costs money—and costs the most at exactly the moment the provider is degrading. Three mechanisms Nygard does not have, because there a retry is free for the caller.

Mechanism 1: a 429 arrives along several axes at once. The provider limits not "requests" but several independent counters. Anthropic has three—requests per minute (RPM), input tokens per minute (ITPM), and output tokens per minute (OTPM); exceeding *any* of them yields a 429 stating which limit was hit. OpenAI's axes are its own (requests and tokens per minute plus daily ones), and the documentation states outright: whichever limit is exhausted first is the one that fires. The practical consequence for a client-side limiter: a bucket that counts *calls* will faithfully keep catching 429s on long prompts—the token axis was hit, and the bucket cannot see it. You have to count tokens.

And here is a vendor divergence you must not carry over by analogy. OpenAI reserves the token quota *before* execution: the limit is computed as the maximum of `max_tokens` and an estimate from the request's character count, so an inflated `max_tokens` eats quota even if the answer came out short. Anthropic is the opposite: OTPM is counted in real time from the tokens actually produced, `max_tokens` does not enter the limit calculation at all, and inflating it costs nothing as far as limits go. The same setting punishes you at one provider and is free at another—the same vendor specificity as the economics of prefix caching (§20.2). A useful side effect of that same cache: at Anthropic, tokens read from the cache do not count toward ITPM on most models—the cache improves not only the bill but also throughput within the same limit.

Mechanism 2: a 429 usually comes with a ready answer to "when." The provider returns a `retry-after` header—the number of seconds until the window resets. Anthropic's documentation leaves no room: "The number of seconds to wait before retrying. Earlier attempts will fail." Blind exponential backoff when the reset window is known is not caution but a series of attempts guaranteed to fail. The rule: if the header arrived, time the pause by it; exponential backoff with jitter stays for the cases where the provider named no window (5xx, 529 "overloaded," a dropped connection).

Mechanism 3: the failed attempt has already been consumed. A drop at the 900th output token out of 1,000 rolls back neither the input nor those 900: there is no useful result, and there is consumption. The ratio of the cost of failure to the cost of success:

$$\frac{C_{fail}}{C_{success}} = \frac{N_{in} + k\phi N_{out}}{N_{in} + kN_{out}}$$

Where:

- N_{in}—input tokens in the request; in the §2.2 example, 1500.
- N_{out}—output tokens in the full answer; there too, 400.
- k—how many times more an output token costs than an input token (§2.2). Across the whole Claude lineup as of mid-2026, exactly 5 ($5/$25 per million for Opus 4.8, $3/$15 for Sonnet 4.6, $1/$5 for Haiku 4.5).
- ϕ—the share of the answer that got generated before the drop (0…1).
- The numerator and denominator are expressed in notional units of the input-token price; the ratio is dimensionless.

Live numbers. A drop at 90% of the answer (ϕ = 0.9): numerator 1500 + 5 × 0.9 × 400 = 3300, denominator 1500 + 5 × 400 = 3500, ratio 3300/3500 = 0.94—the failed attempt consumed 94% of a successful one and delivered nothing. The retry will spend the full 3500 all over again: 6800 instead of 3500 for one delivered answer.

On a long context it is worse, and the prefill/decode asymmetry (§2.2) is to blame. An input of 100,000 tokens, the same 400-token answer, a timeout right after prefill—not a single output token ($\phi = 0$): 100000/102000 = 0.98. An attempt that produced **not one** useful token consumed 98% of a successful one. In plain terms: you pay first and foremost for processing the input, and that happens in full regardless of whether the request survived to an answer; the longer the input relative to the answer, the closer the cost of failure gets to the cost of success, and the less "try once more" differs from "make another full request." A caveat on billing: whether the provider refunds a request cut off mid-generation is vendor-specific, there is no reliable public rule here, and it gets checked against the provider's documentation and your own invoice. What does not depend on the vendor: a *retry* is a full new request with a full prefill, and it is billed in full.

The retry storm and the inversion of the circuit breaker's meaning. The chain closes on itself: latency degrades → timeouts → retries → load grows → the bill spikes at exactly the moment quality is at rock bottom. The SRE book describes the dynamics in numbers: 100 retries per second in the first second turn into 200, then 300, and an ever-smaller share of the requests does useful work. More dangerous is that retries at different layers do not add, they multiply:

$$A = \prod_{i=1}^{L} a_i$$

Where:

- L—the number of layers, each retrying on its own.
- a_i—attempts at layer i (the one original plus retries).
- A—attempts reaching the bottom layer per single user action. The unit is attempts.

The primary source's live numbers: three layers (frontend, backend, JavaScript), each doing 3 retries, that is 4 attempts—one user action produces $4^3 = 64$ hits on the database. In an LLM stack one such layer is easy to overlook, because it is on by default: the official Anthropic SDKs retry transient errors on their own with an exponential pause—**twice by default**, that is 3 attempts—and they respect `retry-after`. An application that conscientiously wraps the call in "three attempts" gets $3 \times 3 = 9$ hits on the provider per request; add a retry at the gateway (§20.6) and it becomes 27. Before adding a retry of your own, find out how many the library already does, and leave exactly one level.

Hence an inversion worth naming outright. In Nygard the circuit breaker protects the *callee*—it keeps you from finishing off a failing service. Your traffic will neither save nor kill the model provider: the capacity is someone else's. The breaker here protects **the caller's wallet**—it stops paying for attempts at the moment their expected return is close to zero. The mechanics are the same, the motivation is different, and the tuning follows from the motivation: the threshold is set by the budget's burn rate, not by the provider's health.

The engineer's iron rule: set the retry budget in tokens and as a share of traffic, not as a number of attempts per request. A "3 attempts" limit does not bound the storm—under full degradation it authorizes tripling both the load and the bill. The second, independent breaker from the SRE book is a client-side budget: the share of retries in the overall request stream is kept below 10%, and when it goes over, retries stop until recovery. The carry-over to the model is direct, with one correction: count the share in tokens, not in requests (§2.2)—you pay for tokens.

Sources. M. Nygard, *Release It!* (2018); Google SRE Book (2016)—"Handling Overload" (the retry budget: 3 attempts per request, retry share below 10%) and "Addressing Cascading Failures" (attempts multiplying across layers, $4^3 = 64$; the storm growing 100 → 200 → 300 requests per second); the Anthropic documentation—Rate limits (the RPM/ITPM/OTPM axes, `retry-after`, ITPM with the cache taken into account), Errors (the SDKs' default retries, 529), and Pricing (the ratio of output price to input price); the OpenAI documentation—Rate limits (the limit axes, quota reservation by `max_tokens`).

2.6 Hosted versus self-host: a decision framework

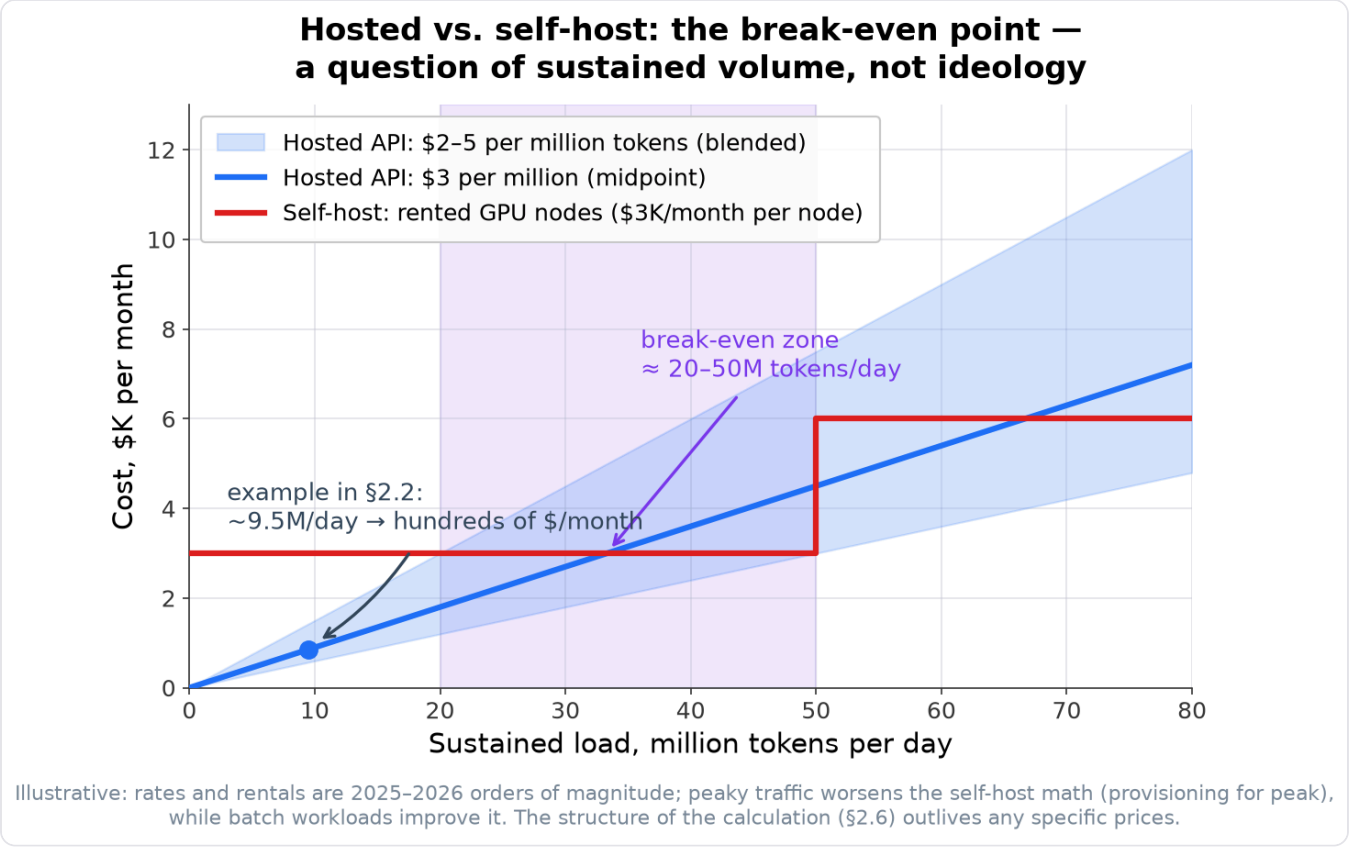
Not an ideological choice but an engineering trade-off along axes:

Axis	Hosted API	Self-host (your own infrastructure)
Cost at scale	Per call; grows linearly with volume	Capex + utilization; cheaper at high steady volume
Latency	Depends on the provider and the network	Under the operator's control
Data residency/compliance	Data leaves the premises	Data never leaves the perimeter
Customization	Limited	Full (fine-tuning open-weight models, Chapter 21)
Model choice / lock-in	The provider's catalog, lock-in	Any open-weight model
Operational burden	Minimal	Running GPU inference (vLLM and the like)

Self-host presumes a mature infrastructure practice: orchestrating GPU workloads and operating inference servers. With that in place it is realistic, but it is not justified for low or spiky volume, where hosted is simpler and cheaper.

The middle tier: managed inference for open models. Between the table's poles sits a third option—serverless GPU inference of open-weight models at specialized providers (Together, Fireworks, Baseten, and the like): open weights and freedom to pick the model (including your own LoRA adapters), without owning GPU infrastructure. It covers the case of "we need a specific open or fine-tuned model but have no team to run vLLM"; on residency and compliance, though, it sits closer to hosted (the data goes to the tier's provider) than to self-host.

The break-even point (an illustrative calculation). Orders of magnitude: renting a node with one server-class GPU runs a few thousand dollars a month; the break-even point ≈ rent / blended price per million hosted tokens. At $3,000/month and a blended rate of $2–5 per million, that is 0.6–1.5 billion tokens per month—that is, **20–50M tokens per day of sustained load**—several times more than a typical single feature (the §2.2 example: ~9.5M tokens per day, which at a blended rate of $1–2 per million comes to on the order of $300–600 a month on the API). Hence the practical conclusion: self-host rarely pays for itself on one feature—what pays for it is a *portfolio* of workloads at high utilization; a spiky profile makes the bill worse (you provision for the peak), background batch workloads make it better. The figures are illustrative; the structure of the calculation is not.



Illustrative: rates and rentals are 2025–2026 orders of magnitude; peaky traffic worsens the self-host math (provisioning for peak), while batch workloads improve it. The structure of the calculation (§2.6) outlives any specific prices.

The hybrid (the standard 2026 pattern). A hosted frontier model for hard, rare tasks + a self-hosted small model for high-frequency narrow tasks (field extraction, for example). Cost optimized while quality is preserved where quality is needed. Details in Chapter 22.

Sources. Chip Huyen, *AI Engineering* (2025)—build vs buy, the economics of serving.

2.7 Specifying a feature in budgets (the architect's artifact)

The chapter's material comes together in an artifact—a **nonfunctional specification for a feature**, adapted to the peculiarities of a language model:

> **Template: NFR specification for an LLM feature**
>
> - **Latency:** SLO p95 (TTFT and total), streaming yes/no.
> - **Cost:** budget per request and per month; model tier; caching.
> - **Quality:** eval thresholds (faithfulness/relevance/schema validity), a gate in CI.
> - **Determinism:** the required level; temperature; pinning the model version.
> - **Availability:** the target + a failure strategy (fallback model, degradation).
> - **Security/compliance:** data residency, guardrails (Chapter 19).

The filled-in specification goes to design review. It turns the intent to "add an LLM" into an engineering decision with measurable properties—the thing missing from most pilot projects, the ones that never make it into production.

Sources. Google SRE Book (2016)—SLO/SLI/budgets; Chip Huyen, *AI Engineering* (2025).

In the Margins: Chapter 2

> ↩ **Reasoning model**—a model trained to produce a long internal chain of reasoning before answering (the o-series, DeepSeek-R1 and successors); the reasoning is billed as output tokens. → §13.8; ⇐ Glossary.
>
> ↩ **SGLang**—an open-source LLM inference server (a program that holds the model on the accelerator and serves a stream of requests); known for RadixAttention prefix caching. → Chapter 22.

Part II. The Interface to the Model

Part I treated the model as a component with a defined set of properties (§1.10) and budgets (Chapter 2). Part II is about its **interface**—how intent is communicated to the model and how its output is constrained. Three chapters: designing the prompt (Chapter 3), designing the context (Chapter 4), and structured output with reliable parsing (Chapter 5).

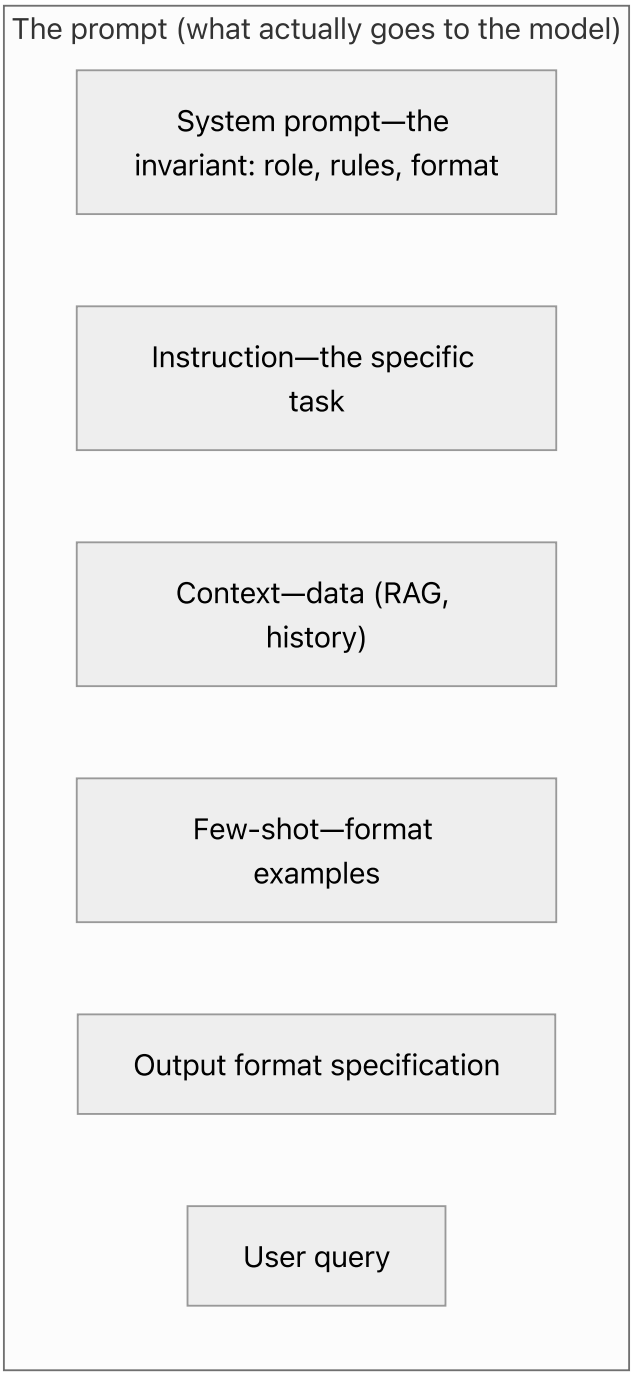
Chapter 3. Prompt Engineering as Contract Design

3.1 The framing: a prompt as an interface specification

Basic prompt engineering is usually understood as "writing good prompts." For system design, a different framing matters more: a prompt is an interface specification for a probabilistic component that (a) is nondeterministic, (b) reads instructions and data in a single text channel, and © may satisfy a requirement only partially. The job, then, is not to hunt for lucky wordings but to *specify* the input, *constrain* the output, and *handle* the cases of incomplete or incorrect execution. The sections that follow work through the tools for each of these three tasks; their limits are drawn in §3.7—and beyond those limits lie RAG° (knowledge), structured output (format guarantees), and guardrails (safety).

3.2 Anatomy of a prompt

A production prompt is not a query string but an assembled structure. Its components and their roles:



Component order matters: because of positional bias (§1.8), the model attends more fully to the beginning and the end. The typical layout puts the invariant and the required format at the start (the system part), and critical data at the beginning or end of the user part—never in the middle.

Instructions and data share one channel. The defining property of this interface: the model receives both instructions and data as a single text stream and does not tell them apart automatically. So the boundaries between them must be drawn explicitly—with delimiters (XML tags, labeled blocks), marking untrusted content (user input, retrieved documents, tool results) as data. This improves clarity and doubles as the first line of defense against prompt injection (Chapter 19), in which instructions embedded in the data hijack control.

Instruction hierarchy. Part of the same problem is addressed at the level of model training: modern models are trained to rank instruction sources by priority—system above developer, developer above user, and content from tools and data below all of them (E. Wallace et al., "The Instruction Hierarchy," OpenAI, 2024; the line of work continues to evolve through 2025–2026). This improves resistance to data attempting to hijack control, but like any learned defense it is probabilistic and yields to an adaptive attack (§19.5)—structurally isolating untrusted content remains mandatory.

3.3 A catalog of prompting patterns

Mini-patterns, each with its conditions of applicability:

- **Role / Persona priming**—assigning a role ("a support specialist who answers strictly according to policy"). Narrows the answer space. *When:* almost always useful in the system part.
- **Few-shot / demonstrations**—defining format and behavior through examples rather than verbal description. The number of examples, their selection (representativeness, edge-case coverage), and their order all matter—each measurably affects the result. *When:* format-sensitive tasks, a specific output style. *When not:* if the model handles the task from the instruction alone, examples just burn tokens.
- **Chain-of-Thought (CoT)**—instructing the model to produce intermediate reasoning steps. Improves accuracy on multi-step tasks (§3.4). *The price:* extra tokens and latency.
- **Decomposition**—splitting one large prompt into a sequence of narrow steps (the Prompt Chaining pattern, Chapter 11). Each step is easier to specify and to verify.
- **Output format specification**—describing the schema (a soft measure) or enforcing it (a strong one, Chapter 5). For output that feeds a parser or a tool, use enforcement.
- **Delimiters and structure**—explicitly separating instructions from data (see §3.2); doubles as the first line of defense against prompt injection (Chapter 19).
- **Prefilling**—supplying the beginning of the model's response turn to shape how it continues (an opening brace `{` forces JSON; a heading sets the structure). An effective way to constrain the form of the output without any external mechanism.
- **Permission to refuse**—explicitly allowing the model to answer "not enough information" or "I don't know." Reduces the tendency to fabricate on unanswerable inputs (§1.9), and matters especially in RAG, where the answer must rest on the context (§6.7).
- **Positive framing over negative**—stating what the model *should* do works more reliably than the prohibition "don't do X": the model follows a prescribed action more faithfully than a negation.

3.4 Chain-of-Thought, in depth

What it is. Chain-of-Thought (CoT) is a technique in which the model generates intermediate reasoning steps before the final answer. Wei et al. (2022) showed that this markedly improves accuracy on arithmetic, logic, and multi-step tasks.

Why it works. The explanation comes down to compute per token. In a single forward pass the model performs a fixed amount of computation—a transformer is not a CPU with a loop: within one pass there is no mechanism to repeat a computation an arbitrary number of times (a fixed number of layers, one pass through each)—so a task that requires many steps has no room to be "folded" into a single step. CoT moves the intermediate steps out into output tokens, increasing the amount of *sequential* computation: each generated reasoning step becomes part of the input for the next. The intermediate reasoning acts as the external scratchpad the model lacks within a single pass.

Variants:

- **Zero-shot CoT** (Kojima et al., 2022)—an instruction to reason step by step is enough, no examples needed.
- **Few-shot CoT**—examples with the reasoning explicitly shown.
- **Self-consistency** (Wang et al., 2022)—generate several reasoning chains (at elevated temperature) and take the majority answer; more expensive, but more accurate on hard tasks.

When to use it. CoT pays off on tasks that require multi-step inference (calculation, logic, planning); on simple extraction or classification tasks it only spends tokens and latency for no gain.

The shifting landscape (2025+). Reasoning models—specialized at first (the o-series, DeepSeek-R1), by 2026 the standard operating mode of most frontier models—are trained with reinforcement learning to carry out extended reasoning *internally*; for them, an explicit "reason step by step" instruction is often redundant and sometimes hurts the result. The lever you control is no longer whether CoT appears in the prompt but the model's internal reasoning budget (§13.8).

Sources. J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in LLMs" (2022); T. Kojima et al., "Large Language Models are Zero-Shot Reasoners" (2022); X. Wang et al., "Self-Consistency Improves Chain of Thought Reasoning" (2022).

3.5 The system prompt as an invariant

What goes into the system prompt—the long-lived role, hard constraints, output format, safety rules: everything that does not change from request to request. What does not belong there: volatile data (that belongs to the request context) and, critically, **secrets**. The system prompt is subject to leakage (prompt leakage, OWASP risk LLM07[°], Chapter 19), so keys, connection strings, and internal addresses have no place in it—access control is implemented outside the model.

Stability and caching. A stable system prompt is the ideal candidate for prompt caching (§2.2): its constant part goes into the cacheable prefix.

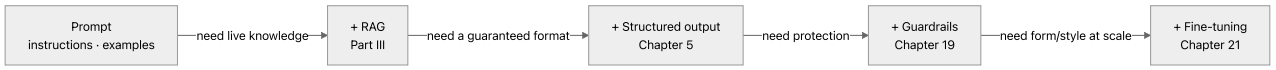
3.6 Prompts as code

Prompts are production artifacts and demand to be treated as such: versioning, templating (variables instead of string concatenation), change review, testing through evaluation (Chapter 17). A prompt change is a deployment: it can alter system behavior just as a code change can. Hence the standard change-management discipline applies—pinned versions, review, regression testing, canary rollouts. Editing a prompt directly in production, outside the deployment process, carries the same risk of uncontrolled behavior change as any change that bypasses the pipeline.

3.7 The limits of prompt engineering

Being honest about the limits is what positions Part II as a whole. A prompt **cannot**:

- reliably carry a large body of changing knowledge → that is **RAG** (Part III);
- guarantee a valid output schema → that is **structured output** (Chapter 5);
- fully prevent prompt injection → that takes **guardrails** (Chapter 19);
- durably instill, at scale, a new form/style/tool-calling format → that is **fine-tuning** (Chapter 21).



The sequence (the same ladder): **prompting first; escalate only when it hits a wall.** Most tasks are covered by a good prompt plus RAG; fine-tuning and multi-agent are the top of the ladder, climbed only on proven need. Prompt engineering is the cheap, instant, reversible first tool—which is exactly why it comes first.

Sources. J. Wei et al. (2022), T. Kojima et al. (2022), X. Wang et al. (2022); Z. Sprague et al., "To CoT or not to CoT?" (2024)—a meta-analysis: CoT's gains are concentrated in math and symbolic tasks; the Anthropic/OpenAI prompting documentation; Chip Huyen, *AI Engineering* (2025).

In the Margins: Chapter 3

> ↩ **RAG**—Retrieval-Augmented Generation: relevant knowledge is retrieved from external sources and fed into the context, instead of hoping the model "remembers." → Chapter 6; ⇐ Glossary.
>
> ↩ **OWASP LLM07**—an entry in the "OWASP Top 10 for LLM Applications," the ranked list of LLM-application risks from the Open Worldwide Application Security Project; LLM07 is system prompt leakage. → §19.3.

Chapter 4. Context Engineering and Context Management Patterns

4.1 A shift in the unit of design

Prompt engineering (Chapter 3) optimizes how the task is phrased. **Context engineering** optimizes the *entire* set of tokens assembled into the window at every inference step. The shift was no accident: once agents became multi-step and windows became large, the bottleneck stopped being “how do I word the request” and became “what information, in what form, where in the window, and within what budget do I supply.” Phrasing the task takes a fraction of a percent of an advanced agent’s context; the rest is the system instruction, retrieved knowledge, history, tool definitions, and intermediate results.

A precise definition. **Context engineering** is the discipline of dynamically constructing the model’s input context: selecting, formatting, ordering, and compressing the information the task needs (instructions, retrieved knowledge, history, tool definitions, examples) within a token budget, separately for each inference step.

A useful framing: context is state assembled for a stateless component. The model retains nothing between calls (§1.10); everything it operates on at a given moment was placed into the context on that step. Context engineering is therefore state management for a stateless component under a hard budget: every step decides anew what state to reconstruct in the window, and in what form.

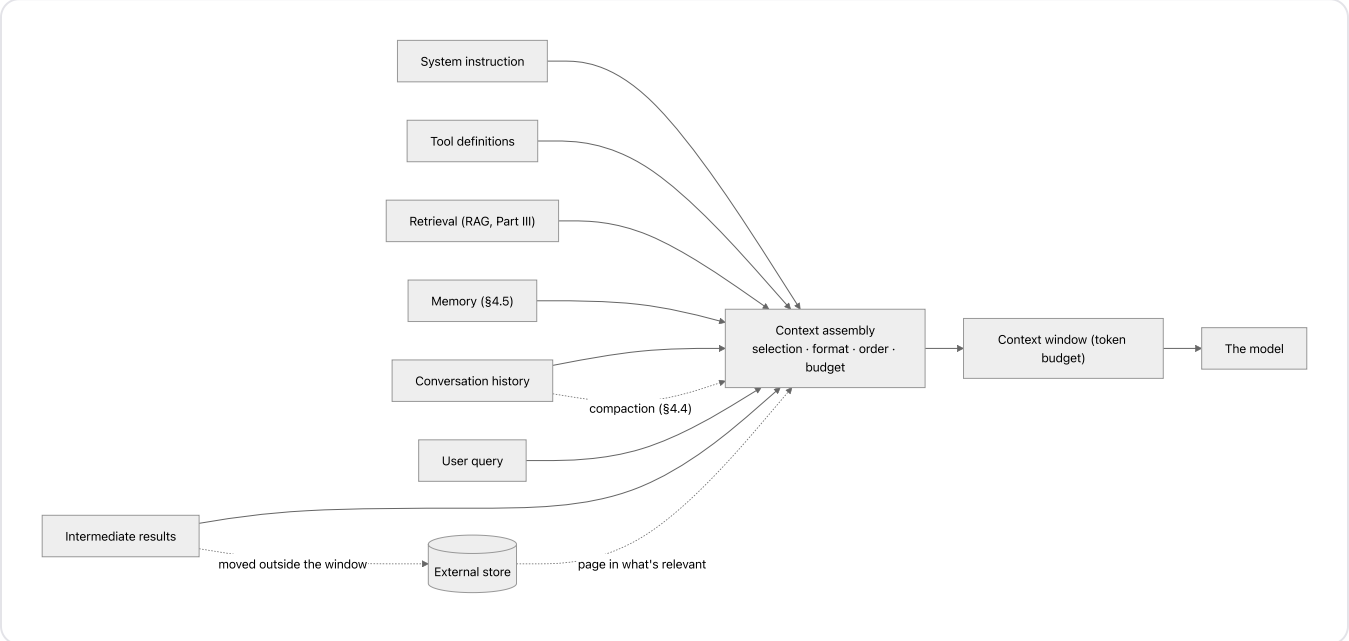
4.2 The anatomy of context: who competes for the budget

The window is a finite budget, and its consumers compete for it with different “value/cost” profiles:

Consumer	What it provides	What it costs
System prompt	A behavioral invariant, the format	Constant tokens on every call (cacheable, §2.2)
Tool definitions	The ability to act	Can run large; many tools = many tokens and confusion over which to pick
Few-shot examples	Format/behavior	Expensive once there are many examples
Conversation history	Continuity	**Grows without bound**—the main source of overflow
Retrieved chunks (RAG)	Fresh/private facts	Tokens + the risk of dilution when overdone
Scratchpad / intermediate results	Multi-step reasoning	Accumulate over the agent loop
User query	The task itself	Small
Generation (the model's answer)	The result itself	Shares **the same** budget with the input: `max_tokens` is subtracted from the window (§1.8)

The design job is to allocate that budget: what to include, what to compress, what to move outside the window.

Structurally, context engineering is an *assembly* layer that funnels heterogeneous sources into a single window under budget:



4.3 Forces: why this is not trivial

Context engineering is **constrained optimization**, because these collide:

- **A finite budget**—the window's hard ceiling.

- **Degradation with volume** (§1.8)—"lost in the middle" and attention dilution mean *more context ≠ better*; past a certain volume, adding tokens makes the answer *worse*. And *placement* matters: what's critical goes at the edges.
- **Cost** (§2.2)—every token is paid for on *every* call; and in an agent loop the context keeps growing on top of that.
- **The relevance ↔ recall trade-off**—include more (don't miss what you need) versus include less (don't add noise). The same trade-off as in search.
- **Staleness**—history and tool results go stale within a session; an old result can contradict a new one.

The goal: maximum relevant signal for minimum tokens and noise.

Deep Dive: the mechanics and the empirics of context degradation

The claim "more ≠ better" needs a mechanism and measurements behind it, or it's just faith.

The mechanics: the attention budget is normalized to one. The cause of the degradation was already assembled back in §1.4—all that's left is to retrieve it. Step 3 of the attention-block trace: `softmax` turns compatibility scores into weights that **sum to exactly 1**—a token's "attention budget," distributed across every token it can see. What matters here is that the budget **does not grow with context length**: a thousand tokens and a hundred thousand tokens divide the same unit. Dilution is therefore not a metaphor but a zero-sum game: every token added takes weight away from the material already there. Don't confuse this quantity with the "finite budget" from the list above: that one is a window ceiling measured in *tokens*, which a vendor is free to raise to a million; this one is normalization to *one*, which nobody will ever raise.

How much it takes away can be computed exactly. Suppose the fact you're after (the "needle") scored Δ higher on compatibility than each of the n tokens of the irrelevant "haystack":

$$w^* = \frac{e^s}{e^s + n \cdot e^{s-\Delta}} = \frac{1}{1 + n \cdot e^{-\Delta}}$$

Legend. w^* is the share of the attention budget the needle gets: dimensionless, from 0 to 1. n is the number of distractor tokens in the window (a count; 10^3–10^6 in working windows). Δ is the gap in compatibility scores between the needle and an ordinary haystack token, already divided by $\sqrt{d_{head}}$ (§1.4): dimensionless, and it has no "typical" tabulated value—so we fix Δ = 10 not as a measurement but as a premise of the problem: the gap at which, across 1000 tokens, the needle collects 96% of the budget, meaning the model sees it confidently. s is the needle's raw score; it cancels on the right-hand side, and that is the first conclusion: the model's absolute "confidence" means nothing, only the gap does any work.

Live numbers at Δ = 10, where $e^{-10} = 4.5 \cdot 10^{-5}$. The denominator $1 + n \cdot 4.5 \cdot 10^{-5}$ grows along with the haystack:

n (haystack tokens)	Denominator	w^*
1,000	1 + 0.045 = 1.045	**0.96**
10,000	1 + 0.45 = 1.45	**0.69**
100,000	1 + 4.54 = 5.54	**0.18**

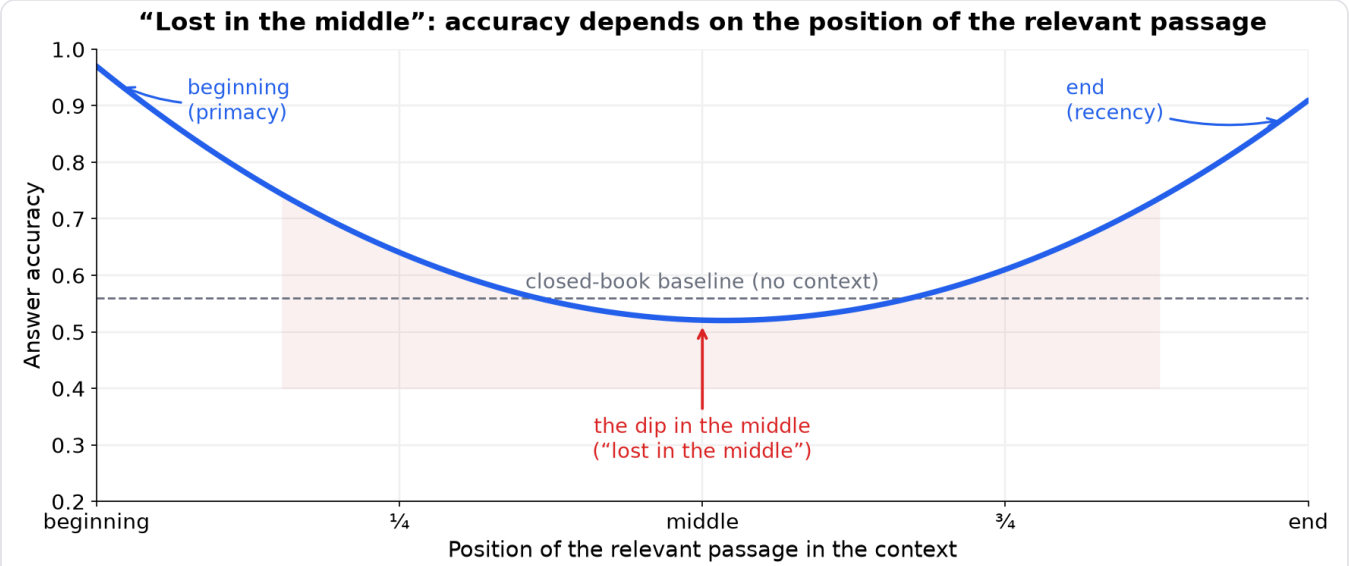
In plain terms: the model did not change its mind. Δ stayed at 10, the needle's score did not move an inch—only the denominator grew. A haystack of a hundred thousand tokens, each one individually negligible (e^{-10} against one), took 82% of the budget between them: by count, not by quality. This is the derivation of what §1.8 asserts without proof: attention spreads "thinner" precisely because the sum is fixed.

Why the model's confidence cannot cure this. To hold the same 96% at 100,000 tokens, the gap has to grow from 10 to 14.6—by exactly ln 100 = 4.6: holding a share requires Δ to grow proportionally to ln n. There is nowhere for it to grow from, though the reason is subtler than "the weights are frozen." The upper bound on Δ is a product of four factors (up to a factor of 2): the spectral norms of the matrices W_q and W_k—that is, the largest stretch a matrix can impose on a vector—and the norms of the representation vectors themselves, the query and the keys (Veličković et al., 2025, Prop. 3.1). At inference only the first two are frozen: the norms of the representations depend on the input and, left to themselves, could grow along with it. What holds them down is layer normalization (RMSNorm, §1.4), which levels the scale of the vectors at every block—and only both facts together close the ceiling. The original paper states the same thing from the other side, titling the section "sharpness requires large weights": you can hold sharpness at growing n only if something is allowed to grow—and in a frozen, normalized model there is no such factor. Hence the proven result: with bounded score spread, any attention weight is pinned into the corridor $\frac{1}{n}e^{-\delta/\theta} \le w \le \frac{1}{n}e^{\delta/\theta}$ and behaves as Θ(1/n)—as n grows it inevitably slides toward a uniform distribution (ibid., Lemma 2.1; δ is the spread between the maximum and minimum scores, θ is the softmax temperature, equal to 1 in ordinary attention). The needle's advantage is a constant factor $e^{\Delta} \approx 22\,000$, and a constant factor does not outrun a linearly growing denominator. At 100k tokens the uniform share is 10^{-5}: the needle lives a constant factor above that floor, but it sinks along with it.

The edges: normalization explains one of the two. A sum of 1 implies uniform dilution—but not the U-shaped asymmetry. It does explain one edge, and by the same mechanism: since the weights are obliged to sum to one, they cannot all be zeroed—when a token has nothing to look for in the past, the surplus budget has nowhere to go, and the model **dumps** it on the first tokens of the sequence, which every later token can see by virtue of autoregression. This is the **attention sink**, already familiar from the role of BOS (§1.2); in Xiao et al.'s measurements on 4096-token sequences, the first token's share exceeds 50% of attention in most layers (all but the bottom two). The beginning is "hot" not because anything important sits there but because it serves as the dump for the surplus. Recency (how well the end survives) and the dip specifically in the *middle*, however, do not follow from normalization: positional encoding and the distribution of lengths in the training data are what account for those. The honest boundary of the mechanism: a sum of 1 explains why everything goes blind as n grows and why the beginning holds its weight; the U-curve below remains an empirical finding, not a consequence.

The engineering invariant. Δ is not tunable at inference: the weights are frozen, and a more insistent prompt wording will not raise a fact's weight against a hundred-thousand-term denominator. The only quantity in the engineer's hands is n. That is why most of the catalog that follows (§4.4) is about shrinking the denominator—retrieve instead of feeding everything in (§1.8), trim tool output, isolate sub-agents—and not about asking the model to pay closer attention.

The U-shaped curve (Liu et al., 2023). The original "Lost in the Middle" study tested models on multi-document tasks, varying the *position* of the relevant document. The result is a characteristic U-shaped dependence of accuracy on position: high when the needed fact sits at the beginning or the end of the context, and noticeably sagging in the middle. The dip in the middle can be severe enough that a model with the relevant fact in the middle of a long context answers *worse than a model given no such context at all*—"open book" turned out to be worse than "closed book." Primacy (the beginning) and recency (the end) are attention's two "hot" edges.



Schematic, after Liu et al. (2023). Dip depth varies by model and task; in the original study, mid-context accuracy sometimes fell below the closed-book level.

The needle-in-a-haystack (NIAH) methodology. A practical way to measure the effect on a specific model and task: insert a “needle” (a unique fact, say “the secret code is 7492”) into a “haystack” of irrelevant text at varying depths and context lengths, ask for the fact, and plot a heatmap of “depth × length → found/not found.” This became the standard smoke test for long context (popularized by G. Kamradt, 2023). An important caveat: basic single-needle NIAH is an *easy* test; real tasks demand *several* facts and *reasoning* over them, where degradation is stronger. Hence the multi-needle and reasoning-over-context variants.

Differences between models, and “context rot.” How much degradation you get depends heavily on the model: some hold 100k+ with almost no dip on plain NIAH, others start drifting at 32k. But even models strong on NIAH degrade on *hard* tasks as length grows—the effect practitioners call **context rot**: reasoning quality falls as the window fills, even when everything formally “fit” and simple lookup works. The practical conclusion: do not take the advertised window size for the working one; measure degradation on your actual task (multi-needle and reasoning over the context), not on simple lookup. A “1M token” window does not mean effective reasoning over 1M. A counterintuitive detail from the same study: *coherent*, well-structured long context can degrade quality more than shuffled context—being well-ordered is no automatic hedge against degradation.

Sources. N. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts” (2023); G. Kamradt, “Needle In A Haystack” (2023)—a methodology for evaluating long context; Chroma Research, “Context Rot: How Increasing Input Tokens Impacts LLM Performance” (Hong, Troynikov, Huber, 2025)—18 models, including the effect of how structured the context is. P. Veličković, C. Perivolaropoulos, F. Barbero, R. Pascanu, “Softmax is not Enough (for Sharp Size Generalisation)” (arXiv 2024; ICML 2025)—Lemma 2.1: with bounded scores, attention weights behave as $\Theta(1/n)$. G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis, “Efficient Streaming Language Models with Attention Sinks” (2023)—the attention sink as a consequence of softmax normalization.

4.4 A catalog of context management patterns

- **Retrieval (RAG) as context selection**—instead of holding all knowledge in the window, retrieve only what’s relevant to the query (Part III). The dominant pattern for knowledge.
- **Sliding Window / truncation**—keep the last `N` turns of history, drop the older ones. Simple, cheap; loses long-range memory. Fine when the old turns genuinely aren’t needed.

- **Externalized state / scratchpad**—intermediate results are written to an external store, and only pointers or a digest stay in the context. The window is freed of bulky working state, which is reconstructed on demand.
- **Context isolation / sub-agents**—each sub-agent gets *its own clean, minimal context* instead of one growing, polluted window (multi-agent, Chapter 14). Separating contexts keeps one long interaction from adding noise to the rest (in detail—the deep dive below).
- **Tool-result pruning**—don't keep a tool's full verbose output in the context; extract or compress what's needed. Otherwise one chatty tool eats the budget.
- **Placement / ordering**—what's critical goes at the edges of the window (§1.8), untrusted data goes in an isolated block (§4.6).
- **Context caching**—reuse a stable prefix for cost and latency (§2.2).

One pattern in the full template—the least obvious one, and the most practical for long sessions:

Pattern: Context Compaction

Context. A long multi-step dialogue or agent loop: history and intermediate results grow and approach the window limit. **Problem.** How do you preserve continuity (the agent remembers what has been done and decided) without overflowing the window or paying for context bloat? **Forces.** Keeping the whole history means overflow and dilution; bluntly truncating the old means losing important early decisions and facts. **Solution.** As the budget threshold approaches, **compress the older part of the history into a summary** (a rolling summary): the agent periodically "takes notes" on what it has been through, and the context retains `summary of the past + the last N turns verbatim`. The summary is updated incrementally. **Mechanics.** Trigger on reaching a fraction of the budget → call the model with "compress this dialogue down to key facts, decisions, open questions" → replace the old turns with that summary. Summarization is usually done by a cheap model (a tier, §2.2). **Example.** An incident-triage agent running over many steps: it keeps an "investigation summary" plus the most recent actions, not the entire log. The same mechanism operates in consumer chat interfaces: a long conversation doesn't run into a length refusal (§1.8) only because the shell quietly compresses the history on the user's behalf. It helps to see two sides of one limit here: the API refuses hard and in the open, while the product shell hides that same limit—and hides the loss of detail along with it. **Resulting Context.** Constant context size with the substance preserved; the price is that summarization is lossy (you can lose a detail that turns out to matter later) and costs a model call. **When to Use.** Long sessions and loops where history substantially exceeds any sensible budget. **When Not To.** Short interactions—simpler to keep everything; or when *every* detail matters verbatim (then externalize state with exact retrieval, not compression). **Related Patterns.** Sliding Window (the crude alternative); Externalized State (when you need precision, not compression); Retrieval-based memory (§4.5). **Sources.** Anthropic, engineering notes on effective context (2024–2025); agent memory patterns in V. Dibia, *Designing Multi-Agent Systems* (2025).

Deep Dive: how to compress (hierarchy and structured notes)

The Compaction pattern answers "when"; this section answers "how," because naive summarization loses the wrong things and accumulates error.

- **What to keep vs. compress.** Not everything in the history is of equal worth. Keep *verbatim*: decisions made and their rationale, established facts and constraints, open questions, identifiers (order ids, file names). Compress: draft reasoning, dead ends, chatter. The rule: *decisions and facts verbatim, the path to them in the summary*.

- **Recursive/hierarchical summarization.** For very long histories, compress not "all at once" but in levels: blocks → summaries of blocks → summaries of summaries (like map-reduce). More robust against the window limit of the summarization step itself, and it yields multi-level memory (detailed for the recent, coarse for the distant).
- **Structured notes instead of prose.** A prose summary is itself subject to dilution and distorting paraphrase. It's more reliable to maintain *structured* state—not a paragraph but a structure with fields `decisions[]`, `facts[]`, `open_questions[]`, `entities{}` (a link to structured output, Chapter 5). It's more compact and more resistant to meaning drifting under retelling: what you maintain is not a retelling but a structured working model of the task.
- **Error accumulation.** Every re-summarization is a lossy operation over an already lossily compressed summary, so errors accumulate from generation to generation. The mitigation: compress the *original* history rather than summaries of summaries, and keep "anchor" facts verbatim, outside the compression loop.

Native provider primitives (2025+). Before building compaction and memory yourself, check what the API already gives you. At Anthropic that's **context editing**—automatically evicting stale tool results from a growing context—and the **memory tool**—file-based memory that the model manages through a tool and that survives the session (beta `context-management-2025-06-27`); by Anthropic's numbers on a 100-turn scenario, context editing alone cuts token consumption by up to ~84%, and adding the memory tool raises the quality gain from ~29% to ~39%. Comparable mechanisms are appearing at other providers. These primitives don't retire this chapter's patterns—they implement them for you, off the shelf; the design decision that remains is policy: what counts as stale and what deserves to be remembered (the memory tool is an off-the-shelf implementation of the write-read loop from §4.5).

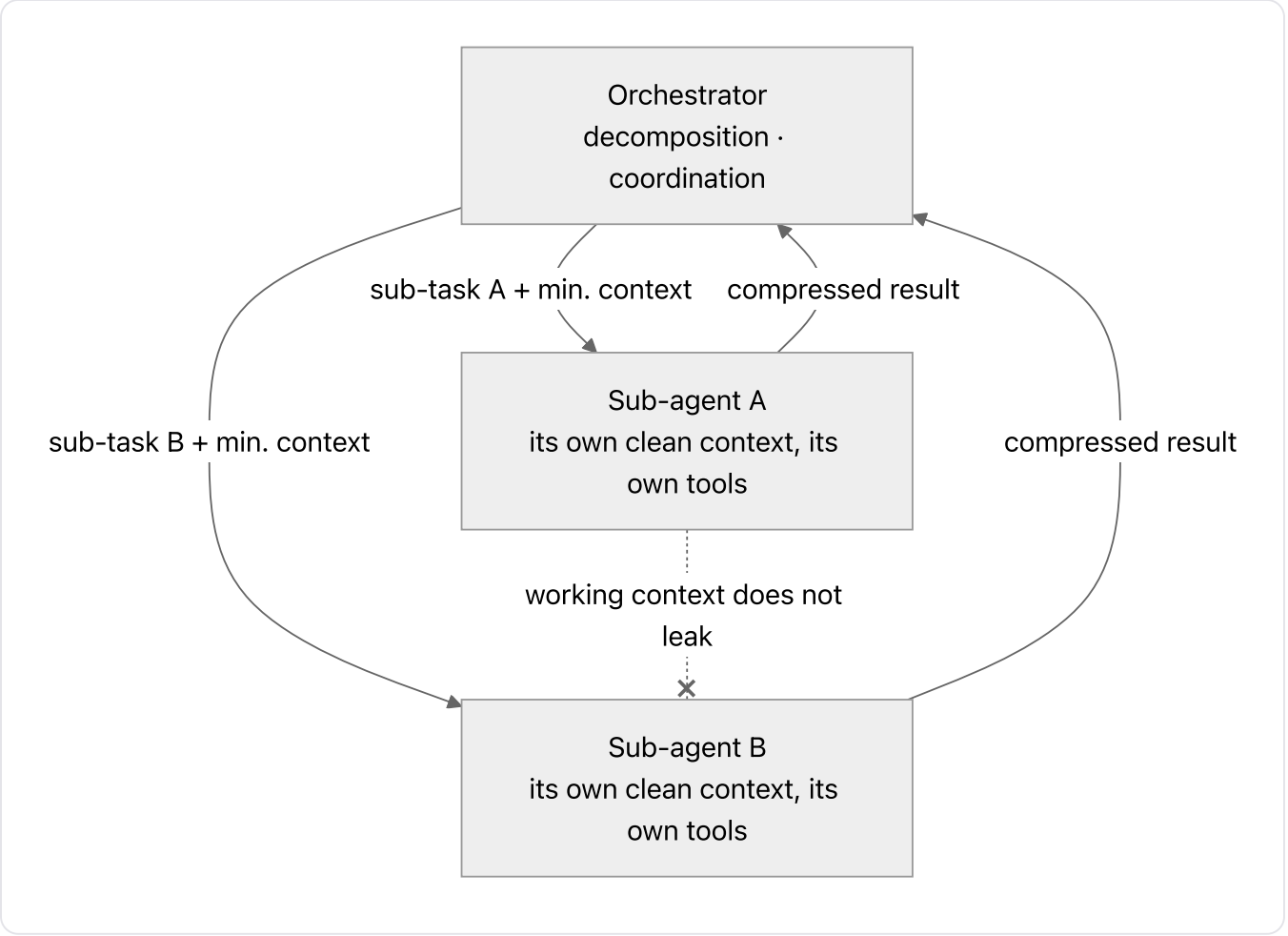
Deep Dive: context isolation with sub-agents

One of the most important modern techniques for scaling agents—hence the mechanics, not a one-liner.

The problem. As a task grows, a single agent accumulates everything in its window: instructions, intermediate results from a dozen sub-tasks, tool output. The context picks up noise—dilution sets in (§4.3) and cost climbs. On top of that, different sub-tasks need different tools and instructions, all competing for one window.

The mechanics. An orchestrator (Chapters 11, 14) decomposes the task and launches **sub-agents, each with its own clean, minimal context**: only the tools it needs, only the data that's relevant, only its own instruction. The sub-agent returns a *compressed result* (not its whole internal trajectory) to the orchestrator. The sub-task's "dirty" working context leaks neither into the orchestrator nor into its neighbors.

Mechanically, the isolation looks like this:



The sub-task's working ("dirty") context leaks neither into the orchestrator nor into neighboring sub-agents—only the compressed result goes out. In essence this decomposes one monolithic context into isolated regions, each with a clear zone of responsibility.

Trade-offs. Upside: every agent runs on a short, focused context (less dilution, cheaper per step, a simpler tool set). Downside: calls multiply (cost, §2.2), coordination and debugging get harder (Chapter 14), and there's a risk of losing context *between* agents (a neighbor doesn't see what another one knows). The rule (the ladder): isolation is justified when a single context has genuinely picked up noise or when sub-tasks need incompatible tools; for a simple linear process it's over-engineering.

Sources. Anthropic, material on multi-agent research systems and context isolation (2024–2025); V. Dibia, *Designing Multi-Agent Systems* (2025).

Deep Dive: the mechanics of prompt/prefix caching

Since context caching is a cost lever (§2.2), you need to understand what exactly gets cached and what breaks the cache.

- **What gets cached is the shared *prefix*, as a KV-cache°.** The provider saves the prefill (§1.6) result for the start of the prompt and reuses it on subsequent requests with the same start—you pay only for the "tail." This saves latency (no recomputing the prefill of a long system prompt) and money (cache tokens are much cheaper).
- **The condition is an exact prefix match, character by character, from the start.** The cache works on a match *from the beginning* of the sequence; any difference at the start invalidates the cache for everything after it. The consequence: **stable up front, volatile at the back.** The order

`[system] → [tools] → [RAG chunks] → [query]` caches well; drop a changing timestamp or user-id at the start of the system prompt and you kill the cache for the whole request.

- **What breaks the cache:** any change at the start (even a space or a date), a model version change, TTL expiry (usually short), exceeding limits.
- **The conflict with §4.3, and how it resolves.** §4.3 advises putting what's critical *at the end* (recency), while the cache likes what's stable *at the beginning*. There is no contradiction: the beginning takes the stable *instructions and tools* (the cache), the end takes the volatile *critical content of the request* (recency). These are different parts of the context.

Worked example: laying out a 200k window

Say a support agent has a 200k-token window. The naive read is "everything will fit." The engineering layout of the budget:

Component	Budget (example)	Strategy
System + rules + format	~2k	Stable, goes at the start → cacheable
Tool definitions (8 of them)	~3k	Only the ones the sub-task needs (pruning)
RAG chunks (policy documents)	~6k (top-5)	Retrieve what's relevant, not the whole policy; edges, not the middle
Conversation history	~4k	Sliding window + compaction of the past into a structured summary
Scratchpad / tool output	~3k	Prune the verbose; the bulky goes to an external store
Query + critical content	~1k	At the end (recency)
Working total	**~19k**	—
+ Reserve for the answer (`max_tokens`)	~4k (the model's ceiling is 64k)	Output shares the window with the input (§1.8)
Occupied in a 200k window	**~23k**	The remainder is headroom for peaks, not a target

The key takeaway: even with a 200k window, the target is around 19k of meaningful context, not filling the window. The hard limit isn't remotely close here: 19k of input plus even the model's maximum 64k of output is 83k out of 200k (§1.8). The constraint that determines the layout is not arithmetic but qualitative. The grounds are §4.3 (dilution and context rot hurt quality long before the limit) and §2.2 (every token is paid for on every call across the whole loop). Filling the window just because it's big is an anti-pattern: a big window is headroom for peak cases, not an operating point.

4.5 Memory as long-term context

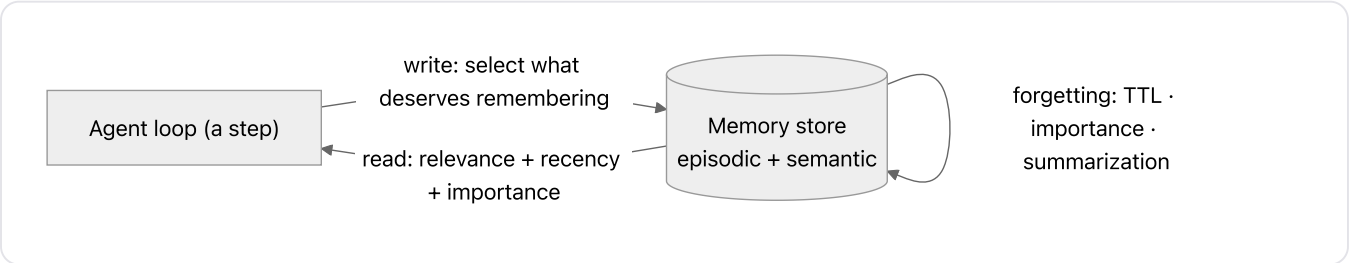
The notion connects to agent memory (Chapter 13). If the context window is the state available on the current step, then **memory is the state outside the window**, from which what's needed is brought back into the window. Three kinds, following the CoALA taxonomy (Sumers et al., 2023; its fourth kind, working memory, is the context window itself, §4.2):

- **Episodic**—the history of interactions, usually in a vector database; for the current query you retrieve a *relevant slice* of the past (retrieval over your own history), not the entire history.
- **Semantic**—durable facts and knowledge (a customer profile, learned preferences), often in a graph or relational database.
- **Procedural**—learned *ways of acting*: debugged step sequences, tool combinations that work; stored as reusable instructions or skills and pulled in by the agent loop (§13.1). The least intuitive category—and often the biggest lever: an agent that remembers "how to solve this class of task" is cheaper than an agent that rediscovers it every time.

The key idea: what comes back into the context from memory is constructed. The "long-term memory" of a stateless component is the pattern "store outside the window → retrieve what's relevant back into the window"—the same retrieval as in RAG, applied to session or user state.

Deep Dive: memory architecture

"Retrieve what's relevant from memory" requires a designed subsystem. The components and the data paths:



- **Memory store.** Storage outside the window: a vector database for episodic memory (semantic retrieval), a relational or graph database for semantic memory (profile, facts, relationships). Both are covered by, for example, pgvector° on Postgres.
- **The write path.** Not everything gets written. The policy: what counts as "worth remembering" (important facts, decisions, preferences), and in what form (the raw turn vs. an extracted fact vs. an embedding). A separate step—"extract the facts worth remembering"—is often placed on the write path: the same idea as the structured notes above.
- **The read path.** On every step, retrieve a *relevant slice* of memory for the current context and return it to the window. Ranking is not by semantic proximity alone: in systems like generative agents (Park et al., 2023) the score combines **relevance + recency + importance**—the recent and the important surface even at equal relevance. That's richer than naive top-k by cosine.
- **Consolidation / reflection.** Periodically the system synthesizes higher-level memories out of low-level ones ("the user prefers short answers," from a dozen observations)—reflection in generative agents. The analogue is a materialized view, or a rollup over raw events.

- **A forgetting policy.** Memory without eviction degrades—it grows, adds noise, and costs more. The policies: TTL, deleting entries by low importance or age, summarizing the old into compact form. The eviction criterion here is semantic (an entry's importance and relevance), not only its age or access frequency.

The ranking formula. A memory's score for the current query is the sum of three normalized terms:

$$score = \alpha_{rec} \cdot recency + \alpha_{imp} \cdot importance + \alpha_{rel} \cdot relevance$$

- ***recency***—freshness: $0.995^{\Delta t}$, where Δt is the number of hours since the **last access** to the memory, not since it was created. The difference is fundamental: retrieving a memory thereby "refreshes" it, and on the next step it ranks higher—reinforcement of the same nature as LRU in a cache. The half-life is $\ln(0.5)/\ln(0.995) \approx 138$ hours, about 5.8 days.
- ***importance***—an integer score of 1–10 that the model itself assigns at write time, in answer to a direct question, "how significant is this" (in the original, a scale from "brushed teeth" = 1 to "a breakup" = 10).
- ***relevance***—the cosine between the query embedding and the memory embedding: the one thing naive top-k confines itself to.
- ***α***—the term weights; in the paper all three equal 1. Each term is first mapped into [0,1] by min-max scaling over the candidate pool, which makes the score relative to that pool: it's a ranking quantity, not a threshold comparable across queries.

A live calculation. Two memories at *equal* relevance of 0.6 (the values are given before min-max: with a large pool it shifts them only slightly and does not change the conclusion). A—"the user's payment failed": importance 8/10 → 0.8, last accessed 72 hours ago → $0.995^{72} \approx 0.70$; score = 0.70 + 0.8 + 0.6 = **2.10**. B—"the user said hello": importance 2/10 → 0.2, accessed an hour ago → $0.995^{1} = 0.995$; score = 0.995 + 0.2 + 0.6 ≈ **1.80**. The important-but-cooled beat the fresh-but-empty, even though their cosines are identical and top-k would not have told them apart at all.

In plain terms: the formula lifts to the top not "what resembles the query" but "what resembles it—and either surfaced recently or was judged significant to begin with." Three independent axes, each able to pull an entry into the window on its own.

The engineer's iron rule: the decay constant is a configuration parameter tied to your product's real cadence, not a number from a paper. The 0.995 per hour was fit to a simulation with a densely packed in-game day. For an assistant used once a week, a two-week-old fact yields $0.995^{336} \approx 0.19$—effectively evicted, even though it might be the customer profile. Hence the second consequence: it is **episodic** memory that should decay; semantic memory (durable profile facts) is excluded from the recency term entirely—otherwise the system "forgets" a customer's enterprise plan just because they haven't logged in for a month.

The paper vs. the code. In the published reference implementation the weights are not equal to one: the code multiplies the terms by `gw = [0.5, 3, 2]` (recency, relevance, importance)—relevance weighs six times more than recency—and the authors themselves admit in a comment that the weights ought to be learned rather than hand-picked. In the same code, recency is computed not from elapsed hours but from an entry's ordinal position in a list sorted by last access. The conclusion for your project: those three weights and the decay constant are yours to tune and measure on your own traffic (Chapter 17), not to inherit from a 2023 paper.

The **MemGPT** framework (Packer et al., 2023) organizes memory into two tiers—"main context" in the window and "external context" in storage—and the model manages moving data between them through tools (pulling what's relevant into the window, evicting what's stale); the authors draw a direct analogy to an operating system's memory hierarchy.

Sources. C. Packer et al., "MemGPT: Towards LLMs as Operating Systems" (2023); J. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior" (2023)—memory stream, recency/importance/relevance, reflection; T. Sumers et al., "Cognitive Architectures for Language Agents (CoALA)" (2023)—a taxonomy of the kinds of memory. The generative-agents reference implementation (github.com/joonspk-research/generative_agents, `persona/cognitive_modules/retrieve.py`)—the actual `gw` weights and the recency computation.

4.6 Context boundaries and trust (a bridge to security)

Back to the fact from §1.4: the model does not separate instructions from data in a single channel. Inside the assembled context, **untrusted content**—retrieved documents, tool output, user data—must be *structurally isolated* (with delimiters or tags) and never treated as an instruction. Context engineering is **where defense against prompt injection begins**: correct markup and isolation of untrusted blocks. The defense is fully implemented at the guardrails level (Chapter 19), but it is laid down here, at context assembly. The invariant: data from the outside world goes into an isolated (quarantine) block—say, `<untrusted_data>…</untrusted_data>`, with the model explicitly told that the block's contents are data, not instructions—and instructions arrive only from a trusted source. The trained instruction hierarchy (§3.2) works the same problem from the model's side; context assembly works it from the system's side, and it is context assembly you should treat as load-bearing.

4.7 Chapter summary

Context engineering is state management for a stateless probabilistic component under a token budget. It sits between designing the prompt (Chapter 3) and RAG (Part III): **the prompt states the requirement, RAG supplies the knowledge, and context engineering decides what exactly lands in the window on each step, and in what form**—relevant, within budget, correctly placed and isolated. In essence it is the discipline of state management, applied to a probabilistic component.

Sources. Anthropic, engineering material on context engineering and effective use of context (2024–2025); N. Liu et al., "Lost in the Middle" (2023); Chip Huyen, *AI Engineering* (2025); the RAG literature (Part III).

In the Margins: Chapter 4

> ↩ **K and V**—the Key/Value tensors of the attention mechanism, computed for every token of the context (§1.4). With an unchanged prefix they do not change—the provider stores them and skips the prefix's prefill entirely. → §1.6.
>
> ↩ **pgvector**—a PostgreSQL extension: a vector type, proximity operators, and ANN indexes (HNSW, IVFFlat) in an ordinary table. → §8.5.

Chapter 5. Structured Output and Reliable Parsing

5.1 The problem: the boundary between probabilistic and deterministic

The model emits free-form text. Downstream components—a parser, a tool call, a database write, UI rendering—demand **typed, valid data**. The boundary between these two worlds is exactly where most agentic systems break in production.

The naive approach—"ask for JSON and parse it"—does not survive production, for concrete reasons: the model wraps the JSON in prose ("Here's the answer: …"), fences it in markdown (```json), leaves trailing commas, invents or omits fields, puts in the wrong type (a string where a number belongs), truncates output at the token limit (§1.8). Every one of these cases is an unhandled exception in the parser—or worse, silently wrong data flowing further downstream.

At its core, this is contract enforcement at a trust boundary: the output comes from a probabilistic component that *can* violate the required shape. Structured output is the set of techniques that brings model output into conformance with a schema you can rely on. It matters because structured output is the junction where the nondeterministic core (§1.10) meets deterministic code; the tighter the junction, the fewer production incidents.

5.2 The spectrum of techniques: from weak to guaranteed

The techniques line up in order of increasing guarantee (and, as a rule, cost/complexity):

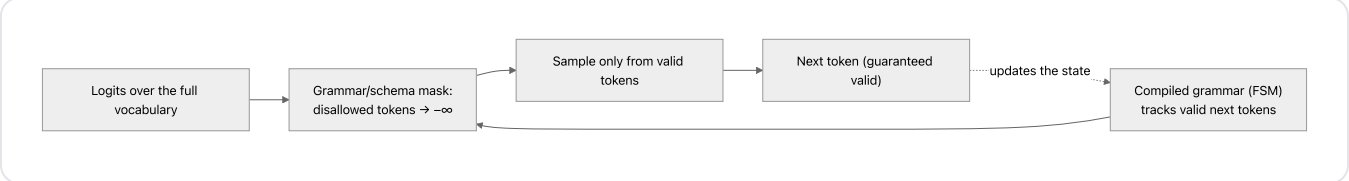
Level	Technique	What it guarantees	Cost / limitation
1	**Prompt only** (“return JSON matching this schema…”)	Nothing (it’s a request)	Prose, fences, invalid JSON
2	**Few-shot format examples**	Nothing, but noticeably more reliable	Tokens spent on examples
3	**Post-hoc parsing + repair (reask)**	Validity after retries	Extra model calls, latency
4	**JSON mode** (provider-side)	Syntactically valid JSON	Valid JSON ≠ *the right schema*
5	**Constrained / grammar decoding** (schema-guided)	Conformance to the *schema* at generation time	Requires logit access; small overhead
6	**Function / tool calling**	A structured call against a declared schema	Specialized for invoking functions (Chapter 12)

The practical takeaway: do not rely on levels 1–2 anywhere the output feeds a parser or a tool. Level 3 (validation + reask) is the minimum, and the best available option is level 5 (constrained decoding) or level 6 (tool calling) for actions.

5.3 How constrained decoding works (in depth)

This is the single most powerful technique of the lot, and its mechanics are worth understanding because they are a direct generalization of §1.7.

Recall: at every decoding step the model produces **logits**—a score for each token in the vocabulary, which softmax turns into a distribution (§1.7). Constrained (grammar-guided) decoding inserts one step between the logits and the sampling: a **compiled grammar** (a finite-state machine / pushdown automaton° for JSON plus your schema) tracks which tokens are *permitted* next given what has been generated so far, and **masks the logits of every disallowed token to −∞** before the choice is made. After the mask, the model *physically cannot* pick a token that breaks the structure.



This is the same “mask the invalid logits” principle from §1.7, scaled up to a full grammar: not “forbid one character,” but “at every step, allow only what leads to a schema-valid document.” The essential subtleties:

- **You need access to the logits.** That is why constrained decoding is available out of the box on self-hosted stacks (Outlines, GBNF grammars in llama.cpp, XGrammar in vLLM)° or from providers who implement it themselves (structured outputs with a JSON schema). With a pure black box that lacks the feature, it is simply unavailable—you fall back to level 3.
- **The overhead is small**; modern implementations compile the grammar efficiently.

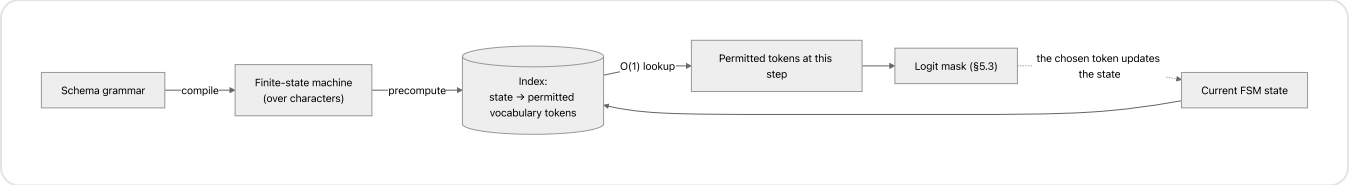
- **It guarantees structure, not semantics** (see §5.6)—the key limitation.

Deep Dive: token alignment and FSM indexing

The mechanics of §5.3 (grammar-driven logit masking) hide a nontrivial engineering problem that matters when choosing a constrained-decoding engine for self-hosting.

The token alignment problem. A grammar or regex defines valid *character strings*. But the model generates *tokens*—subword units, each covering *several* characters and potentially straddling grammar boundaries. A single valid next character can begin hundreds of different tokens; a single token can "leap across" several grammar characters at once. So the naive "allow the valid characters" doesn't work—at every step you need to know *which vocabulary tokens* keep the generation on a valid path through the automaton. That is the alignment between the character-level grammar and the token vocabulary.

The Outlines contribution (Willard & Louf, 2023)—indexing the FSM against the vocabulary. The naive solution is to scan the entire vocabulary (tens to hundreds of thousands of tokens) at every step, checking each token for compatibility with the grammar. Expensive on *every* generated token. The Outlines idea: compile the grammar into a finite-state machine (FSM) and **precompute an index** mapping each FSM state to its set of permitted tokens. Masking at each step then becomes an O(1) lookup—"permitted tokens for the current state"—rather than a vocabulary scan. This is what made grammar-guided decoding practical speed-wise (later engines—XGrammar and others—build on the same idea with JSON-specific optimizations).



Token healing. An adjacent subtlety: greedy tokenization can "break the boundary" at the seam between the fixed prefix and the generation. Example: if the context ends with `"url": "htt`, the model would naturally continue with the token `https`, but the boundary has already cut off `htt` as a separate token—the optimal continuation is unreachable. **Token healing** rolls back the last token(s) of the prefix and regenerates so that the tokenization matches the "natural" one. The technique was popularized by Microsoft Guidance; it matters for the correctness of constrained/template-driven generation at boundaries.

The limits of grammars. Not everything can be constrained cheaply. A flat object with enums compiles into a compact automaton. But unbounded recursion (arbitrarily nested JSON with no depth limit) requires a pushdown automaton (a context-free grammar), not a finite one, and indexing it efficiently is harder; a deep or recursive schema bloats the automaton. One more reason to keep the schema flat and bounded (§5.4): not just for the model's quality, but for the cost of the constraint engine itself.

Deep Dive: when constrained decoding hurts

Guaranteeing the structure is not free where *quality* is concerned:

- **An over-complicated schema strangles the model.** Constrained decoding forces the model to emit only tokens that lead to a valid structure. If the schema is rigid and "unnatural" for the task, the model's distribution gets herded into a narrow corridor, forcing it to pick tokens that would not otherwise have been picked—and that can degrade the *content* (filler hallucinations in required fields). A simple schema with enums is fine; a baroque schema with a dozen required nested fields is a way to get a valid but bad answer.

- **Why a rigid schema imposed early gets in the way of reasoning.** Force the final schema *from the very first token* and the model loses its "room to think" (§3.4: CoT works by spilling the reasoning out into tokens). It has to emit the answer fields immediately, without having reasoned. Hence two remedies: two-step (§5.5) and a reasoning field inside the schema (§5.5).

Source: Z. Tam et al., "Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of LLMs" (EMNLP 2024)—empirical evidence for reasoning degrading under format restrictions.

5.4 Schema design as contract design

Design the output schema as a contract: reliability depends on it directly.

- **Keep the schema simple and, where possible, flat.** Deeply nested structures come harder to the model; every level of nesting raises the probability of an error. The same "don't complicate the contract without need" principle.
- **Enums for categorical fields.** If a field draws from a fixed set of values (status, category), declare it as an enumeration. The model is then obliged to choose from the set—sharply more reliable than free text, and it removes the entire class of "the model wrote the status slightly differently." One of the cheapest and most powerful techniques there is.
- **Required/optional fields and default values**—explicitly, as in a contract.
- **Field descriptions in the schema act as inline instructions.** A good JSON schema with a `description` on each field is both a specification for the validator and documentation that *the model reads*. A dual-purpose schema.
- **Be careful with types.** The model garbles numbers because of tokenization (§1.2)—avoid critical arithmetic in the output where you can; for dates, state the format explicitly (ISO 8601), or you will get a mishmash.
- **Always validate post-hoc with a real validator** (JSON Schema, Pydantic; on the JVM—Jackson + Bean Validation or a schema library)—even under constrained decoding, as defense-in-depth. The grammar guarantees the structure, but the validator catches edge cases and serves as a second line.

Vendor schema limits (check them against the documentation as of your implementation date). Provider-side structured outputs constrain the schema not only by common sense but by hard implementation limits that shift from release to release: for OpenAI, the 2024–2025 ballpark is on the order of a hundred properties per schema, a handful of nesting levels, and caps on the total size of names and enum values; support for JSON Schema's composite features (composition, recursion) is limited and differs between providers. More consequential than the limits is another gotcha: **the value-validation keywords—`pattern`, `minLength`, `minimum`, `format`—may be accepted in the schema by the provider yet not enforced during generation**; checking them falls entirely to the post-hoc validator from the bullet above—not knowing this costs weeks of debugging "the schema is right there, where are the invalid values coming from?" Gemini has its own set of restrictions: an overly complex schema is rejected with an `InvalidArgument` error, and an enum comes back as a string. The rule: verify the semantics of every keyword and the limits against the provider's documentation—not against the JSON Schema spec, and not from memory.

5.5 Reliability patterns

- **Constrained Decoding**—a structural guarantee at generation time (§5.3); the strongest single technique when you have logit access.

- **Two-step generate-then-format**—first let the model reason freely (CoT, §3.4), then format the result into the schema with a *separate* constrained call. It pulls reasoning and formatting apart: a rigid schema imposed too early gets in the way of reasoning, and a late formatting step relieves that tension. More expensive (two calls), but better on hard tasks.
- **Validate-then-Retry**—the baseline pattern when constrained decoding is unavailable:

> ### Pattern: Validate-then-Retry (Reask)
>
> **Context.** The model's output feeds a parser or a tool, but constrained decoding is unavailable (a black box with no schema feature). **Problem.** How do you get schema-valid output without a guarantee at generation time? **Forces.** Parsing it straight breaks on invalid output; accepting invalid output breaks downstream; retrying forever costs money and latency. **Solution.** Generate → validate against the schema → on failure, **hand the model the validator's error text and ask it to fix the output** (reask), with a **cap on the number of attempts**. Given a concrete error message, the model usually repairs its own output. **Mechanics.** `generate → validate → (invalid?) → append(error) → regenerate`, with an attempt counter (typically 1–3), then a fallback (deterministic handling or escalation). Idempotency matters if the downstream action has already partly happened. **Example.** A number of frameworks (Spring AI and LangChain4j, for instance) build the pattern in: the typed converter runs validation and the reask internally. **Resulting Context.** Reliability without logit access; the price is extra calls (cost/latency) and the absence of a *guarantee* (after N attempts it can still fail → you need a fallback). **When to Use.** A black box with no constrained mode. **When Not To.** When constrained decoding or tool calling is available—they give the guarantee more cheaply than reasks do. **Related Patterns.** Constrained Decoding (a guarantee instead of retries); Fallback (what to do once the attempts run out, §2.5); Two-step generate-then-format. **Sources.** B. Willard, R. Louf, "Efficient Guided Generation for Large Language Models" (2023, Outlines); provider documentation on structured outputs and function calling.

Technique: the reasoning field first (the CoT ↔ structured output bridge)

A cheap alternative to two-step when you need a single call with a guaranteed schema: **move the reasoning inside the schema itself, as the first field.**

```
{
  "reasoning": "string—a step-by-step analysis before the answer",
  "category": "enum",
  "answer": "…"
}
```

Because JSON fields are generated in order, the model fills `reasoning` first (getting its "room to think," §3.4) and only then the structured answer, now leaning on its own reasoning. This embeds CoT directly into structured output in a single call. Caveats: the `reasoning` field burns tokens (cost/latency) and is usually discarded after parsing; field order matters (reasoning goes *before* the answer fields, or there is no point).

The tension: streaming ↔ structured output

A real production problem people often forget. Streaming (§2.3) hands over tokens as they are generated, for the sake of perceived latency. But **partial JSON is invalid**: until the closing `}` arrives, you cannot parse or validate it, and you cannot render it "halfway." The conflict: either streaming (good latency, but nothing to validate or use

until the end) or structured output (validatable, but the user waits for the full answer).

Practical resolutions: (1) stream only free-form text *to the human reader*, and generate the structured part with a separate non-streaming call (see two-step); (2) use **incremental / partial-JSON parsers** that close off unfinished structures on the fly (off-the-shelf libraries exist)—for progressive rendering; (3) stream *by field* (some APIs deliver structured output in fragments). The choice depends on whether the output is for a human (streaming matters) or for code (validity matters more).

On the empirical evidence (honestly)

How much any given technique lifts reliability depends on the model, the task, and the schema; there is no "universal percentage." The order of the effect, reproducible in reports and in practice: moving from "prompt only" to validate-with-reask removes most *parsing* errors; constrained decoding / JSON-schema mode removes them *entirely* by construction (§5.3) as far as structure goes; enums and a flat schema noticeably cut *value* errors in categorical fields. But no technique touches factual correctness (§5.6)—that is a separate axis, measured by evals (Part VI). The moral: measure format reliability and content correctness *separately*, with different metrics.

5.6 The semantic gap: structure ≠ correctness

The critical caveat you cannot skip. Constrained decoding guarantees that the JSON is **well-formed and conforms to the schema**—but the *field values* can still be a hallucination. The model will hand you a flawlessly valid `{"order_status": "SHIPPED", "tracking": "…"}` in which the status is simply invented.

Hence a clean split of responsibility:

- **Structured output solves the parsing/integration problem**—"the output plugs into my code."
- It **does not solve the accuracy problem**—"the output is *right*."

Accuracy is the job of grounding (RAG, Part III) and evaluation (evals, Part VI). Never confuse "it parsed" with "it's correct." This is one of the classic traps: a team adds structured output, sees valid JSON, relaxes—and ships confidently wrong data in perfect packaging. Structured output closes the *format* boundary, not the *truth* boundary.

5.7 Implementation on the JVM

The JVM ecosystem gives you typed output with almost no extra effort:

- **Spring AI:** structured-output converters (`ChatClient….entity(MyRecord.class)`, `BeanOutputConverter`) generate the schema from the Java type, put it in the prompt, and parse the response back into a record or POJO. Where the model supports schema-guided output, that is what gets used. For actions—`@Tool` (tool calling, Chapter 12).
- **LangChain4j:** `AiServices` with a typed return value—you declare a Java interface that returns a POJO, and the framework takes on the schema, the parsing, and the reask.
- **Validation:** Jackson for parsing and Bean Validation (`@NotNull`, `@Pattern`, …) as the post-hoc line (§5.4).

At the code level this comes down to “declare the return type, get a typed result,” and Validate-then-Retry is often already implemented inside the converter. The §5.6 caveat still holds: type-safe output does not mean correct output.

5.8 Summary—the boundary is closed

Structured output is how you close the boundary between a probabilistic model and the deterministic consumption of its output. The spectrum runs from asking for JSON (weak) to constrained decoding (a structural guarantee) and tool calling (a guarantee for actions). But structure does not equal correctness: the format is guaranteed, the truth is not.

That closes Part II—the interface to the model: **designing the request sets the requirement** (Chapter 3), **context engineering assembles the input** (Chapter 4), **structured output closes the output boundary** (Chapter 5). What remains unsolved is the correctness of the *content*, which format does not provide. Part III is devoted to it: RAG—grounding the model’s output in verifiable facts.

Sources. B. Willard, R. Louf, “Efficient Guided Generation for Large Language Models” (2023)—the FSM approach to constrained decoding (Outlines). OpenAI and Anthropic documentation on structured outputs and function calling; GBNF grammars in llama.cpp; XGrammar (2024) for efficient grammar-guided decoding. Spring AI and LangChain4j documentation on structured output.

In the Margins: Chapter 5

> ↩ **Pushdown automaton**—a finite-state machine with a stack. The stack is the reason it turns up here: JSON’s nesting (an object inside an array inside an object) requires remembering the depth, which an automaton without a stack cannot do.
>
> ↩ What’s what here: **llama.cpp**—a local inference runtime; **GBNF**—its grammar format (GGML BNF); **Outlines** and **XGrammar**—constrained-decoding libraries with a precomputed vocabulary index; **vLLM**—an inference server that integrates them. → Chapter 22.

Part III. Grounding Knowledge: RAG

Part II tamed the interface. But structured output guarantees *format*, not *truth* (§5.6), and the model's parametric knowledge is static, unverifiable, and prone to hallucination (§1.9)—it also holds none of the organization's private data and knows nothing of access control. Part III is about grounding the answer in data that is retrievable, verifiable, and access-controlled. Four chapters: framing RAG (6), a catalog of retrieval patterns (7), vector indexes and infrastructure (8), and evaluating RAG quality (9).

Chapter 6. Retrieval-Augmented Generation: The Framing

6.1 The problem RAG solves

Back to §1.9. The model's parametric knowledge is baked in at training time, which makes it **static** (it stops at the cutoff date), **unverifiable** (no source—there is no way to check where a "fact" came from), and **prone to hallucination** (the model confidently produces plausible falsehood). On top of that it **holds no private data** belonging to the organization (none of it was in the training set) and **takes no account of access control** (the model has no idea that a particular user is not allowed to see a particular document).

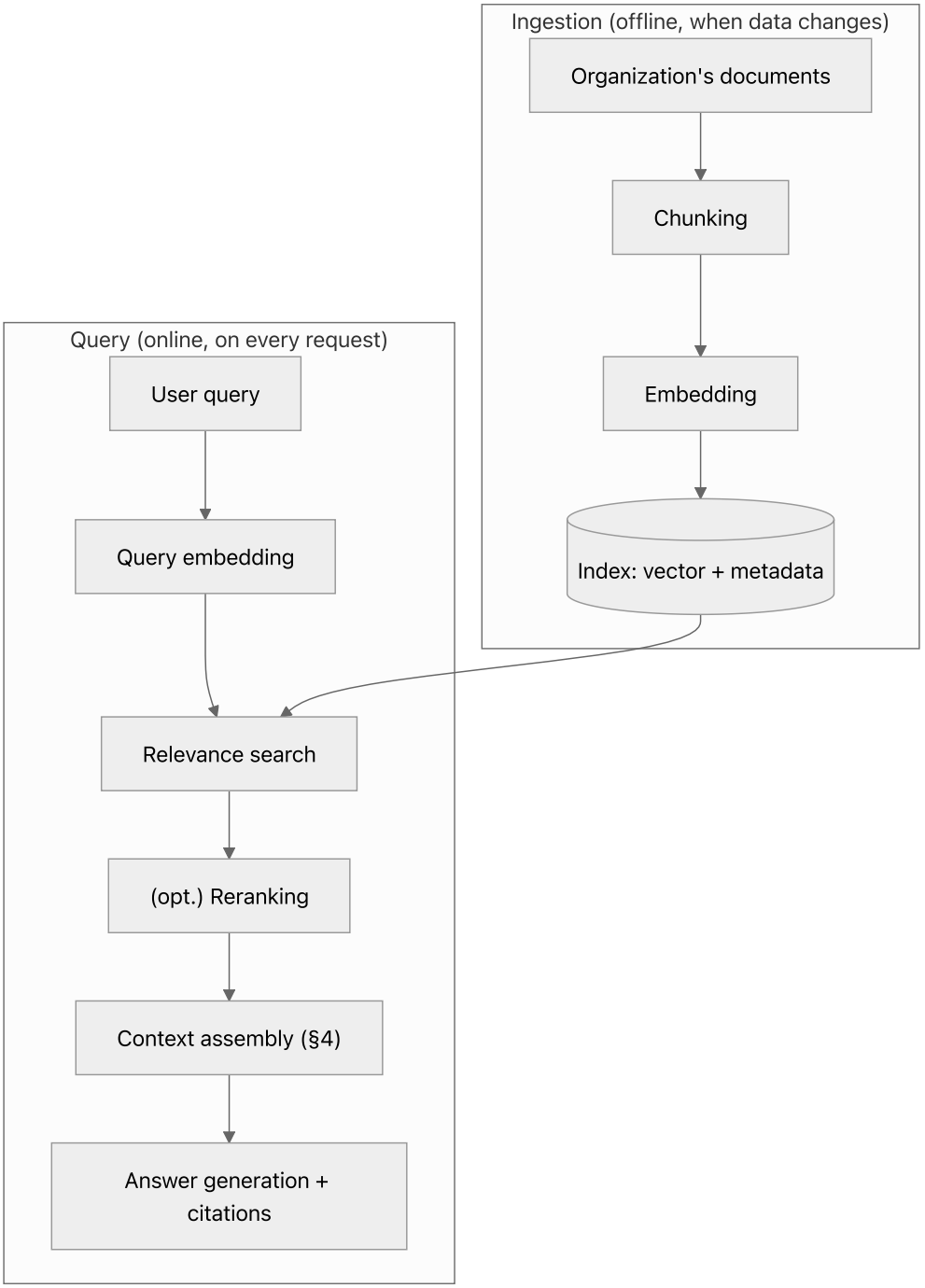
The architectural question: how do you make a probabilistic component answer from data that is *fresh*, *private*, *verifiable*, and *access-controlled*? **RAG is the architectural answer.** It does not improve the model's memory—it replaces reliance on memory with reliance on retrieved data.

6.2 What RAG is

Retrieval-Augmented Generation (RAG)—a pattern in which *retrieved* relevant data is fed into the model's context at inference time, and the answer is formed from that data rather than (only) from parametric memory. Structurally it is two phases:

- **Retrieve**—find the data relevant to the query.
- **Generate**—generate an answer conditioned on that data.

And two pipelines, offline and online:



Notice that retrieval is exactly what Part II called *context selection* (§4.4). RAG is the concrete, dominant implementation of context engineering for knowledge.

One qualification the diagram needs: the vectors live only inside the search block. What reaches generation is the retrieved *text*—the model re-cuts it into tokens and builds its own internal vectors; it never sees the RAG vectors at all (§1.3).

6.3 RAG versus the alternatives—the architect's key decision

Three ways to give the model knowledge or behavior it lacks by default—one of the main design forks:

Axis	RAG	Fine-tuning (Chapter 21)	Long context
Data freshness	Update the index and you're done	Requires retraining	Per request
Volume of knowledge	Huge (the entire corpus)	Limited, "blurs"	Only what fits the window
Verifiability / citations	Yes (there is a source)	No	Yes (the data is in the prompt)
Access control	Yes (RBAC° at retrieval)	No (the knowledge sits in the weights, for everyone)	On the assembly side
Cost	Index + retrieval per request	Expensive training, cheap inference	An expensive long prompt
What it's for	*Knowledge* (facts, documents)	*Form* (style, schema, tone)	Small / one-off

The rule (the same ladder `Prompt → RAG → Fine-tune`): for *knowledge*, almost always RAG; fine-tuning is for *form*, not facts (§1.9, Chapter 21); long context is for a small one-off injection, not for a knowledge base (§1.8: dilution and cost). The two are often combined: RAG for knowledge plus a small fine-tuned model for the form of tool calls.

6.4 Anatomy of a RAG system

Every component is a failure surface, and it pays to see them separately:

1. **Ingestion pipeline**—load → chunking° → embedding → indexing (+ metadata for filtering/RBAC).
2. **Retrieval**—embed the query → search → (opt.) reranking.
3. **Augmentation**—assemble what was retrieved into the context (placement, isolation, §4).
4. **Generation**—an answer conditioned on the context, ideally with citations.

Most RAG problems in production are a *retrieval* failure (components 1–2), not a generation failure—hence §6.5–6.6.

6.5 Retrieval is the bottleneck (the key shift)

Naive RAG misses at retrieval, by widely circulated industry estimates, in **on the order of 30–40%** of cases in production (the figure floats and traces back to no single primary source—all the more reason to measure it on your own data, Chapter 9), and it misses **silently**: the model confidently generates an answer from the *wrong* retrieved fragments, and the output is indistinguishable from a correct one. The key shift: **the bottleneck is not generation but retrieval.** Modern models generate well; what breaks is *what* gets handed to them. The practical consequence: retrieval quality is treated as a measurable SLO (Chapter 9), and improving it (Chapter 7) is the main lever on the quality of a RAG system.

6.6 Deep Dive: RAG failure taxonomy

Let's break "RAG doesn't work" down into concrete failure modes:

Failure mode	What happens	Mitigation
Recall miss	The relevant document is not retrieved	Hybrid search, query rewriting (Chapter 7)
Precision miss	Irrelevant noise gets retrieved	Reranking, filters (Chapter 7)
Bad ranking	What you need is there, but not in the top-k	A reranker (Chapter 7)
Silently wrong context	The model confidently answers from the *wrong* fragments	The most dangerous one; surfaced only by evaluation (Chapter 9) and citations (§6.7)
Unfaithfulness	The model ignores or distorts the context, or hallucinates beyond it	A separate axis (§6.7); the faithfulness metric (Chapter 9)
Chunk fragmentation	The fact you need is cut in two by a chunk boundary; the chunk makes no sense out of context	Chunking strategies, contextual retrieval (Chapter 7)
Staleness	The index has fallen behind the source	A reindexing pipeline
Security	Something the user may not see gets retrieved (an access-control breach at retrieval); index poisoning	RBAC/ABAC at retrieval (Chapters 8, 19)
Indirect injection	A legitimately accessible but maliciously crafted document carries an instruction that hijacks the model/agent	Isolating retrieved content as data (§4.6); taint-tracking and the rule of two (§12.5); Chapter 19

The key point: "improve RAG" is not one action but targeted work on a specific *measured* failure mode (Chapter 9). Without measurement you are fixing blind.

6.7 Deep Dive: grounding and faithfulness—two orthogonal axes

The critical distinction, and the one most often confused. The quality of a RAG system is **two independent axes**, and they have to be measured *separately*:

- **Retrieval quality**—did we pull the right documents? (context recall—did we miss what we needed; context precision—did we drag in noise).
- **Faithfulness / groundedness**—did the answer use what was retrieved *correctly*, without inventing beyond it?

They are orthogonal, and all four combinations are real:

- Good retrieval + faithful → the ideal.

- Good retrieval + not faithful → we pulled the right thing and the model hallucinated anyway (a generation problem).
- Bad retrieval + faithful → the model *honestly* restated the *wrong* chunks (a retrieval problem, masked by a confident tone).
- Bad retrieval + not faithful → everything is broken.

Two conclusions follow. First: you cannot fix "RAG" in general—you have to know *which axis* sagged (Chapter 9 measures both separately). Second, a design one: **citations as a design goal.** Make the model name *which chunk* backs each claim (through structured output, Chapter 5: a `citations` field with source ids). That buys three things at once—verifiability (the user can check), fewer hallucinations (the model is "pinned" to a source), and faithfulness debugging (you can see where a claim came from). Citability is not decoration but an architectural requirement for trusting RAG.

Deep Dive: knowledge conflict (parametric vs. retrieved)

RAG assumes the model will prefer the retrieved context to its own memory. In practice that is not guaranteed—what arises is a **knowledge conflict**: what does the model do when the context *contradicts* its parametric prior?

Two symmetric failures:

- **The model ignores the context in favor of the prior.** If a retrieved fact contradicts something firmly fixed in the weights (common knowledge), the model may ignore the context and answer from memory—with a fact that is stale, or wrong for this particular organization. The stronger the prior and the more "surprising" the context, the higher the risk. This bears directly on the case where an organization's internal policy diverges from the generally accepted one.
- **The model blindly trusts wrong context.** The reverse: a wrong, stale, or poisoned fragment gets retrieved—and the model faithfully reproduces the error. Faithfulness, in this case, works against correctness.

Design measures: **prioritize the context explicitly** in the instruction ("answer *only* from the documents; if they are not enough, say 'I don't know'"); **citations** (above) make a fallback onto memory visible (a claim without a source = a flag); **retrieval freshness/quality/security** (Chapters 7–8) closes half the problem (wrong context); **admitting uncertainty**—the system is obliged to say "that is not in the documents" rather than filling the gap with the prior (checked with a golden set containing unanswerable questions, Chapter 9).

The architectural conclusion: RAG does not switch parametric knowledge off, it competes with it. You design the system so that the context wins the conflict, and you measure that.

Source. S. Longpre et al., "Entity-Based Knowledge Conflicts in Question Answering" (2021); context-faithfulness research (2023–2024).

Deep Dive: RAG versus long context—do 1M+ windows make RAG itself obsolete?

A live argument: if windows are growing to 1M+, why retrieve at all—"just stuff the whole corpus into the context"? The answer: long context *changes* RAG's design but does not abolish it:

- **Cost (§2.2).** Feeding 1M tokens on *every* request is linearly expensive, permanently; retrieving the relevant 5k is orders of magnitude cheaper. Under a stream of requests this is the decisive argument.
- **Latency (§1.6, §2.3).** A long prefill hits TTFT; a short retrieved context is faster.
- **Accuracy (§1.8).** Dilution and lost-in-the-middle: quality *drops* as irrelevant content grows. Less but more relevant is often *more accurate*, not just cheaper.

- **Scale.** An organization's corpus (terabytes) does not fit any window at all—retrieval is mandatory.
- **Access and freshness.** RBAC at retrieval (Chapter 8) and index updates without retraining—properties a long window does not give you on its own.
- **"What about prompt caching?"** A prefix cache (§2.2, §20.2) removes part of the cost of resending the corpus, but the argument does not save the day: the cache is invalidated on every corpus update; every user has their own RBAC slice—there is no common cacheable prefix to assemble; the TTFT of the uncached tail remains; and the quality degradation from dilution (§1.8) a cache does not cure at all.

What long context *does* change: you can afford **larger chunks** and a **less aggressive top-k** (20 rich fragments instead of 5 narrow ones), suffering less from fragmentation (§7.6). A long window, in other words, is *extra budget* that makes the RAG pipeline gentler, not unnecessary. Empirically (Xu et al., 2023) RAG and long context are *complementary*: RAG gives comparable quality more cheaply, and the combination is best.

The conclusion: the thesis "the window is big → RAG is unnecessary" does not survive scrutiny. Economics, accuracy, scale, and access control keep RAG mandatory; a long window merely tunes its parameters.

Source. P. Xu et al., "Retrieval meets Long Context Large Language Models" (2023).

6.8 Implementing the retrieval layer (a bridge to Chapter 8)

The retrieval layer is an additional index over the data. At the start you can build it without a dedicated infrastructure component: **pgvector in Postgres** for vector search; hybrid search as a combination of pgvector and Postgres full-text search; a cache for embeddings and answers (including semantic caching, §2.2)—in Redis, for example. Fragment metadata (owner, clearance, date) in a relational database gives you filtering and access control (RBAC/ABAC) at retrieval time, by way of the access-management system. The infrastructure details (ANN, HNSW, parameters, the conditions for moving out to a specialized store like Qdrant or Weaviate) are Chapter 8.

The opposite end of the build-vs-buy scale is **managed retrieval**: provider offerings that hide ingestion, chunking, embeddings, and the index behind a single API (Amazon Bedrock Knowledge Bases, Vertex AI Search, OpenAI vector stores / file search, Azure AI Search). You start faster and without infrastructure of your own; the price is limited control over exactly the levers Chapter 7 is made of (chunking, hybrid, reranking), opacity about retrieval quality, and your data tied to the provider; the access-control model (§8.7) has to be expressible by the provider's means. The working rule: managed for a fast start and ordinary corpora; your own layer when retrieval quality becomes a product differentiator or the data demands it (privacy, specific filtering).

6.9 Chapter summary

RAG grounds the probabilistic component in retrievable, verifiable, access-controlled data instead of leaning on static parametric memory. The bottleneck is retrieval, not generation (§6.5). Quality is two orthogonal axes—retrieval quality and faithfulness (§6.7)—measured separately (Chapter 9). Next: **Chapter 7—the patterns that improve retrieval; Chapter 8—the infrastructure; Chapter 9—the measurement.**

Sources. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (2020)—the original RAG paper. Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey" (2023)—a map of the field. Chip Huyen, *AI Engineering* (2025)—the engineering frame.

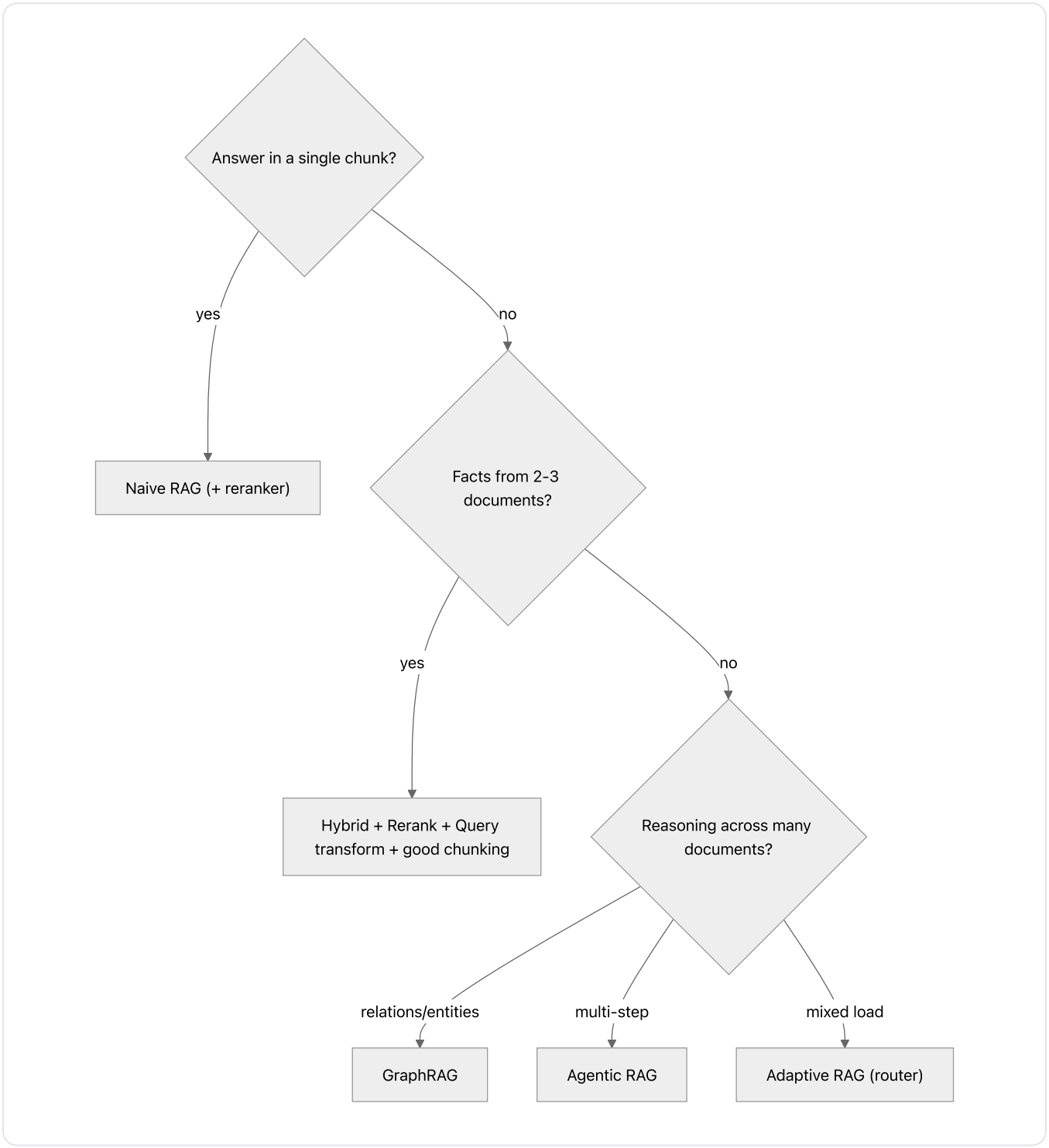
In the Margins: Chapter 6

> ↩ **RBAC / ABAC**—Role-/Attribute-Based Access Control: access by role versus access by attribute (department, clearance, project). In RAG it is checked at *retrieval*, not at generation. → §8.7.
>
> ↩ **Chunking**—splitting documents into fragments for indexing; fragment size and boundaries are a design decision in their own right. → §7.6; ⇐ Glossary.

Chapter 7. A Catalog of Retrieval Patterns

7.1 The retrieval pattern ladder

The patterns are ordered by increasing complexity; the principle running through all of them is to climb exactly as far as measurement (Chapter 9) says you must, and not to pick the complicated option up front.



The practical rule: **the default is Hybrid + Reranker + competent chunking.** That closes off most of the problems. GraphRAG, Agentic, and multi-query are the top of the ladder, and you climb up there on proven need—otherwise you buy 10× the cost with no gain in accuracy.

7.2 Pattern: Naive RAG (the baseline)

Context. Simple question-answering over a corpus; the answer usually sits in one fragment. **Solution.** One pass: embed the query → dense top-k search → feed it into the context → answer. **When to Use.** The answer is localized in a single chunk, the corpus is small, accuracy requirements are moderate. **When Not To.** Exact terms and codes (dense search loses them—§7.3), multi-document or multi-step questions. **Role.** This is the *baseline* against which every improvement is measured (Chapter 9). You start here, you measure, and you climb the ladder only to where a failing metric points.

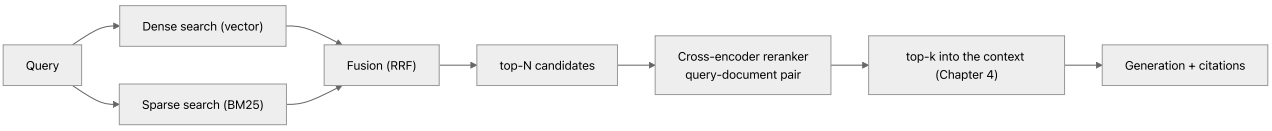
7.3 Pattern: Hybrid Search

Context. Queries carry both semantics ("how do I file a return") and exact tokens—error codes, SKUs, names, IDs—that must not be lost. **Problem.** Dense (vector) search catches meaning beautifully but misses exact rare terms; keyword search catches the exact thing but does not understand a paraphrase. How do you get both? **Forces.** Dense: semantics, robustness to synonyms, but it blurs rare exact tokens. Sparse (BM25°): exact term matching, but blind to meaning. **Solution.** Run **both** searches—dense (vector) and sparse (BM25/full-text)—and **fuse** their results into a single ranked list. **Mechanics—fusion via RRF.** The most dependable way to merge two lists is **Reciprocal Rank Fusion**: each document accrues `Σ 1/(k + rank_i)` over its positions in each list (k is a constant, e.g. 60)°. RRF works *on ranks*, not on raw scores, so it needs no normalization of incomparable scales (cosine vs BM25 score)—a large practical convenience. *Reading the formula:* `rank_i` is the document's position in the i-th list (1 is first place); the term `1/(k + rank_i)` is the "points" for that position—a reciprocal, so a high position earns more points, but the damper k keeps it from earning overwhelmingly more. A mini-example at k = 60: a document that came back 1st on BM25 and 5th on vector search earns `1/(60+1) + 1/(60+5) ≈ 0.0164 + 0.0154 = 0.0318`—both scales contributed comparably, even though adding their raw scores (BM25 ≈ 12.7 and cosine ≈ 0.83) would have been meaningless. **Implementation Example.** Vector search supplies the dense candidates (pgvector, say), full-text search supplies the sparse ones (`tsvector` / `ts_rank` in Postgres, say), and RRF is computed in SQL or in the application; one relational database covers both branches. **Alternative to Fusion.** If the score scales are calibrated and comparable, a weighted (convex) combination of normalized scores stands in for RRF, with a coefficient `α` tuning the dense and sparse contributions; RRF is preferred where calibration is out of reach, since it requires no normalization. **Resulting Context.** A noticeable rise in recall on mixed queries; the price is two searches and fusion logic. **When to Use.** Almost always for real corpora—they always hold both meaning and exact tokens. **When Not To.** Purely semantic short FAQs with no terminology. **Related Patterns.** Reranking (the next step after fusion); Query transformation. **Sources.** G. Cormack et al., "Reciprocal Rank Fusion outperforms Condorcet…" (2009); S. Robertson, H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond" (2009).

7.4 Pattern: Reranking

> **Context.** The first-stage search returned top-N candidates, but their order is inaccurate—what you need may be sitting at position 15, and only top-5 will fit into the context. **Problem.** How do you get an accurate relevance ordering when fast vector search is lossy? **Forces.** The key distinction here is between two architectures (expanding on §1.3):
>
> - **Bi-encoder** (the search model) encodes query and document *independently* into vectors and compares them by cosine. Fast and scalable (the vectors are precomputed), but **lossy**—there is no query↔document interaction.
> - **Cross-encoder** (the reranker) feeds the **[query, document] pair jointly** into the model and emits a relevance score. Accurate (it sees the interaction), but **expensive**—it runs on every pair and cannot be precomputed. **Solution.** Two stages: the bi-encoder cheaply pulls top-N (say, 50), the cross-encoder reranks precisely down to top-k (say, 5). **Reranking works precisely because it restores the query-document interaction the bi-encoder threw away for speed. Mechanics/Placement.** After retrieval (dense or hybrid), before generation. Latency grows (a cross-encoder pass over N candidates), but N is small—tens—so it is tolerable. The reranker is a separate small model (self-hosted) or an API. **Resulting Context.** Top-k accuracy rises noticeably; the price is extra latency and one more component, the reranker. **When to Use.** Almost always, if top-k accuracy matters. **When Not To.** Ultra-low latency outweighs accuracy, or the corpus is so small that ranking is beside the point. **Related Patterns.** Hybrid Search (before the rerank); Naive RAG (without one). **Sources.** R. Nogueira, K. Cho, “Passage Re-ranking with BERT” (2019); N. Reimers, I. Gurevych, “Sentence-BERT” (2019)—bi- vs cross-encoder.

The combined default pipeline—hybrid search followed by reranking—looks like this:



Choosing `N` is a trade-off: a larger `N` raises the odds that the relevant document survives to the reranking stage (first-stage recall), but it drives the cross-encoder’s cost up linearly. Beyond cross-encoders there are **LLM rerankers** (the model ranks a list of documents by relevance, listwise): more flexible, but pricier and slower. Practical reference points (mid-2026): the default open pairing is BGE-reranker-v2 over BGE-M3 embeddings; from the APIs, Cohere Rerank. Confirm the choice on your own golden set (Chapter 9), not on someone else’s benchmarks.

Deep Dive: late interaction (ColBERT)—the third retrieval paradigm

Between the bi-encoder (fast, lossy) and the cross-encoder (accurate, expensive) sits a third paradigm—**late interaction**—that dissolves their dilemma.

The idea (ColBERT, Khattab & Zaharia, 2020). Instead of one vector per document (bi-encoder) or a joint pass over the pair (cross-encoder), ColBERT encodes query and document into a *set* of vectors—one per token (**multi-vector**). Relevance is the **MaxSim** operation: for each query token, take the maximum similarity across all

document tokens, then sum the maxima. It is "late" because the query-document interaction happens *late* (at the level of precomputed token vectors) rather than early (a joint pass, as in a cross-encoder).

Where it sits on the spectrum. More accurate than a bi-encoder (it keeps token granularity instead of compressing to a single vector), yet more scalable than a cross-encoder (document vectors are *precomputed*—no pair pass on every query). The price is **memory**: a vector per document token is expensive (an order of magnitude more than a single vector); hence the storage optimizations (compression of token vectors, PLAID[°]).

When to consider it. When a bi-encoder plus reranker falls short on quality, a cross-encoder over the whole corpus is unrealistic, and the memory budget for multi-vector is there. ColBERT is often used either as a strong retriever or as a reranker. It is, in essence, one more point on the "quality ↔ memory ↔ speed" curve.

Source. O. Khattab, M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT" (2020).

7.5 Family: query transformation

Sometimes the problem is not the index but the query itself—it is murky, conversational, or lexically far from the documents. A family of techniques that rewrite the query before searching:

- **Query rewriting**—reformulate a conversational or incomplete query into a good search query (resolve pronouns against the history, strip the noise).
- **HyDE (Hypothetical Document Embeddings)**—generate a *hypothetical answer* to the query and search on the embedding of the *answer* rather than the question. The idea: a hypothetical answer is lexically closer to real documents than a question is (it bridges the "question ↔ document phrasing" gap).
- **Multi-query**—generate several variants of the query and union the results (raises recall).
- **Decomposition**—break a complex multi-part question into sub-queries and search on each.
- **Step-back**—generalize the query to pull in broader context (useful when a narrow question needs background knowledge).

When: a lexical gap between query and document; complex, multi-part, or conversational queries. *The price:* an extra model call (or several) for the transformation—cost and latency. *Source*. L. Gao et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels" (2022, HyDE); the Gao et al. survey (2023).

7.6 Deep Dive: chunking

Chunking is an ingestion-time decision, but it sets the ceiling on retrieval quality: you cannot retrieve what was badly cut. Strategies, in increasing order of quality:

- **Fixed-size**—cut every N tokens. Simple, but it tears meaning apart mid-sentence, mid-thought.
- **Recursive / structural**—cut along the structure: headings, paragraphs, and for code, functions and classes via the AST (not by characters). Respects the boundaries of meaning.
- **Semantic**—cut where the embeddings of neighboring sentences *shift* (a topic boundary). The cut rule is not an absolute threshold but a **percentile of the document's own distribution**: cosine distances (1 – cos) are computed between the embeddings of neighboring sentence groups, and a cut goes wherever the distance exceeds the 95th percentile of the distances within *that same* document. 95 is the default in both of the going implementations (`SemanticSplitterNodeParser` in LlamaIndex, `SemanticChunker` in LangChain); lower the

threshold and you get more cuts and smaller chunks. *Worked numbers:* a 200-sentence document has 199 seams; the 95th percentile cuts off the top 5%—about 10 cuts and about 11 chunks of roughly 18 sentences each. Which means the percentile directly sets the **number** of chunks, not the "quality of the boundary": it is a granularity knob, and it should be tuned as a granularity knob. The threshold is relative out of necessity: absolute cosine values are not comparable across embedders—anisotropy (§1.3) squeezes the vectors into a narrow cone, so "distance 0.3" under one model does not mean what it means under another, and a hardcoded constant transfers neither to another model nor to another corpus.

- **Parent-document (small-to-big)**—index small, precise chunks (for an accurate *hit*), but feed the larger *parent* fragment into the context (for *intelligibility* at generation time). It resolves the conflict "a small chunk is found more precisely but is poorer in context." The mechanics in one line: *child* chunks go into the vector index (256 tokens, for example), each carrying a `parent_id` field; the search runs over the children, while the context is assembled from the parent (2048 tokens, for example), pulled by `parent_id` out of ordinary storage and **deduplicated**—several children of one parent landing in top-k must yield a single parent, or the same text will occupy the window twice.
- **Contextual retrieval (Anthropic, 2024)**—before embedding, attach to each chunk a short LLM-generated blurb that situates it within the document ("This is the returns section of policy X, covering defective goods"). It works because an isolated chunk is often unintelligible without the document's context. The size of the gain has been measured: on Anthropic's runs the miss rate (the needed fragment absent from top-20) falls from 5.7% to 3.7%—**35% in relative terms**; fold the same blurb into the BM25 branch of the hybrid (§7.3) as well and it drops to 2.9% (−49%); add a reranker on top (§7.4) and you get 1.9% (−67%). The absolute percentages are deceptive and have to be read on the relative scale: the baseline already doesn't miss 94% of the time, so "−2 percentage points" is a third of every failure that was left. The price is one model call per chunk at ingestion.

Why "a call per chunk" doesn't bankrupt you: prompt caching. Naively, contextual retrieval looks unaffordable: to write the blurb for a chunk, the model needs the whole document—which means the document is fed in again for every one of its chunks. What saves it is that the document goes into the **cacheable prefix** (§2.2): it is written to the cache once, and every subsequent call *reads* it. Anthropic's pricing is set as multipliers on ordinary input: a cache write costs 1.25 of the input price, a cache read 0.1 (§20.2). Only the tail of the prompt changes—the chunk itself. Indexing cost is computed in two steps. First, the cost of a single document. Dividing by 10^6 here is not decoration but a unit conversion: our lengths are in tokens, while the price list is quoted per million tokens:

$$C_{doc} = \frac{(L_{doc} + L_{ins}) \cdot p_w + n \cdot ((L_{doc} + L_{ins}) \cdot p_r + L_{chunk} \cdot p_{in} + L_{ctx} \cdot p_{out})}{10^6}$$

Then multiply by the number of documents in a million tokens of corpus, $10^6 / L_{doc}$. The two factors of 10^6 cancel exactly, and a clean fraction is left:

$$C_{1M} = \frac{10^6}{L_{doc}} \cdot C_{doc} = \frac{(L_{doc} + L_{ins}) \cdot p_w + n \cdot ((L_{doc} + L_{ins}) \cdot p_r + L_{chunk} \cdot p_{in} + L_{ctx} \cdot p_{out})}{L_{doc}}$$

Symbol	What it is	Value in Anthropic's calculation
L_{doc}	document length, tokens	8000
L_{ins}	wrapper instruction, tokens	50
L_{chunk}	chunk size, tokens	800
L_{ctx}	length of the generated blurb, tokens	100
n	chunks per document, L_{doc}/L_{chunk}	10
p_w	cache write, $ per 1M tokens	0.30
p_r	cache read, $ per 1M tokens	0.03
p_{in}	ordinary input, $ per 1M tokens	0.25
p_{out}	output, $ per 1M tokens	1.25
C_{doc}	cost of one document, $	0.0081
C_{1M}	cost of a million tokens of corpus, $	1.01

The prices are the Claude 3 Haiku price list—the model Anthropic actually ran the numbers on. An honest caveat about the source: the model was retired in April 2026 and its rows are gone from Anthropic's current price list—there is nowhere to check the base $0.25/$1.25 today, these are figures from the 2024 price list. Something else is verifiable, and it is enough: the cache multipliers are still in the price list, and the calculation itself reproduces Anthropic's published figure to within a percent. Substituting, for one document:

- writing the document to the cache, once: `8050 × 0.30/10⁶ = $0.0024`
- 10 cache reads (one per chunk): `10 × 8050 × 0.03/10⁶ = $0.0024`
- 10 chunks as ordinary input: `10 × 800 × 0.25/10⁶ = $0.0020`
- 10 blurbs on the output: `10 × 100 × 1.25/10⁶ = $0.0013`
- total C_{doc} ≈ `$0.0081` per document; a million tokens of corpus holds `10⁶/8000 = 125` documents → C_{1M} = `8080/8000` = ≈ **$1.01** (Anthropic publishes $1.02)

Reading the formula in plain terms: the blurb costs one model call per chunk, but the document is not fed into that call again—it is read from the cache, and the dominant term, "document × number of chunks," gets roughly eight times cheaper (`0.25/0.03`). Eight, not the nominal ten, because the Haiku 3 table is rounded to cents: a read by the multiplier is `0.1 × 0.25 = 0.025`, while the price list says $0.03 (the same penny-sized discrepancy on the write: `1.25 × 0.25 = 0.3125` → $0.30). And the rounding works in our favor: with exact multipliers (`0.3125` for a write, `0.025` for a read) the same calculation gives $0.97, and it is the price list's rounding that lifts the estimate to $1.01—that is, nudges it toward the published figure. The remaining cent up to $1.02 cannot be reconstructed from Anthropic's published data; the gap between the rounded calculation and the publication is within a percent and does not affect the shape of the conclusion. Without the cache, the same calculation gives `10 × 8850 × 0.25/10⁶` of input plus output ≈ `$0.023` per document, or **$2.92** per million—nearly three times more. The cache here is not an optimization layered on top of a working scheme; it is what makes the scheme economically sensible in the first place, and that is the non-obvious engineering point of the technique. What is

worth carrying into your own calculation is not the absolute figures (the model is gone, and cache structure differs across providers—§20.2) but the shape: "document × number of chunks" dominates, and the cache is exactly what removes it.

Parameters and problems. Size is usually 256–1024 tokens; **overlap** (the shared span between neighboring chunks) preserves boundary context. The numbers here are empirical rather than derived, and you can see it right in the source. Microsoft's chunking guidance for Azure AI Search recommends, **as a starting point, a 512-token chunk (≈ 2000 characters) and 25% overlap—those very 128 tokens**; the same page's defaults for built-in splitting (2000 characters, overlap 500) give the same 25%. But in the table of techniques on that same page, in the description of fixed-size splitting, an overlap of "for example, 10–15% of the content" is mentioned in passing—with no status as a recommendation at all. One vendor page, two inconsistent numbers—the best proof there is no law here: take 25% as a starting point and tune it to your data. Overlap costs you twice, and the second half of the bill is not obvious: the index grows roughly in proportion to the overlap, and **near-duplicates** arrive in top-k—two neighboring chunks sharing a stretch of text—which silently eat the context budget and dilute it (§1.8). Hence the rule: if there is overlap, deduplication after retrieval is mandatory. The main affliction is **fragmentation**: the fact you need is severed by a boundary, or the chunk is unintelligible outside its document (cured by parent-document and contextual retrieval). And always **attach metadata** to the chunk—source, owner, clearance, date—because you need it for filtering, citations (§6.7), and RBAC at retrieval time (Chapter 8).

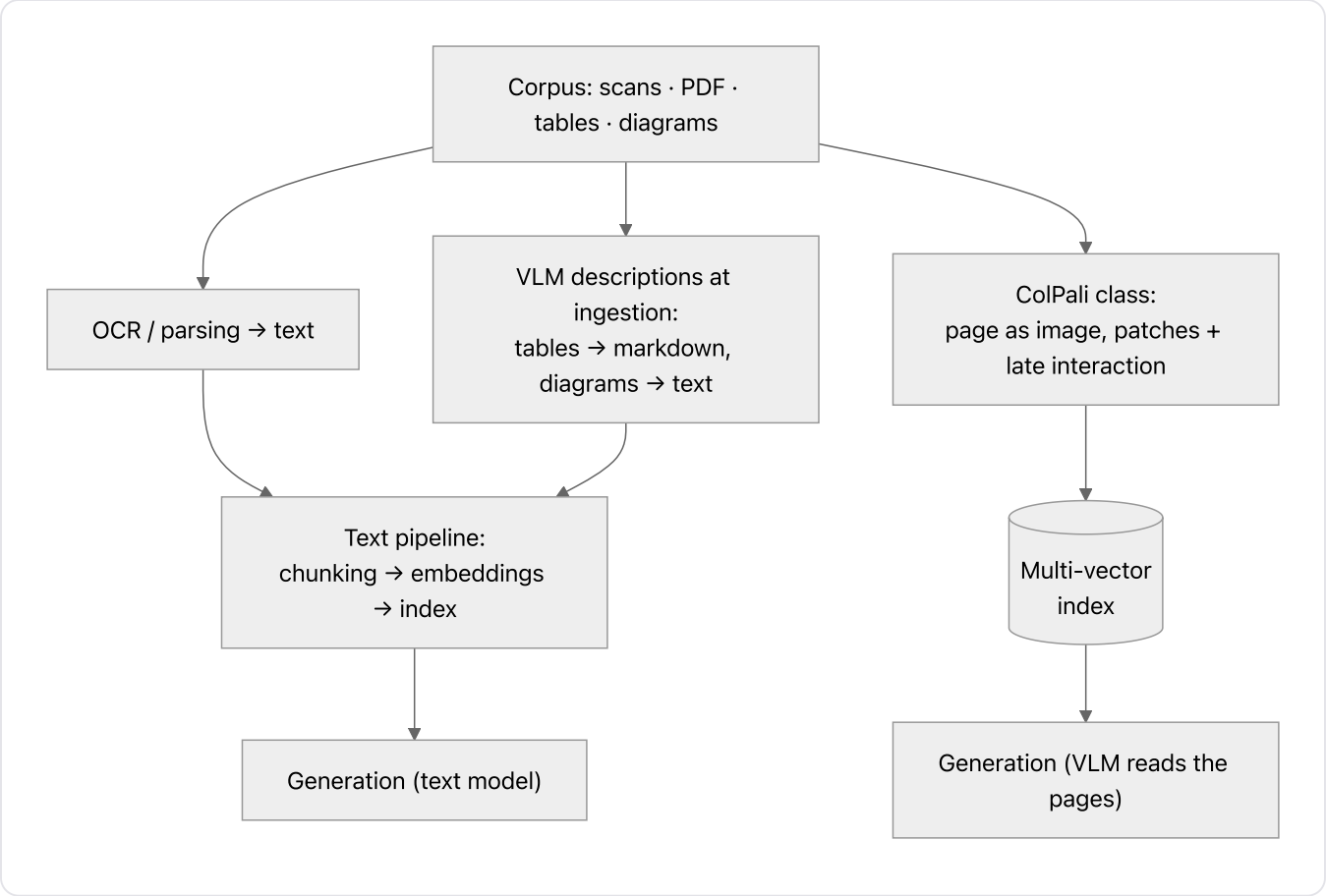
The engineer's iron rule: Chunking is an **index-time** decision: size, overlap, and strategy are baked into the vectors already sitting in the index, and there is no extracting them after the fact. So changing any of the three is not a config edit but a **full reindex of the corpus**—the same class of migration cost as changing the embedder (§1.3, §8.8). Two practical consequences follow. First: chunking parameters are tuned on the golden set (Chapter 9) *before* the corpus is loaded, not after. Second: on a corpus that has grown, the cost of the experiment "let's try 512 instead of 1024" is counted in hours of reindexing and in the embedding bill, not in minutes of config editing—which is why such experiments are planned in batches, not one at a time.

Sources. Anthropic, "Contextual Retrieval" (2024)—the miss rates, the calculation's parameters (chunk 800, document 8k, instruction 50, blurb 100 tokens), and the $1.02-per-million result. The cache multipliers (write 1.25, read 0.1 of input)—Anthropic's current price list (§2.2, §20.2); the Claude 3 Haiku base rates—the 2024 price list; the model was retired in April 2026 and is absent from the current price list. The 95th-percentile default—the `SemanticSplitterNodeParser` (LlamaIndex) and `SemanticChunker` (LangChain) implementations. The 512/25%/128 overlap—Microsoft, chunking guidance for Azure AI Search, section "Content overlap considerations." The Gao et al. survey (2023).

Deep Dive: multimodal RAG (scans, tables, diagrams)

A corporate corpus rarely consists of clean text: scanned PDFs, tables, diagrams, screenshots. The classic pipeline—OCR/parsing into text → chunking → text embeddings—loses table structure (naive chunking slices tables across the grain of their meaning) and is blind to diagrams; on corpora like these that is the leading source of retrieval misses, and neither hybrid search nor a reranker cures it. Two ways out beyond the text pipeline:

- **VLM descriptions at ingestion**—a vision-language model converts images and tables into structured text (markdown tables, textual descriptions of diagrams), after which the ordinary text pipeline takes over. A practical first step: it preserves the whole text infrastructure, and the cost is one-time, at indexing.
- **Retrieval over page images** (the ColPali/ColQwen family, 2024–2025)—the page is encoded *as an image*: visual patches play the role of tokens, with late interaction on top of them (§7.4). Indexing without any OCR pipeline at all; robustness to layout, tables, handwriting. The price: memory for the multi-vectors (the same pain as ColBERT), the generator must be able to read page images in its context, and the infrastructure is younger than the text one.



The choice: clean text → the classic pipeline; layout and tables are critical → VLM descriptions as the first rung; the ColPali class—when measurement (Chapter 9) shows that retrieval quality on visual documents is bounded by OCR losses specifically.

Sources. M. Faysse et al., "ColPali: Efficient Document Retrieval with Vision Language Models" (2024); the ColQwen models (2025); §7.4—late interaction.

Deep Dive: choosing an embedding model (the foundation that affects everything)

Retrieval quality is bounded by embedding quality, yet the embedding model is often chosen at random. The axes of the choice:

- **Domain fit.** A model trained on the general web encodes a specific domain (legal, in-house jargon) worse; evaluate it on that domain's data, not against the general leaderboard.
- **The MTEB benchmark**—the standard starting point (retrieval, clustering, and other tasks), but a reference point, not the final criterion: the top row of MTEB does not guarantee the best result on your particular task. Reference points as of mid-2026: at the top of the leaderboard, Gemini Embedding and the open Qwen3-Embedding; among API models, Cohere embed-v4 and OpenAI text-embedding-3-large are widespread; the open default is BGE-M3. When comparing, account for MTEB's move to v2 (v1 and v2 scores are not comparable). The names go stale fast—the axes of choice from this list go stale more slowly.
- **Multilingualism.** A Russian-language system needs a model with strong Russian support (many strong English-language models are weaker in Russian); check both monolingual and cross-lingual retrieval on Russian pairs.

- **Dimensionality and length.** A larger `d` is richer but costs more in memory and search (Chapter 8); the maximum input length caps the chunk size. **Matryoshka embeddings** let you truncate the dimensionality without retraining (flexibility along "memory ↔ quality").
- **Fine-tuning the embedder**—the most powerful lever on in-domain retrieval quality: fine-tuning the embedding model on domain pairs (query, relevant document) often buys more than switching retrieval patterns. It needs labeled pairs (from clicks and feedback, or synthetic ones with review).
- **The operational consequence (§1.3, §8.8).** Changing the embedder means a full reindex, so the choice is made with long-term use in mind.

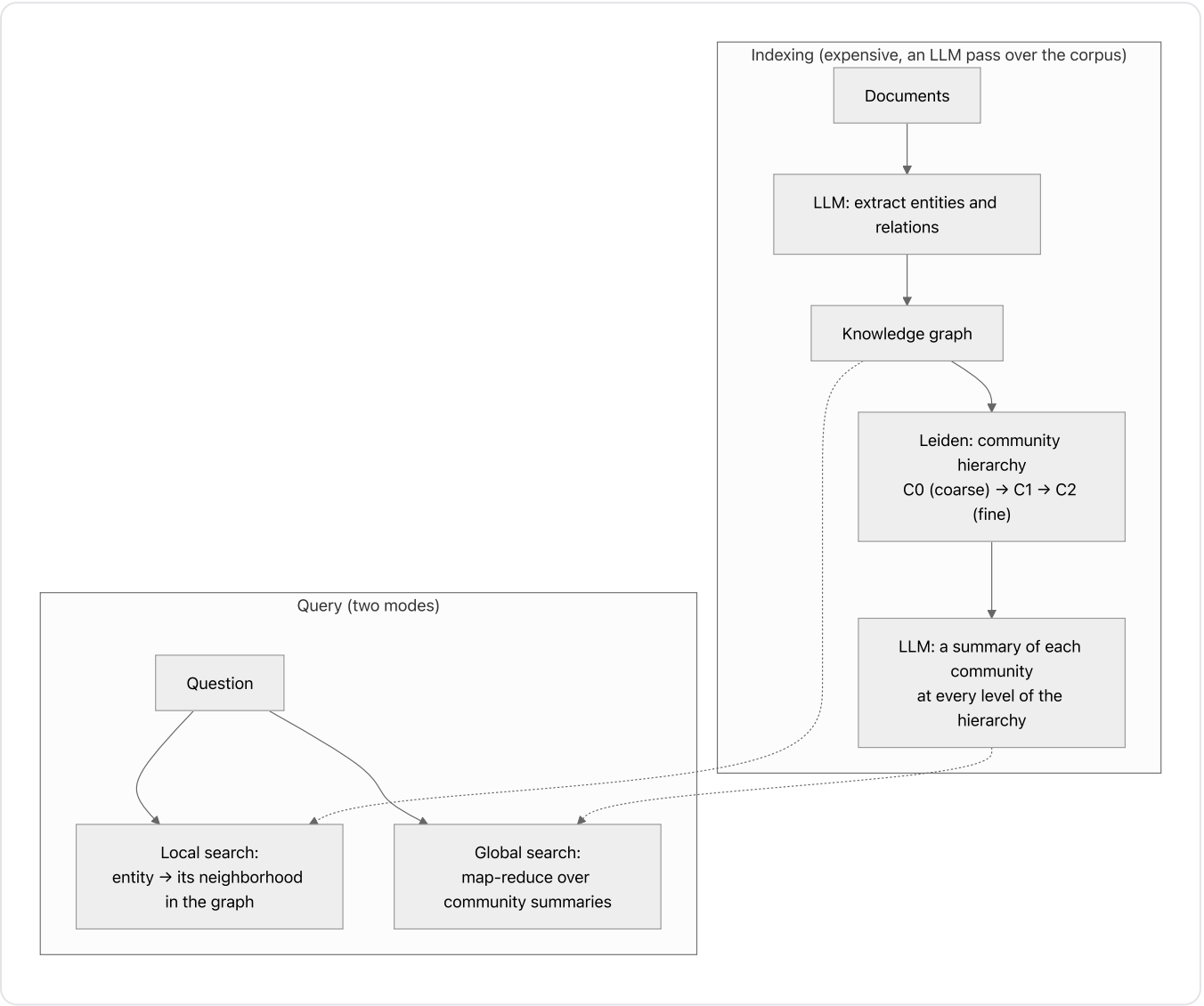
The practical order: a strong multilingual model as the base → measurement on the golden set (Chapter 9) → if the domain is specific and the data exists, fine-tune the embedder. This is the foundation: no reranker compensates for base embeddings that do not reflect the domain.

Source. N. Muennighoff et al., "MTEB" (2022); Matryoshka Representation Learning (2022); the literature on domain fine-tuning of embedders.

7.7 Pattern: GraphRAG

> **Context.** The questions require "connecting the dots" across many documents: supply chains, org structures, who-knows-whom links, citations. **Problem.** Vector search retrieves *similar* fragments, not the *relations* between entities; it will never assemble "through which subcontractors is A connected to B." **Solution.** At ingestion, extract **entities and relations** into a knowledge graph; at query time, walk the graph (traverse the relations) rather than only search for similarity. **Mechanics (the Microsoft GraphRAG variant).** Indexing: an LLM extracts entities and relations → a graph is built → **community detection** (the Leiden algorithm) carves it into a hierarchy of communities → an LLM writes a summary of each community. Query: **local search**—the neighborhood of a specific entity (pinpoint questions)—or **global search**—map-reduce over the community summaries (questions "about the whole corpus," which fragment-based RAG cannot answer even in principle). Indexing is expensive (an LLM pass over the corpus); the cheaper variant **LazyGraphRAG** (2024) defers the LLM work from indexing to query time. **Resulting Context.** It answers multi-step relational questions; the price is expensive entity extraction and *graph maintenance* (schema, updates, extraction quality). **When to Use.** Relational, multi-document questions where relations matter more than similarity. **When Not To.** Ordinary factoid QA—GraphRAG is pricier there and no more accurate (a common mistake is reaching for a graph where hybrid would have done). **Source.** Microsoft Research, "GraphRAG" (2024).

The full pipeline is the only multi-stage one in this chapter, so let's draw it:

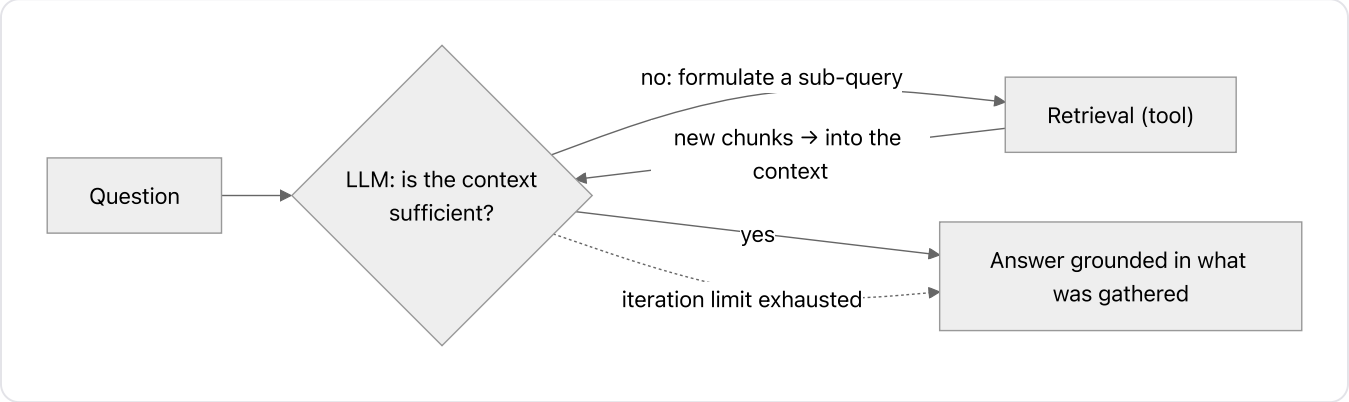


What Leiden actually does. Community detection partitions a graph into groups of vertices with many edges inside and few between. The formal criterion is **modularity**: the fraction of edges falling inside communities minus its expectation for a random graph with the same vertex degrees; the higher it is, the more "real" the partition. Maximizing modularity exactly is NP-hard, so greedy algorithms do the work: the classic **Louvain** (2008) alternates local moves of vertices between communities with aggregation (a community collapses into a supervertex and the process repeats—hence the hierarchy of levels). Louvain has a documented defect: it can produce an *internally disconnected* community—a bridge vertex moved off into a neighboring group, and the remaining pieces are formally counted as one community while having no path to each other. **Leiden** (Traag, Waltman, van Eck, 2019) inserts a refinement phase between the move and the aggregation and *guarantees* that every community is connected, while running faster besides. For GraphRAG that guarantee is not an abstraction: the summary of a disconnected "community" would describe two unrelated groups of entities as one, poisoning global search. The hierarchy of levels gives you a granularity knob: global search over C0 is cheap and broad-brush, over C2 pricier and more detailed; you pick the level to fit the query's token budget.

7.8 Pattern: Agentic RAG

> **Context.** The question is multi-step: a single search pass cannot in principle assemble the necessary context (you need to fetch one thing, understand it, refine, fetch another). **Problem.** A static `retrieve → generate` does not adapt when the first retrieval wasn't enough. **Forces.** A single pass is cheap and fast but inflexible; a model-driven loop is flexible but expensive and slow. **Solution.** Let **the LLM drive the retrieval loop**: retrieve → judge sufficiency → reformulate or fetch more → repeat until it's enough. This is already an agent on top of retrieval (the bridge to Part IV). In the trained variant (**Self-RAG**, Asai et al., 2023) the model itself decides *when* to retrieve and *critiques* the retrieved passages for relevance. **Mechanics.** Retrieval becomes a tool (tool calling, Chapter 12) that the model uses in a loop, with an **iteration limit** (otherwise you risk an infinite loop). **Resulting Context.** It handles multi-step questions; the price is latency and cost (several calls plus several retrievals). **When to Use.** Multi-step questions where one pass is not enough. **When Not To.** A simple factoid—that's over-engineering. **Related Patterns.** Adaptive RAG (route only the hard ones *into* agentic); the agent patterns of Part IV. **Source.** A. Asai et al., "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection" (2023).

The loop is structurally identical to Evaluator-Optimizer (§11.6)—generation and evaluation alternate until sufficiency or the limit:



Worked example: what the loop costs. Say one Naive RAG pass is ~2000 input tokens (the question plus the top-k chunks) and ~300 output; at a rate of $1/M input and $4/M output that comes to ≈ $0.0032. A 3-iteration agentic loop *accumulates* context: each iteration is a full call carrying everything retrieved so far, so the input grows in steps of ~2k → 4k → 6k (~12k in total) plus ~0.9k of output for the intermediate decisions and the answer ≈ **$0.0156—five times more expensive**—and 3–4× slower (the iterations are sequential and cannot be parallelized: each sub-query depends on the previous result). Notice the *superlinearity*: the cost of k iterations grows as $O(k^2)$ in input tokens—the same arithmetic of accumulating context as in a long conversation (§4.4). Hence the practical conclusion: an iteration limit is not only protection against non-termination (§10.5) but a budget lever as well, and routing only the hard queries into agentic (Adaptive RAG, §7.9) is what makes the pattern economically viable at volume.

Self-RAG: how "decides" and "critiques" work on the inside. Prompt-based Agentic RAG keeps the loop outside, in the harness (§10.2): instruction, tool call, check. Self-RAG moves control *inside decoding*. Special **reflection tokens** are added to the model's vocabulary, and the fine-tuned model emits them in the ordinary generation stream, like any other token: `Retrieve` (is retrieval needed right now), `ISREL` (is the retrieved passage relevant to the question), `ISSUP` (is the generated claim supported by the passage—in effect a built-in faithfulness check, Chapter 9), `ISUSE` (the answer's overall usefulness). Training has two stages: first a *critic*

model labels the training corpus with these tokens, then the *generator* is fine-tuned to predict the labels along with the text. At inference the tokens do double duty: `Retrieve` switches retrieval on mid-generation, while the probabilities of `ISREL` / `ISSUP` / `ISUSE` serve as scores for ranking candidate segments generated against different passages (segment-level beam search)—the model itself picks the continuation better supported by sources. The price of the elegance is that this is a *fine-tuned* model needing access to the vocabulary and the logits: you cannot do this against a hosted API, so Self-RAG is a pattern for self-hosted deployments (Chapter 22), while in the hosted world the prompt-based variant with an external loop rules.

7.9 Pattern: Adaptive RAG

> **Context.** The query stream is heterogeneous: simple factoids and hard multi-step questions mixed together. Running everything through expensive agentic/graph is wasteful; running everything through naive is risky on the hard ones. **Solution.** A classifier judges the query's complexity and **routes** it to the appropriate strategy: simple → naive/hybrid, hard → agentic/graph. This is exactly Model Routing (§2.2) applied to retrieval—a cost/quality optimizer. **Resulting Context.** A balance of cost and quality across the whole stream; the price is the cost and the errors of the classification itself. The anatomy of the classifier (implementations, labeling, the silent failure mode) is the Deep Dive in §2.2. **When to Use.** An explicitly heterogeneous stream. **When Not To.** A uniform load—there is nothing to route. **Source.** S. Jeong et al., "Adaptive-RAG…" (2024).

7.10 Summary and decision frame

The catalog, in increasing order of complexity and cost:

Pattern	Solves	Cost	Take it when
Naive	The baseline	Minimal	The answer is in one chunk; the start
Hybrid Search	Missing exact tokens	Low	Almost always (the default)
Reranking	A bad top-k ordering	Medium	Almost always (the default)
Query transform	The query-document gap	+a call per query	Murky or complex queries
Contextual chunking	Fragmented or unintelligible chunks	+a call per chunk (ingest)	Isolated chunks lose their meaning
GraphRAG	Relational multi-doc questions	High	Relations matter more than similarity
Agentic RAG	Multi-step questions	High	One pass is not enough
Adaptive RAG	A heterogeneous load	+a router	A mixed stream of complexity

The decision frame: start with Naive (the baseline) → measure (Chapter 9) → by default, **Hybrid + Rerank + competent chunking** → and only if measurement shows a shortfall on a specific class of questions do you reach for query transform, graph, agentic, or adaptive. Complexity is earned by the demonstrated failure of the simple solution, not chosen in advance.

Sources. The Y. Gao et al. survey (2023); the primary sources for each pattern (see the individual sections); V. Traag, L. Waltman, N. J. van Eck, "From Louvain to Leiden: Guaranteeing Well-Connected Communities" (2019)—the mechanics of community detection (§7.7); Chip Huyen, *AI Engineering* (2025).

In the Margins: Chapter 7

↩ **BM25** (Okapi BM25)—the classic of lexical ranking: TF-IDF with term-frequency saturation and normalization by document length. "Sparse" search = matching on words, not on meaning. ⇐ Glossary.

↩ The `k` constant in RRF is a damper: it keeps first place in one list from automatically outweighing consistently high (but not first) positions in the others. Working on ranks rather than scores does away with normalizing incomparable scales. ≠ A notation warning: the letter k is overloaded throughout this book—the RRF constant, retrieval's top-k, HNSW's ef_search candidates (§8.3), the k in recall@k (Chapter 9), and, earlier still, top-k sampling during decoding (§1.7) and the top-k experts in MoE routing (§1.5)—six different quantities; the meaning is fixed by context.

↩ **PLAID** (Santhanam et al., 2022)—a storage and search engine for late-interaction models: it compresses ColBERT's token vectors (centroids + residuals) and speeds up search by an order of magnitude. → §7.4 (Deep Dive).

Chapter 8. Vector Indexes and Retrieval Infrastructure

The retrieval layer rests on a special kind of index over the data, and most of this chapter comes down to choosing the index structure and tuning it to the load. What makes it peculiar: “distance” here means geometric proximity in a high-dimensional space, and the index itself is *approximate* by construction.

8.1 The problem: nearest-neighbor search in high dimensions

Retrieval reduces to **nearest-neighbor search**: given a query vector, find the vectors in the corpus closest to it. Exact (brute-force) search costs `O(N·d)` per query: compare the query against each of the `N` vectors of dimension `d`. At millions of vectors the latency is unacceptable. So you need **ANN—Approximate Nearest Neighbor**: trading hit accuracy for speed. The operative word is *approximate*: an ANN index can, by construction, miss the true nearest neighbor. That is not a defect but a deliberate trade-off (§8.4).

8.2 Deep Dive: the curse of dimensionality

Why not reach for a familiar tree (a kd-tree) and stop thinking? Because in high dimensions (embeddings run to `d` = hundreds or thousands) the **curse of dimensionality** takes over: distances *concentrate*—the gap between the nearest and the farthest neighbor collapses, and almost everything ends up roughly equidistant. Tree structures (kd-tree, ball-tree), efficient in 2–3D, degrade into a brute-force scan: you have to walk nearly every branch. That is exactly why vector search got its own *purpose-built* ANN structures (graph-based and clustering-based) instead of reusing a database’s spatial indexes. Grasping this explains why pgvector adds HNSW/IVFFlat rather than building an ordinary B-tree over a vector.

The concentration criterion. “Collapses” is not a metaphor but a theorem with a checkable condition. Beyer et al. (1999) state it this way: concentration sets in if the *relative* spread of distances—the variance, normalized by the distance’s own average scale—tends to zero as dimension grows:

$$\lim_{d\to\infty} var\left(\frac{D_d^p}{E[D_d^p]}\right) = 0 \quad \Longrightarrow \quad \lim_{d\to\infty} P\left[D_{max} \le (1+\varepsilon)D_{min}\right] = 1$$

Where:

- d—the vector’s dimension; embeddings run 384–4096, and from here on we take $d = 1536$.
- D_d—the distance from the query to a random corpus point at dimension d, in units of the metric (for L2 in the cube $[0,1]^{1536}$, about 16; the arithmetic is below).
- p—the exponent constant, $0 < p < \infty$; for squared Euclidean distance $p = 2$. It does not affect the substance.
- $E[\cdot]$ and $var(\cdot)$—the mean and the variance over the corpus. Dividing by $E[D_d^p]$ makes the quantity dimensionless: it is the spread of distances *as a fraction of their own scale*.
- $P[\cdot]$—the probability of an event; here, the fraction of queries for which the bracketed inequality holds.

- D_{min}, D_{max}—the distances from the query to the nearest and to the farthest of the n corpus points; n is **fixed**—the theorem is about d growing with the corpus held constant, not the other way around.
- ε—the relative margin; the statement holds for any arbitrarily small $\varepsilon > 0$.

One caveat about citation accuracy: in the original the theorem is stated for an abstract index m along which the sequence of distance distributions converges; dimension is its canonical case, and from here on we write d.

In plain terms: if, as dimension grows, distances stop differing from one another as a fraction of the average distance, then with probability tending to one the farthest point in the corpus lies no more than $(1 + \varepsilon)$ times as far as the nearest. Not "the far neighbor is a bit closer" but "far and near are indistinguishable": the word "nearest" stops singling anything out.

Live numbers. Uniform random points in the cube $[0, 1]^d$ are exactly the case where the condition holds. Per coordinate, $E[(u - v)^2] = 1/6 \approx 0.167$ and $var = 7/180 \approx 0.039$. The coordinates are independent, so at $d = 1536$: $E[D^2] = 1536/6 = 256$, $var(D^2) = 7 \cdot 1536/180 \approx 59.7$, standard deviation $\sqrt{59.7} \approx 7.7$. The relative spread of the squared distance is $7.7/256 \approx 3\%$, and of the distance itself (the square root roughly halves the deviation)—**about 1.5%**, that is, $D \approx 16 \pm 0.24$. The whole corpus lies in a thin spherical shell around the query.

A direct simulation (n = 200,000 points, averaged over several runs) confirms the estimate: at $d = 1536$ it gives $E[D] = 16.05$ and $\sigma = 0.224$, a spread of 1.4%, with $D_{min} \approx 15.0$ and $D_{max} \approx 17.0$. The same simulation exposes the mechanism of concentration in a single comparison: as dimension grows, the **absolute** spread does not change at all—$\sigma = 0.224$ at $d = 100$ and at $d = 3072$ alike, and the gap $D_{max} - D_{min} \approx 2.0$ likewise stays put—while the distance itself grows as $\sqrt{d/6}$: from 4.08 to 22.70. The contrast $(D_{max} - D_{min})/D_{min}$ is a frozen constant divided by a growing denominator, so it melts away roughly as $1/\sqrt{d}$: 0.70 at $d = 100$, 0.13 at $d = 1536$, 0.09 at $d = 3072$. What collapses is not the spread—what collapses is its *share* of the distance.

Why a tree ends up worse than a brute-force scan, not merely slower. A kd-tree saves work in exactly one way—**pruning branches**: if the ball around the query with radius "out to the best candidate found so far" does not intersect a cell, that whole branch goes unread. But once contrast has collapsed, that radius nearly equals the distance to *all* the points at once, and the ball intersects practically every cell. There is nothing left to prune—the tree walks almost every leaf, doing the same work as a brute-force scan (the same N distance computations), plus chasing pointers instead of running linearly through memory: sequential access, prefetching, and vectorization are all lost. Hence the result of Weber, Schek, and Blott (1998): above roughly **10** dimensions a simple sequential scan beats partitioning and clustering indexes on average. The tree here is not "slower"—it strictly loses to the `O(N·d)` brute-force scan (§8.1), and that is the concrete reason not to drag a spatial index into vector search.

The paradox—and its resolution. An attentive reader is obliged to trip here: if at $d = 1536$ everything really is equidistant, then HNSW's greedy descent (§8.3) is doomed exactly as the kd-tree is—yet it works. The theorem's premise resolves the contradiction. Beyer's condition holds for points whose coordinates behave independently (uniform noise); real embeddings do not behave that way. The model was trained to *pull* semantically close texts toward one another, so the vectors occupy not the cube's full volume but a thin **low-dimensional surface** inside it (a manifold—a surface embedded in a space of higher dimension: 1536 coordinates, but only dozens of independently varying directions). This is not a retrofitted excuse: Beyer et al. themselves put both exceptions in a separate section of their paper—clustered data (their section 4.1) and *implicitly low dimensionality* (their section 4.2).

The gap between the **nominal** dimension (how many numbers are in the vector) and the **intrinsic** one (intrinsic dimension—how many independent directions are actually in play) is measurable. LID estimates (local intrinsic dimensionality—the intrinsic dimension measured locally, in the neighborhood of each point) on standard corpora:

Corpus	Nominal d	LID (mean)
MNIST	784	14.0
Fashion-MNIST	784	15.6
GLOVE	100	18.0
GLOVE-2M	300	26.1
SIFT	128	21.9

The curse acts on the nominal dimension, while the ANN graph navigates the intrinsic one—which is why HNSW stays alive at $d = 1536$. Chen et al. (2024) show this most directly: for *random* vectors the relative contrast (the ratio of the mean distance to the distance to the nearest neighbor; a value of 1 means "everything is equidistant, search is meaningless") collapses to 1 already at $d \approx 512–768$, whereas for text embeddings at the same dimensions it holds within 1.75–2.05. The same nominal d—the opposite outcome; the whole difference is in the distribution of the data, not in the dimension.

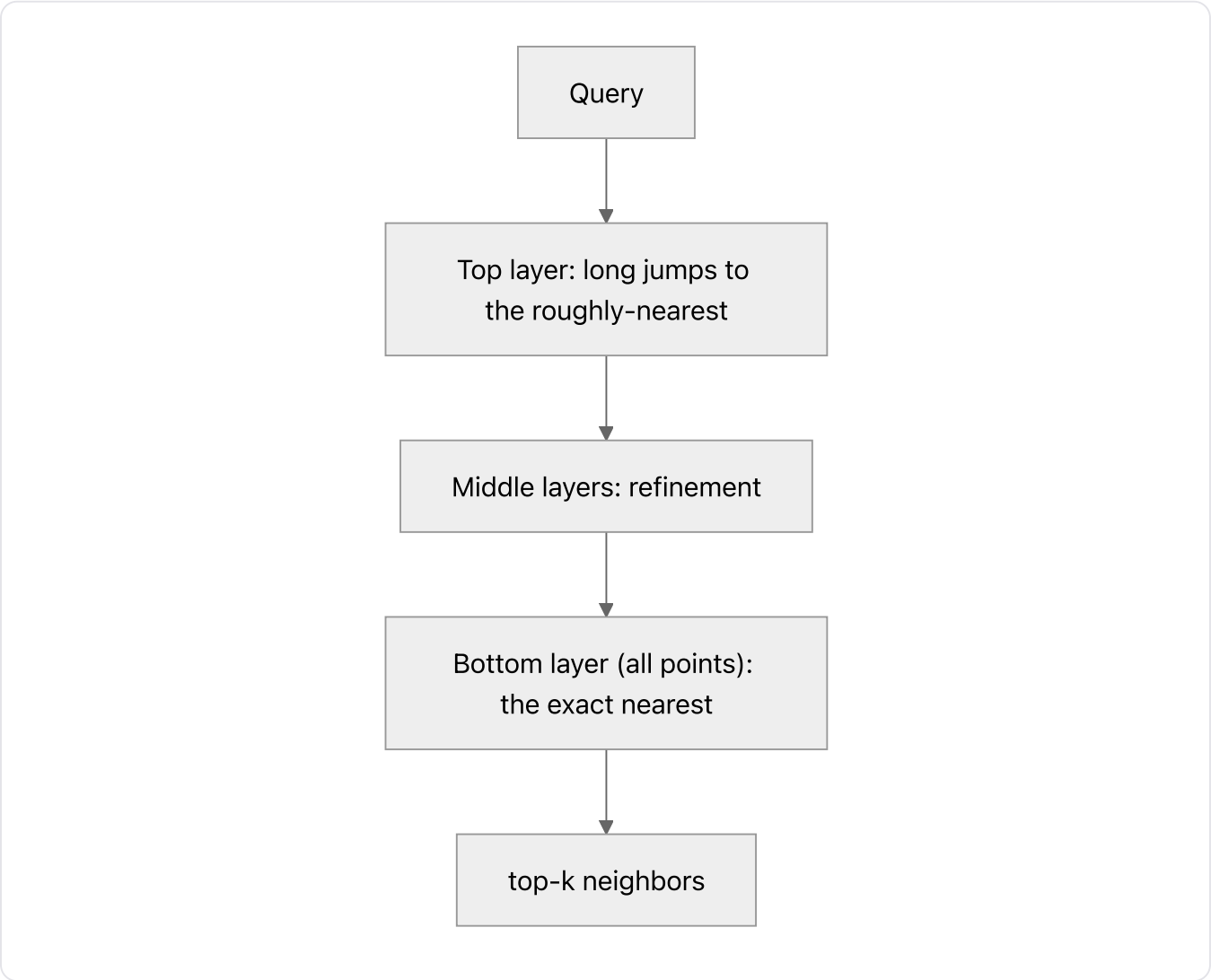
The operational consequence. Whether ANN works is a property of *your data*, not a guarantee of the algorithm. Within a single corpus, queries with high local dimensionality consistently take more time to reach the same recall, and on the hardest corpus (GLOVE-2M) not every implementation manages recall close to one (Aumüller & Ceccarello, 2019). But this cannot be carried across corpora, and the authors state the prohibition separately: LID by itself does not explain performance differences across datasets—SIFT, at LID 21.9, turns out in practice to be markedly easier than its LID predicts. Which only sharpens the conclusion: on data with honestly high intrinsic dimensionality—hashes, random vectors, poorly trained embeddings—HNSW's recall will sag just as the kd-tree's pruning does, and tuning `ef_search` will not cure it; but there is exactly one way to learn this about your own corpus—measuring recall against brute-force ground truth (§8.4) on *your* vectors. No published LID estimates for the typical 1536-dimensional embeddings of the latest generation could be found—all the more reason to measure intrinsic dimension and contrast on your own corpus instead of taking "dozens" on faith.

Sources. K. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft, "When Is 'Nearest Neighbor' Meaningful?" (ICDT, 1999)—theorem 1 and sections 4.1–4.2 on the exceptions. R. Weber, H.-J. Schek, S. Blott, "A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces" (VLDB, 1998)—the ~10-dimension threshold. M. Aumüller, M. Ceccarello, "The Role of Local Intrinsic Dimensionality in Benchmarking Nearest Neighbor Search" (SISAP, 2019)—the LID table, LID's effect on the cost of recall within a corpus, and the caveat that it does not carry across corpora. Z. Chen, R. Zhang, X. Zhao, X. Cheng, X. Zhou, "Exploring the Meaningfulness of Nearest Neighbor Search in High-Dimensional Space" (arXiv preprint, 2024)—relative contrast of embeddings versus random vectors.

8.3 Deep Dive: families of ANN indexes

Three main approaches, like different index types for different profiles:

HNSW (Hierarchical Navigable Small World)—graph-based, the dominant default. It builds a multilayer proximity graph: the upper layers are sparse (long "jumps"), the bottom one holds every point (a dense graph). Search is a greedy descent: start at the top layer, hop to the nearest neighbor, descend layer by layer.



Tuning parameters: **M**—the number of links per node (higher M → higher recall and more memory); **ef_construction**—the size of the candidate list during construction (higher → a better graph, a longer build); **ef_search**—the list size at query time (higher → higher recall, higher latency). Profile: high recall and a fast query, but **heavy memory** (the graph lives in RAM) and a **slow build**.

IVF (Inverted File)—clustering-based. K-means partitions the space into `nlist` (Voronoi) cells; a query searches only the `nprobe` nearest cells rather than the whole corpus. Parameters: **nlist** (the number of cells), **nprobe** (how many cells to probe—more → higher recall, slower). Profile: a fast build, less memory than HNSW; recall/speed is tuned through nprobe.

PQ (Product Quantization)—compression. It splits a vector into sub-vectors and encodes each with a code from a small codebook, cutting memory sharply (vector → m bytes: the exact arithmetic and the mechanics of the speed are in the deep dive below). Often combined: **IVF-PQ** (clustering + compression) for billion scale. Profile: dramatically less memory, paid for in recall.

DiskANN and disk-based indexes—a graph on SSD for billions of vectors on a single node, when it all will not fit in RAM.

Index	Recall	Query latency	Build	Memory	When
HNSW	High	Low	Slow	High (RAM)	The default: quality + speed, memory available
IVFFlat	Medium-high (nprobe)	Medium	Fast	Medium	Fast build, memory savings
IVF-PQ	Medium (compression loses some)	Low	Medium	Very low	Billions of vectors, memory is expensive
DiskANN	High	Medium (disk)	Slow	Low (RAM)	Billions on a single node

Sources. Y. Malkov, D. Yashunin, "Efficient and robust ANN search using HNSW graphs" (2016/2018); H. Jégou et al., "Product Quantization for Nearest Neighbor Search" (2011); S. Subramanya et al., "DiskANN" (2019); J. Johnson et al., "Billion-scale similarity search with GPUs" (2017, FAISS).

The table's qualitative labels translate into orders of magnitude like this: at typical parameters HNSW holds recall@10 in the high 90s with millisecond search over millions of vectors; IVF shifts the balance toward less memory and greater sensitivity to parameters; PQ variants sacrifice recall for a multiple-fold saving. The specific numbers depend critically on the data and the parameters—take them from a benchmark on *your* vectors (methodology: ann-benchmarks) instead of carrying them over from someone else's table.

Deep Dive: HNSW mechanics from the inside (including the pain of deletion)

HNSW is the default, so it pays to understand *why* it is fast and where it hurts.

Small-world and layers. HNSW builds a graph with the "small-world" property: most nodes are connected by short paths through a small number of "long-range" links (as in "six degrees of separation"). Layering adds a hierarchy: a node lands on layer `l` with *exponentially decaying* probability, so the upper layers are sparse (few nodes, long jumps) and the bottom one is dense (every node). Search starts at the top (coarse navigation in big jumps) and descends, refining. The "small-world + hierarchy" combination gives **logarithmic** search complexity instead of linear—that is where the speed comes from.

Greedy search with backtracking (ef). On each layer, a greedy walk: move to the neighbor closest to the query. But pure greed gets stuck in local minima, so the search maintains a **dynamic candidate list of size** `ef` (a beam/backtracking, in effect): higher `ef_search` → a wider search, higher recall, higher latency. This is the main recall ↔ speed trade-off knob at query time.

Construction. Inserting a node: find its `M` nearest neighbors (with that same search) and link to them on every layer where it is present. Hence `ef_construction` (the quality of the neighbor search at insertion) and `M` (the graph's degree): higher means a better graph, but costlier memory and build.

Deletion—the real operational downside. HNSW **supports deletion poorly**: removing a node risks breaking connectivity (paths ran through it), which lowers recall. In practice deletion is **soft** (a tombstone: mark it deleted, filter at retrieval) with a **periodic full rebuild** of the index to purge physically. The consequence: where

data gets deleted or changed often (GDPR deletion requests, document rotation), the rebuild strategy is planned up front, on the understanding that a vector index is not a table you can cheaply delete a row from; deletion here takes a separate operation.

Source. Malkov & Yashunin (2016/2018); the pgvector/FAISS documentation on deletion limitations.

Deep Dive: the quantization spectrum (memory is often the main limit)

Memory for vectors is a common limit at scale; quantization cuts it, paying in recall. The spectrum, by aggressiveness:

- **Scalar quantization (SQ)**—a component as int8 instead of float32. ~4× savings, a small recall loss. The cheap default.
- **Product quantization (PQ)**—split the vector into sub-vectors, encode each with an index into a trained codebook. High compression (vector → m bytes at $k^* = 256$—96 bytes at $d = 1536$, $m = 96$; the arithmetic is in the deep dive below), a more noticeable recall loss; often paired as IVF-PQ for billions (§8.3). Rescoring candidates with full vectors (see the next item) applies to PQ as well—comparing methods is only fair when both are equally two-stage.
- **Binary quantization (BQ) + rescoring**—the most aggressive and the most fashionable (2024): a component → *1 bit* (its sign), similarity by Hamming distance. Up to ~32× savings and a very fast search. The precision loss is offset by **rescoring**: pull candidates with the binary vectors (coarse, cheap), then re-score the top-N with the *full* (float) vectors. The two-stage "coarse-binary → exact-full" gives nearly full recall on a fraction of the memory—conceptually the same move as retrieve-then-rerank (§7.4), but at the level of the vector's representation.

Method	Memory savings	Recall loss	When
Scalar (int8)	~4×	Small	The cheap default
Product (PQ)	High: $4d/m$ at $k^* = 256$ (64× at d=1536, m=96)	Noticeable	Billions of vectors
Binary + rescoring	~32×	Small (with rescoring)	Memory is the main limit, speed is needed

The conclusion: when memory (rather than latency or quality) becomes the constraint, quantization is the first lever; **binary + rescoring** today gives the best memory ↔ quality trade-off for many cases. Recall (§8.4) is measured at every compression level—that is the price of compression.

Source. Jégou et al. (2011, PQ); materials on binary quantization + rescoring in vector databases (2024).

Deep Dive: PQ mechanics from the inside (compression and ADC)

PQ is the main lever at billion scale, so it pays to be able to compute *how much* it actually saves, to understand why a compressed index turns out to be fast as well, and to know where it breaks silently. All three answers come out on paper, before any benchmark.

The arithmetic of compression. The vector is cut into m disjoint sub-vectors (d must be divisible by m). For each piece, *separately*, its own **codebook** is trained—a set of k^* typical representatives (centroids). The vector itself is then not stored: what is stored is m numbers—"which of the k^* stock shapes each piece resembles."

$$L = m \cdot \log_2 k^* \text{ bits}, \qquad \text{compression} = \frac{32d}{m \cdot \log_2 k^*} = \frac{4d}{m} \text{ (only when } k^* = 256)$$

- d—the embedding's dimension, typically 1536 (§1.3);
- m—how many pieces the vector is cut into; **this is the tuning knob**;
- $D^* = d/m$—the dimension of one piece;
- k^*—the codebook size for one piece; 256 by convention—and the short form $4d/m$ holds **only** at that value (why: below);
- L—the code length of one vector;
- 4—bytes per component of the original float32;
- 32 = 4 bytes × 8 bits—the conversion factor: it brings the original vector's bytes and the code's bits to a common unit.

Live numbers at $d = 1536$, $m = 96$: a piece holds $D^* = 16$ components; $\log_2 256 = 8$ bits—**exactly one byte**, and that is precisely why $k^* = 256$ became the convention: a centroid's number is addressed by a byte with no bit packing. The vector's code = 96 × 8 = 768 bits = **96 bytes**, against the 1536 × 4 = 6144 bytes of the original vector. Compression: 6144/96 = 64×.

The same place shows both where the short form comes from and where it stops. At $k^* = 256$ the factor $\log_2 k^*/8 = 1$ collapses, and $4d/m$ is all that remains of the general formula. Move k^* and it starts lying: at $k^* = 1024$ a piece costs $\log_2 1024 = 10$ bits = 1.25 bytes, the code = 96 × 1.25 = 120 bytes, compression = 6144/120 = 51.2×, not 64×. The short form would have overstated it by 25%. $4d/m$ is not a law of PQ but the arithmetic of one particular convention, "a centroid = exactly a byte"; outside it you compute with $32d/(m \log_2 k^*)$.

In plain terms: what is left of the vector is 96 bytes of pointers into a table of stock shapes—64 times less than it weighed. The knob works like this: larger m → a longer code → weaker compression, but a more accurate approximation and higher recall; smaller m → more aggressive and coarser. The codebooks themselves weigh $k^* \cdot d$ float32 = 256 × 1536 × 4 ≈ 1.5 MB per index—a one-time quantity, negligible across millions of vectors.

ADC: why PQ is not only small but fast. The §8.3 table gives PQ "low latency," and that is not a consequence of small size in itself—a separate mechanism is responsible for the speed: **ADC (asymmetric distance computation)**. It is "asymmetric" because only the database is encoded and the **query is not quantized**: a live query vector is compared against a document's code (which is also more accurate—quantization noise enters once, not twice). The scheme: once per query, a table is computed of the squared distances from each piece of the query to all the centroids of the corresponding codebook—$m \times k^* = 96 \times 256 = 24{,}576$ numbers at a cost of $k^* \cdot d = 256 \times 1536 \approx 393$ thousand operations. After that, the distance to **any** document is $m = 96$ table reads and additions; not a single multiplication over d. In the original's own words: "only m additions are required per distance calculation."

Live numbers on a corpus of $n = 10$ million: the scan costs $n \cdot m = 10^7 \times 96 = 960$ million additions, while building the table costs 393 thousand—0.04% of the work (the original also gives the break-even point: the table stops being noticeable once $n > k^* D^* = 256 \times 16 = 4096$). An exact brute-force scan on the same data is $n \cdot d = 10^7 \times 1536 \approx 15.4$ billion multiply-adds. So there are two wins at once: $d/m = 16$ times less arithmetic, and it is additions instead of multiplications; and 64 times less memory traffic (96 bytes against 6144 per vector). Since the scan is bound by memory bandwidth rather than by the ALU, the second is what decides: "small" and "fast" are one property of PQ, not two. A caveat from the same paper: what stops you from growing k^* past 256 is not only the lengthening code but also the volume of the centroids ($k^* \cdot d$ floats), which is efficient only as long as it fits in cache.

The failure mode: distribution drift. A codebook is not a constant of the algorithm but a model **trained on data**: m independent k-means runs over a *sample* of the corpus. The sample is small and hard-capped by default: FAISS trims the training set to `max_points_per_centroid` = 256 points per centroid, that is, to 256 × 256 =

65,536 vectors—however many millions sit in the corpus (and below `min_points_per_centroid` = 39 per centroid it emits a warning). The original paper puts it plainly: a codebook is "to some extent, still adapted to the data distribution to represent"—it is fitted to the distribution it was shown.

Hence a failure mode specific to PQ. A new document domain arrives—a different language, a different subject area—its vectors land far from every centroid, quantization error grows, recall drops. **And no error is raised at any point**: the index does not crash, queries answer, codes get written, the metrics are green. The only detector is measuring recall against brute-force ground truth (§8.4), and *on the new domain separately*, not on the pooled sample. IVF's cells are also built by k-means over a sample and also drift, but IVFFlat stores full vectors: distances inside the probed cells stay exact, and re-clustering needs nothing beyond what already sits in the index. With PQ the loss is baked into the stored representation itself.

The cost of the fix is the key point for a migration plan. Drift is cured by retraining the codebooks and **re-encoding the whole corpus**, but there is nothing to re-encode from: a PQ code is irreversible, the vector cannot be recovered from it—which is exactly why the index takes 96 bytes instead of 6144. So you need the original float vectors, which PQ exists in order not to store—meaning a fresh run of the embedder over the corpus. That is the same class of cost as switching the embedding model (§8.8).

The invariant. Choosing PQ means accepting not only the "64 times less memory ↔ lower recall" trade-off but also an obligation: either the original vectors sit in cold storage, or the plan carries a budget for re-embedding the entire corpus. The first is cheaper and has a second, rarely named bonus—those same vectors are what rescoring needs (above, in §8.3). And recall gets measured whenever a new domain arrives: distribution drift keeps quiet.

Sources. H. Jégou, M. Douze, C. Schmid, "Product Quantization for Nearest Neighbor Search" (IEEE TPAMI, 2011)—the split into m sub-vectors, $l = m \log_2 k^*$, "$k^* = 256$ and $m = 8$ are often a reasonable choice," ADC/SDC, Table II (a cost of $k^* D$ for the table, nm for the scan), codebook training by Lloyd's algorithm; the FAISS sources (`Clustering.h`: `min_points_per_centroid` = 39, `max_points_per_centroid` = 256).

8.4 Deep Dive: the universal trade-off

ANN has no free lunch—it has a trade-off surface between four quantities: **recall ↔ query latency ↔ memory ↔ build time**. You cannot maximize all of them: raising ef_search increases recall and latency; moving to PQ lowers memory and recall. Tuning ANN means picking a point on that surface for a specific load profile.

The key engineering move people so often skip: **measuring recall@k against brute-force ground truth.** Because ANN is approximate, you have to know *how much* recall was given up for speed. For that you assemble a small set of queries, compute the exact neighbors by brute force (offline, once), and measure what fraction of them the ANN index finds at the current parameters. Without this, tuning proceeds blind and the size of the loss is unknown.

8.5 Deep Dive: pgvector

pgvector is the pragmatic default: it stores vectors *in Postgres*. That buys transactions, JOINs against business tables, access control through the mechanisms you already have, and—above all—**no new datastore** until you actually need one.

- **Index types:** HNSW and IVFFlat (the same families from §8.3).

- **Parameters:** HNSW— `m` , `ef_construction` (at index creation), `hnsw.ef_search` (at query time, via SET); IVFFlat— `lists` (at creation), `ivfflat.probes` (at query time).
- **Distance operators:** `<=>` cosine, `<#>` (negative) inner product, `<->` Euclidean (L2). The operator you pick matches the similarity measure you chose (§1.3).

When pgvector is enough, and when to move out to a dedicated database (Qdrant/Weaviate/Milvus): pgvector covers the start and mid-scale beautifully, especially where the tie-in with relational data and filtering is valuable. Moving out makes sense at: very large scale (hundreds of millions and up), heavy load with hard latency requirements, or a need for the advanced capabilities of dedicated databases (complex filtering with ANN, sharding, built-in hybrid engines). The rule: **start with pgvector and move out on proven need.**

Source. The pgvector documentation.

8.6 Deep Dive: hybrid search infrastructure and the filtering pitfall

Hybrid (§7.3) is implemented inside one relational database: pgvector (dense) + Postgres full-text search (`tsvector` with a GIN index, sparse) + an RRF merge in SQL or in the application.

But there is an **important pitfall—how metadata filtering interacts with ANN.** You often need a search *with a filter*: "the nearest chunks, but only from sales-department documents, no older than a year." It naively looks trivial, but ANN and the filter conflict:

- **Post-filtering** (ANN top-k first, then filter by metadata)—can return *too few*: if 2 survive the filter out of the top-50 and you need 5, the result is short and the search has to be widened.
- **Pre-filtering** (select by metadata first, then search among the survivors)—"breaks" the HNSW graph: the traversal assumes the full graph, and on a subset connectivity is broken and recall drops; the alternative is a brute-force scan over the subset.



Real systems use **iterative / filtered ANN** (a graph traversal that skips nodes failing the filter and tops up candidates), and different vector databases implement this differently and with differing quality. A concrete reference point: pgvector 0.8.0 (October 2024) formalized it as `hnsw.iterative_scan` (modes strict_order / relaxed_order) with a top-up budget of `hnsw.max_scan_tuples` —filtered ANN went from "implicit behavior nobody talks about" to a tunable parameter with documented semantics. The practical conclusion: **filtering with ANN is neither free nor trivial**; where the filter is highly selective, recall (§8.4) gets checked precisely *with the filter on*, not only on a clean search.

8.7 Deep Dive: RBAC/ABAC at retrieval (a bridge to security)

A critical and underrated topic. The index holds chunks from documents at *different clearance levels*, and a query is obliged to return only what a particular user is *allowed* to see. Skip this and you have an access-control breach at retrieval: the agent will retrieve and display (or use in its answer) a confidential chunk the user has no rights to.

Approaches:

- **Filtering on an ACL in the metadata**—attach an owner/clearance/access list to every chunk and filter at query time by the user's rights. Simple, but see §8.6 (filter + ANN).
- **Postgres Row-Level Security (RLS)**—RLS policies on the chunk table: the user physically does not see rows outside their rights. Powerful and dependable, because access is enforced at the database level rather than in application code (which you can forget to write).
- **An index per tenant/level**—separate indexes/collections for isolated groups (hard isolation, paid for by duplicated infrastructure).

Integration with the access-management system: the user's roles and rights from the token map onto a filter predicate or an RLS policy. The key invariant: **access control is applied at retrieval, not at generation**—you cannot rely on the model to "just not show" what is off-limits: the model is not a security mechanism.

8.8 Operational concerns

- **Reindexing when the embedding model changes.** Vectors from different models are not comparable (§1.3)—switching the embedder means a *full* reindex of the corpus. This goes into the migration plan as an expensive operation.
- **Build cost and incremental updates.** HNSW builds slowly; plan for incremental insertion and rebuilds.
- **Freshness and staleness.** A pipeline that keeps the index in agreement with the source (§6.6, the "staleness" mode).
- **Capacity planning.** Memory for the HNSW graph and for the vectors themselves is a new consumer of RAM, counted when you size capacity; the arithmetic is below.

Deep Dive: capacity planning in bytes

The threshold for moving out of pgvector in §8.5 is stated in rows—"hundreds of millions." In rows it is deceptive: you pay in bytes, and you can hit a node's ceiling two orders of magnitude earlier. This is napkin arithmetic, done before you buy hardware.

What a vector weighs. pgvector stores the `vector` type densely, without compression:

$$S_{vec} = 4d + 8$$

- S_{vec}—bytes per vector.
- d—the embedding's dimension (§1.3), a count: 1536 for the common OpenAI models (text-embedding-3-small, ada-002), 1024 for BGE-M3.
- 4—bytes per component: each number is stored as `float32`.
- 8—bytes of the value's header (length and dimension).

At $d = 1536$: $4 \cdot 1536 + 8 = 6152$ bytes. **In plain terms:** one chunk costs ~6 KB before any index at all.

What the graph weighs. Malkov and Yashunin give the average memory per element directly:

$$S_{graph} = (M_{max0} + m_L \cdot M_{max}) \cdot b$$

- M_{max0}—the cap on links for a point on layer zero (the dense one, where *all* the points live, §8.3). The authors recommend $2M$, that is, 32 at $M = 16$ (in pgvector this parameter is spelled with a lowercase `m`, default 16).
- M_{max}—the same for the upper, sparse layers; equal to $M = 16$.
- m_L—the factor that normalizes the level draw for a new point: the level is drawn as $\lfloor -\ln(U) \cdot m_L \rfloor$, where U is uniform on (0,1). The authors recommend $1/\ln M = 1/\ln 16 \approx 0.36$. It is the scale of the draw, and only that: m_L is not the number of layers per point. Layers thin out exponentially—$e^{-1/m_L} = \sim 1/16$ of the points reach layer 1, ~1/256 reach layer 2—so nearly the whole mass of links sits on layer zero.
- b—bytes per link. For the authors it is 4: they cap the index at roughly four billion elements and store a neighbor as a 32-bit number. **In pgvector the figure is different**—there a link is written as an `ItemPointerData`, Postgres's standard pointer to a row ("a page number plus an offset within it"), and that takes 6 bytes. Below, the authors' arithmetic is reproduced first at $b = 4$, then recomputed for pgvector at $b = 6$.

Substituting at $M = 16$: $(32 + 0.36 \cdot 16) \cdot 4 = 37.8 \cdot 4 \approx 151$ bytes. A convergence check: across the whole working range $M = 6$–48 the formula gives 61–434 bytes—which is exactly the estimate the authors published, "about 60–450 bytes per object, *not counting the data itself*." Of those ~38 links per point, layer zero holds 32, and the entire superstructure above it holds $0.36 \cdot 16 \approx 6$.

A caveat about the upper term. It is overstated. The average number of upper layers per point is the sum of the probabilities of landing on layer 1, on layer 2, on layer 3, and so on: $\sum_{k \geq 1} 16^{-k} = 1/15 \approx 0.067$—that is, ~1 link from above, not 5.8. The figure agrees with the authors' own formulation for the choice $m_L = 1/\ln M$—"average single element overlap between the layers," an overlap of one element between layers. The original's arithmetic is nevertheless kept here: it gives an upper bound—for capacity planning an error in that direction is safe—and it reproduces the published range. Anyone computing more precisely takes ~33 links instead of ~38: the graph comes out ~12% lighter, which does not affect the section's conclusion.

Recomputed for pgvector. The same ~38 links, but at 6 bytes each: $(32 + 0.36 \cdot 16) \cdot 6 \approx 227$ bytes per element. That is the figure used in the arithmetic from here on. **In plain terms:** the graph is roughly 38 neighbor references per point, and it does not depend on d at all.

Hence the section's main conclusion. 227 bytes against 6152—the graph is ~3.7% of the footprint. Which means `m` **is not a memory lever**: doubling it costs another ~3.5% of RAM and pays for itself in recall; cutting it to save memory is pointless. Memory is the *vectors*, and there are exactly two levers against them, both already named. The first is quantization (§8.3): `halfvec` stores a component in two bytes instead of four ($2d + 8 = 3080$ B, ~2×), and `binary_quantize` into the `bit` type stores it in one bit ($d/8 + 8 = 200$ B, ~30×, with mandatory rescoring of candidates against full vectors). The second is trimming the dimension (§7.6, Matryoshka): 1536 → 768 halves the bill with no retraining of the embedder and no reindexing from another source.

The whole corpus. $N = 10$ million chunks, $d = 1536$, $m = 16$: vectors $10^7 \cdot 6152$ B ≈ 61 GB, graph $10^7 \cdot 227$ B ≈ 2.3 GB.

And that is a lower bound—in pgvector it is understated by roughly a factor of two. The HNSW index does not reference the table row; it keeps its **own full copy** of each vector inside its element (the `Vector data` field in the `HnswElementTupleData` struct, `src/hnsw.h`). So the table is ≈61 GB *and* the index is ≈63 GB on top of it. Storage granularity piles on: a Postgres page is 8 KB, an index element at $d = 1536$ is over 6 KB, two will not fit in a page, so in effect a whole page goes on each vector. The result: on the order of an estimated 130–150 GB instead of the naive 61; the exact figure you take from `pg_relation_size` on your own data rather than from this back-of-the-

envelope (the same discipline as with recall, §8.4). That same 8-kilobyte limit also explains the 2000-dimension ceiling on `vector` for HNSW ($4 \cdot 2000 + 8 = 8008$ B—right up against the page): $d = 3072$ (text-embedding-3-large) cannot be indexed as `vector` at all and requires `halfvec` (whose ceiling is 4000).

The invariant: capacity here is a cliff, not a slope. An HNSW traversal is random access across a graph: dozens of hops per query into unpredictable places in the index. While everything sits in RAM, a hop costs on the order of 100 ns and the search fits inside milliseconds (§8.3). The moment the vectors plus the graph stop fitting in the node's RAM, every cache miss turns into an SSD read—on the order of 100 µs, a thousand times more expensive—and a hundred hops then gives an estimated tens of milliseconds instead of one. The degradation is not gradual: the index does not "slow down a bit," it falls through by an order of magnitude or more, all at once, out of nowhere. The practical consequence: 10 million chunks is fifty times short of the "hundreds of millions" from §8.5, and yet a 64-gigabyte node is already finished on them. Plan in bytes, and take the decision from §8.3: either shrink the vector (quantization, Matryoshka) or take an index designed to live on disk (DiskANN)—HNSW was never built for it.

Sources. Y. Malkov, D. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs" (2016/2018)—the graph memory formula and "60–450 bytes per object" (§4.2.3), the four-byte link in the same place, the recommendations $M_{max0} = 2M$ and $m_L = 1/\ln M$ with "average single element overlap" (§4.1), the level draw (alg. 1). The pgvector documentation and source code (`src/hnsw.h`)—the sizes of the `vector` / `halfvec` / `bit` types, storing links as `ItemPointerData` (`HNSW_NEIGHBOR_TUPLE_SIZE`), the default value of `m` (`HNSW_DEFAULT_M` = 16), the 2000-dimension limit (`HNSW_MAX_DIM`). The PostgreSQL source code (`src/include/storage/itemptr.h`)—the size of `ItemPointerData`: "designed to be six bytes long."

8.9 Chapter summary

Vector search is a special kind of index; ANN is *approximate* by construction and sits on a trade-off surface of recall ↔ latency ↔ memory ↔ build (§8.4), tunable to the load. HNSW is the default (quality and speed, paid for in memory), pgvector the pragmatic store to start with. Mandatory in production: hybrid, metadata filtering (accounting for its conflict with ANN, §8.6), and **access control at retrieval** (§8.7). Recall is measured against ground truth, filters included.

Sources. Malkov & Yashunin (2018, HNSW); Jégou et al. (2011, PQ); Johnson et al. (2017, FAISS); the pgvector documentation; the survey by Gao et al. (2023).

Chapter 9. Evaluating RAG Quality

Chapter 6 established it: retrieval is the silent bottleneck, and quality is two orthogonal axes. This chapter is about how to *measure* them, because fixing RAG without measuring (§6.6) is done blind. At bottom this is regression-testing and SLO discipline applied to a probabilistic system.

9.1 Why measure: eval-driven RAG

Naive RAG misses silently on a noticeable share of queries (§6.5), and a confident wrong answer is indistinguishable by eye from a correct one. The only way to manage quality is to **measure it as a metric**. Hence eval-driven RAG development: pin down what "good" means as measurable metrics, measure a baseline, and iterate against the metrics. Since quality is two axes (§6.7), you measure *both separately*—otherwise it is unclear *what* to fix.

9.2 Deep Dive: a two-axis metric framework

We measure retrieval and generation separately (this is essentially the RAGAS framework):

The retrieval axis—"did we pull the right thing?"

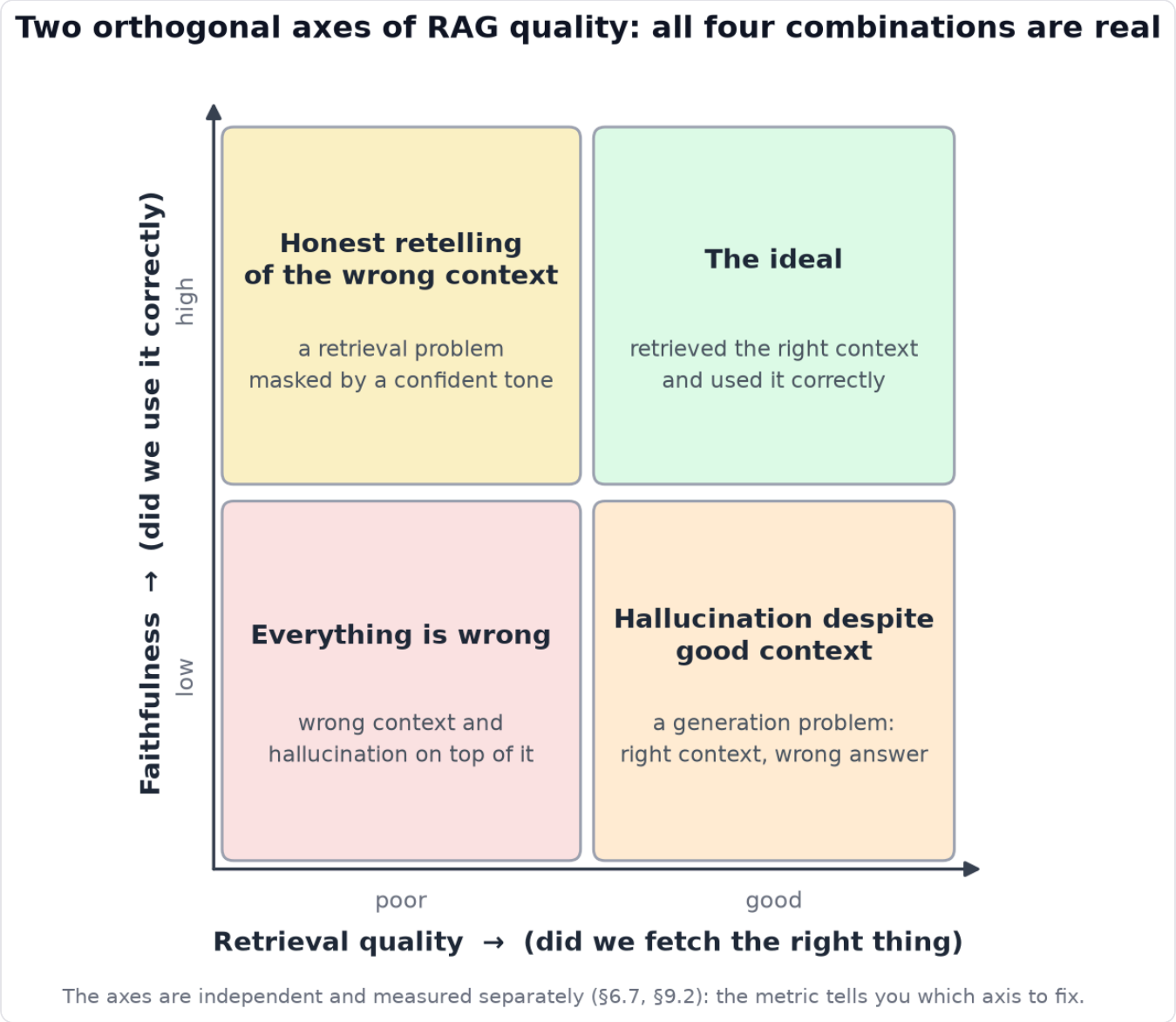
- **Context Recall**—the share of the relevant information that retrieval actually *pulled* (did we miss what was needed). Low → a recall problem (§7.3, §7.5).
- **Context Precision**—is what we pulled relevant, and is it well ranked (did we drag in noise, is the needed thing buried). Low → you need a reranker/filters (§7.4).
- **Classic IR metrics** (when you have labeled relevance judgments): **Hit Rate** (did we find at least one relevant item in the top-k), **MRR** (how high the first relevant one sits), **NDCG**° (the quality of the whole ranking).

The generation axis—"did we use it correctly?"

- **Faithfulness / Groundedness**—does *every claim* in the answer rest on the retrieved context, with nothing invented beyond it. Low → the model hallucinates despite good context (a generation problem, §6.7).
- **Answer Relevancy**—does the answer address *the question asked* (and not a neighboring one).
- **Answer Correctness**—does it match the reference (when a reference exists).

Axis	Metric	What it diagnoses	Where to fix
Retrieval	Context Recall	Missed something relevant	Hybrid, query transform, chunking
Retrieval	Context Precision	Noise / bad ranking	Reranking, filters
Generation	Faithfulness	Hallucination beyond the context	Prompt, citations, temperature
Generation	Answer Relevancy	Answer off the question	Prompt, query rewriting

Keeping them separate is exactly what gives you a diagnosis: low Context Recall with high Faithfulness means "the model honestly restated an incomplete context" → fix *retrieval*, not generation.



Deep Dive: how the metrics are actually computed

"Faithfulness" and "context precision" sound abstract; knowing the *algorithm* is what lets you trust the number and debug it. Here is how RAGAS (and its analogues) does it:

- **Faithfulness.** (1) Break the answer into atomic *claims*—with a separate LLM call. (2) For each one, ask the judge: "does this follow from the retrieved context?" (yes/no). (3) `faithfulness = (confirmed claims) / (total claims)`. The metric is the share of the answer that actually rests on the context. Debugging: you can see *exactly which* claims went unconfirmed.
- **Answer Relevancy.** (1) From the generated answer an LLM works *backward* and produces several questions this answer would be answering. (2) The similarity of those questions to the *original* one gets computed (embeddings). (3) High similarity → the answer is on point; low → it drifted off. Catches "answered the wrong question" without a reference.
- **Context Precision.** Looks at the *ranking*: do the relevant chunks sit *above* the irrelevant ones in the retrieved list (going by the positions of the relevant ones). Low → you need a reranker (§7.4).

A faithfulness micro-example. The answer: “A return is filed within 14 days; for electronics the window is 30 days; the courier comes free of charge.” The decomposition: (1) “a return—14 days”—present in the context ✓; (2) “for electronics—30 days”—present ✓; (3) “courier free of charge”—absent from the context ✗. Faithfulness = 2/3 ≈ 0.67, and you can see immediately *which* claim was invented—that is exactly what separates a diagnosable metric from eyeballing it.

- **Context Recall.** Requires a reference answer: every claim in the *reference* is checked for *attributability* to the retrieved context. `recall = (reference claims covered by the context) / (total reference claims)`. Low → retrieval missed what was needed (§7.3/7.5).

The key observation: almost all of these metrics *use an LLM themselves* (to break text into claims and to check entailment)—which makes them LLM-as-judge under the hood (§9.4), with every calibration requirement that carries. So “RAGAS = 0.82” is not an absolute but a *measurement made by a measuring instrument*, and it has to be validated (see below).

Source. S. Es et al., “RAGAS” (2023).

9.3 Deep Dive: the golden set

The foundation of any evaluation is a **golden set** (golden dataset): a curated collection of tuples `(question, relevant context, reference answer)`. It is the regression test suite for RAG; without it, evaluation is impossible.

How to build it:

- **From real queries**—collect actual user questions (the most valuable source—it reflects the real load).
- **Expert labeling**—an SME marks up which context is relevant and what the ideal answer is.
- **LLM generation + human review**—have the model generate questions from the documents, then validate them with a human, *without exception* (otherwise the eval inherits the generator’s errors).

Requirements: **coverage** (easy factoids *and* hard multi-step ones, edge cases, questions with no answer in the corpus—to check that the system admits “I don’t know” instead of inventing); enough **size** for statistical significance, though the quality of the examples matters more than their count. A golden set is versioned and grows along with the system. The external reference point for the retrieval side is **BEIR** (Thakur et al., 2021), a heterogeneous IR benchmark: useful for sanity-checking baseline retrieval quality before you have domain labels, but it does not replace a golden set of your own.

9.4 Deep Dive: LLM-as-judge for RAG

Faithfulness and relevancy are semantic properties; a boolean assert cannot check them. The answer is **LLM-as-judge** (in detail in Chapter 17): hand the judge the question, the context, the answer, and a *rubric*, and ask for a score.

- **Faithfulness through a judge:** break the answer into claims and ask of each one, “is this confirmed by the retrieved context?”—the share confirmed is the faithfulness. This operationalizes “did it make something up.”
- **Answer Relevancy through a judge:** score how well the answer addresses the question actually asked.

- **Reference-free vs reference-based:** faithfulness/relevancy can be measured *without a reference* (the judge sees only question + context + answer)—which is convenient, and a reference is not always needed. Correctness requires one.
- **Calibrating the judge is mandatory** (Chapter 17): explicit scoring steps, a strict mode for pass/fail, and validation against human labels on a subsample—otherwise you are trusting an uncalibrated instrument.

In production: light/cheap metrics on 100% of traffic + an expensive LLM judge on a sample (Chapter 17), so you are not paying to evaluate every request.

Deep Dive: statistical rigor and validating the judge

An eval is a measurement, and a measurement has error. Academic honesty demands you reckon with it.

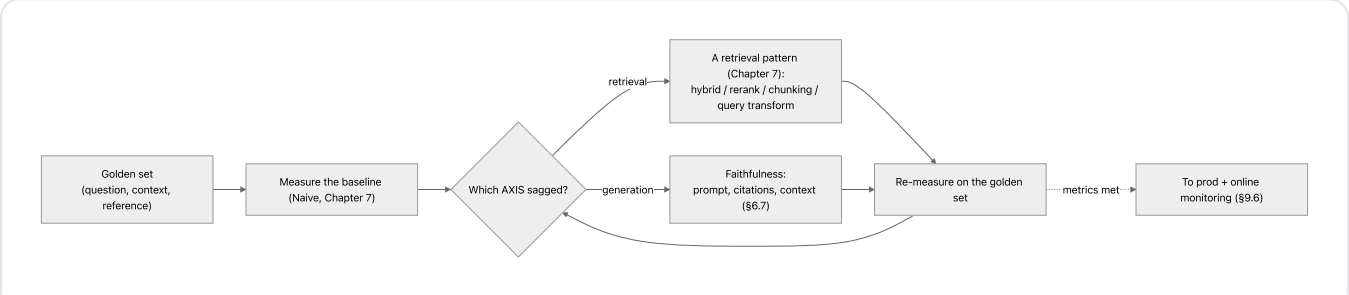
- **The variance of an LLM judge.** The judge is nondeterministic (§2.4) and noisy: the same answer can draw a different score from run to run. Mitigation: average over several runs for decisions that matter; pin the judge to a low temperature; prefer binary/discrete verdicts over continuous ones (they hold up better).
- **Sample size and significance.** A metric moving from 0.80 to 0.83 on 30 examples is probably noise. To tell a real improvement from a chance one you need a golden set of sufficient size and, ideally, **confidence intervals** or a significance test on the difference. The rule: an improvement that fits inside the spread does not count as an improvement. This is standard statistical-significance discipline applied to evaluation.
- **Validating the judge against humans (eval the evaluator).** Before you trust an LLM judge, you have to show that it agrees with a human: label a subsample by hand and compute **agreement** (Cohen's kappa°, for instance) or the correlation "judge ↔ humans." Low agreement means the judge's rubric needs fixing, not that its numbers deserve trust. Skip this step and the SLO rests on an uncalibrated instrument.
- **Regression mode.** A golden set with thresholds in a CI gate (§9.5) catches *regressions* when the prompt, the model, or the embedder changes (§2.4); pin the baseline and set an alert on a drop.

The conclusion: treat evaluation as a measuring system that has error—average out the judge's noise, check the significance of differences, and *validate the judge itself against humans* before building decisions on its numbers. Otherwise "faithfulness 0.9" is a number with no confidence interval and no calibrated instrument behind it.

Source. L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (2023); the classical statistics of rater agreement.

9.5 Deep Dive: the eval-driven RAG loop (ties Chapters 6–9 together)

Here is the working loop that stitches all of Part III together:



This operationalizes the "ladder" (§7.1): you climb the retrieval patterns **not at random but on the strength of a measurement**—the metric tells you *which axis* sagged and therefore *which* pattern to apply. That is exactly how the principle "start simple, add complexity on proven need" turns from a slogan into a process.

9.6 Online evaluation and monitoring

Offline evals (a golden set as a CI gate before deploy) are necessary but not sufficient. In production, add **online evaluation**: sample real traffic, run the judge over it, track **quality drift** over time (the corpus changes, queries drift, the model gets updated—§2.4). The signals: faithfulness dropping, "not found" rising, anomalies in retrieval latency. This is the bridge to observability (Chapter 18): tracing the RAG pipeline + quality metrics on a dashboard.

9.7 Summary of the chapter and of Part III

Evaluating RAG means measuring **the two orthogonal axes separately** (retrieval quality and faithfulness) on a **golden set** with an **LLM judge**, and using the measurement to decide *which pattern* (Chapter 7) to apply. That closes Part III: **RAG = ground it (Chapter 6) → improve it with patterns (Chapter 7) on the right infrastructure (Chapter 8) → guided by measurement (Chapter 9), iteratively.** Format was settled in Part II; knowledge and its correctness, in Part III. Next comes Part IV: how to make this grounded component *act*, by applying the patterns of orchestration and reliability.

Sources. S. Es et al., "RAGAS: Automated Evaluation of Retrieval Augmented Generation" (2023); the Y. Gao et al. survey (2023); Chip Huyen, *AI Engineering* (2025); the classic IR metrics (MRR, NDCG, recall@k).

In the Margins: Chapter 9

> ↩ **NDCG**—Normalized Discounted Cumulative Gain: a relevant document's contribution is discounted by the logarithm of its position, and the sum is normalized against the ideal ordering (1.0 = the best possible). Penalizes "the right thing is there, but at the bottom of the list."
>
> ↩ **Cohen's kappa**—the agreement of two raters, corrected for chance matches: $\kappa = (p_o - p_e)/(1 - p_e)$. Fairer than a raw percentage of matches, especially when the class distribution is skewed.

Part IV. Action: Agents and Orchestration

Parts I–III turned the model into a reliable, grounded component. Part IV makes it *act*. The central thesis: an agent system is a distributed system in which the control logic of one node is a probabilistic model. So the familiar distributed-systems patterns apply to it directly—orchestration and choreography, compensations, idempotency, tolerance of partial failures—augmented by two new dimensions: **nondeterministic control flow** and **nontermination**. Five chapters: the agent as a distributed system (10), a catalog of agent patterns (11), tool calling as a contract (12), memory and reasoning strategies (13), multi-agent systems and a taxonomy of failures (14).

Chapter 10. The Agent as a Distributed System

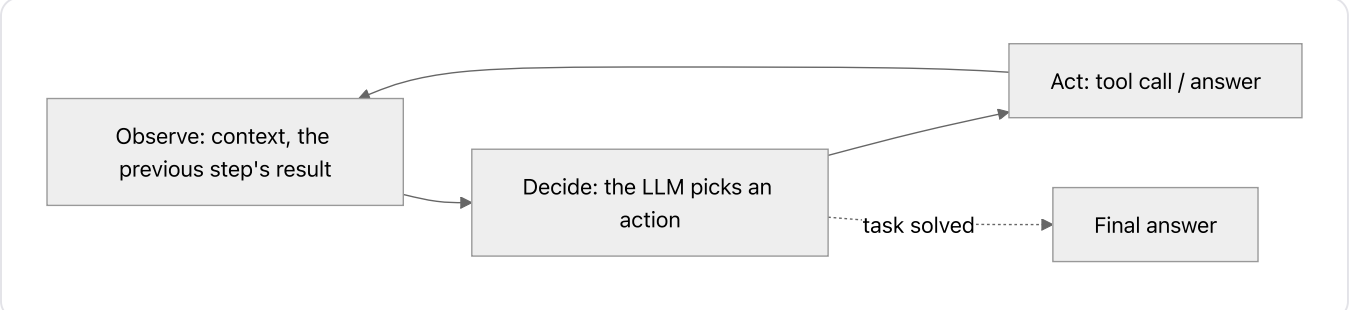
10.1 What an agent is

Agent—a system in which a language model serves as the decision-making core of an **observe → decide → act** loop and holds the autonomy to choose which actions to take and when to stop. It stands opposed to a **workflow**—a system whose sequence of steps is fixed in advance by a graph: the model is called at individual steps but does not decide what the next one will be.

The distinction is not binary; it forms a spectrum of autonomy—from "the model fills in one step of a fixed graph" to "the model determines the entire sequence of actions on its own." What matters about an agent system is that it is **a distributed system in which the control logic of one node is nondeterministic**. It follows that the known results about distributed systems apply to it—partial failures, coordination, state consistency—while what is specifically new is that the control flow is probabilistic and not laid down statically.

10.2 Augmented LLM—the basic building block

The basic building block of an agent is the **augmented LLM**: a model extended with tools (tool calling, Chapter 12), memory (Chapter 13), and retrieval (Part III). The control loop spins on top of it:



This is the ReAct loop (in detail, Chapter 13): reasoning interleaves with action until the task is solved or the budget runs out. Note that, unlike a workflow, *the number and order of iterations are not fixed in advance*—the model chooses them.

Who executes this loop is not a rhetorical question. The model itself is a pure function "tokens → tokens": it executes nothing, holds no state between calls, and cannot stop its own loop. All of that is done by the **harness** (agent harness; also host application, runtime)—the *deterministic* application code that assembles the context and feeds it to the model, parses the returned tool calls and executes them, carries state across iterations, and imposes budgets and timeouts (§10.5). The agent of §10.1's definition is always a pair: **the LLM as the nondeterministic decision-making core + the harness as the deterministic execution scaffolding**. The split carries architectural meaning: everything that must be guaranteed—authorization, limits, logging, approval gates (§10.6)—lives in the harness, because a probabilistic core cannot be trusted with guarantees.

10.3 Workflow vs agent—the fundamental decision

The first and foremost architectural decision of this part. The trade-off:

	Workflow (the graph is fixed in advance)	Agent (the model decides)
Predictability	High	Low
Cost/latency	Lower, controllable	Higher (many calls), variable
Reliability/debugging	Simpler	Harder
Flexibility	Low	High

A practical principle: prefer a workflow until the task genuinely needs an agent's dynamism. An observation from practice: most systems called agents are in fact workflows with one or two points of autonomy. Full autonomy is justified only on proven need, because every extra degree of autonomy lowers predictability and raises cost and risk. The "autonomy ↔ reliability" trade-off runs through the entire part.

10.4 Deep Dive: the agent as a distributed system

An agent system has the properties and failure modes of a distributed system, plus a nondeterministic decision node. The mapping is direct:

Distributed-systems property	How it shows up in an agent	Standard mechanism to address it
Partial failures	A tool call failed and the loop carried on	Explicitly check the result of every action
Retries and idempotency	A step repeats; the side effect must not be duplicated	Idempotent operations under an idempotency key
Timeouts	A tool or the model does not respond	A per-step timeout + a fallback path (§2.5)
State management	Working state between steps	External state storage, compaction (§4)
Coordination	Several agents or workers (Chapter 14)	Orchestration or choreography
Observability	Reconstructing the sequence of steps and decisions	Distributed tracing across steps (Chapter 18)
Compensations	Rolling back a multi-step process on failure	Compensating operations (the saga pattern)

What is fundamentally new compared with a classic distributed system is **the nondeterminism of the control flow itself**. In a deterministic system the transition graph is laid down statically, and you can reason about it formally: prove termination, enumerate reachable states. In an agent the next step is chosen by the model at runtime—no static graph exists. Hence two classes of problem that deterministic systems do not have: the control loop may fail to terminate (§10.5), and actions are executed by a nondeterministic component that may err or be manipulated (§10.6). The remaining failure modes, and the methods for addressing them, coincide with those common to distributed systems.

10.5 Deep Dive: the control loop and nontermination

In a deterministic system whose transitions follow a fixed graph, termination is guaranteed by construction. The agent loop offers no such guarantee: the model can loop (repeating the same action), drift from the original goal over a long trajectory, or refine a solution endlessly without converging. This is a failure class specific to systems with nondeterministic control.

The remedies are external limits on the loop. Conceptually they are the same limits applied to any repeated operation involving external calls (a retry cap and a timeout), but here they act at the level of the decision loop rather than the individual call:

- **An iteration limit and a step budget**—a hard upper bound on the number of cycles.
- **A cost or token budget**—stop when it runs out (§2.2).
- **A progress check**—detect the absence of progress (passing through the same states again) and abort or escalate. The mechanics when state is free text: compare action signatures—the tool name plus a hash of the normalized arguments—for an exact repeat; a softer heuristic is high embedding similarity across consecutive observations.
- **A timeout on the task as a whole**, not just on an individual step.

- **Escalation to an operator** when the budget runs out, instead of a silent failure or an endless loop.

A hierarchy of budgets: which limit must fire first. The limits above are not independent fuses but an ordered system, and the order comes from arithmetic, not taste. The notation runs through the whole section:

- N_{max}—the iteration limit, count. The order of magnitude for a ReAct loop with tools is tens, not hundreds; there is exactly one hard reference point in vendor documentation: LangChain's `AgentExecutor` defaults to `max_iterations = 15` (this is the classic `AgentExecutor`—it lives in the `langchain-classic` package: LangChain 1.x moved the agent part over to LangGraph, so the current documentation does not carry this value under the old name).
- t_{model}—the time of one model call within an iteration, seconds; on the order of a few seconds.
- t_{step}—the tool's execution time within an iteration, seconds; bounded above by the per-step timeout.
- T_{task}—the timeout on the task as a whole, seconds.
- k—the number of iterations the loop has run, count; here and to the end of the section, k counts iterations and nothing else.
- Δ—how many observation tokens one step adds to the context, tokens; for a retrieval pass, thousands (in the example in §7.8, about 2000).
- k_{rep}—how many consecutive *identical* action signatures the detector requires before declaring a loop, count; the value is derived below, typically 3.
- m—how many consecutive observations the embedding heuristic compares, count; typically 3.

One iteration takes $t_{model} + t_{step}$ of wall-clock time. Hence a two-sided condition on T_{task}:

$$N_{max} \times (t_{model} + t_{step})_{\text{typ}} \;<\; T_{task} \;<\; N_{max} \times (t_{model} + t_{step})_{\max}$$

In plain terms: the task timeout must be large enough for a healthy loop to spend its step limit, and small enough that a pathologically slow loop gets killed before it counts up to that limit.

Live numbers. Let $N_{max} = 20$; at typical latencies $t_{model} \approx 2$ s and $t_{step} \approx 1$ s an iteration takes 3 s, while in the worst case $t_{model} \approx 10$ s and t_{step} runs into the per-step timeout of 30 s—40 s in total. The bounds: $20 \times 3 = 60$ s and $20 \times 40 = 800$ s, so T_{task} is chosen inside the interval (60 s, 800 s)—300 s, for example. Both violations, checked. At $T_{task} = 900$ s even the worst-case loop reaches N_{max} in 800 s—the timeout is unreachable, a dead setting, and the iteration counter is left as the only real limiter. At $T_{task} = 45$ s it is N_{max} that becomes the dead setting: the loop is amputated by the clock without ever reaching its step limit, and a healthy long task stops being distinguishable from a looping one.

The roles in this system divide up as follows: N_{max} is the **routine** limiter—it fires in the expected degenerate case (the model circling), gives a cheap, unambiguous diagnosis, "out of steps," and does not depend on latencies; T_{task} is the **emergency kill**. The 800-s ceiling and the verdict that "900 s is a dead setting" hold only while every step stays within the per-step timeout; on exactly that assumption T_{task} looks redundant. Its real job is where the assumption breaks and a single step breaches the per-step bound: a model call with no timeout, a tool that ignores cancellation, a hung external call. Such a step does not fit inside 800 s, the iteration counter freezes on it and cannot see it at all—and only T_{task} finishes the task off. How easy it is to end up with no kill is shown by that same vendor documentation: `max_iterations = 15` with `max_execution_time = None`—the step counter is set, the clock is off.

The fourth limit—the token budget—behaves unlike all the others: it alone grows **superlinearly** with the number of iterations, because every iteration feeds the model the whole accumulated context (the same arithmetic as in §7.8). If a step adds Δ observation tokens, the total input across k iterations is $\Delta \times k(k+1)/2$, that is, $O(k^2)$. At $\Delta \approx$ 2000 tokens (the order of magnitude from the example in §7.8): 20 iterations—$2000 \times 210 = 420\,000$ input

tokens; 40 iterations—2000 × 820 = 1 640 000, almost four times as much for a doubled limit. The practical consequence: the cost budget can bind before N_{max} does, and raising the iteration limit "just in case" gets quadratically more expensive.

The signature detector, failure mode one: a false positive. Signature comparison has two failure modes, and both get closed explicitly. The false positive: legitimate status polling—`check_status(order_id=42)` while waiting for the transition to `SHIPPED`—produces a byte-identical signature on every iteration and gets killed as "stuck" while working exactly as designed. Two remedies. The first: require not one repeat but k_{rep} consecutive identical signatures—a repeat counter, not to be confused with the iteration counter k. The value of k_{rep} is derived, not assigned: the detector spends k_{rep} iterations of the budget confirming a loop, so the sensible range is $k_{rep}/N_{max} \lesssim 0.15$; at $N_{max} = 20$ that gives $k_{rep} = 3$ (15% of the budget on the diagnosis), whereas $k_{rep} = 10$ all but nullifies early detection: confirming a loop eats 10 of the 20 iterations, so any gain over N_{max} survives only for loops that begin in the first half of the budget—the value of early detection falls in proportion to k_{rep}. The second: mark idempotent reads and polls in the tool's metadata (the slot already exists, §12.5)—for those, a repeated signature is normal, not a symptom. Polling exempted from the detector is not left unsupervised: N_{max} and T_{task} still hold it—but precisely as fuses, not as a diagnosis.

Failure mode two: a false negative. The model really is stuck, but it rephrases the arguments on every turn: `"yesterday"` → `"2026-07-15"` → `"July 15"`. Semantically this is one and the same call, but the hash yields three different values—an exact-repeat detector sees nothing. Normalizing the arguments (dates to ISO, case, key order) pushes the boundary out but does not close it: you can rephrase things the normalizer knows nothing about, too. This is exactly the case the embedding heuristic is for—it compares the meaning of the observations rather than their bytes: it fires when cosine similarity (§1.3) stays above a threshold across the last m consecutive observations ($m = 3$—by the same budget-share arithmetic as k_{rep}).

You cannot take the cosine threshold off the shelf. The absolute value of the threshold does not carry across embedders, and that is geometry, not a matter of taste: a transformer's representation spaces are **anisotropic**—the vectors occupy a narrow cone, so the average cosine similarity of *random* pairs is far from zero (Ethayarajh, 2019: in GPT-2's last layer the anisotropy is such that two random words give, on average, "almost perfect" cosine similarity). Contrastive training of the embedder (§1.3) weakens the effect but does not remove the model-specific character of the scale. The working recipe is the one Ethayarajh proposed for measurement: measure not the raw similarity but the excess over your own embedder's **baseline**—over the average similarity of observations known to be different, taken from your own traces (Chapter 18). A threshold of the form "cos > 0.95" means something only after such a calibration: where random pairs average 0.1, it means "practically the same thing"; where the baseline is 0.8, it means merely "a little closer than two arbitrary texts."

Hence the order in which to apply them—by price, not by coverage. Signatures are computed locally, cost one hash per step, and catch the dumb case—turn them on always, and first. Embeddings catch the paraphrasing case but add one embedder call per agent step—turn them on when the traces (Chapter 18) show real loops that slipped past the signatures. And a subtlety: the similarity heuristic is a probabilistic detector on top of a probabilistic loop, so its verdict is good enough for escalation but not for silently killing the task; N_{max} and T_{task} remain the hard, deterministic ones.

What the harness does at the moment of exhaustion. "Escalation to an operator" as a one-line bullet hides the main thing: when the budget runs out, the trajectory is **half executed**. An agent stopped at step twenty of twenty has already created the request, already reserved the goods, and possibly already charged the money. To cut the loop off silently is to leave those side effects without an owner: an unfinished saga with no coordinator. So budget exhaustion is not a `return` but the same rollback the harness runs for a failed step: walk back through the **step log**—that same saga log the orchestrator keeps, with a defined compensation for every step (Preface)—run the compensations for the reversible actions, and route the irreversible ones into a **decision card for a human** (§10.6). A compensation is itself a network call and can fail: it is idempotent and retried with backoff (§2.5), and

one that cannot be recovered escalates into the same decision card for a human as the irreversible actions. The distinction comes from the same place: the reversible is undone by a compensating operation, the irreversible is not compensated at all, so a human decides on it. Hence a requirement on the harness that is cheaper to build in up front than to add after an incident: the step log is kept from the first iteration, and a compensation is defined for every action *at the moment it executes*, rather than reconstructed after the fact from the trace. Once the budget is gone, it is too late to work out what exactly the agent got up to.

The essential point: because the transition graph is formed by the model at runtime, termination is not guaranteed by default and must be imposed from outside—by budgets and timeouts at the level of the control loop.

Sources. K. Ethayarajh, "How Contextual are Contextualized Word Representations?" (EMNLP 2019, arXiv:1909.00512)—representation anisotropy and the baseline correction. LangChain, the `AgentExecutor` reference (the `langchain-classic` package)—the defaults `max_iterations = 15` and `max_execution_time = None`. C. Richardson, *Microservices Patterns* (2018), Chapter 4—the step log and saga compensations (the source the book ties the step log to in the Preface).

10.6 Deep Dive: reliability and the reversibility principle

Actions are executed by a nondeterministic component that can err or be driven out of its normal behavior by instructions embedded in the input data (prompt injection, Chapter 19). Hence the design principles that raise reliability (formulated, among other places, in Anthropic's recommendations):

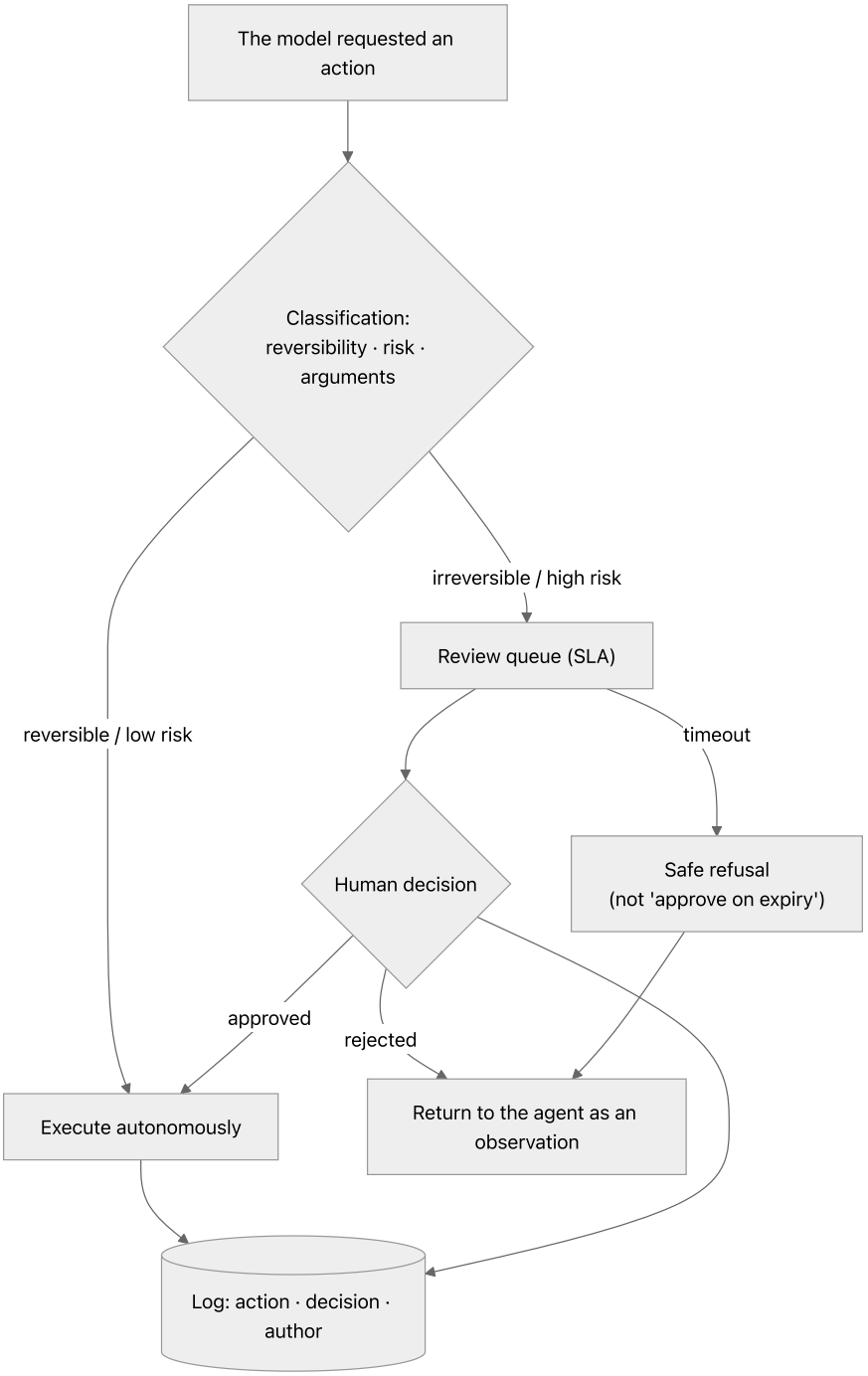
- **Prefer reversible actions to irreversible ones.** A reversible action can be undone by a compensating operation; an irreversible one (sending a message, charging money, publishing, deleting) cannot.
- **Human confirmation (human-in-the-loop) before actions with significant consequences.**
- **A minimal set of permissions and tools (least privilege)**—the link to the excessive agency class of risks (Chapter 19).

The rationale. In a deterministic system, trust rests on the code having been written and tested by a developer. In an agent, the role of the code that picks the action is played by a probabilistic model whose behavior can be distorted by the input data. So defense in depth is built around it: reversibility provides the rollback when a decision is wrong, human confirmation catches the error before irreversible consequences land, and restricting permissions shrinks the blast radius. The split by reversibility matches the standard split in the saga pattern: compensating operations apply to reversible steps; for irreversible ones the answer is prior confirmation, not compensation (you cannot unsend a sent message). Honoring that split is what separates an agent fit for production from a demo prototype.

Human confirmation is not a line in a checklist but a pattern in its own right, with its own mechanics and its own failure modes; the rest of the book refers to it in short as the **human-gate**.

Pattern: Human-in-the-Loop Gate (human-gate)

Context. The agent executes actions with consequences; some of them are irreversible (sending messages, charging money, deleting, changing configuration), and the decision is made by a probabilistic model whose behavior can be distorted by an injected instruction (Chapter 19). **Problem.** How do you get the benefit of autonomy on reversible actions without accepting the risk of an irreversible mistake or a compromised action? **Forces.** Autonomy buys speed and scale, but the cost of a mistake on an irreversible action is asymmetrically large. Blanket manual confirmation kills throughput and the reviewer's attention. Confirmation requests without context invite mindless approval. **Solution.** A deterministic **gate** at the point between "the model requested an action" and "the action executed" (§12.2): actions are classified by reversibility and risk, the flagged ones are held until a human decides explicitly, the rest execute autonomously. **Mechanics.** (1) Classification: static labeling of the tool (risk metadata, §12.5) plus rules on the arguments (amount, recipient, environment). (2) A review queue with an SLA and safe behavior on timeout—reject, not "approve on expiry." (3) A decision card for the human: exactly what will be done, the agent's justification, a preview of the outgoing content, the provenance of the data in the context (has it been contaminated by an untrusted source, §12.5). (4) The decision and its author are written to the log (audit); a rejection goes back to the agent as an observation, not as a silent cutoff. (5) Gate metrics: the approval rate and the time to decide—approvals approaching 100% with seconds spent per decision signal that oversight is degrading (**automation complacency**—the canonical name from the literature on the human in automation; colloquially, rubber-stamping) and that the classification of actions needs revisiting.


The model requested an action
Classification: reversibility · risk · arguments
irreversible / high risk
reversible / low risk
Review queue (SLA)
timeout
Human decision
Safe refusal (not 'approve on expiry')
approved
rejected
Execute autonomously
Return to the agent as an observation
Log: action · decision · author

Example. In the scenario from the Preface, processing the refund and notifying the warehouse go through the gate (they are irreversible), while checking the order status executes autonomously (a read). The reviewer sees the refund amount, the draft notification, and the source of the request. **Resulting Context.** An irreversible error is caught before the consequences land; the "rule of two" (§12.5, §19.4) is satisfied cheaply. The price: latency on gated actions, load on the reviewers, and the degradation of their attention when gating is excessive. **When to Use.** Irreversible or expensive actions; configurations close to the lethal trifecta (§12.5); the early stage of trusting an agent. **When Not To.** Fully reversible actions with cheap compensation—the gate only eats throughput; reversibility and auditing are enough there. **Related Patterns.** Levels of agency (§10.7—the gate is precisely the mechanism of level L2); the rule of two (§12.5); Orchestrator-Workers (the gate on irreversible steps, §11.5); Evaluator-Optimizer (§11.6—an automatic evaluator as a pre-filter ahead of the human). **Sources.** Anthropic, "Building Effective Agents" (2024)—checkpoints involving a human; Meta, "Agents Rule of Two" (2025)—confirmation as the way out of the trifecta; OWASP Top 10 for LLM Applications (2025)—Excessive Agency mitigations; R. Parasuraman, V. Riley, "Humans and Automation: Use, Misuse, Disuse, Abuse" (1997)—automation complacency and the degradation of oversight.

10.7 Levels of agency

How much autonomy you grant is a design decision in its own right, driven by the cost of an error and the blast radius—not a choice of "agent on or off."

Level	Who decides / executes	Blast radius	When
L0—Suggest	The LLM *suggests*, the human executes everything	Zero	High risk, getting started, an untrusted domain
L1—Reversible-only	The LLM executes *only what is reversible*	Bounded (compensable)	Medium risk
L2—Approval-gated	The LLM executes; the irreversible goes through the human-gate	Controlled	Production with consequences (the typical choice)
L3—Full autonomy	The LLM executes everything	Maximal	Low risk / sandbox / internal

The scale is this book's didactic frame (industry analogues differ in the number of levels; the substance is not in the numbers but in the axes "who executes" and "how large is the blast radius"). The level is chosen by the cost of an error. The working default for running an agent inside an organization is **L2** (reversible actions autonomous, irreversible ones through confirmation): a direct application of the reversibility principle of §10.6.

10.8 Chapter summary

An agent is a distributed system with a nondeterministic control node. The two key design decisions—workflow or agent (§10.3) and the level of autonomy (§10.7)—are settled by the cost of an error. The methods that make distributed systems reliable (idempotency, compensations, timeouts, tracing) apply directly (§10.4); what is specifically new is two dimensions: the control loop may fail to terminate, which external budgets address (§10.5), and actions are executed by a nondeterministic component, which demands reversibility, human confirmation, and restricted permissions (§10.6). Next comes the catalog of patterns such systems are built from.

Sources. Anthropic, "Building Effective Agents" (2024); V. Dibia, *Designing Multi-Agent Systems* (2025); C. Richardson, *Microservices Patterns* (2018)—orchestration, choreography, compensations, and idempotency as general distributed-systems patterns.

In the Margins: Chapter 10

> ↩ **Blast radius**—the maximum damage from a single mistaken or compromised action; bounding it is the goal of the levels of autonomy and of least privilege. → §12.5, Chapter 19.
>
> ↩ **Agency** and **autonomy** are used interchangeably in this book: agency is the property of the system (hence "levels of agency" and the OWASP risk Excessive Agency, Chapter 19), autonomy the measure of that property. This is one axis, not two: agency *is* the autonomy granted. ⇐ Glossary (Excessive Agency).

Chapter 11. A Catalog of Agent Patterns

11.1 The ladder and composition

Patterns split into **workflow patterns** (a deterministic graph, laid down by the developer) and **agentic autonomy** (the model decides the flow). The Anthropic canon has five patterns; the first three are workflows at heart, the last two introduce autonomy. The key property: **they compose**—a real system is assembled from them like building blocks (§11.7). You climb the ladder from the simple (chaining) to the autonomous (orchestrator-workers) only when you have to (§10.3).

11.2 Pattern: Prompt Chaining

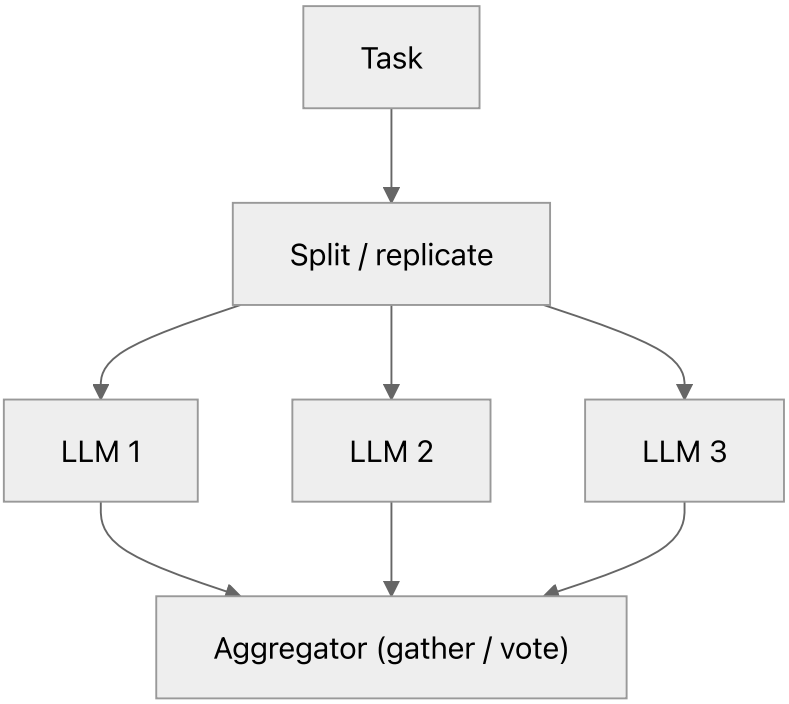
> **Context.** The task decomposes into a *fixed* sequence of steps, each running on the output of the one before. **Problem.** One mega-prompt does everything badly (too many goals at once) and is hard to verify and debug. **Solution.** Break it into a chain of narrow LLM calls: the output of step N is the input of step N+1. Between steps you can insert programmatic checks (a gate: is the intermediate result valid—if not, abort). **Mechanics.** The deterministic graph is fixed up front (by the developer, not the model)—that makes it a workflow; structurally it matches a sequential pipeline. **Example.** "Extract entities → check them against a reference catalog → compose the answer": three narrow calls instead of one broad one. **Resulting Context.** Higher quality and better debuggability (every step is narrow and checkable); the price is latency (the steps run in sequence) and more calls. **When to Use.** The decomposition is fixed and linear. **When Not To.** The number or order of steps depends on the data → Orchestrator-Workers (§11.5). **Related Patterns.** Orchestrator-Workers (the dynamic version); Evaluator-Optimizer (checking a step). **Source.** Anthropic, "Building Effective Agents" (2024).

11.3 Pattern: Routing

Context. The input is heterogeneous: different request types need different handling, a different model, or a different prompt. **Problem.** One handler for everything is either overloaded or suboptimal (an expensive model on an easy request, a generic prompt on a specific one). **Solution.** A classifier determines the input's type and **routes** it to the right branch (a specialized prompt, tool, or model). **Mechanics.** Structurally this is a content-based router. The classifier is a cheap LLM or a trained classifier (for the component's anatomy, see Deep Dive §2.2). The branching is fixed up front, which makes it a workflow. It connects to Model Routing (§2.2) and Adaptive RAG (§7.9)—the same move in different layers. **Example.** Incoming requests → {refund, complaint, product question} → a branch of its own for each. **Resulting Context.** Every branch is optimal; the price is the cost of classification and its errors (misrouting: an easy request went down the wrong branch). **When to Use.** The input has clearly distinguishable categories. **When Not To.** A homogeneous stream. **Related Patterns.** Model Routing (§2.2), Adaptive RAG (§7.9). **Source.** Anthropic (2024).

11.4 Pattern: Parallelization

Context. The task splits into *independent* subtasks (sectioning), or you need several independent attempts at the same one (voting). **Problem.** Running independent parts one after another piles up latency for nothing; a single attempt may be unreliable. **Solution.** Two variants: **Sectioning**—hand out independent subtasks in parallel and gather the results (scatter-gather). **Voting**—run one task several times (safety checks with different prompts, say) and aggregate by majority or consensus—the same idea as self-consistency in prompting (Wang et al., 2022; §3.4), lifted to the orchestration layer. **Mechanics.** Structurally this is scatter-gather. The split is fixed up front, which makes it a workflow.



Resulting Context. Lower wall-clock latency (§2.3) and/or higher reliability (voting); the price is cost (several calls) and the aggregation logic. **Deep Dive—partial failures.** Scatter-gather breeds a problem familiar from distributed systems: one of the N parallel branches *silently* failed or returned garbage. Naive "gather whatever showed up" aggregation silently loses part of the result. The remedy is the standard distributed-systems approach: track the status of *every* branch explicitly, define a policy for coming up short (wait, retry, degrade), and do not treat the aggregate as complete until every branch is confirmed (or knowingly accept a partial result). **When to Use.** Independent subtasks, or you need reliability through redundancy. **When Not To.** The steps depend on each other (you need chaining). **Related Patterns.** Evaluator-Optimizer (voting as a form of checking). **Source.** Anthropic (2024).

11.5 Pattern: Orchestrator-Workers

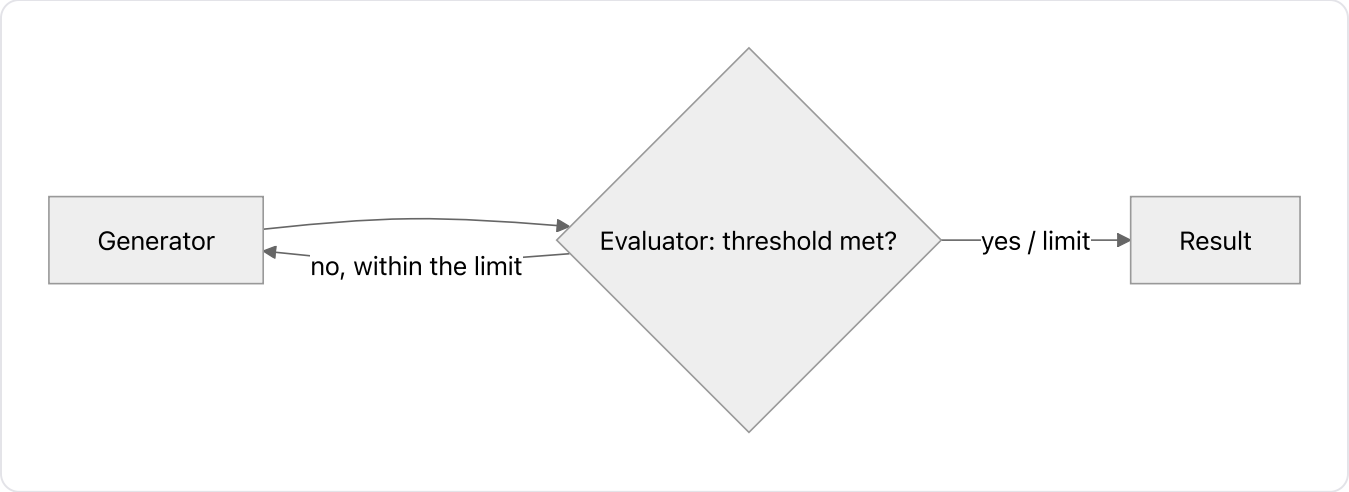
This is **the catalog's first pattern to introduce agentic autonomy**: the LLM decides *which* steps are needed. The Preface demonstrates the format at length on this very pattern (with a diagram and a full example); what follows is the working card, so the chapter stands on its own when read for reference.

Pattern: Orchestrator-Workers

Context. The task does not decompose into a fixed set of steps up front: the number and nature of the subtasks depend on the input and are known only at runtime. **Problem.** How do you execute a task with dynamic decomposition while preserving reliability, observability, and the ability to roll back—the properties saga orchestration routinely provides? **Solution.** An LLM **orchestrator** analyzes the task at runtime, generates a list of subtasks, and hands them out to **workers** (each a separate LLM or tool call with a narrow responsibility), then gathers and synthesizes the results. Architecturally this is **Saga orchestration** (in Richardson's terms) with two differences: the coordinator is nondeterministic, and the set and order of steps are determined dynamically rather than by a state machine. **Mechanics.** A step log (much like a saga log); idempotency of the worker step keyed by task; a timeout and a compensation for every step; a mandatory **iteration limit/budget**—a nondeterministic coordinator can loop forever (§10.5). Unlike Chaining (§11.2), the graph is not fixed, so observability is built on the trajectory (§17.5, Chapter 18). **Resulting Context.** The flexibility of dynamic decomposition while auditing, idempotency, and rollback survive. The price: a nondeterministic plan (two runs on the same input can produce different decompositions—test by trajectory, §17.5), cost (every worker is a model call), and latency. **When to Use.** The decomposition genuinely depends on the input, the number of subtasks varies, and there are reliability/audit requirements. **When Not To.** A fixed decomposition—a static workflow (§11.2) is cheaper, more predictable, and easier to debug. **Related Patterns.** Prompt Chaining (the static alternative, §11.2); Routing (the degenerate case—the orchestrator only picks a branch, §11.3); Evaluator-Optimizer (checking step quality, §11.6); Human-in-the-Loop Gate (irreversible steps, §10.6); Trajectory Evaluation (§17.5). **Sources.** Anthropic, "Building Effective Agents" (2024)—orchestrator-workers. C. Richardson, *Microservices Patterns* (2018), Chapter 4—saga orchestration, compensations, idempotency. V. Dibia, *Designing Multi-Agent Systems* (2025)—supervisor/handoff orchestration.

11.6 Pattern: Evaluator-Optimizer

Context. The first-pass answer is not good enough, but you can *evaluate* it and improve it iteratively. **Problem.** A single generation does not reach the quality you need; you need a "produce → check → improve" loop. **Solution.** Two LLMs in a loop: a **generator** produces the answer, an **evaluator** critiques it against criteria and returns feedback; the generator reworks it. Repeat until it clears the threshold or hits the **iteration limit**. **Mechanics.** ≈ a retry with a quality check. The evaluator is a separate call with a rubric (a relative of LLM-as-judge, Chapters 9/17). An iteration limit is *mandatory* (otherwise you get an infinite loop, §10.5).

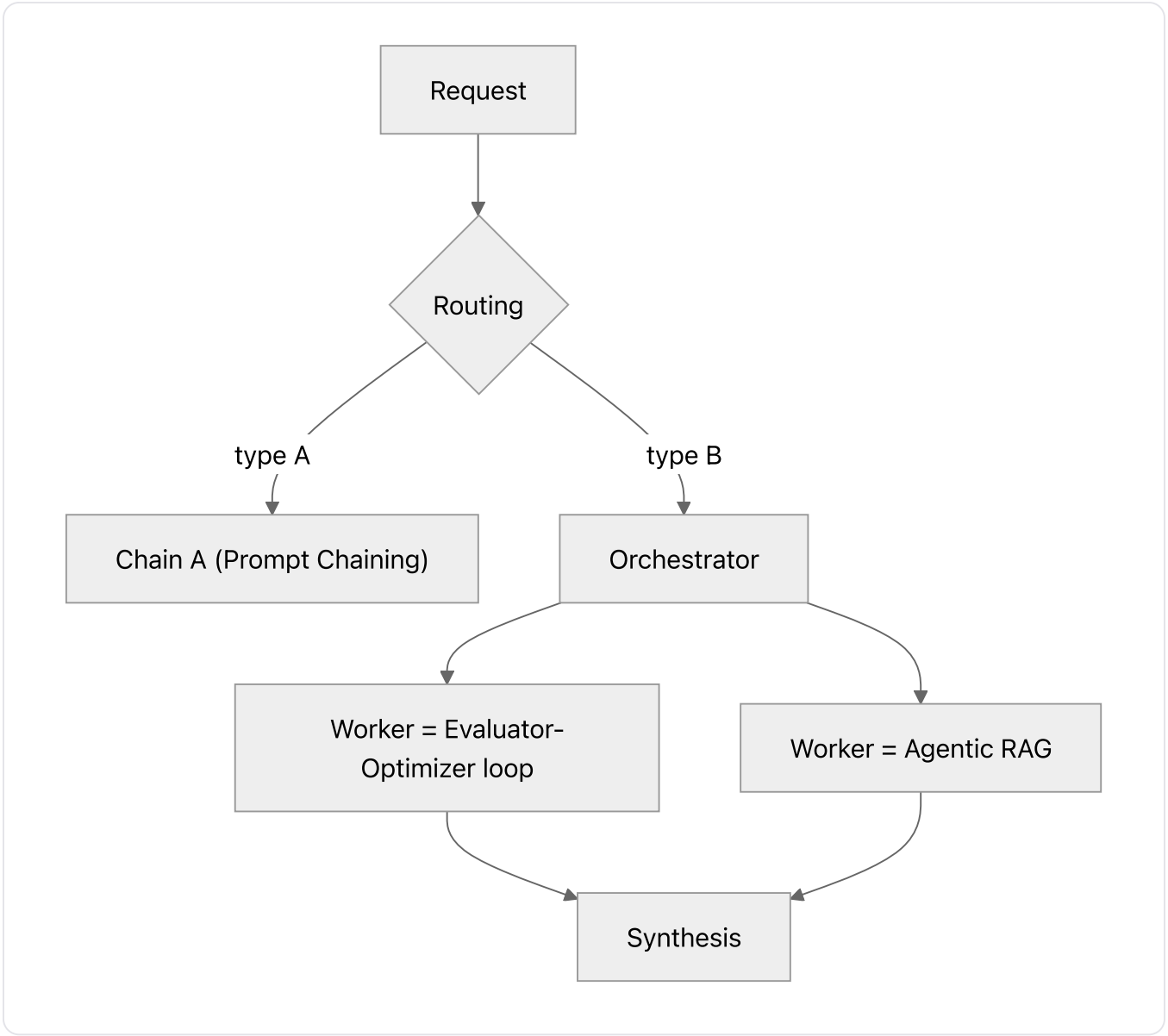


Example. Draft a reply to a customer → the evaluator checks tone, completeness, and compliance with policy → rework. Or iterative improvement of code or a translation. **Resulting Context.** Higher quality on tasks with a clear evaluation criterion; the price is cost and latency (several rounds). **When to Use.** There is a measurable quality criterion and iteration is worth something. **When Not To.** The criterion is unclear, or the first pass is already good enough. **Related Patterns.** Voting (§11.4), Reflection (Chapter 13), LLM-as-judge (Chapters 9/17). **Source.** Anthropic (2024).

11.7 Deep Dive: composing patterns

The central idea of a "pattern language": real systems are **assembled out of patterns**—they do not pick one. Examples of composition:

- **Router → Chains.** The router (§11.3) sends a request into one of several chains (§11.2), each tuned to its own type.
- **An orchestrator whose workers are Evaluator-Optimizers.** The orchestrator (§11.5) hands out subtasks, and each worker internally runs a generator-evaluator loop (§11.6) for quality.
- **Parallelization inside a chain step.** A chaining step that itself hands out subtasks in parallel (§11.4) and aggregates them.
- **Agentic RAG (§7.8) as a worker.** The orchestrator tells a worker to "find the information," and that worker is an agentic RAG loop.



Designing an agent system means *composing* these blocks to fit the task, not choosing one pattern. The same discipline of restraint applies: every added layer of autonomy or pattern raises cost, latency, and risk, so you compose when the task requires it, not by default.

11.8 Deep Dive: what here is a workflow and what is an agent

Drawing the line clearly pays off—it determines how predictable the system is:

- **Workflow patterns** (the graph is fixed up front, the flow is deterministic): Prompt Chaining, Routing, Parallelization. Predictable, cheaper, easier to debug. Most “agents” running in production are these.
- **Agentic autonomy** (the *LLM* decides the flow at runtime): Orchestrator-Workers (dynamic decomposition) and the ReAct control loop itself (Chapter 13). More flexible, but nondeterministic—with every consequence from §10.4–10.6 (nontermination, a nondeterministic actor).

The practical takeaway: know *which* parts of your system are workflow and which are autonomous. Introduce autonomy surgically, and only where a fixed graph genuinely cannot cope. The more of the system you hold inside deterministic workflow patterns, the more predictable, cheaper, and safer it is; fence the autonomous stretches

with budgets (§10.5) and a human gate (§10.6).

11.9 Summary and a decision framework

Pattern	Type	Solves	Take it when
Prompt Chaining	Workflow	A fixed sequence	The decomposition is linear
Routing	Workflow	Heterogeneous input	The categories are distinguishable
Parallelization	Workflow	Independent parts / reliability	It parallelizes, or you need redundancy
Orchestrator-Workers	Autonomy	Dynamic decomposition	The steps depend on the input
Evaluator-Optimizer	Workflow loop	Iterative quality	There is an evaluation criterion

A workflow loop is an intermediate form between the two poles of §11.8: bounded autonomy. At runtime the evaluator decides whether to keep iterating (the model picks the next step), but the space of choice collapses to "another iteration or stop" inside a fixed template and a hard cap on cycles.

The decision framework: start with the simplest pattern that covers the task (usually chaining or routing) → introduce autonomy (orchestrator-workers) only once a fixed graph has demonstrably failed to cope → compose (§11.7) to fit the task's real shape → cap every autonomous stretch with a budget (§10.5), and put irreversible actions behind a human gate (§10.6). Complexity goes in on proven need, not in advance.

Sources. Anthropic, "Building Effective Agents" (2024)—the five patterns; V. Dibia, *Designing Multi-Agent Systems* (2025)—orchestration patterns; C. Richardson, *Microservices Patterns* (2018)—the basis for mapping onto sagas/EIP°.

In the Margins: Chapter 11

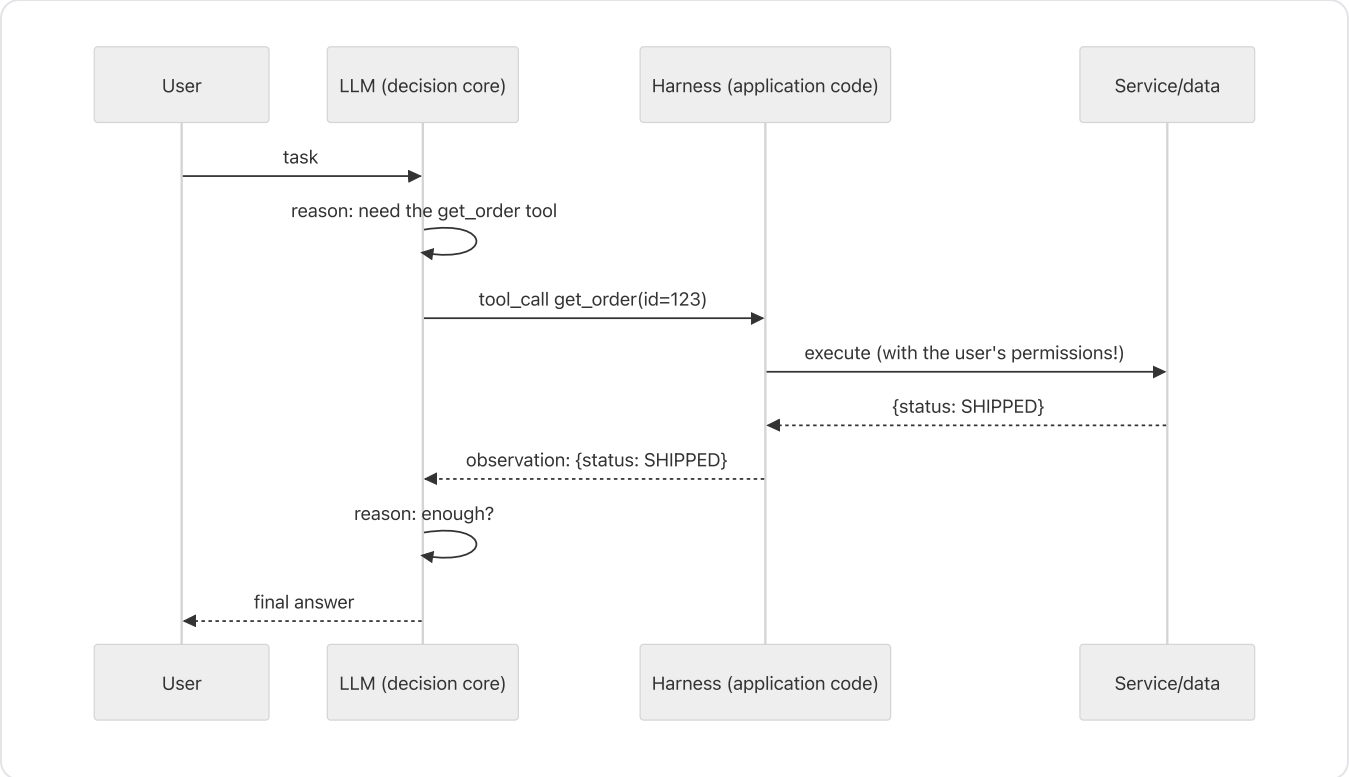
> ↩ **EIP**—Enterprise Integration Patterns (Hohpe, Woolf, 2003): the canonical catalog of messaging patterns (router, splitter/aggregator, dead letter channel); Routing and Parallelization are their direct relatives.

Chapter 12. Tools and Tool Calling as a Contract

Tool calling lets a nondeterministic model *initiate* calls into the application's functions: the model composes a structured call request, and the harness (§10.2)—deterministic application code—executes it. Two threads run through this chapter: a tool's interface is a contract the model reads, and its quality determines whether the calls come out correct; and tools are the agent's main security surface, because they are how it acts on the outside world.

12.1 What tool calling is

Tool calling (function calling)—a mechanism in which tools are *declared* (name, description, parameter schema); the model, handed a task, *decides* which tool to call and with what arguments, and returns a structured call; the harness (§10.2) executes it and returns the result into the context; the model continues. The direction of the call here is the reverse of the familiar one: the calling side is the nondeterministic model, and the executing side is deterministic application code.



12.2 The mechanics of tool calling

How it works:

1. **The tools are described in the context**—their schemas (name, parameters, types) are handed to the model as a specification (a relative of structured output, Chapter 5).
2. **The model returns a structured call**—it executes nothing; it *composes a request*: "call X with arguments Y." Constrained/structured decoding (§5.3) guarantees the call conforms to the tool's schema.
3. **The harness (§10.2) dispatches** the call to a real function and executes it.
4. **The result goes back into the context** as an *observation*, and the loop (§10.2) continues.

A few API-level details matter in practice:

- **The wire format.** Call and result travel as separate message-block types: the model returns a call-request block (`tool_use`, carrying a name and arguments), and the application returns a result block (`tool_result`), matched to it by call id. Binding request to result explicitly is what lets several calls coexist within one turn.
- **Steering the choice (`tool_choice`).** Providers expose a mode: the model decides for itself (`auto`), must call some tool (`required` / `any`), must call one specific named tool, or is barred from calling anything (`none`). This is a lever of determinism: where a call is mandatory, `required` removes the "the model answered with text instead of a call" outcome.
- **Parallel calls.** In a single turn the model can request several independent calls, executed in parallel, which cuts wall-clock latency (§2.3) for independent actions (the analogue of Parallelization, §11.4, but initiated by the model itself). The flip side: parallel calls complicate rollback on partial failure (some calls went through, some did not).
- **Native versus emulated.** Native function calling (the model is trained to return structured calls, and decoding is constrained by the schema) is more reliable than emulating it through the prompt—asking for `CALL tool(...)` and then parsing the text.

Critical for security (§12.5): the model does not perform the action—it requests it; the harness performs it. That split is the control point: between "the model requested a call" and "the call happened" sit argument validation, authorization, and, where needed, human confirmation.

12.3 Deep Dive: agent-computer interface (ACI) as contract design

A tool's name, its description, and its parameter schema are **the contract the model reads**. A bad tool interface makes the model call the wrong tool or pass the wrong arguments. Anthropic frames this as a requirement to invest in the **agent-computer interface (ACI)** as much as in the interface for a human (HCI).

Principles for designing a tool interface:

- **Clear, unambiguous descriptions.** The model reads a tool's description as documentation; ambiguity leads to the wrong choice.
- **Informative parameter names and descriptions** and **constraints through types and enums** (Chapter 5): the stricter the schema, the fewer wrong calls.
- **Poka-yoke (mistake-proofing by construction).** Design the interface so that a wrong call is *hard to make*: requiring absolute paths instead of relative ones, for instance, eliminates the whole "wrong working directory" class of errors. Make the tool hard to use incorrectly rather than only documenting correct use.
- **A small set of well-designed tools beats a pile of overlapping ones.** An excess of tools blurs the choice (**tool confusion**): the model loses track of which one to call. Order of magnitude: up to a couple dozen well-separated tools are usually manageable; past that the curve of selection errors climbs fast—a signal to group

them, impose a hierarchy, or select tools dynamically (which has a dead-end failure mode of its own: if the retriever left the needed tool out of the candidates, the model will not recover—unlike the recoverable loop errors of §12.4).

- **A compact, structured result** instead of a verbose dump—otherwise it burns context budget (§4.4, tool-result pruning). The result is part of the contract too.
- **Examples in the description** help the model the same way examples in documentation help a developer.

Selecting tools at scale. As the number of tools grows, the model's choice degrades (tool confusion) and their schemas take up more and more context (§4.4). Beyond curating the set, there are structural techniques: grouping and tool namespaces; and **dynamic selection**—retrieving into the context only the subset of tool definitions relevant to the specific task (retrieval over a tool catalog) instead of presenting all of them at once. This carries the logic of context selection (Part III, §4.4) over to the tool definitions themselves.

What separates designing for a model from designing for a human is that the consumer is probabilistic: given a bad contract it errs and, unlike a human, does not ask a clarifying question—it silently calls wrong.

12.4 Deep Dive: tool errors and the feedback loop

Tools fail (wrong arguments, an external error, "not found"). What differs from classical error handling: **the caller can *reason* about the error.** Return the error *usefully*—as an observation with a clear message—and the model often recovers on its own: it retries with corrected arguments, tries another tool, or gives up cleanly. This is a **feedback loop**: error → the model adapts.

In practice:

- **Errors come back as observations, not as exceptions that topple the loop.** The model fixes a message like "parameter date must be in YYYY-MM-DD format"; an unhandled stack trace ends the loop.
- **Telling recoverable errors from terminal ones.** Return a recoverable error (a malformed argument) so the model can retry with a fix; return a terminal one (access denied, resource missing) so the model stops trying rather than looping on fixing the unfixable. Blurring these two categories is a common cause of useless retry loops. Put it in the contract: an explicit `retryable` field (or error classes along the lines of HTTP 4xx/5xx—a bad request versus a failing executor) in the tool's result, so the distinction is drawn by the loop's code and not by the model's reading of an error string.
- **Bounding the loop** (§10.5): cap the attempts, or the "fix → error → fix" sequence runs off to infinity.
- **Validate tool arguments as untrusted input:** the model can generate invalid or dangerous arguments. The schema (Chapter 5), a validator, and the security checks (§12.5) apply before execution. The model is not a trusted source of arguments.

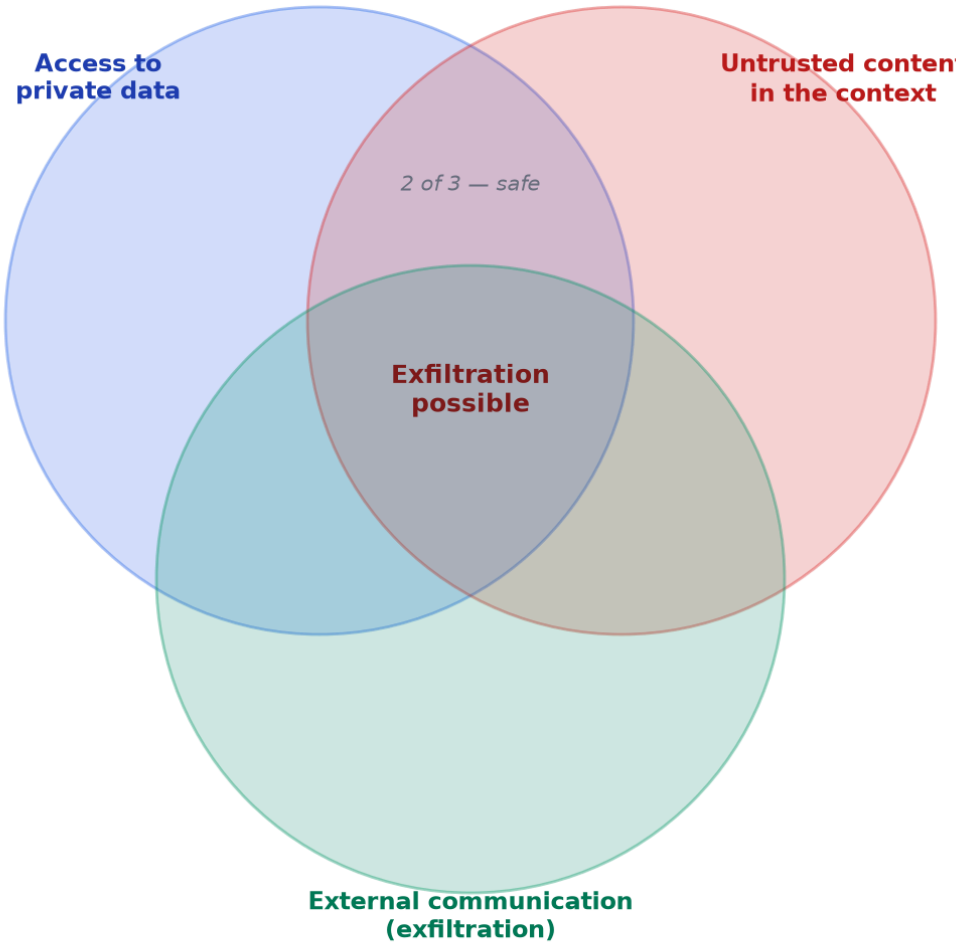
12.5 Deep Dive: tool security (a bridge to Chapter 19)

Tools are where the agent gains a *real ability to act on the world*, which makes them its main attack and risk surface.

Lethal trifecta. The most compact frame for assessing the risk was proposed by S. Willison (2025): the danger of data exfiltration arises when **three** properties coincide within one agent session—(1) access to private data, (2) untrusted content present in the context, and (3) the ability to communicate externally (to send data out). Any *two*

of the three are safe; all three together let an attacker who controls the untrusted content make the agent pull private data and send it out. The mechanism is indirect prompt injection (Chapter 19): instructions embedded in data hijack the behavior of a model that does not tell instructions from data (§1.4, §3.2). The trifecta is not the mechanism itself but a trust-boundary frame indicating *when* an injection escalates from a "weird answer" into a leak.

The "lethal trifecta": danger requires all three properties combined



S. Willison (2025). The Rule of Two (Meta, 2025): an autonomous agent may hold at most two of the three properties; with all three, human confirmation is mandatory. Removing any one leg breaks the data-theft chain.

The combination is real and has been observed in systems already in operation; the communication channel is often non-obvious—an outbound HTTP request, an image loaded from a URL with the data in its parameters, even a rendered clickable link (the click becomes the leak channel). Incidents of this class have been analyzed publicly: access to private repositories through the GitHub MCP server via an instruction planted in a public issue; exfiltration from enterprise assistants through the parameters of an image URL (the "zero-click"[9] class).

The Agents Rule of Two. A practical budget proposed by Meta (2025) and endorsed by Willison: within a single session an autonomous agent must hold no more than **two** of the trifecta's three properties; if a task genuinely requires all three, the agent **must not run autonomously**—it needs human confirmation or another reliable

check before any consequential action. Removing any one of the three "legs" breaks the theft chain: no private data—nothing to steal; no external channel—nowhere to send it; no untrusted content—nothing to plant the command with.

Taint tracking as a deterministic measure. The trifecta implies a concrete architectural measure: treat the presence of untrusted content as a *taint event*—once the agent has taken in attacker-controlled input, the rest of the turn counts as tainted. In the tainted state, actions with exfiltration potential (outbound HTTP, sending messages, opening a PR, rendering a clickable link) are blocked or require explicit confirmation. Tools carry metadata (`reads_private_data`, `sees_untrusted_content`, `can_exfiltrate`), and the runtime forbids all three from meeting in a single execution path.

The concrete controls that implement this frame:

- **Least privilege and a minimal tool set (§10.6).** Grant only the necessary tools, with the narrowest scope; a tool kept around "just in case" widens the blast radius. This maps to the OWASP class LLM06 (Excessive Agency).
- **Audit the set, not the individual tools.** Tools that are harmless on their own combine into an attack channel: reading a file plus an outbound call is an exfiltration path. Assess the agent's tools *as a whole* against the trifecta, not each tool in isolation.
- **A tool result is untrusted data, not instructions.** Isolate a tool's output (a web page, file contents, an API response) in the context (§4.6) and never let text from a result steer the agent (indirect injection, OWASP LLM01).
- **Authorization at the tool level (the "confused deputy" problem).** A tool must apply the permissions of the *user* the agent is acting for, not the agent's own. To the question "what privileges is this call executing with?" the answer must be "those of a specific user." Otherwise the agent becomes a *confused deputy*: through the agent, a user performs an action they have no right to. Pass the user context through into the tool's execution and check authorization *at the point of execution*. How that delegation is expressed technically (token exchange, on-behalf-of, the agent's own identity) is the Deep Dive in §15.7.
- **Human confirmation on dangerous and irreversible tools (§10.6).**

An important caveat. Safety training, guardrails, and the instruction hierarchy *reduce* an injection's success rate but do not *eliminate* it: an adaptive attacker finds a way around by working through variants (shown, in particular, in the work on adaptive attacks against published defenses). The defense therefore has to be **architectural**—constraining the trifecta, taint-gating, and human confirmation—not merely "model-level." Tools are the agent's security perimeter; what makes them peculiar is that the calling side (the model) is nondeterministic and can be driven out of normal behavior by a planted instruction, so authorization and validation are enforced by the tool's *executor*, not by the model. Chapter 19 covers this in detail.

The industry frame: OWASP Top 10 for Agentic Applications (the 2026 edition, published in December 2025). The discipline described here is fixed in an industry taxonomy as well: alongside the LLM Top 10 (§19.3), OWASP released a separate list of agentic-system risks (ASI01–ASI10). Among the categories: hijacking the agent's goal (ASI01 Agent Goal Hijack), tool misuse, misuse of the agent's identity and privileges, compromise of the agentic supply chain, unexpected code execution, memory and context poisoning (ASI06), and insecure inter-agent communication (ASI07); the full list is at genai.owasp.org. The document's through-line is **Least-Agency**: give an agent the minimum level of autonomy the task requires—a direct generalization of least privilege onto the axis of autonomy (§10.7). The categories map onto this part's material almost one to one: goal and non-termination—§10.5; tools and privileges—this section; memory—Chapter 13; inter-agent communication—Chapter 14.

Sources. S. Willison, “The Lethal Trifecta for AI Agents” (2025); Meta, “Agents Rule of Two” (2025); K. Beurer-Kellner et al., “Design Patterns for Securing LLM Agents against Prompt Injections” (arXiv:2506.08837, 2025); OWASP Top 10 for LLM Applications (2025) and Top 10 for Agentic Applications (2026)—© the OWASP Foundation, licensed under CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/); the categories (LLM01 Prompt Injection, LLM06 Excessive Agency, LLM10 Unbounded Consumption, and ASI01–ASI10 in §12.5) are reproduced as published, the mapping to this book’s sections is added; N. Hardy, “The Confused Deputy” (1988).

12.6 Deep Dive: scaling the tool surface—code execution and Skills

§12.3 ended on the point that as the number of tools grows, both sides of the contract degrade: the schemas eat context, and the model’s choice blurs (tool confusion). Dynamically selecting definitions is a palliative with a dead-end failure mode of its own. In 2025–2026 the industry produced two structural answers, and both change the very shape of the contract.

Code execution: tools through code

Code execution (also “code mode”; the idea traces back to **CodeAct**—Wang et al., 2024, with the industrial form in Anthropic, “Code execution with MCP,” 2025) inverts the mechanics of §12.2: instead of presenting the model with JSON schemas and getting one structured call per turn, the harness presents the tools as a **code API**—a tree of wrapper files (TypeScript modules generated from those same MCP schemas, for instance)—and the model *writes code* that calls those wrappers. The code runs in a sandbox (§19.6); only the result of the run comes back into the context.



Why this works shows in each of four mechanical consequences:

1. **Progressive disclosure instead of presenting everything.** The schemas are not loaded into the context all at once—the model discovers tools as files (listing a directory, reading a specific wrapper) and reads only the ones it needs. Hundreds of tools stop costing context; Anthropic reports a reduction from ~150k to ~2k tokens on a real scenario (the order of magnitude matters more than the exact figure).
2. **Intermediate data does not pass through the model.** In the classic loop, every call’s result is an observation in the context: a JSON blob thousands of lines long is paid for in tokens and dilutes attention (§4.4). In code, filtering and aggregation happen *in the sandbox*: the model gets the five rows it needs, not the whole dump.
3. **Loops and conditionals execute deterministically.** “Repeat for each of 40 items” in an agent loop means 40 model turns (§10.2), each with its own probability of error and the full price of a call; in code it is an ordinary `for` loop, executed in a single turn. Part of the control logic moves out of the probabilistic core into deterministic code—the same vector as all of §10.6.
4. **The native language of composition.** The model was trained on a colossal corpus of code; composing operations *as a program* is a more natural task for it than a long sequence of isolated JSON calls—which is CodeAct’s original observation.

The price is symmetric with the gains: you need an execution sandbox as infrastructure (and model-written code inside it is untrusted by definition—the whole discipline of §19.6 is mandatory); generated code is harder to debug than a sequence of explicit calls in a trace (Chapter 18); a model weak at code loses the main advantage. The practical frame: single calls to narrow tools—classic tool calling; broad surfaces (hundreds of tools, large intermediate data, bulk operations)—code execution.

Skills: competence as an artifact

A tool answers the question "what the agent *can do*." The question "how it is *normally done*"—procedural knowledge: sequences, conventions, templates—is answered by a **skill** (Agent Skills; an open format, Anthropic, 2025): a directory with a `SKILL.md` manifest—a YAML header (name, description) plus instructions in Markdown—and, where needed, supporting files and scripts. A skill is not executed by the harness as a function—it is *read by the model* as a procedure.

The mechanics are a three-stage progressive disclosure, and in it lies the whole point of the format:

1. The system prompt always carries only the **name and description** of each installed skill—tens of tokens apiece;
2. When the model recognizes that a task matches a description, the harness loads the **body** of `SKILL.md`—a few thousand tokens;
3. The files the body references (reference material, templates, scripts) are read **as needed**—as ordinary files, through tools.

Context budget is spent in proportion to relevance (§4.4): a hundred installed skills cost what one loaded skill costs. Compare the alternative—"sew every procedure into the system prompt"—which is paid for in full on every call.

Architecturally a skill is **externalized procedural memory** (§13.1): know-how pulled out of the weights and the prompt into a versioned git artifact that can be reviewed, tested, moved between agents, and rolled out like code. A procedure worked out by an agent (or a human) crystallizes into a file—instead of being relearned from scratch in every session.

Security follows from the nature of the format: a skill is instructions *loaded into the context*, which makes it an injection surface (Chapter 19); a skill with scripts is executable code. A third-party skill is an untrusted supply-chain component, under exactly the same discipline as a third-party MCP server (§15.7).

Tools, Skills, MCP—three different axes

The three words often get conflated, because all three "give the agent capabilities." The axes are different:

Axis	Tool (Chapter 12)	Skill	MCP (Chapter 15)
Nature	An executable contract: schema + function	A declarative procedure: instructions + resources	A protocol for connecting sources
Answers	“What can be done”	“How it is normally done”	“How to discover and call”
Who consumes it	The harness executes the call	The model reads it as text	The harness’s client connects the server
Cost in context	The schema (always, or by selection)	Name+description always, body on demand	The server’s tool schemas
Versioned as	Application code	A file in git	A server/spec version
Engineering analogue	An API method	A runbook	A connector standard (M+N, §15.1)

They compose rather than compete: an MCP server supplies the tools, code execution changes how they are called, and a skill explains to the model in what order and by what conventions to use them.

12.7 Implementation on the JVM

- **Spring AI:** a method is annotated `@Tool`, the framework generates the schema from the signature (the way structured output comes from a type) and passes it to the model; calls are dispatched to the corresponding bean.
- **LangChain4j:** the same—typed tool methods with a schema derived from the signature.
- **MCP (Chapter 15):** `@McpTool` exposes a method as an MCP tool—the standard way to offer tools to any MCP-compatible agent (a contract-first integration layer).

In every case, authorization and validation (§12.4–12.5) are enforced *in the body of the tool method*: the framework only relays the call, and security stays on the application’s side.

12.8 Chapter summary

Tool calling inverts the direction of the call: the calling side is the nondeterministic model, the executing side is the harness. A tool’s interface (the ACI) is the contract the model reads, and its quality determines whether the calls come out correct: clear descriptions, strict schemas, poka-yoke, a small curated set of tools, compact results. Tool errors form a feedback loop that demands telling recoverable cases from terminal ones and bounding the number of retries. On broad surfaces the shape of the contract changes: code execution replaces a sequence of JSON calls with writing code in a sandbox, and Skills package procedural knowledge into versioned artifacts with progressive disclosure (§12.6). Tools are the agent’s main security surface: the risk concentrates in the *lethal trifecta* (private data + untrusted content + external communication), bounded by the Rule of Two, taint tracking, least privilege, authorization with the user’s permissions, and human confirmation on irreversible actions.

Sources. Anthropic, “Building Effective Agents” (2024)—ACI (“invest in the ACI as much as in the HCI”), poka-yoke; provider documentation on function calling (the `tool_use` / `tool_result` formats, `tool_choice`, parallel calls); X. Wang et al., “Executable Code Actions Elicit Better LLM Agents” (CodeAct, 2024); Anthropic, “Code execution with MCP” (2025) and “Agent Skills” (2025); tool security—the sources in §12.5 (Willison, Meta, Beurer-Kellner et al., OWASP); Spring AI, LangChain4j, MCP.

In the Margins: Chapter 12

> ↩ **Zero-click**—a class of attacks in which exfiltration requires no action at all from the victim: the client, for instance, renders an image on its own, and the attacking instruction has packed the stolen data into that image’s URL. The countermeasure is control over outbound channels. → §15.7, Chapter 19.

Chapter 13. Memory and Reasoning Strategies

13.1 Memory in the agent loop (in brief, see §4.5)

§4.5 works through the architecture of memory (the memory store, the write and read paths, consolidation and reflection, forgetting, the MemGPT framework). Here: its place in the loop, and the full taxonomy.

Agent memory divides along several axes. By content: **episodic** (the history of specific interactions), **semantic** (durable facts and knowledge), and **procedural** (learned ways of acting—debugged step sequences for a routine task, say). By horizon: **short-term**—the context window of the current session (Chapter 4)—and **long-term**—state outside the window, from which what's needed comes back into the context. Long-term memory feeds the *observe* phase of the loop (§10.2), returning the slice of the past relevant to the current step.

One architectural variant matters in particular: **memory as a tool**. Instead of retrieving implicitly, the agent manages memory explicitly, through read and write tool calls (the MemGPT approach, §4.5), deciding for itself what to save and what to load; managing memory becomes part of the agent's reasoning. The key idea does not change: the long-term memory of a stateless agent is the pattern "store outside the window → retrieve what's relevant into the window"—the same retrieval as in RAG, but over session and user state.

When to consolidate: sleep-time compute. Consolidation and reflection (§4.5)—reworking raw episodic memory into durable facts and procedures—cost LLM calls themselves, and in a synchronous loop they compete with the user's task for latency and the turn's budget. A 2025–2026 direction (the term **sleep-time compute** comes from Letta; the name has not settled yet, the mechanics are already practice) moves this work *into the background*: a separate process, idle between sessions, deduplicates episodes, generalizes what recurs into semantic facts, crystallizes debugged sequences into procedures (the natural form to offload them into is a skill file, §12.6), and demotes the weight of what has gone stale. The economics work out on their own: background reworking has no latency requirement, so it runs on a cheap tier at the batch rate (§2.2)—memory consolidation turns out to be a classic offline batch workload, familiar to any backend engineer. The analogy with how human memory consolidates during sleep is only scaffolding: what it shares with neurobiology amounts to exactly this much—the reworking is separated from acting.

For the mechanics, see §4.5; from here the chapter is about how the agent *reasons*.

13.2 Reasoning strategies—the family

A reasoning strategy sets *how* the agent structures thinking and planning inside the loop. They line up in order of increasing cost/complexity; you choose to fit the task (§13.7).

13.3 ReAct (Reason + Act)

ReAct (Yao et al., 2022)—the canonical agent loop: interleaving **Reason → Act → Observe**. The model reasons ("the order status is needed"), acts (a tool call), observes the result, reasons again—and so on until it has an answer. At the level of the trace this is alternating blocks of *Thought* (the reasoning), *Action* (the tool call), and *Observation* (the call's result).

Why interleaving works. Reasoning alone, without actions, has no access to external facts and tends to hallucinate; actions alone, without reasoning, have no planning. ReAct joins them: every observation informs the next round of reasoning, and the reasoning directs the next action. Grounding the reasoning in actual observations reduces hallucination compared with reasoning that gets no external check (CoT in its pure form). This is the dominant control-loop pattern (§10.2).

Mechanics and the trade-off. The model picks each step *dynamically* (agent autonomy, §11.8), under an iteration limit (§10.5). Flexible and reactive, but nondeterministic and one model call per step (cost and latency). The characteristic failures are getting stuck (repeating the same action) and goal drift over a long trajectory; hence the need to check progress and to bound the step budget (§10.5).

Source. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (2022).

13.4 Plan-and-Execute

Plan-and-Execute—a strategy in which the agent *first builds a plan* of all the steps, *then executes* it, re-planning when things deviate. The contrast is with ReAct, which decides step by step. The prototype at the prompting level is Plan-and-Solve (Wang et al., 2023: "first devise a plan, then solve step by step"); the agent architecture generalizes it into an explicit planning phase followed by execution.

The trade-off against ReAct:

- Plan-and-Execute is **cheaper** (planning happens once, and execution needs fewer "thinking" calls), more **structured**, and holds up better on **long tasks** (an explicit plan reduces goal drift, §10.5).
- But it is **less adaptive**: the plan goes stale once reality has diverged; a step failure has to trigger a re-planning mechanism.

Optimizing execution. When the plan contains independent steps, run them in parallel: the LLMCompiler approach, for one, builds a graph of task dependencies and fires independent calls at the same time, cutting latency —the same reasoning as in Parallelization (§11.4).

Which when. ReAct—short reactive tasks with an unpredictable trajectory; Plan-and-Execute—long, relatively predictable multi-step tasks where structure and savings pay. The two are often combined: a plan on top, ReAct execution of the individual steps.

Source. L. Wang et al., "Plan-and-Solve Prompting" (2023); S. Kim et al., "An LLM Compiler for Parallel Function Calling" (2023)—LLMCompiler.

13.5 Reflection / Self-Critique

Reflection—the agent *analyzes its own* output or trajectory and improves the result. This is Evaluator-Optimizer (§11.6) applied to the agent's own reasoning. The development of it is **Reflexion** (Shinn et al., 2023): the agent reflects on *failures*, states the lesson in words, and retries with that lesson in hand, keeping the reflection in episodic memory across attempts (verbal "reinforcement" without updating weights).

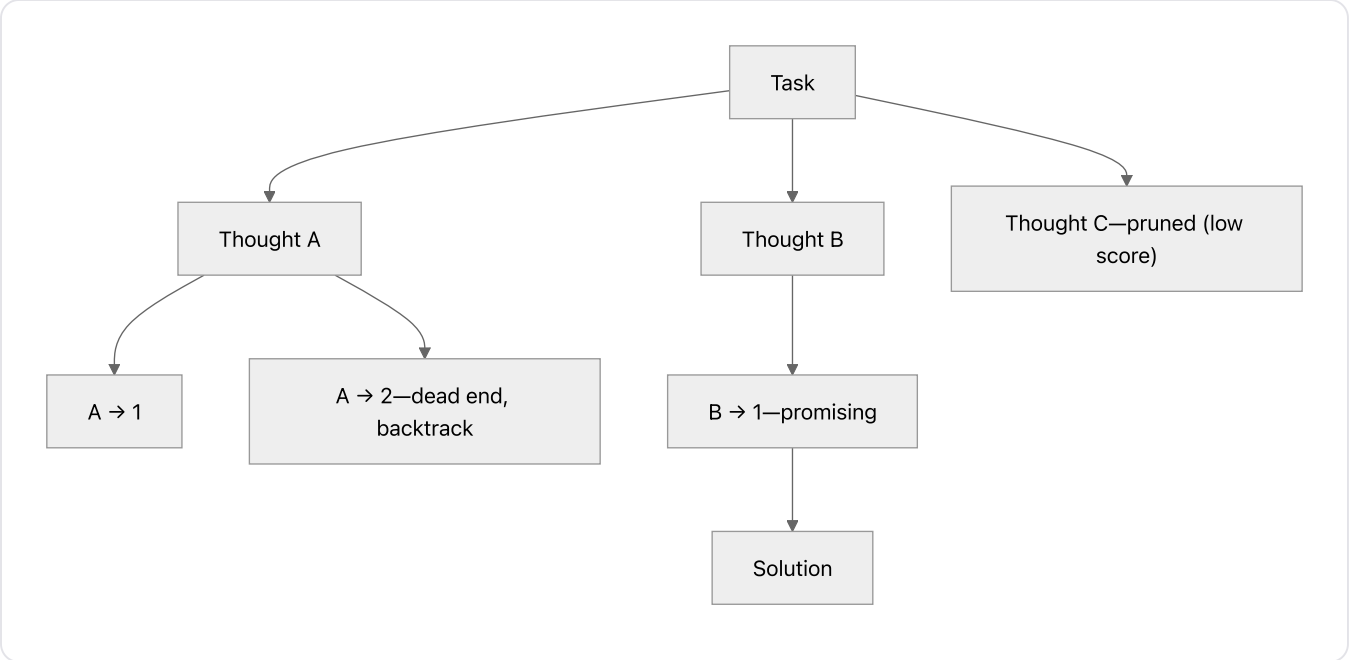
A serious limitation: self-correction without an external signal is unreliable. The source of the feedback is what fundamentally matters. *Internal* self-correction—trying to fix an answer relying only on the model's own abilities, with no external signal—is unreliable on reasoning tasks: Huang et al. (2024) showed that quality after this kind of self-correction often *gets worse rather than better*. Tyen et al. (2024) sharpened the reason: models are bad at *finding* their own errors but are able to *fix* them once the error's location is pointed out from outside. The practical conclusion follows: reflection works when an *external* signal is available—an execution result, a failing test, a compiler error, a verifier's verdict. That is exactly what the success of Reflexion (an external signal of task success or failure) and of Evaluator-Optimizer (an external evaluator) rests on. Self-critique without an external check is a weak and potentially harmful technique.

When. Tasks with a *checkable* criterion that yields an external signal (code with tests, proofs, answers with a reference). **The price:** extra rounds (cost and latency).

Sources. N. Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning" (2023); J. Huang et al., "Large Language Models Cannot Self-Correct Reasoning Yet" (ICLR 2024); G. Tyen et al., "LLMs cannot find reasoning errors, but can correct them given the error location" (ACL Findings 2024).

13.6 Tree-of-Thought and search

Tree-of-Thought (ToT) (Yao et al., 2023)—instead of a single reasoning chain, the agent *explores several branches*, scores them, and backtracks, like a tree search. ToT has three components: *thought generation* (several candidates for the next reasoning step), *state evaluation* (the model itself heuristically scoring how promising each branch is), and *a search algorithm* (breadth-first or depth-first traversal, pruning unpromising branches). It applies to tasks where a single line of reasoning often hits a dead end and the solution space needs scouting (hard planning, puzzles). The generalization is Graph-of-Thoughts, where the intermediate results form a graph rather than a tree, allowing branches to merge.



The price is high: the number of calls grows as the product of branching, depth, and the cost of evaluation. Apply the strategy only when a single chain (CoT or ReAct) has demonstrably failed to cope.

Source. S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models” (2023); M. Besta et al., “Graph of Thoughts” (2023).

13.7 Deep Dive: choosing a reasoning strategy

The frame is the same ladder—“don’t add complexity before you need it”—in order of increasing cost:

Task	Strategy	Cost
A simple fact/classification	A single call (no strategy)	Minimal
Tool use, reactive	**ReAct**	Medium
Long multi-step, predictable	**Plan-and-Execute**	Medium
Self-correction against a criterion is needed	**Reflection**	Higher
Scouting the solution space is needed	**Tree-of-Thought**	High

The rule: start with the cheapest option that covers the task; climb only when measurement (Chapters 9/17) shows it falling short. ToT and Reflexion are the top of the ladder, for hard cases—not a default choice.

13.8 Deep Dive: reasoning models change the calculus (2025+)

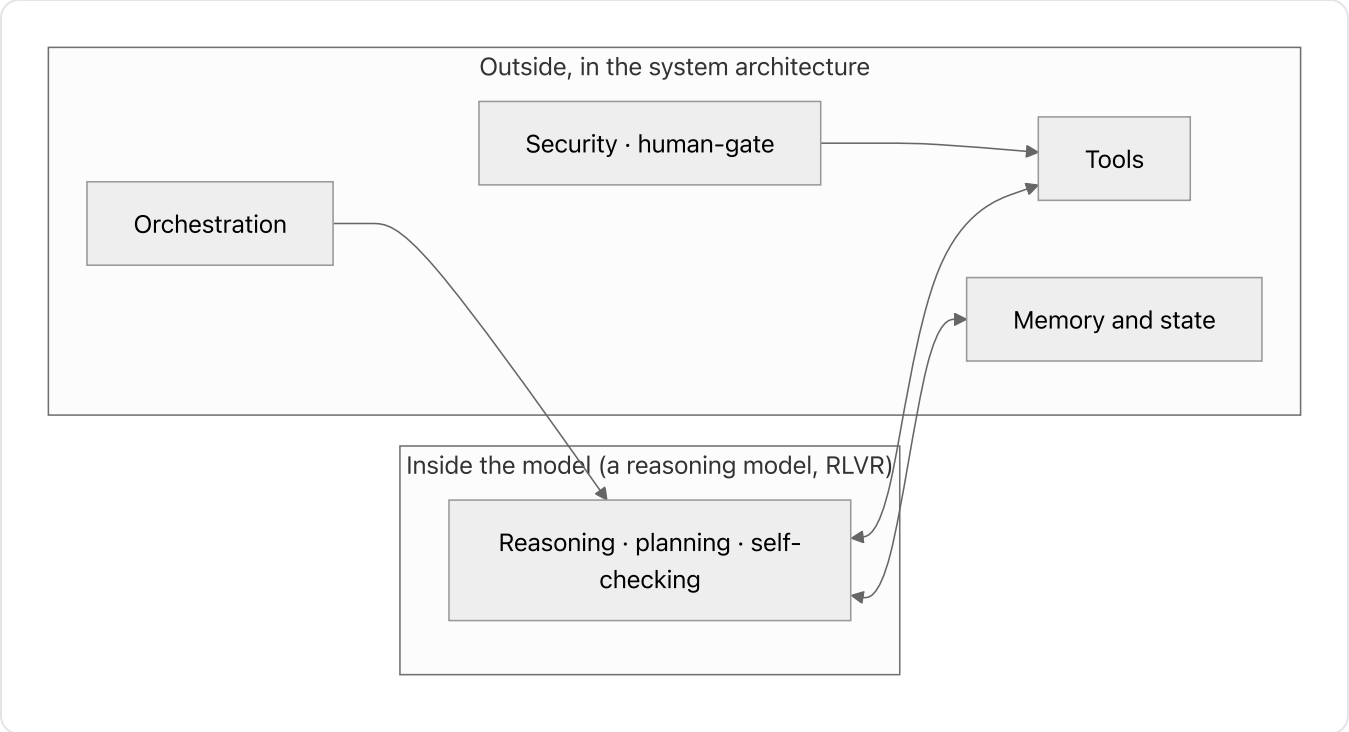
A major shift in the landscape. **Reasoning models**—specialized at first (the OpenAI o-series, DeepSeek-R1), and by 2026 the standard operating mode of most frontier models, open ones included—are trained to carry out a large share of the reasoning *internally*.

How they are trained: RLVR. At the base is **reinforcement learning with verifiable rewards (RLVR)**. Unlike RLHF, where the reward comes from a reward model trained on human preferences, RLVR computes the reward with a *deterministic verifier* that checks whether the answer is correct: matching against a reference in math, running unit tests in programming. The model produces a long reasoning chain as a sequence of actions and receives a signal about whether the outcome is right. OpenAI o1 (2024) was the first to show that reasoning quality scales with compute at inference time; DeepSeek-R1 (2025) demonstrated that a rule-based reward can elicit self-checking and reflection without dense human supervision (the GRPO° algorithm—an extension of PPO° that scores a group of answers to get a relative advantage).

Inference-time scaling. Hence the *inference-time scaling* regime: quality grows with the amount of compute spent per generation (longer reasoning chains, multiple samples), and this scaling differs in kind from pre-training scaling. The parameter you control becomes the internal reasoning budget, not hand-built CoT.

Reasoning interleaves with action (2025–2026). The first generation of reasoning models reasoned in a single block before answering; the next interleaves reasoning with tool calls: the model thinks, calls a tool, keeps reasoning over its result (interleaved thinking, "thinking with tools"). The control surface shifted too: instead of a hard token budget for reasoning, providers offer setting an *effort level*, within which the model itself decides how much to think about a given step. For the architect this means the boundary "reasoning inside, orchestration outside" has become porous to tools—part of the ReAct loop now runs inside a single model call—but the system still provides tool control, memory, budgets, and the human-gate.

A caveat. RLVR is most effective in domains where correctness is checkable (math, code); carrying it over to open-ended tasks with no verifier is a subject of active research. There is evidence (Yue et al., 2025) that RLVR mostly *redistributes probability mass across reasoning paths already present in the model* rather than extending its fundamental abilities, and that the gain saturates on finite data. Reasoning models amplify what the base already knows how to do; they do not create abilities out of nothing.



The consequence for architecture. The split is clean: **reasoning moves inside the model; orchestration stays outside, in the system architecture.** A reasoning model does not retire the tool-use loop, coordination, memory, the human-gate, or security (Chapters 10–12)—the system still provides all of it. Explicit ReAct prompting for the *reasoning* part is often redundant with a reasoning model, and sometimes makes the result worse (§3.4); at the same time, on simple tasks reasoning models tend to over-think, burning tokens and latency for no gain. The stronger the models' internal reasoning, the less external reasoning "scaffolding" is required—and the more the system's role in orchestration, state, and security matters.

Sources. OpenAI, "Learning to Reason with LLMs" (o1, 2024); DeepSeek, "DeepSeek-R1" (2025); Z. Shao et al., "DeepSeekMath" (2024)—the GRPO algorithm; N. Lambert, *RLHF Book*, the chapter on reasoning and inference-time scaling (2025); Y. Yue et al. (2025)—an analysis of the limits of RLVR; provider materials on interleaved thinking and adaptive effort levels (2025–2026).

13.9 Chapter summary

Memory (the architecture is in §4.5) feeds the loop with the relevant past; it divides into episodic, semantic, and procedural, and into short-term and long-term (§13.1). Reasoning strategies, in order of increasing cost: a single call → **ReAct** (reactive, with tools) → **Plan-and-Execute** (long, structured) → **Reflection** (self-correction, but only with an external signal) → **Tree-of-Thought** (scouting the solution space). Choose by the task, by what measurement shows (§13.7). The 2025+ shift: **reasoning moves inside reasoning models (trained through RLVR), while orchestration, state, and security remain the external architecture of the system** (§13.8).

Sources. S. Yao et al. (ReAct 2022; ToT 2023); N. Shinn et al. (Reflexion 2023); J. Huang et al. (2024); G. Tyen et al. (2024); L. Wang et al. (Plan-and-Solve 2023); OpenAI (o1 2024); DeepSeek-R1 (2025); Z. Shao et al. (GRPO 2024); Letta—sleep-time compute (2025), §13.1.

In the Margins: Chapter 13

↩ **PPO** (Proximal Policy Optimization, Schulman et al., 2017)—an RL algorithm: the "policy" here is the LLM itself (picking a token = an action), and updates are constrained so the policy cannot run far from the previous one. It requires a separate value model to estimate the advantage—how much better an action is than the expected level. → §21.6.

↩ **GRPO**—Group Relative Policy Optimization (Shao et al., 2024): the advantage is computed relative to the mean reward of a *group* of answers to the same prompt—no value model needed; cheaper in memory, the standard of the RLVR line. → §21.6.

Chapter 14. Multi-Agent Systems and the Taxonomy of Failures

The closing chapter of Part IV: when to split a system into several agents, how to coordinate them (patterns of orchestration and choreography), and the full taxonomy of agent failures—the one that distributed-systems discipline clears away.

14.1 What a multi-agent system is and when you need one

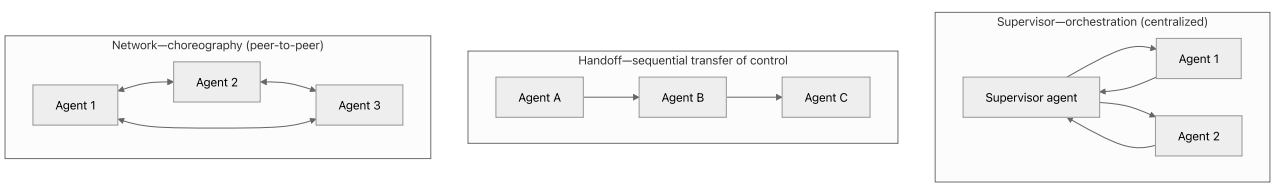
A multi-agent system—several specialized agents coordinating to solve a task, instead of one agent. When to break it up:

- **The single agent's context is picking up noise** (§4.4, isolation with sub-agents): spread the sub-tasks across agents with clean contexts.
- **Sub-tasks require incompatible sets of tools or instructions**—specialize the agents.
- **Specialization improves quality**—a "researcher agent," an "executor agent," a "critic agent."

When **not** to split (the same ladder): you do not multiply agents until a single agent has demonstrably failed to cope. Every additional agent adds cost, coordination, and debugging difficulty. Multi-agent is not a goal but a tool for a proven need; most tasks are covered by one well-designed agent (Chapters 10–13), and multi-agent is the top of the ladder. Empirically, the quality gain of multi-agent systems over a single agent on common benchmarks is often small (Cemri et al., 2025, §14.4), which strengthens the argument against splitting without an explicit need.

14.2 Deep Dive: coordination patterns (orchestration vs. choreography)

Coordinating agents reproduces the general dichotomy of distributed systems—orchestration against choreography (Richardson):



- **Supervisor / Orchestrator (centralized)**—a lead agent coordinates the workers and collects the results. ≈ the **orchestration saga** (§11.5), scaled up to agents.
- **Handoff / Sequential**—an agent hands control to the next one. ≈ pipeline / partial choreography.
- **Network / Peer**—agents call each other directly, decentralized. ≈ **choreography**.

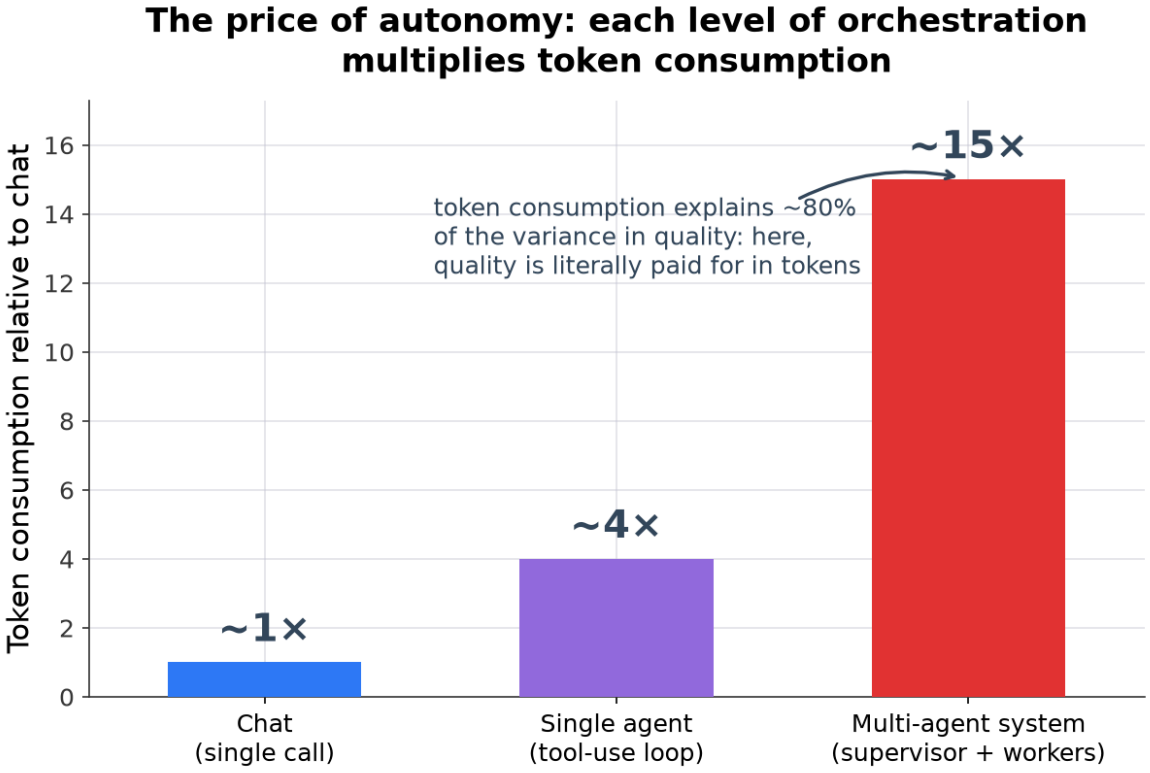
- **Hierarchical**—supervisors above supervisors. It pays off once a single supervisor can no longer hold the coordination (context, throughput); the price is that the second level itself becomes a bottleneck and a point of failure, and the trace grows longer by another level.

The key trade-off is the same one as orchestration ↔ choreography: a centralized supervisor is easier to reason about, debug, and observe (a single point of coordination), but it is a bottleneck and a single point of failure; a decentralized network is more flexible, but *harder to trace and analyze* (emergent behavior°, cyclic handoffs). The practical conclusion: **orchestration (supervisor) is the default preference**—it is more predictable and more observable; you move to choreography once centralization genuinely becomes the bottleneck. This is the same logic that governs the choice between an orchestration saga and a choreography saga. For agents communicating *across organizations* (different owners, a supervisor impossible), a protocol standard is taking shape—A2A (§15.6).

14.3 Deep Dive: the price of multi-agent

Be honest about the price, because multi-agent is seductive and overrated. It **multiplies**:

- **Cost**—each agent = LLM calls; coordination adds still more (§2.2). Concrete figures from Anthropic's report on a multi-agent research system (2025): ~15× the tokens relative to chat (a single agent—~4×), and token spend explained ~80% of the variance in quality—here you literally pay for quality in tokens.
- **Latency**—coordination round-trips between agents add up (§2.3).
- **Debugging and observability difficulty**—tracing behavior *across* agents is harder by a multiple (Chapter 18); emergent coordination bugs.
- **Context loss between agents** (§4.4)—an agent does not know what its neighbor knows; only what is passed explicitly gets passed. A subtle source of misalignment errors.

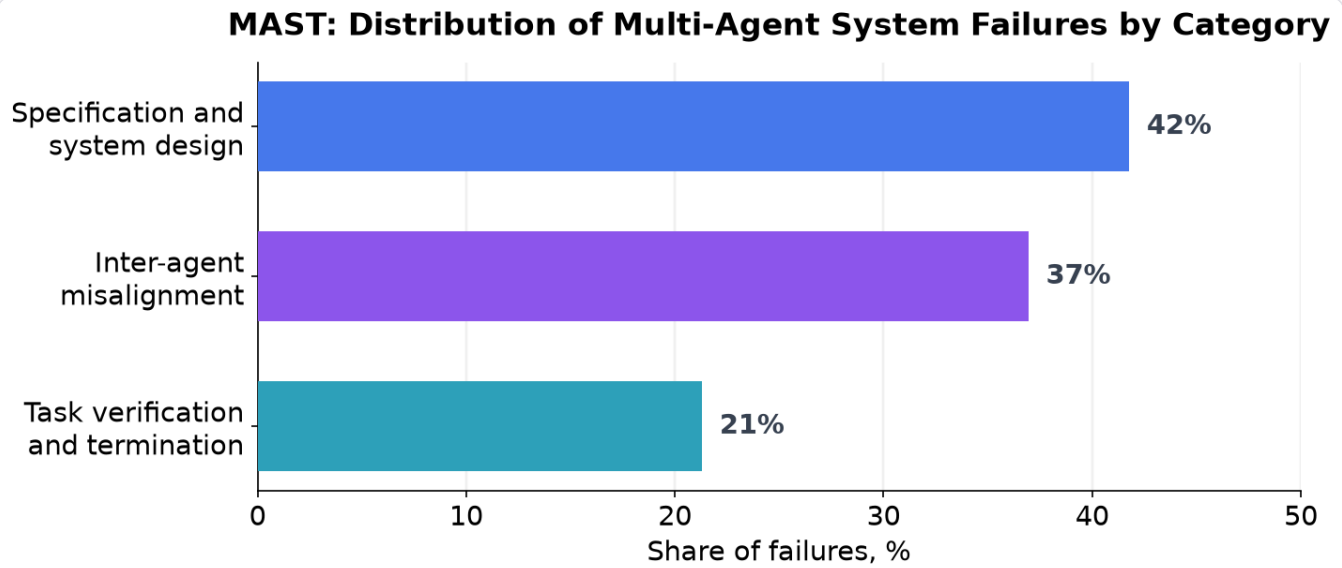


Based on Anthropic's report on its multi-agent research system (2025). Multi-agent designs pay off on highly parallelizable, high-value tasks — and only when the value justifies the multiplied cost (§14.3).

The conclusion (Anthropic's, and practice's): multi-agent is justified for *parallelizable* tasks of high value (broad research with independent sub-tasks, for instance), but only when the value justifies the multiplied cost. That same point dissolves the apparent contradiction with the data on multi-agent systems' weak gains on typical benchmarks (Cemri et al., §14.4): both results are true—the gain is small on sequential tasks and large on broadly parallelizable ones, which is exactly where Anthropic measured. **Multi-agent is a distributed system with every coordination cost, plus nondeterministic nodes.** You do not move to it by default: first exhaust what a single agent and the workflow patterns can do (§11.8).

14.4 Deep Dive: the full taxonomy of agent failures

Failures of agent systems can be cataloged systematically. The empirical basis comes from **MAST (Multi-Agent System Failure Taxonomy)** (Cemri et al., 2025)—the first empirically grounded taxonomy of multi-agent system failures, built on an analysis of more than 200 tasks across seven frameworks (over 1,600 annotated traces, annotator agreement $\kappa = 0.88$). MAST identifies 14 failure modes in three categories: (i) *specification and system design* (badly posed tasks, wrong assignment of roles, weak error handling), (ii) *misalignment between agents* (communication breakdowns, conflicting goals, context lost in the handoff), and (iii) *task verification and termination* (insufficient validation of the output, an error propagating down the chain).



Cemri et al. (2025), 14 failure modes across 3 categories (7 frameworks, 1,600+ traces, κ=0.88). Failures are predominantly design flaws, not artifacts of base-model quality.

The study's key finding: most failures are *design* failures (specification and coordination dominate) rather than a matter of base-model quality—in the authors' formulation, improving the model's capabilities is not enough to close out the whole taxonomy. This directly supports the position of §14.1 and §14.3: the price of multi-agent is not tokens alone but a whole class of coordination design failures.

Below is a composite practical catalog of agent failures (covering both the single agent and multi-agent), grouped by subsystem; most entries are distributed-systems failures with a nondeterministic surcharge.

Category	Failure	Analog in distributed systems	Mitigation (and where in the book)
Control loop	Looping / nontermination	Infinite retry	Iteration cap, budget (§10.5)
	Goal drift over a long trajectory	State drift	Periodic re-grounding, a plan (§10.5, §13.4)
	Premature stop	Early exit	Termination criteria, a completeness check
Tools/actions	Hallucinated call / wrong arguments	Contract violation	Schema + validation (§5, §12.4)
	Silent partial failure under parallelism	Scatter-gather partial failure	Explicitly track every branch (§11.4)
	Silent success when a sub-step fails	Swallowed error	Propagate errors, check the result
	Dangerous composition of tools	Confused deputy, excessive privileges	Audit the *set*, least privilege (§12.5, §19)
Reasoning	Hallucination	— (LLM-specific)	RAG grounding (Part III), evals (Chapters 9, 17)
	A wrong plan	Wrong business logic	Plan-and-Execute + re-plan, evaluation (§13.4)
	Error cascade (compounding step by step)	Cascading failure	Checks between steps, evaluator (§11.6)
Context/memory	Context overflow	Working-state overflow	Compaction, sliding window (§4.4)
	Lost-in-the-middle degradation	— (LLM-specific)	Curation, placement (§1.8, §4)
	Stale memory	Cache staleness	Forgetting/refresh policies (§4.5)
	Context pollution	—	Isolation with sub-agents (§4.4)
Coordination (multi-agent)	Context loss between agents	State desync	Explicit handoff, shared state (§14.3)
	Cyclic/deadlocked handoffs	Deadlock	Supervisor instead of peer, limits (§14.2)
	Responsibility gap	Nobody owns the step	Clear boundaries of agent responsibility

Category	Failure	Analog in distributed systems	Mitigation (and where in the book)
Security	Prompt injection (direct/indirect)	Injection (SQLi / stored XSS)	Segregation, guardrails (§19)
	Excessive agency	Excessive privileges	Least privilege, human-gate (§10.6, §19)

The table serves as a checklist during design and incident review: for each agent, determine which of these failures are possible and what covers them.

An industry security framework complements MAST's empirical taxonomy—the "OWASP Top 10 for Agentic Applications" (2026, §12.5): at least two of its categories cover the same modes as the table above (ASI06 Memory & Context Poisoning ↔ stale memory and context pollution; ASI07 Insecure Inter-Agent Communication ↔ context loss and deadlocked handoffs between agents). Running both frameworks is the practical move: MAST as quality diagnostics, OWASP as threat analysis.

14.5 Deep Dive: treating failures with distributed-systems discipline

The meta-conclusion of all of Part IV. As the table in §14.4 shows, the overwhelming majority of failures are **distributed-systems failures with a nondeterministic surcharge**. So the reliability discipline of distributed systems applies almost in full:

- **Idempotency** (agents retry often)—by task key (saga).
- **Timeouts and budgets**—on a step *and* on the whole loop (§10.5).
- **Compensations** for the reversible, a **human-gate** for the irreversible (§10.6)—you do not compensate the irreversible.
- **Circuit breaker and fallback** (§2.5)—for external dependencies and for the provider.
- **Observability/tracing**—by spans per step (Chapter 18).
- **Bulkhead**—isolating context and sub-agents (§4.4).

What is genuinely **new** beyond the classics (and these are three axes absent from deterministic systems):

1. **Bounding nondeterminism**—budgets on reasoning loops, because there is no transition graph (§10.5).
2. **Evals for probabilistic behavior**—tests have become distributions (Chapters 9, 17); `assertEquals` does not work.
3. **Guardrails for a manipulable actor**—defense against injection and excessive agency (Chapter 19), because the "code" of the decision can be knocked off course by data.

The frame in one phrase: **agent reliability = distributed-system reliability + the discipline of probabilistic control.** The first term is settled engineering practice; the second is the new one, and Part VI is devoted to it. Designing distributed systems covers a substantial share of what agent systems require, leaving the specifics of probabilistic control a separate but bounded layer.

14.6 Summary of the chapter and of Part IV

Multi-agent is a distributed system; coordination runs through orchestration (supervisor, more predictable) or choreography (peer, more flexible), with a preference for the first (§14.2); you do not multiply agents without proven need—the cost is a multiple (§14.3). The failure taxonomy (§14.4) is mostly distributed failures plus nondeterminism, predominantly design failures (MAST); they are cleared by existing discipline (idempotency, timeouts, compensations, observability, bulkhead) combined with the new discipline of probabilistic control (budgets, evals, guardrails, §14.5).

Part IV in sum. The model now *acts*: as a component (Part I), with a tamed interface (Part II) and grounded knowledge (Part III), organized by agent patterns (Chapters 10–11), armed with tools-as-contracts (Chapter 12), with memory and reasoning strategies (Chapter 13), coordinated and defended by distributed-systems discipline (Chapter 14). Next comes **Part V: integration** (MCP as the standard and the JVM stack, Spring AI/LangChain4j)—that is, how to embed all of this in a real organization's architecture.

Sources. Anthropic, "Building Effective Agents" (2024) and material on multi-agent research systems (2025)—coordination patterns, the price in tokens; V. Dibia, *Designing Multi-Agent Systems* (2025)—supervisor/handoff/network; M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" (MAST, arXiv:2503.13657, 2025)—the empirical taxonomy of failures; C. Richardson, *Microservices Patterns* (2018)—orchestration vs. choreography, the basis for mapping the failures.

In the Margins: Chapter 14

> ↩ **Emergent behavior**—behavior of a system that is built into none of its components and arises out of their interaction; in multi-agent systems, the main source of failures invisible when agents are tested one at a time. → §14.4 (MAST, the misalignment category).

Part V. Integration: Embedding Into the Architecture

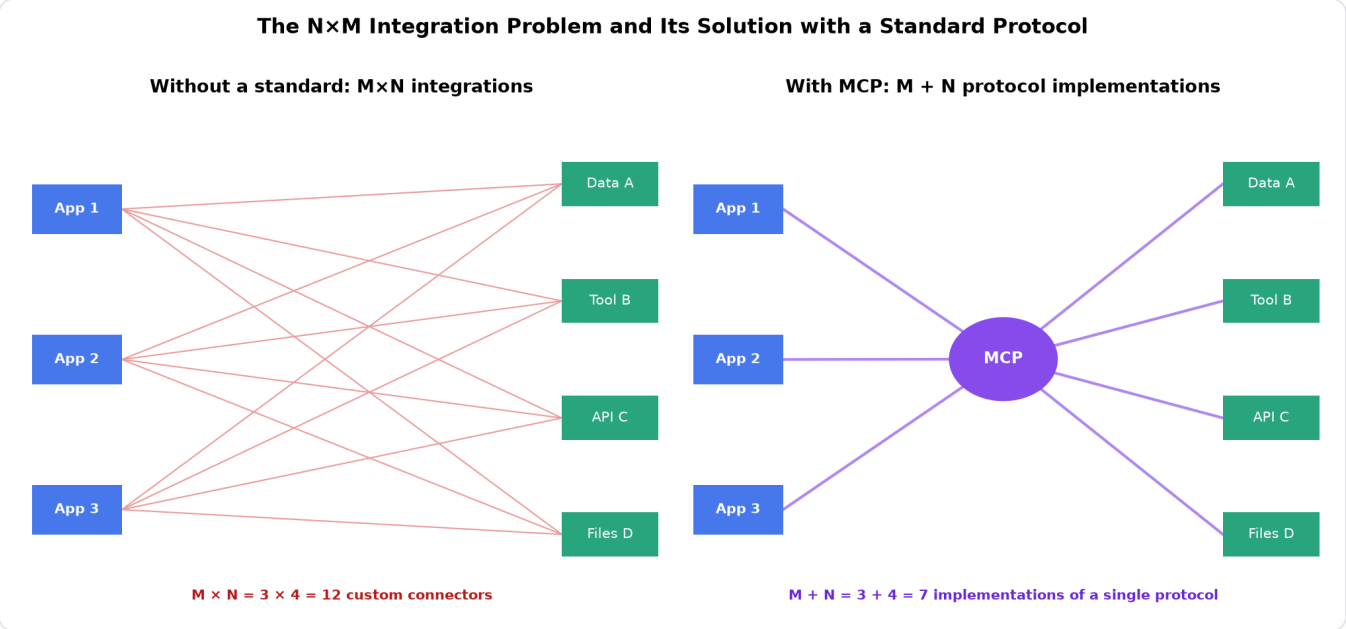
Parts I–IV built the agent as a system. Part V is about fitting that system into an existing architecture: through a standard protocol for integrating with tools and data (Chapter 15, Model Context Protocol) and through a concrete implementation layer on the JVM (Chapter 16). The shift in Part V is from the question “how is an agent built” to the question “how does an agent connect to the rest of the organization’s landscape by contract rather than through one-off integrations.”

Chapter 15. Model Context Protocol (MCP)

Chapter 12 showed that a tool is a contract the model reads. MCP answers the next question: how to *standardize* that contract across different applications and different sources of tools and data, so that integrations get reused instead of rewritten for every pair. MCP is an open protocol Anthropic proposed in November 2024; since December 2025 the specification has been governed by the Linux Foundation (the Agentic AI Foundation, co-founded by Anthropic, Block, and OpenAI, with support from Google, Microsoft, and AWS)—for an architect making a multi-year bet on a protocol, that removes a substantial part of the vendor lock-in risk. The protocol is built on JSON-RPC 2.0 and standardizes how an AI application discovers and calls tools, reads data, and fetches prompt templates from external servers.

15.1 The N×M problem

Before the standard, integration suffered from the **N×M problem**: M AI applications, each of which needs access to N tools and data sources, require in the worst case M×N individual connectors—every "application ↔ source" pair implemented separately and incompatibly. As both sides grow, the number of integrations grows multiplicatively.



A standard protocol turns this into **M + N**: each application implements the client side of the protocol once, each source implements the server side once, and after that any client works with any server. MCP moves tool integration off the level of private, framework-specific abstractions and onto the level of **protocol interoperability**—with capability discovery, negotiation, and reuse across heterogeneous systems. The analogy, by role, is a standard peripheral interface: a device plugs into any host that supports the same connector, instead of a separate cable for every pair.

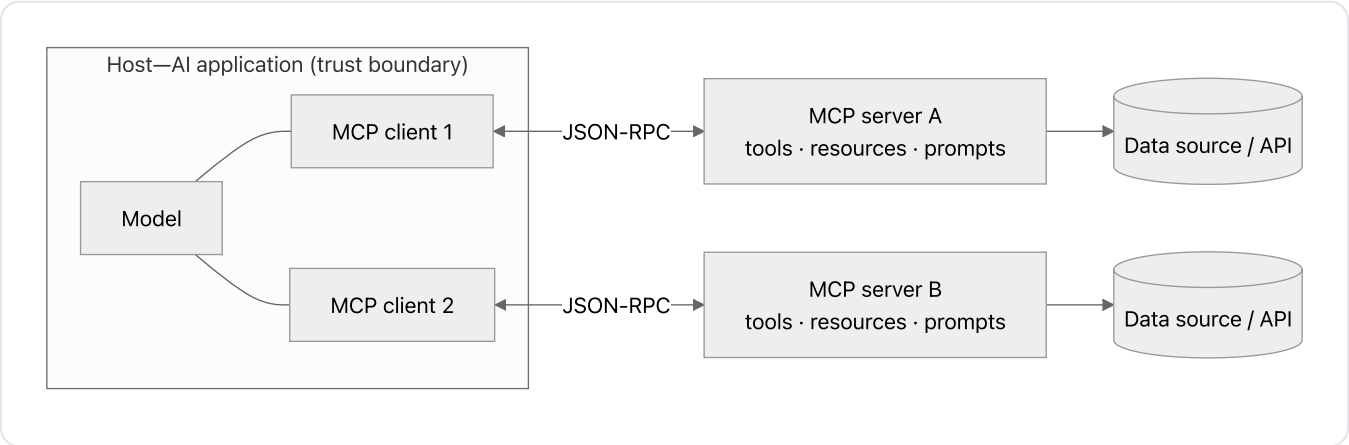
The protocol's infrastructure companion is the **MCP Registry** (the official server registry, in preview since September 2025; by the end of the year, on the order of two thousand entries with a discovery API). The registry settles the discovery question ("which servers exist and where to get them") but not the trust question: publishing to the registry is not an audit, and the discipline of §15.7—treating a third-party server as an untrusted supply-chain component—extends to registry servers as well.

15.2 Architecture: host, client, server

MCP defines three roles.

- **Host**—the user-facing AI application (for example, an IDE assistant, a chat application, an agent environment). The host holds the model and orchestrates the work.
- **Client**—the protocol component *inside* the host; it maintains a one-to-one connection with a server. Each connected server gets its own client.

- **Server**—a process that exposes capabilities (tools, data, templates); it wraps a specific tool or data source. The server **does not interact with the model directly**—all communication with the model is mediated by the host through the client.



Separating the roles matters for security and for reasoning about the system: the server is a potentially untrusted third-party component (§15.7), while the host remains the trust boundary and the point of control. The server sees only what the host passed to it through the client, and has no direct access either to the model or to the rest of the host's context.

15.3 Primitives

MCP describes capabilities through a small set of primitives; the terseness of the taxonomy is a deliberate decision that covers most practical cases.

Server primitives:

- **Tools**—executable operations with side effects: a call changes the state of the world (a database write, a message sent, an API call). Each tool is described by a JSON schema for its parameters and its result (kin to tool calling, Chapter 12). In short: *tools act*.
- **Resources**—*read-only* access to data: a user profile, a file's contents, a document, a configuration. A resource returns context but does not change state—a low-risk primitive. In short: *resources read*.
- **Prompts**—reusable parameterized templates that standardize routine interactions (for example, a code-review template with "language" and "file path" parameters). In short: *prompts standardize*. In practice servers most often expose tools, resources less often, and prompts almost never—even though prompts are exactly what lets you load the resources you need into the context explicitly, as part of a reproducible workflow.

Client primitives (capabilities the host provides and the server can request):

- **Roots**—the boundaries (filesystem or URI) within which the server is invited to operate; the host declares which areas the server has access to. An important caveat: in the base protocol, roots are a *declaration of expectation*, not isolation; honoring them is left to the server, and hard enforcement comes from outside the protocol—at the OS/container level.
- **Sampling**—the server can ask the client for *a call to the model* (a generation). This inverts the usual direction: the server uses the host's model without holding an access key of its own, and the host controls whether to allow the request and which model to apply.

- **Elicitation**—the server can ask the *user* for additional information mid-operation (for example, a missing parameter). Both client primitives keep the user and the host in the control loop.

15.4 The session lifecycle and capability negotiation

MCP works through a **stateful session**, not through one-off REST calls. A session opens with an `initialize` handshake, in which:

- **The protocol version is negotiated**—a version field (date strings of the form `2025-06-18`, `2025-11-25`) guarantees the two sides are compatible; if there is no compatible version, the connection terminates.
- **Capabilities are negotiated**—each side declares which primitives and features it supports (the server: tools, resources, prompts, subscriptions; the client: sampling, notification handling). Both sides must honor the declared capabilities for the duration of the session.

Session state enables **dynamic updates**: the server notifies the host that its capability set has changed (the tool list changed, say) through notifications. This is what separates MCP from a static tool catalog hardwired into the application.

15.5 Transports

The specification defines two transports over JSON-RPC:

- **stdio**—local, within a single machine: the server runs as a subprocess and the exchange goes through the standard I/O streams. Simple, and a fit for local integrations.
- **Streamable HTTP**—remote, over HTTPS (using SSE° for streaming from server to client). It replaced the early HTTP+SSE transport and targets remote, multi-client deployments.

Topology picks the transport: a local tool—stdio; a network service shared by several hosts—Streamable HTTP, with the corresponding authentication and encryption requirements (§15.7).

15.6 MCP and tool calling: where the line falls

MCP does not replace tool calling (Chapter 12); it standardizes its *transport and packaging*. The model still performs function calling—deciding which tool to call and with which arguments (§12.2). MCP defines how tools (and resources and prompts too) are *discovered, described, and invoked* across the "host ↔ server" boundary, uniformly for any compatible server. Put differently: tool calling is *the model's decision to call*; MCP is *the delivery standard* for getting tool definitions into the context and calls out to the executor. The tool-interface design principles from §12.3 (clear descriptions, strict schemas, compact results, a curated set) apply to MCP tools unchanged.

Deep Dive: agent interoperability—A2A

MCP standardizes the "agent ↔ tools and data" axis. For the "agent ↔ agent" axis, **A2A (Agent2Agent)** is taking shape—a protocol Google proposed (April 2025) and handed to the Linux Foundation (June 2025). The mechanics: an agent publishes an **Agent Card**—a JSON manifest of its capabilities and its endpoint; interaction is organized

around **tasks** with a lifecycle (submitted → working → input-required → completed/failed), with streaming and push notifications for long-lived operations—that is, the protocol is designed from the start for asynchronous, multi-minute agent tasks rather than a synchronous function call. The protocols are complementary: internally an agent uses tools over MCP; outward it exposes a task interface over A2A. For multi-agent systems (Chapter 14), A2A is a candidate standard for *cross-organizational* choreography, where supervisor orchestration is impossible: the agents have different owners. Caveats: the ecosystem is markedly younger than MCP's, and agent-to-agent communication is a risk class of its own (ASI07, §14.4)—authenticating agents, trusting someone else's Agent Card, taint through inter-agent messages; the disciplines of §15.7 apply here too.

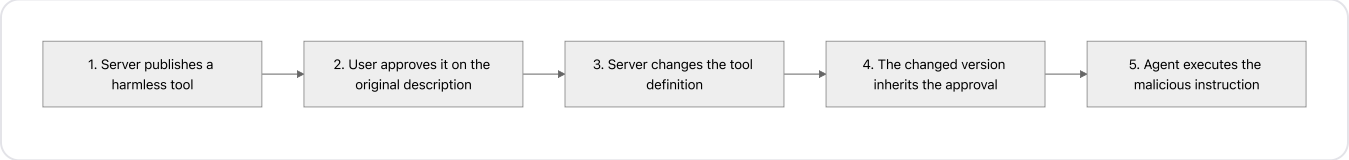
Sources. A2A Project (Linux Foundation, 2025 specification); Google, the Agent2Agent announcement (2025).

15.7 Deep Dive: MCP security (a bridge to §12.5 and Chapter 19)

MCP widens the agent's attack surface, and its own threat model matters. The key observation: **MCP's very design—plugging in tools from different sources—makes the lethal trifecta easier to assemble** (§12.5). Mixing servers, a system readily combines access to private data, untrusted content, and external communication channels in a single session; so the Rule of Two and taint tracking (§12.5) apply to MCP directly.

Attack classes specific to MCP:

- **Tool poisoning.** Malicious instructions are planted in a tool's *description* or metadata—which the model reads but the user may never see—and the model follows them. Unlike a one-off injection, poisoning is not confined to a single session: it hits every agent that reads the poisoned tool. The specification explicitly directs that tool annotations be treated as untrusted unless the server itself is trusted.
- **Rug pull.** The server changes a tool's definition *after* the user has approved it. The approval covered the original configuration, and the base protocol has no mechanism for tracking definition changes or requiring re-approval—so the modified version inherits the trust. The class parallels attacks on package indexes (a package is uploaded harmless, then modified).



- **Tool shadowing and cross-server shadowing.** A malicious server registers a tool whose name or description matches a legitimate one, intercepting the calls; in cross-server shadowing, one server's tool description targets a *different* trusted server in the same session. Demonstrated (Invariant Labs, April 2025): through instructions embedded in a description, a malicious server made an agent pull message history out of a legitimate server and ship it outward over a trusted channel—transport encryption is no defense, because the leak happens above the encryption boundary, through the agent's legitimate access.
- **Return value poisoning.** A tool call's result carries planted instructions that hijack the reasoning that follows (indirect injection, §12.5).
- **Real CVEs (2025).** OS command injection through an untrusted connection (`mcp-remote`, CVE-2025-6514); configuration substitution in an IDE following the rug pull pattern (CVE-2025-54136) and execution via injection from an external source (CVE-2025-54135).

The defenses build up across several levels:

- **Authorization.** The 2025-06-18 revision of the specification classifies MCP servers as OAuth Resource Servers and requires clients to implement Resource Indicators (RFC 8707), so that a token issued for one server cannot be used against another—and that same revision introduces elicitation (§15.3). The 2025-11-25 revision extends authorization: OIDC Discovery, incremental scope consent, OAuth Client ID Metadata Documents.
- **Definition integrity.** Pin approved tool descriptions by hash and reject mismatches; the ETDI proposal (Bhatt et al., 2025) binds tool definitions to signed tokens and versions them, so that any change invalidates the signature (not yet part of the core specification, but the principle is applicable today).
- **The user loop.** Showing the user what a tool is about to send outward is an effective defense against exfiltration; draw a clear visual line between what the user sees and what the model sees.
- **Server hygiene.** Input validation, rate limiting, output sanitization, access control on resources; treating a third-party MCP server as an untrusted supply-chain component.

Deep Dive: agent identity—who exactly is acting

The authorization above answered the question "is this call allowed." The question one level down—**on whose behalf** it is made—has three bad default answers and one right one.

Three candidate identities. (1) *An application service account*: one account with broad rights across all users—a confused deputy waiting to happen (§12.5): the agent executes a narrowly privileged user's request with the system's rights, and any injection inherits those rights. (2) *The user's identity*: the agent is an invisible client; the rights are correct, but the action log cannot tell a human's actions from those of their agent—the audit goes blind, and compromising the agent equals compromising the user. (3) *The agent's own identity* (**non-human identity, NHI**—the established industry term for machine accounts: services, bots, now agents): with no link to a user, the agent cannot act on that user's behalf. The right answer is **the pair**: every action carries both *who initiated* it (the user) and *what executed* it (the specific agent)—and both identities land in the tracing spans (Chapter 18).

The delegation mechanics—OAuth 2.0 Token Exchange (RFC 8693). How to express "agent A acts on behalf of user B" technically: the harness exchanges the user's token at the authorization server for a *new* downstream token in which the subject is the user, the actor is the agent (the `act` claim), the scope is *narrowed* to what the next step needs, and the audience is a specific service (a resource indicator, RFC 8707 above). In the Microsoft ecosystem this flow is called **on-behalf-of**. The key discipline along the user → agent → MCP server → internal API chain: at every hop the token is *exchanged and narrowed*, not forwarded onward—the MCP specification explicitly forbids passing the user's token straight through (passthrough): a server must accept only tokens issued *to it*. Otherwise a token stolen at a far hop opens everything the user could reach.

Where the agent's own identity comes from. The same question as for any workload with no human at the keyboard, and the industry's answer is the same—**workload identity**: the platform (SPIFFE/SPIRE as the open standard, for example) issues the workload a short-lived cryptographic credential based on its *attributes* (which pod, which service, which code) rather than on a secret in a config file. An agent is no different from a microservice here—three things are different. First, **nondeterminism**: a microservice's rights can be derived from its code; an agent's rights are set *up front* and *from outside*—least privilege over the declared tool set (§12.5), not over observed behavior. Second, **duration**: an agent's trajectory can outlive a token's TTL—refreshing tokens mid-task falls to the harness. Third, **scale**: dozens of agents per organization quickly produce an explosion in the number of NHIs, and without a registry (Chapter 25) they turn into an unmanageable fleet of accounts—which is exactly why OWASP Agentic names agent identity abuse a risk category of its own.

The upshot: MCP introduces no new security *principles* beyond Chapters 12 and 19, but it makes following them critical, because it lowers the barrier to plugging in untrusted third-party tools. The discipline of the lethal trifecta and the Rule of Two (§12.5) is the base frame for MCP as well; identity and delegation are its foundation one level

down.

15.8 Implementation on the JVM

The JVM platform has an official MCP Java SDK (built with the Spring team's involvement). **Spring AI** provides both client and server support for MCP: an application can act as an MCP client, connecting to external servers, or expose methods annotated `@Tool` as an MCP server—offering its capabilities to any MCP-compatible agent through a standard protocol rather than through a private integration. This lifts the contract-first approach to tools (Chapter 12) up to the level of cross-system integration.

15.9 Chapter summary

MCP is an open protocol (JSON-RPC) that standardizes how AI applications integrate with tools, data, and templates, turning the N×M problem into M+N. The architecture is three roles: the host (the application with the model), the client (the protocol component inside the host, one per server), and the server (which exposes capabilities without talking to the model directly). Capabilities are described by primitives—server-side (tools act, resources read, prompts standardize) and client-side (roots, sampling, elicitation)—negotiated in the `initialize` handshake within a stateful session; the transport is stdio locally or Streamable HTTP remotely. MCP does not replace tool calling; it standardizes its delivery (§15.6). The main caveat is security (§15.7): MCP makes the lethal trifecta easier to assemble and introduces attack classes of its own (tool poisoning, rug pull, shadowing), so it demands authorization (OAuth, RFC 8707), definition integrity, and treating servers as untrusted components. Organizationally the protocol is vendor-neutral (under the Linux Foundation since December 2025) and has acquired an official server registry (§15.1); the adjacent "agent ↔ agent" axis is covered by the A2A protocol (§15.6). Next, Chapter 16: assembling the full stack on the JVM.

Sources. Anthropic, the Model Context Protocol specification (modelcontextprotocol.io; revisions 2025-06-18, 2025-11-25)—architecture, primitives, lifecycle, transports; X. Hou et al., "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions" (2025); Invariant Labs, the tool poisoning and rug pull advisories (2025); M. Bhatt et al., "ETDI: Mitigating Tool Squatting and Rug Pull Attacks in MCP" (2025); S. Willison, "The Lethal Trifecta" (2025, §12.5); Linux Foundation / Agentic AI Foundation—the transfer of MCP governance (December 2025); MCP Registry (registry.modelcontextprotocol.io, preview 2025); identity and delegation—RFC 8693 (OAuth 2.0 Token Exchange), RFC 8707 (Resource Indicators), SPIFFE/SPIRE (workload identity); the Spring AI documentation on MCP.

In the Margins: Chapter 15

> ↩ **SSE**—Server-Sent Events: a one-way streaming channel server→client over plain HTTP; simpler than WebSocket and sufficient for delivering events.
>
> ↩ The OAuth role model: the **Authorization Server** issues tokens, the **Resource Server** accepts and validates them, the **Client** requests access. Classifying an MCP server as a Resource Server means it issues no tokens itself and trusts an external AS.
>
> ↩ **ETDI**—Enhanced Tool Definition Interface: an academic proposal to sign and version tool definitions; not part of the core specification—the practical equivalent is pinning approved descriptions by hash.

Chapter 16. Assembling the Stack on the JVM

The abstract patterns of Parts I–IV have concrete implementations on the JVM platform. This chapter maps the architectural concerns—calling the model, templating, structured output, RAG, tool calling, memory, orchestration, observability, MCP integration—onto the constructs of two mature frameworks: **Spring AI** (1.0 GA—May 2025, 1.1 GA—November 2025) and **LangChain4j** (1.0 GA—May 2025; the agentic module `langchain4j-agentic` is noticeably younger and still in active flux). The goal is not a list of API methods (which changes from release to release) but a durable "pattern → construct" correspondence—enough to assemble a full agent stack and choose tooling deliberately.

16.1 The two options and their philosophies

Spring AI follows Spring's idiom: auto-configuration, starters, beans, dependency injection. Its central construct is the **ChatClient**, a fluent, type-safe API for calling the model (close in spirit to WebClient and RestClient). Cross-cutting concerns (memory, RAG, filtering) are pulled out into **Advisors**—a chain of interceptors around the model call. The framework integrates tightly with Spring Boot and the Micrometer ecosystem.

LangChain4j is a framework-agnostic library that works with Quarkus, with Spring Boot, and with no framework at all. It offers two levels: low-level components (`ChatModel`, `ChatMemory`, `EmbeddingModel`, `EmbeddingStore`) for full control, and high-level **AI Services**—declarative interfaces with annotations (`@SystemMessage`, `@UserMessage`, `@Tool`, `@MemoryId`) from which the framework generates a proxy implementation. The declarative approach takes its cue from Spring Data JPA: the developer describes an interface, the library produces the implementation.

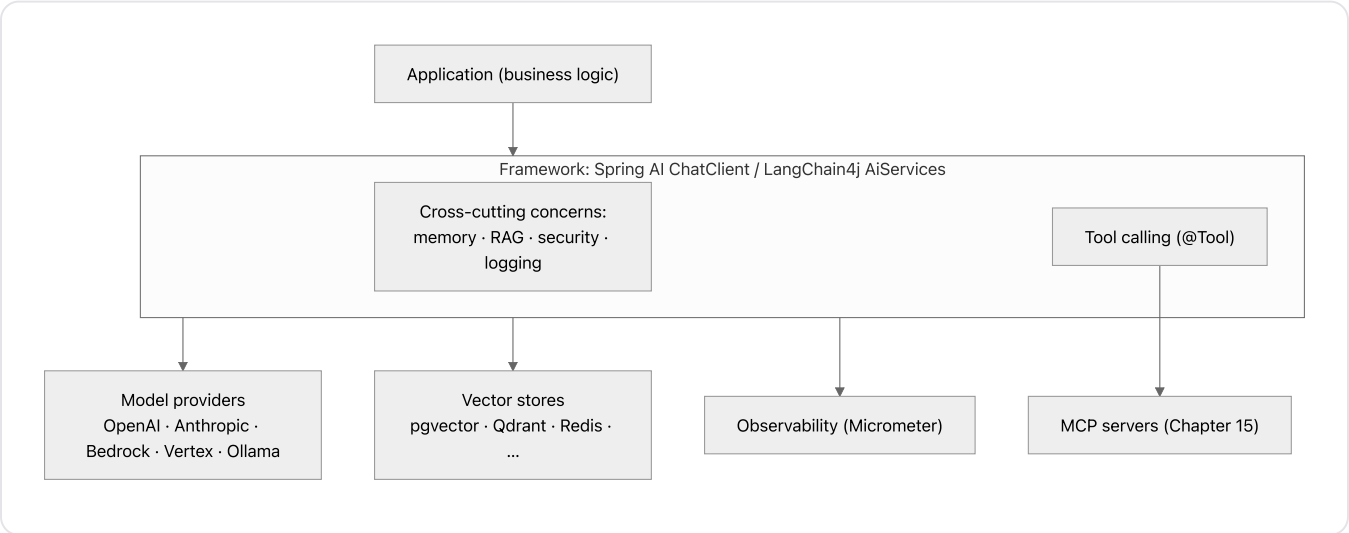
Both libraries put a single API over a range of model providers (OpenAI, Anthropic, Azure OpenAI, Amazon Bedrock, Google Vertex AI, local ones through Ollama) and vector stores, delivering portability and reducing vendor lock-in. On coverage of this book's patterns they are equivalent; they differ in style and in where they integrate.

16.2 Mapping the concerns onto constructs

The chapter's core value is the correspondence between the abstractions of Parts I–IV and concrete constructs.

Concern (chapter)	Spring AI	LangChain4j
Calling the model, portability (Chapters 1–2)	`ChatClient` + provider auto-configuration	`ChatModel` / `AiServices` + a provider module
Prompt templates (Chapter 3)	`PromptTemplate`, system/user messages	`@SystemMessage` / `@UserMessage`, `PromptTemplate`
Structured output (Chapter 5)	mapping to a POJO (structured output)	the method's return type (POJO, enum, `Result<T>`), json mode
Vector store (Chapter 8)	`VectorStore` (a single API, SQL-like filter)	`EmbeddingStore` + `EmbeddingModel`
RAG (Chapters 6–7)	`QuestionAnswerAdvisor` (naive), `RetrievalAugmentationAdvisor` (modular)	`ContentRetriever`, `RetrievalAugmentor`
Tool calling (Chapter 12)	`@Tool` methods, calls dispatched to beans	`@Tool` methods, `ToolProvider`
Memory (Chapters 4, 13)	`MessageChatMemoryAdvisor`, `VectorStoreChatMemoryAdvisor`	`ChatMemory`, `@MemoryId`, `ChatMemoryStore`
Agents and workflows (Chapters 10–11)	agent patterns on top of ChatClient/advisors	`AiServices`, the `langchain4j-agentic` module
Observability (Chapter 18)	Micrometer (metrics, traces)	tokens and latencies emitted as metrics
MCP integration (Chapter 15)	MCP client and MCP server (OAuth2)	MCP support in tool calling
Evaluation (Chapters 9, 17)	Evaluation utilities (relevance, faithfulness)	integration with test infrastructure

What the table means in practice: a pattern introduced abstractly in the earlier parts has a direct embodiment in both libraries; picking a construct is a question of framework style, not of whether the capability exists.



16.3 Deep Dive: interceptors versus declarative services

The two libraries embody different architectural styles of composition, and that difference matters more than any feature list.

Spring AI: a chain of interceptors (Advisors). Cross-cutting concerns take the shape of an ordered chain of advisors around the model call: before the call they run in ascending order, after it in descending order, sharing state through a context. Ready-made advisors cover memory (`MessageChatMemoryAdvisor`), RAG (`QuestionAnswerAdvisor`), filtering unsafe input (`SafeGuardAdvisor`), and logging; you add your own by implementing the interface. This is the classic interceptor pipeline: every layer observes and, where needed, transforms the request and the response. Advisors take part in the observability stack, so their execution shows up in metrics and traces.

LangChain4j: a declarative proxy (AI Services). Concerns are declared on the interface: the method's return type sets the output parser (a POJO, a collection, an enum for classification, `Result<T>` carrying sources and token spend), annotations set the message templates and the tools, and `@MemoryId` partitions memory per user. The library generates the implementation—business code stays declarative and testable (interfaces are easy to swap out in tests). An architecturally useful subtlety: RAG content appended to a message is stored in memory by default, but that behavior is switchable, so retrieved fragments need not bloat the history.

Both styles, illustrated (signatures trimmed to the essentials; check the current API against the documentation):

```
// Spring AI: a chain of interceptors around the model call
ChatClient chat = ChatClient.builder(model)
    .defaultAdvisors(
        MessageChatMemoryAdvisor.builder(memory).build(),   // memory
        QuestionAnswerAdvisor.builder(vectorStore).build()) // RAG
    .build();
Ticket ticket = chat.prompt().user(question).call().entity(Ticket.class);
```

```
// LangChain4j: a declarative interface; the library generates the implementation
interface SupportAgent {
    @SystemMessage("You are a support assistant; answer according to the policies.")
    Result<Ticket> classify(@MemoryId String userId, @UserMessage String question);
}
SupportAgent agent = AiServices.builder(SupportAgent.class)
    .chatModel(model)
    .chatMemoryProvider(memoryProvider)
    .contentRetriever(retriever) // RAG
    .build();
```

The difference comes down to *where* the composition lives: in Spring AI, in an explicit chain of interceptors (an imperative pipeline); in LangChain4j, in the interface declaration (a proxy that generates the behavior). Both models implement the same patterns of this book.

16.4 Deep Dive: agents and workflows on the JVM

The workflow-versus-agent distinction (§11.8) carries straight over into implementation, and both frameworks support the whole spectrum—from a single call to an autonomous loop.

Deterministic workflows are preferable, and the code says so. For processes that need control, the recommended approach on the JVM is to structure the process as a **state machine**: the nodes (steps) and the transition criteria are set programmatically. A node can be a call to an AI service (classifying a request through a guided dialogue, say) or purely algorithmic (a calculation, where a model has no business being). The benefits map directly onto §11.8: control, separation of responsibility (the model does not step outside the node's boundary), the ease of adding safety checks and privilege limits at every node, and the option of running small models on simple nodes and specialized ones on hard nodes. This is the engineering expression of the principle "hold as much as possible inside deterministic workflows."

Autonomous patterns. For agentic autonomy, Spring AI ships implementations of the canonical patterns (chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, Chapter 11) on top of ChatClient. LangChain4j has factored its agent functionality into the `langchain4j-agentic` module—a set of abstractions for multi-agent processes in which agents delegate tasks, share context, and coordinate. Either way, the discipline of Chapters 10 and 14 still holds: iteration limits, timeouts, reversibility, a human-gate.

Curating tools and conditional RAG. Both libraries reflect the concerns of §12.3 and §4.4: a tool included in a call spends tokens and raises the risk of an unintended invocation, so the tool set gets curated. LangChain4j explicitly supports *conditional* RAG—retrieval wrapped in a `@Tool` is invoked only when the model has judged the context necessary, which avoids pointless searches on simple queries (the same principle as Adaptive RAG, §7.9).

16.5 Deep Dive: production concerns on the JVM

Assembling the stack is not only about calls to the model but about operational properties (a bridge to Part VI).

- **Observability.** Spring AI integrates with Micrometer: metrics and traces for model calls, the vector store, and advisors out of the box (Chapter 18). In LangChain4j, tokens and latencies are emitted as metrics; provider and store calls are traced.
- **Blocking and the reactive stack.** A model call is seconds of waiting on the network. On a servlet stack it holds a pool thread for that entire time (the pool drains faster than it seems—do the arithmetic as concurrency × duration; Little's law and the platform's standard answer are worked through below); on a reactive one (WebFlux/Netty), a synchronous `call()` inside the event loop is a direct route to degrading the whole service. The rules: in a reactive pipeline, only the reactive/streaming variants of the API, or offload to a dedicated scheduler; there is no admissible form of a blocking model call in an event loop; a timeout on every network call (§2.5) is mandatory on any stack.
- **Persistent memory and multi-tenancy.** The default memory is simple (a window of messages), but in a multi-user, multi-pod environment the wrong strategy leads to an explosion in tokens, latency spikes, and context leaking between users. The remedies: partition memory by identifier (`@MemoryId` in LangChain4j, a conversation ID in Spring AI), a persistent store (`ChatMemoryStore` over a database), a sliding window, or summarization (§4.4). Memory is treated as a first-class architectural concern, not a detail.
- **Portability and model choice.** A single API lets you swap providers and apply Model Routing (§2.2): a cheap model on a simple node, a strong one on a hard node, inside a single application.

- **Security.** Authorization at the tool, with the user's privileges (§12.5); isolating untrusted data (§4.6); and, for MCP integration, OAuth2 and treating servers as untrusted (§15.7). The framework does not provide these properties automatically: they are designed in.

How many threads a model call eats: Little's law. "Concurrency × duration" is Little's law, and what it gives is not an intuition but an exact ceiling. For steady state:

$$L = \lambda \times W$$

Here L is the average number of requests in processing at once (on a servlet stack that is exactly the number of busy pool threads, in threads); λ is the rate of incoming requests (requests/s); W is the **average** time to process one request end to end (s), dominated by the model call: seconds of network waiting against milliseconds of your own logic. The word "average" here is not a caveat but a condition: Little proves the equality for the mean time a job spends in the system, not for an arbitrary percentile—and that restriction resurfaces below. The pool is finite, so substituting "L = pool size" turns the law into a throughput limit:

$$\lambda_{max} = \frac{L}{W}$$

Live numbers. Spring Boot brings up an embedded Tomcat with a pool of 200 worker threads by default (`server.tomcat.threads.max`; the value is baked into `ServerProperties`), and an agent call with a long generation takes, say, a measured W = 8 s. Then λ_{max} = 200/8 = 25 requests/s per pod—with the CPU idle: the ceiling is set not by computation but by 200 threads parked in `read()` on the provider's socket. Take a different profile—not a single 8-second call but the whole request under the §2.3 budget (an SLO of p95 = 3 s: retrieval, model calls, tools, guardrails): a disciplined agent yields 200/3 ≈ 67 requests/s—and that is a lower-bound estimate: the law works on the mean, and p95 is certainly larger than the mean, so the true ceiling is higher, but stays in the same order of magnitude. For comparison, on ordinary business logic at 50 ms the same pool would give 200/0.05 = 4000 requests/s—a 60–160-fold gap against the agent profile, and it comes entirely out of W. The default of 200 is a constant that knows nothing about your W; it physically cannot be right for a millisecond profile and a second-scale one at the same time. And to serve 100 requests/s at W = 8 s takes L = 100 × 8 = 800 simultaneously busy threads: 800 MB of address space for stacks alone (Linux/x64: 1024 KB per thread by default —reserved up front, physically consumed as pages get touched) plus an OS scheduler shuffling 800 threads.

In plain terms: the pool stops being a pool and becomes a limiter exactly when the desired rate, multiplied by the processing duration, exceeds the thread count. **The engineer's iron rule:** pool size is derived from the target λ and the measured end-to-end duration of a request (§2.3), not taken from the framework's default. And it is more honest to measure at p99 than at the mean—a deliberate departure from the letter of the law: Little gives an average ceiling, but the pool has to survive the tail of the distribution; p99 here is a conscious margin on top of the W the formula calls for.

Virtual threads (Loom)—the platform's standard answer. Since Java 21, a blocking network call inside a virtual thread does not hold an OS thread: the runtime **unmounts** the virtual thread (parking its stack on the heap) and frees the **carrier** OS thread for other work. The profile "thousands of concurrent requests, each of them waiting on the network nearly the whole time" is exactly the one Loom was built for. In Spring Boot (3.2+, on Java 21) this is switched on with a single key, `spring.threads.virtual.enabled=true`, after which Tomcat handles each request on a virtual thread; the worker pool is thereby bypassed and `server.tomcat.threads.max` no longer governs request concurrency (see spring-projects/spring-boot#41937)—the pool size simply stops being a variable in the formula above. The λ ceiling shifts from the thread count to the real constraint—the provider's quota (rate limit, §2.5) and the HTTP client's connection pool. Virtual threads themselves are not pooled: one thread per task, and their number equals the number of tasks.

The trap: pinning. Before Java 24, a virtual thread blocked inside `synchronized` **nailed** the carrier thread to itself (pinning)—the carrier was not released, and the win went to zero: in the worst case concurrency collapsed to roughly the core count (the virtual-thread scheduler keeps by default as many carrier threads as there are available processors, though it may add compensating threads up to `jdk.virtualThreadScheduler.maxPoolSize`, default 256), which is lower than an ordinary 200-thread pool. JEP 491 (Java 24) rewrote the JVM's monitors and removed this case. The practical consequence: on Java 21–23, virtual threads help only if the HTTP client and the provider's SDK do not block under `synchronized` anywhere along the call path—check by running with `-Djdk.tracePinnedThreads=full` (in Java 24 the flag was dropped as unnecessary; diagnosis goes through the JFR event `jdk.VirtualThreadPinned`). Pinning has not vanished entirely even in 24+: native code (JNI, the Foreign Function & Memory API) still nails the carrier thread.

A caveat, so this does not displace the rule above. Virtual threads do not cure the reactive case: a blocking `call()` inside a Netty event loop is inadmissible on any version of Java, because what blocks there is not "a thread from a pool" but one of the few threads serving every connection at once. Loom is an **alternative** to the reactive stack (get simple blocking code back without paying for it in OS threads), not a patch on top of it. The choice frames like this: if the stack is servlet-based → turning on virtual threads lifts the $\lambda_{max} = L/W$ ceiling, at the price of auditing `synchronized` along the call path; if the stack is reactive → the rule stands: the reactive/streaming variants of the API, or offload to a dedicated scheduler.

16.6 Choosing and combining

Since both libraries implement the same patterns of this book, the choice turns not on what capabilities exist but on the fit with the ecosystem and the style:

- **Spring AI** is the natural fit where the application is already built on Spring Boot: auto-configuration, beans, starters, and one observability stack keep the integration effort to a minimum.
- **LangChain4j** suits a need for framework independence (Quarkus, plain Java) or for the declarative AI Services style; it also offers low-level access for full control.
- The libraries are not mutually exclusive: they get used together—LangChain4j's declarative AI Services for individual services inside a Spring application observed through Micrometer, for instance.

The decisive architectural principle stands unchanged: a framework implements patterns but does not stand in for design decisions—the choice of retrieval pattern (Part III), of the level of autonomy (§10.7), of the security boundaries (§12.5, Chapter 19), and of the memory strategy (§4.5) is the architect's, and the framework only embodies those choices idiomatically.

16.7 Summary of the chapter and of Part V

The abstract patterns of Parts I–IV have direct implementations on the JVM in Spring AI and LangChain4j: calling the model (ChatClient / AiServices), templates, structured output, the vector store, RAG, tool calling, memory, agent patterns, observability, MCP integration, and evaluation (§16.2). The libraries differ in composition style—a chain of advisor-interceptors against the declarative AI Services proxy (§16.3)—but cover the same set of patterns, so the choice comes down to the ecosystem rather than to capabilities (§16.6). Control over agent processes comes from structuring them as a state machine with deterministic and model nodes (§16.4), while the operational properties—observability, persistent memory, portability, security—are designed separately (§16.5, a bridge to Part VI).

Part V in sum. The agent built across Parts I–IV is now integrated: it connects to tools and data over a standard protocol (MCP, Chapter 15) and takes concrete shape on a JVM stack through a contract rather than through one-off integrations (Chapter 16). Next comes **Part VI: production**—how to operate this system reliably: evaluation, observability, security, cost, and latency.

Sources. Spring AI 1.0/1.1—the documentation and GA release materials (docs.spring.io/spring-ai; InfoQ, 2025): ChatClient, the Advisors API, vector stores, RAG, tool calling, memory, MCP, observability; LangChain4j—the documentation (docs.langchain4j.dev): AI Services, ContentRetriever/RetrievalAugmentor, ChatMemory, `@Tool`, the `langchain4j-agentic` module; comparative material on Spring AI and LangChain4j (2025). The JVM part of §16.5: J. D. C. Little, "A Proof for the Queuing Formula: L = λW," Operations Research (1961)—the law is proved for the mean time a job spends in the system; OpenJDK JEP 444 "Virtual Threads" (Java 21) and JEP 491 "Synchronize Virtual Threads without Pinning" (Java 24); Oracle, "Core Libraries: Virtual Threads," JDK 21—pinning and `jdk.tracePinnedThreads`; Oracle, the `java` Command Reference, JDK 21—the `-Xss` defaults; Spring Boot—Release Notes 3.2 (virtual threads) and the `ServerProperties` source (the Tomcat pool default of 200; the field javadoc reads only "Maximum amount of worker threads", while the nullifying of its effect under virtual threads is recorded in spring-projects/spring-boot#41937).

Part VI. Production: Operating the System

Parts I–V built the agent and integrated it. Part VI is about operating it reliably: measuring the quality of a nondeterministic system (Chapter 17, evaluation), seeing its behavior in production (Chapter 18, observability), defending it against manipulation and abuse (Chapter 19, security), and holding cost and latency inside a budget (Chapter 20). Part VI shifts the question from “how do I build it” to “how do I keep the system working, observable, secure, and economical under real load.”

Chapter 17. Evaluation: Building an Eval System

Chapter 9 applied evaluation to RAG. This chapter builds evaluation as a general production discipline for LLM and agent systems. The thesis: evaluation is not a supporting stage but the central control loop for a nondeterministic system; without it you cannot steer quality, and regressions go unnoticed.

17.1 Why evaluation is the central discipline

An LLM system behaves nondeterministically (§2.4): the same input yields a distribution of outputs, and a confident wrong answer is indistinguishable from a right one by eye. Classic equality testing (`assertEquals`) does not apply—the property under test is not “the output equals the expected value” but “the output *satisfies a criterion* at an acceptable rate.” Tests therefore turn into *distributions*: evaluation measures the fraction of outputs that pass the criterion across a representative set, and tracks that fraction over time and across versions. This moves engineering practice from a single assert to statistical measurement—with golden sets, thresholds, confidence intervals, and drift monitoring.

17.2 Levels and types of evaluation

Evaluation combines three kinds of checks, differing in cost and in what they can measure.

- **Programmatic (deterministic) checks.** Exact, cheap, reproducible: conformance to schema and types (Chapter 5), exact match against a reference, presence of required fields, format bounds; for code—unit tests passing and the exit status; for actions—state changes and respect for access boundaries. Use them wherever the criterion *can be formalized*.
- **Model-based evaluation (LLM-as-judge).** For semantic properties that resist exact checking: relevance, faithfulness, tone, completeness, instruction following. The judge is a separate model call with a rubric (§17.3). Flexible, but nondeterministic and biased (§17.4), so it needs calibration.
- **Human evaluation.** Most reliable for subtle and subjective properties, and it serves as the *reference standard* against which automatic evaluations are calibrated. Expensive and it does not scale, so it runs on a subsample—to validate the judge and to label golden sets.

An orthogonal distinction—**with a reference or without**. Reference-based evaluation compares the output against a known correct answer (correctness). Reference-free evaluation judges the output against the question and the context with no reference (faithfulness, relevancy)—handy where a reference is expensive or missing, which is typical of production traffic (§17.6).

17.3 Deep Dive: LLM-as-judge

LLM-as-judge (Zheng et al., 2023)—using a model to evaluate another model's outputs against a given rubric; a scalable replacement for human labeling that reaches high agreement with experts on many tasks. Modes:

- **Pointwise**—the judge scores *one* output on a scale or against a pass/fail criterion.
- **Pairwise**—the judge compares *two* outputs and picks the better one; more reliable for ranking and for comparing versions, since a relative judgment comes easier than an absolute one.
- **Reference-anchored**—the judgment is tied to a fixed reference; per the original MT-Bench° work this is more reliable than judging from the prompt alone, because it anchors the judgment to a standard.

Building a judge in practice: an explicit rubric with step-by-step criteria; discrete verdicts (pass/fail or a short scale) hold up better than continuous ones; for faithfulness—break the answer into atomic claims and check each one (§9.2). The judge is a measuring instrument, and like any instrument it needs calibration (§17.4).

17.4 Deep Dive: judge biases and how to remove them

The judge is systematically biased, and that is an engineering problem of its own, not noise. The main biases:

- **Position bias**—in a pairwise comparison the judge disproportionately prefers the option in a particular slot (first or last). Confirmed systematically at scale (Shi et al., 2024/2025: around 150,000 judgments, 15 judges); its strength depends on the judge and the task, and it grows when the quality gap between the options is small. Removal: present the pair in *both* orders and average; a verdict that flips under the swap is a direct indicator of position bias. The price of this hygiene is concrete: both orders double the judge calls, and a full pairwise comparison of n candidates costs $O(n^2)$ comparisons; the judging budget is a Chapter 20–grade line item—you plan it, you don't discover it on the invoice.
- **Verbosity bias**—a preference for longer answers regardless of quality (Saito et al., 2023). Removal: a rubric that explicitly penalizes verbosity; length control.
- **Self-preference bias**—the judge inflates the scores of its *own* model's outputs. The strength of the effect scales linearly with how well a model can *recognize* its own text, and that ability runs higher in stronger models (Panickssery et al., 2024); in GPT-4 the bias is especially pronounced, and at bottom it is a preference for "familiar," low-perplexity° answers regardless of authorship (Wataoka et al., 2024; weak models sometimes show even the reverse bias). Removal: make the judge a *different* model from the one under evaluation; use an ensemble of judges.

An important caveat about the limits of removal. The standard moves—ensembling judges, reporting inter-judge agreement, reversing the order—reduce *variance* within the judge population, but they do not remove biases *common* to all judges. Automatic evaluation is therefore not self-sufficient and has to be anchored to a human.

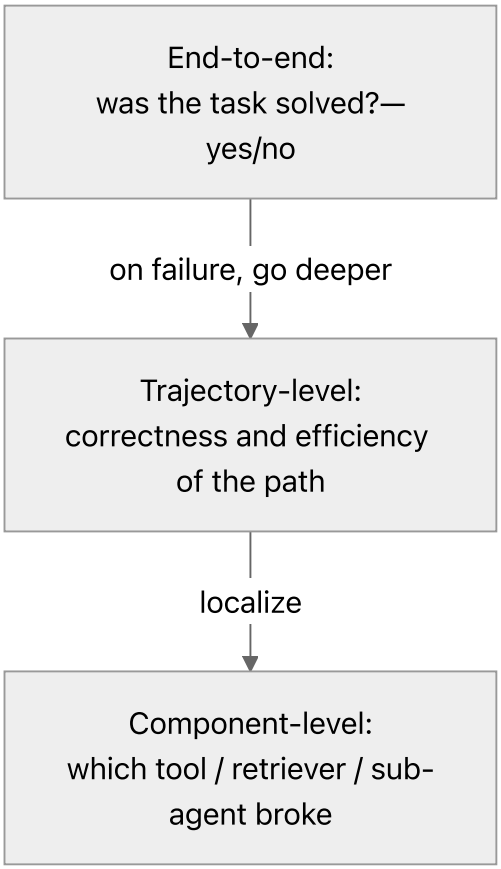
Mandatory engineering practices:

- **Validate the judge against humans.** On a subsample—label by hand and compute agreement (Cohen's kappa) or the correlation "judge ↔ humans"; a divergence above a rough 20–25% means the rubric needs fixing, not that the numbers deserve trust (§9.4). A judge calibrated on dialogue is unfit for evaluating RAG, code, or agents without domain calibration.
- **Pin the judge's version.** A provider can update the model quietly, and the judge will start scoring differently with no warning (temporal drift). The judge model's version gets pinned—an engineering requirement, not an option.

17.5 Deep Dive: evaluating agents and trajectories

Evaluating an agent differs from evaluating a single call: the agent makes autonomous multi-step decisions (tool calls, reasoning, planning), so the final answer is only the last step of a long path. Scoring the outcome alone misses what matters: the agent can reach the right answer by *faulty* reasoning, or look busy and never solve the task, or solve it inefficiently—with superfluous calls, wrong arguments, loops, tokens burned for nothing.

Evaluation runs at **three levels**, which form a sequence that localizes the failure:



- **End-to-end**—was the task solved. Answers yes/no, but does not explain *where* it broke.
- **Trajectory-level**—was the path correct and efficient. Metrics: correctness of tool choice and arguments, plan quality versus plan *adherence* (an agent with a bad plan and high adherence conscientiously executes the bad plan all the way to a wrong answer), efficiency by step count, loop detection, alignment of reasoning and action.
- **Component-level**—which retriever, tool, or sub-agent broke.

Approaches to trajectory evaluation:

- **Matching against a reference trajectory**—a strict comparison of the sequence of steps and calls against an exemplar; suits well-defined workflows with known expected behavior.
- **A judge over the trajectory**—a model scores the trajectory against a rubric (efficiency, soundness); more flexible, used where strict ordering is not required.
- **Final-state evaluation**—check the terminal state (the database state and the final answer, for instance) rather than a trajectory match. This matters because the same goal is often satisfied by *different* correct paths; rigid trajectory matching wrongly rejects them. The approach is standard in agent benchmarks (τ-bench[°] and kin).

The choice: deterministic checks for what can be formalized (tool correctness, state changes, access boundaries); a judge for what depends on content (reasoning quality, appropriateness). Tracing (Chapter 18) is the foundation: it shows *where* the metric fell, and it exposes new failure modes that have no metrics yet.

Public agent benchmarks as a reference point. τ-bench is not the only one; every major benchmark has its own axis, and knowing that axis helps when you read model releases and calibrate your own expectations:

Benchmark	What it measures	How it checks	Why an architect cares
τ-bench (Sierra, 2024)	A conversational agent with tools and domain policies	Final database state; pass^k—stability across k runs	Closest to a support assistant (Chapter 26); pass^k is an exemplar of a stability metric
SWE-bench (Jimenez et al., 2024)	Code agents: real GitHub issues → a patch	The repository's tests run on the patch	Calibrates the choice of model for code tasks and long trajectories
GAIA (Mialon et al., 2023)	A general assistant: web search, tools, files	Exact answer, difficulty levels 1–3	Evaluates the model+tools "assembly," not the model in a vacuum
WebArena (Zhou et al., 2023)	Browser tasks on replicas of real sites	A functional check of the result	A maturity gauge for computer/browser-use (outside this book's scope)
AgentBench (Liu et al., 2023)	Eight heterogeneous environments (OS, DB, knowledge graph...)	Environment-specific	A broad cross-section of one model's agentic abilities

pass^k under the hood. τ-bench introduced the metric and defined it literally: **the probability that *all* k independent runs of one task succeed**, averaged over tasks. For an individual task with a single-run success probability p this is simply a power:

$$\text{pass}^k = p^k$$

Here p is the single-run success probability of a **specific** task (dimensionless, 0...1). Live values: Codex-12B on three HumanEval tasks measured p = 0.9, 0.17, and 0.005 (Chen et al., Figure 2)—each task has its own p, and they spread hundreds of times apart. k is how many times you repeat the same task (1...8 in τ-bench).

In practice p is unknown: all you have is n runs actually performed on the task, of which c succeeded ($c \le n$). From those the paper gives an unbiased estimator—alongside the mirror-image estimator for pass@k:

$$\text{pass}^k = \mathrm{E}_{\text{task}}\left[\frac{\binom{c}{k}}{\binom{n}{k}}\right], \qquad \text{pass@}k = 1 - \mathrm{E}_{\text{task}}\left[\frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

Here E_{task} is the mean across all the benchmark's tasks: each task yields its own value of the fraction, you add them up and divide by the number of tasks (115 tasks in τ-retail, Table 1). In plain terms: the fraction $\binom{c}{k}/\binom{n}{k}$ is the chance that if you draw k runs at random from the n already done, every one you drew turns out to be a success; E_{task} averages that chance over tasks. A live number: at $n = 8$ runs with $c = 6$ successes, $\binom{6}{2}/\binom{8}{2} = 15/28 = 0.536$, whereas the naive substitution $(6/8)^2 = 0.5625$ overstates it. Hence the hard requirement $n \geq k$ (Chen et al., §2.1): you cannot compute pass^8 from five runs—the estimate is undefined, $\binom{5}{8} = 0$, there is nothing to divide by.

The direction in k is the opposite—and this is where people slip. pass@k is the probability that **at least one** of k runs succeeds: $\text{pass@}k = 1 - (1 - p)^k$. The metric's name and its unbiased estimator entered common use with Chen et al., 2021—together with the HumanEval code-generation benchmark; the idea itself of measuring functional correctness over k attempts ("solved if at least one passes") is credited there to Kulal et al., 2019, where it goes by a different name—success rate at B, the fraction of tasks solved within a budget of B attempts. In plain terms: "how many tasks will I close if I'm allowed k attempts and I can pick the lucky one." It **rises** with k, whereas pass^k **falls**. The primary sources give both curves in live numbers: Codex on HumanEval solves 28.8% of tasks with one sample and 70.2% at 100 samples per task (up); gpt-4o on τ-retail—61.2% at k = 1 and around 25% at k = 8 (down). The rule for choosing: **pass@k fits where a cheap verifier exists and you can take the best of k** (ran the tests, accepted the patch); **pass^k fits where the user gets one run**, and a retry costs money and trust. For a support assistant (Chapter 26) only pass^k is valid; mixing the two up means making a production decision on an inverted number.

Why 61% turns into 25% and not 2%. Substitute the published success rate naively into p^k and you get $0.612^8 \approx 0.02$—2%, twelve times below the published ~25%. The error is in the substitution itself: 61.2% is the **mean over tasks** ($\mathrm{E}_{\text{task}}[p]$, which is pass^1), not the p of any real task. pass^k averages p^k **over tasks** rather than raising the mean to a power, and for a convex x^k it always holds that $\mathrm{E}[p^k] \geq (\mathrm{E}[p])^k$ (Jensen's inequality). A live number on real p—those same three HumanEval tasks (0.9, 0.17, 0.005): the mean $\mathrm{E}[p] = 0.358$, the naive eighth power $0.358^8 = 0.0003$; honest averaging of the powers gives $\mathrm{E}[p^8] = (0.9^8 + 0.17^8 + 0.005^8)/3 = (0.430 + 0.0000007 + 0)/3 = 0.143$. A 500-fold divergence, and it rests on a single task: the easy one ($p = 0.9$) survives eight runs 43% of the time and carries the whole aggregate, while the two hard ones contribute zero. The gap is a direct measure of **task heterogeneity**. In τ-bench you can see it head-on: Figure 7 (every τ-retail task run at least 40 times on gpt-4-turbo) shows the success probabilities of individual tasks smeared across the entire 0…1 range. Read it this way: a randomly drawn retail task passes all eight runs roughly a quarter of the time. That is a probability averaged over tasks, not the fraction of "reliable" tasks: the set itself stretches from the nearly failure-proof to the consistently failing (Appendix A openly describes hunting for tasks with zero success). The 61% mean describes no real task, and a benchmark aggregate does not decompose back into "per-step reliability." A caveat on the nature of the spread: in τ-bench the agent's temperature is 0.0 and the simulated user's is 1.0, so the main source of spread is the variety of user phrasings at unchanged intent; the work itself, though, attributes the stochasticity to sampling from both—the user and the agent (zero temperature does not guarantee determinism in practice). In your own system you determine the source of the spread separately—the cure depends on it.

The engineering consequence. The exponent tightens the requirement on a single step harder than intuition suggests: for *one* task to run clean in 9 cases out of 10 across eight repeats, you need $p \geq \sqrt[8]{0.9} = 0.987$—nearly 99% per run. Near that point the metric works as an amplifier: a one-point drop in p (0.987 → 0.977) knocks pass^8 from 90% down to 83%, because the derivative $k\,p^{k-1}$ at $k = 8$ gives a coefficient of about 7. So you set the target level by pass^k at whatever k the business process dictates, and derive the requirement on the reliability of the individual step from that. The reverse order—"we've got a respectable 80% per run, that ought to do"—yields a system that, across eight repeats of one task, runs clean 17% of the time ($0.8^8 = 0.168$).

Two cautions. A public benchmark calibrates *the choice of model and harness*, but it does not replace a domain eval: your corpus, your policies, and your failure modes are not represented in it. And treat leaderboard figures with healthy skepticism: contamination of training data with benchmark tasks and optimizing "for the table" are documented phenomena; the gap between adjacent leaderboard rows is often smaller than one model's pass^k spread.

17.6 Offline versus online

Evaluation operates in two modes, and both are necessary.

- **Offline (before deployment).** A run over a *controlled* golden set with thresholds in a CI gate; it fires before every change to a prompt, a model, or the tool configuration, to catch a regression before deploy (§9.5). Gives reproducibility and comparability across versions.
- **Online (in production).** Evaluation on *real* traces, where the user supplies the input and there is no reference; hence reference-free metrics and traffic sampling. For multi-turn sessions—evaluate at the *thread* level: an individual message can look acceptable while the conversation as a whole never reaches the user's goal. Online evaluation tracks **quality drift** over time (corpus, queries, and model all change, §2.4) and pairs with gradual rollout (canary release, A/B) to compare versions on live traffic.

How the modes link up: offline guards against regressions on known cases, online catches what the golden set does not contain and feeds it back in as real examples (§17.8).

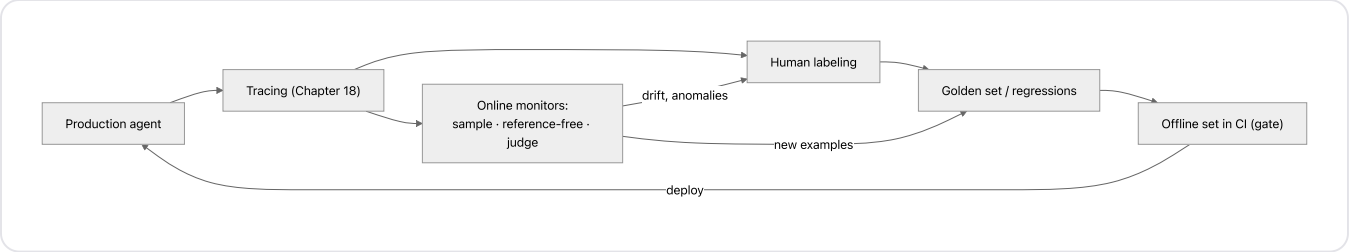
17.7 Deep Dive: statistical rigor

Evaluation is measurement with error, and conclusions demand statistical discipline (§9.4 extended to production scale).

- **Sample size and significance.** A metric moving on a small set is probably noise; to tell a real improvement from a chance one you need sufficient size and, ideally, confidence intervals or a significance test on the difference. An improvement that fits inside the spread does not count as an improvement. Orders of magnitude from the formula for a proportion's interval ($n \approx z^2 \cdot p(1-p)/e^2$): confirming a success rate of ~0.9 to within ±3 percentage points takes $n \approx 385$; *distinguishing* 90% from 85% at $\alpha = 0.05$ with 80% power takes on the order of 700 examples per variant. Hence the practice: a set of a few dozen examples catches only crude regressions; release decisions run into the hundreds.
- **Judge variance versus judge bias.** The judge's nondeterminism (§2.4) produces *variance*—reduced by averaging and a low temperature; systematic *bias* (§17.4) is not cured by averaging and requires anchoring to a human. Keep these two sources of error apart.
- **Judge–human agreement.** Before you decide anything on the judge's numbers, check its agreement with human labels (kappa); low agreement voids the metric.
- **Regression mode.** A golden set with thresholds in CI pins the baseline; wire an alert to a drop (§9.5).

17.8 The architecture of an eval system

The elements above assemble into a closed loop.



- **Tracing is the foundation.** Every call and every agent step is logged (Chapter 18); without traces you can neither diagnose a failure nor gather data for evaluation.
- **Datasets out of production.** Real traces, run through human labeling, turn into golden sets and regression sets that reflect the actual load.
- **The offline set in CI.** Changes to prompts, models, and tools pass a gate with thresholds (§17.6) before deploy.
- **Online monitors.** Reference-free metrics and a judge work on a sample of production traffic, tracking drift and anomalies (retrieval latency, a rising share of "not found," falling faithfulness).
- **The human review loop.** Periodic manual review calibrates the judge as the agent drifts, and it exposes new failure modes, which become new metrics and new examples for the set.

The loop closes: production generates traces → traces become evaluation data → evaluation catches regressions and drift → the findings sharpen the prompts, the tools, and the golden set itself. Evaluation is not a one-off stage but a permanently running measurement subsystem.

17.9 Chapter summary

Evaluation is the central control loop for a nondeterministic system: tests become distributions rather than equality checks (§17.1). It combines programmatic checks, LLM-as-judge, and human evaluation, with a reference and without (§17.2). The judge is a measuring instrument with systematic biases (position, verbosity, self-preference) that are partly removable but still demand anchoring to a human, domain calibration, and a pinned version (§17.3–17.4). Agents are evaluated at three levels (end-to-end, trajectory, component) and by final state, because many paths are valid (§17.5). The offline gate guards against regressions, online monitoring catches drift (§17.6), and conclusions demand statistical rigor (§17.7). All of it assembles into a closed loop built on tracing (§17.8). Next comes Chapter 18: the observability that produces those traces.

Sources. L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena" (2023); L. Shi et al., "Judging the Judges: A Systematic Study of Position Bias in LLM-as-a-Judge" (2024/2025); A. Panickssery et al. and K. Wataoka et al.—self-preference bias (2024); S. Saito et al.—verbosity bias (2023); agent benchmarks (§17.5): τ-bench (Sierra, 2024), C. Jimenez et al., "SWE-bench" (2024), G. Mialon et al., "GAIA" (2023), S. Zhou et al., "WebArena" (2023), X. Liu et al., "AgentBench" (2023); M. Chen et al., "Evaluating Large Language Models Trained on Code" (2021)—HumanEval and the unbiased pass@k estimator (§17.5); S. Kulal et al., "SPoC: Search-based Pseudocode to Code" (2019)—the original source of the idea of evaluating functional correctness over k attempts (§17.5); C. Huyen, *AI Engineering* (2025); offline and online evaluation practices, and agent trajectory evaluation (2025); the tooling landscape of eval platforms (Langfuse, Braintrust, Arize Phoenix, LangSmith, DeepEval—the roster changes fast, the selection criteria outlast the names).

In the Margins: Chapter 17

↩ **MT-Bench**—a set of multi-turn prompts from the same Zheng et al. (2023) work that introduced LLM-as-judge; together with Chatbot Arena, the historical reference standard for pairwise judging.

↩ **Perplexity**—a measure of how "surprised" a model is by a text: the exponential of the average next-token prediction error. The intuition: a perplexity of 20 means the model was choosing at each step as if among 20 equally likely continuations; the more predictable the text is to the model, the lower the perplexity. A "low-perplexity answer" = text the judge would likely have generated itself—hence the mechanism behind self-preference.

↩ **τ-bench** (Sierra, 2024)—an agent benchmark: a simulated user, tools, and a database (retail/airline); it checks the final database state rather than the trajectory; its pass^k metric measures the stability of success across k runs.

Chapter 18. Observability

Chapter 17 measures the system's *aggregate* quality. Observability answers the complementary question: what exactly did a *particular* run do—which steps, calls, and decisions produced the result. It is the primary signal both for debugging and for evaluation itself: without traces you can neither find the cause of a failure nor gather the data for evals (§17.8). Because agentic systems are nondeterministic and multi-step, observability is at once harder and more necessary than in classical services.

18.1 Why observability is harder for LLM systems

A model call produces substantially more telemetry than an HTTP request, and—more importantly—telemetry of a different *shape*. Prompts and responses are large blocks of text; a tool call's arguments have a different structure at every step; an agent's multi-step reasoning does not fit a fixed schema. Beyond the usual "which model was called and how long it took," you need token spend, the cost of the call, and the *quality* of the response (Chapter 17), which classical telemetry conventions do not cover at all. So observability for LLM systems is not APM carried over one to one but a discipline of its own, with its own semantics.

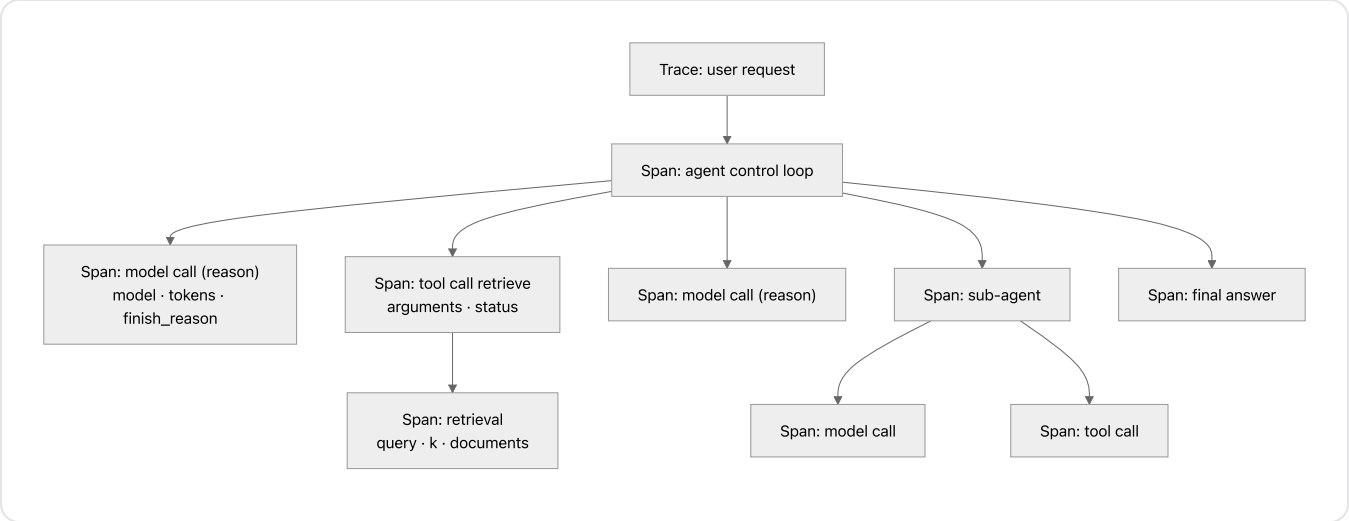
18.2 Three signals

Observability rests on three signals, as in classical distributed systems, but with LLM-specific content.

- **Traces**—the structure of an *individual* run: a tree of nested operations (spans) from input to response, reflecting the actual trajectory (§18.3). The main signal for debugging and the data source for evaluation.
- **Metrics**—*aggregates* over time: token spend, latency, cost, error and refusal rates, cache hits (§18.5). The basis for SLOs, alerts, and cost attribution.
- **Logs**—discrete events (tool errors, guardrail firings, escalations) tied to the trace's spans.

18.3 Deep Dive: tracing an agent—the span hierarchy

The key structure is the **trace** of a single run as a tree of **spans**. The root span is the whole request; nested inside it are the spans of the control loop (§10.2), and inside those, the individual calls to the model, to tools, to retrieval, and to sub-agents. The span tree *is* the trajectory made material (§17.5): it shows what happened and in what order, how long it took, and where it broke.



Every span carries attributes: for a model call—the model name, the number of input and output tokens, the stop reason (`stop` or `tool_calls`), latency; for a tool call—name, arguments, result, status; for retrieval—the query, the count and the identifiers of the documents retrieved. Nesting the spans preserves causality: a failed tool call is visible *inside* a specific loop iteration, not in isolation. That is exactly why a trace localizes a failure down to a specific step, which judging the outcome alone cannot do (§17.5).

18.4 Deep Dive: OpenTelemetry semantic conventions for GenAI

For telemetry to be comparable and portable across frameworks and vendors, span attributes get standardized. The industry standard is the **OpenTelemetry semantic conventions for GenAI**, developed by a dedicated group (the GenAI SIG[9]) since April 2024 under the CNCF umbrella. They set a single vocabulary for model calls, agent steps, vector-database queries, token spend, cost, and quality metrics.

- **Model-call attributes** (the `gen_ai.` prefix): `gen_ai.request.model` (the model), `gen_ai.usage.input_tokens` and `gen_ai.usage.output_tokens` (tokens), `gen_ai.response.finish_reasons` (the stop reason), `gen_ai.provider.name`, `gen_ai.operation.name`. With content capture enabled—`gen_ai.input.messages`, `gen_ai.output.messages`, `gen_ai.system_instructions` (§18.6).
- **Layers of the conventions.** Beyond model calls, the standard covers agent orchestration, tool calls over MCP, content capture, and quality metrics; conventions for multi-agent systems (agent tasks, actions, commands, memory) are under active development.
- **MCP integration** (Chapter 15): MCP spans carry `mcp.method.name`, `mcp.session.id`, `mcp.protocol.version`; when trace context is propagated (W3C Trace Context), the server span nests inside the client span, keeping the trace unbroken across the "host ↔ server" boundary.
- **Maturity.** Most of the conventions are experimental as of 2026, but the major observability platforms already support them; the key value is *portability*: instrument once, analyze in any compatible tool, with no vendor lock-in.

18.5 Metrics and cost attribution

An LLM system's aggregate metrics are its "golden signals":

- **Token spend**—input and output separately (output costs more, §2.2); the reasoning tokens of reasoning models are counted apart (§13.8).
- **Cost**—derived from tokens and the price rate; attributing it by feature, tenant, and user is critical if you want to see *what* is actually consuming the budget (Chapter 20).
- **Latency**—splitting TTFT from total time (§2.3); for agents, latency per step and for the whole task.
- **Error and refusal rates**—tool failures, model refusals, guardrail firings.
- **Cache hits**—how well prefix/prompt caching is working (§2.2), which directly affects cost and TTFT.

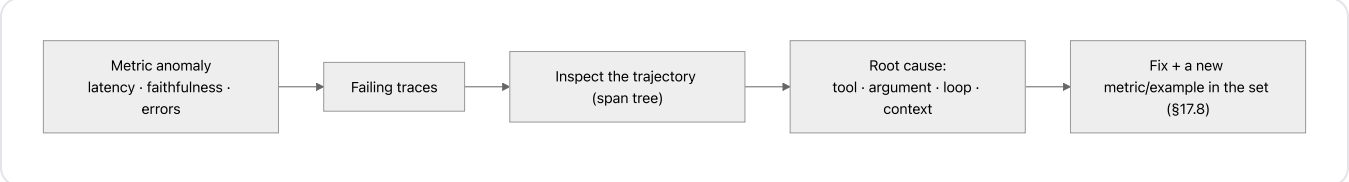
These metrics form the basis for SLOs (latency, availability), for alerts on anomalies, and—crucially for LLM systems—for *cost attribution*, since spend is variable and grows with autonomy (Chapter 20).

18.6 Deep Dive: content capture and privacy

Capturing the *content*—input and output messages, tool arguments and results, as span attributes—is invaluable for debugging: it lets you reconstruct the whole exchange and understand *why* the model behaved the way it did. But it has two costs. First, volume: messages are large blocks, and they bloat telemetry storage. Second, privacy: content can include personal data and secrets. So content capture is made *optional* and controllable: you turn it on selectively and redact sensitive data before it leaves the perimeter—with, for example, OpenTelemetry collector processors that mask and filter attributes. The trade-off "debugging completeness ↔ privacy and storage cost" is settled by a capture policy, not by capturing everything in sight. The second lever against volume is **trace sampling**: head-based (decide whether to keep at the start of the trace—cheap, but it loses rare failures) versus tail-based (decide once the trace finishes—you can keep *every* anomalous and slow trace while keeping only a small percentage of the normal ones). For nondeterministic systems tail-based is the more valuable one precisely because what you care about is the tails of the distribution, not the typical case.

18.7 Deep Dive: debugging nondeterministic trajectories

Observability is the foundation for debugging systems where a rerun does not reproduce the behavior (§2.4). A failed run must be reconstructible step by step: without a trace you are left with the outcome, not the path to it. Diagnosis typically runs *top down*:



- An anomaly in an aggregate metric (latency rising, faithfulness dropping, tool errors spiking) signals a problem.
- Moving to the *failing traces* localizes which step and which component failed (§18.3).
- Inspecting the trajectory exposes the cause: the wrong tool chosen, a corrupted argument, a loop, a poisoned result (§12.5), context lost between agents (§14.4).

What matters is that tracing does not only diagnose *known* failures—it *uncovers new* modes for which no metric exists yet: reviewing real traces surfaces failure patterns that then become new metrics and new golden-set examples (§17.8). That is how observability closes the loop with evaluation.

18.8 Observability and evaluation together

Observability and evaluation (Chapter 17) complement each other; neither replaces the other. The telemetry standard covers model attributes, tokens, cost, and latency, but it does *not* judge quality—faithfulness, safety, policy compliance. Quality is measured by a separate evaluation layer (Chapter 17) that consumes the same traces. Composing the two is what production monitoring is: observability supplies the primary signal (what happened), evaluation renders the judgment (was it any good), and together they give the full picture—where, what, and how well the system did. Online monitoring (§17.6) is exactly that pairing: "observability traces + evaluation on a sample of them."

18.9 Chapter summary

Observability shows what a particular run did—the primary signal for debugging and for evaluation data (§18.1). It is harder than the classical kind because a model call yields more telemetry, of a different shape (§18.1), and it rests on three signals: traces, metrics, logs (§18.2). The central structure is the span tree, which makes the trajectory material and localizes a failure down to the step (§18.3); span attributes are standardized by the OpenTelemetry semantic conventions for GenAI, which deliver portability (§18.4). Token, cost, and latency metrics form the golden signals and the basis for cost attribution (§18.5); content capture is invaluable for debugging but is governed by policy because of privacy and volume (§18.6). Observability is the foundation for debugging nondeterministic trajectories and a source of new metrics (§18.7), and together with evaluation (Chapter 17) it makes up production monitoring (§18.8). Next, Chapter 19: security.

Sources. OpenTelemetry, Semantic Conventions for Generative AI (GenAI SIG, since 2024; experimental status, 2025–2026)—the `gen_ai.*` attributes, the layers of the conventions, MCP integration; material on LLM-system observability and agent tracing (OpenTelemetry, observability platforms, 2025–2026); C. Huyen, *AI Engineering* (2025)—metrics and monitoring.

In the Margins: Chapter 18

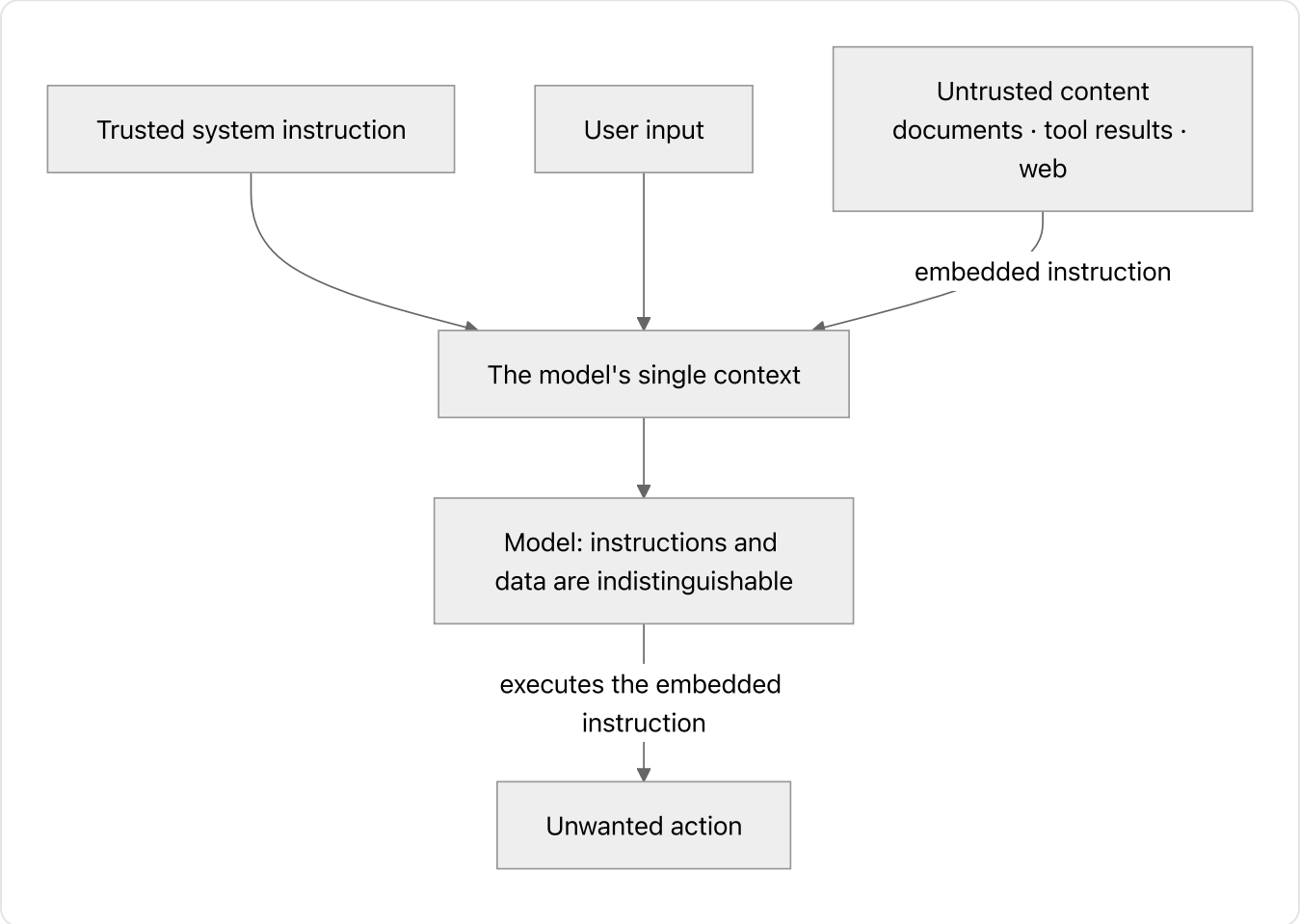
> ↩ **SIG**—Special Interest Group: a standing working group inside a standards consortium (here, OpenTelemetry under the CNCF umbrella) that owns a specific area of the specification.
>
> ↩ **Masking** here is a privacy operation: replacing sensitive values with placeholders/pseudonyms before the data leaves the perimeter (techniques—§25.4). ≠ Not to be confused with masking *logits* in constrained decoding (§5.3): there the "mask" forbids tokens during generation. The two operations share nothing but the word.

Chapter 19. Security: Prompt Injection and Defense in Depth

Security runs through the whole book as a cross-cutting theme: isolating untrusted content (§3.2, §4.6), the tool perimeter and the lethal trifecta (§12.5), classes of MCP attacks (§15.7). This chapter gathers those threads into one systematic treatment, centered on the root problem—prompt injection—and on the architectural answer to it. The key claim: an injection cannot be "patched" at the model level, because it exploits the model's own design; the defense has to be architectural.

19.1 The root problem: instructions and data in one channel

The model processes instructions and data in *one channel*, with no structural separation (§1.4, §3.2): anything that lands in the context is potentially read as an instruction. The model has no built-in notion of "trusted instruction" versus "untrusted data"—the boundary exists only in the developer's intent, never in the mechanism itself. It follows that **prompt injection is a structural defect, not a bug**: any system that accepts untrusted content through the model channel inherits the risk. That is why prompt injection has held first place in the industry's risk catalog (§19.3) for two editions running, and why no point fix removes it—it can only be contained by architecture (§19.6).



19.2 Prompt injection versus jailbreaking

Two classes often conflated, but distinct (the distinction is S. Willison's):

- **Prompt injection**—mixing the trusted (the system instruction) and the *untrusted* (external content) in one context, such that an instruction embedded in the data hijacks behavior. The term is named by analogy with SQL injection: the same essence—code and data mixed in a single channel.
- **Jailbreaking**—getting around the model's policies through direct manipulation by the user themselves (for example, "pretend you don't have to follow the rules"), to elicit forbidden behavior.

The distinction is fundamental to design: jailbreaking attacks the *model's policy*; injection attacks the *application* that trusts the model's output. They divide further:

- **Direct injection**—the attacker supplies the malicious input themselves.
- **Indirect injection**—malicious instructions are embedded in content the model processes as data: retrieved documents (§8), tool results (§12.5), web pages, attachments, email. Indirect injection is the more dangerous of the two, because the source of the untrusted content is not obvious and is often outside the user's control.

19.3 Deep Dive: the threat map—OWASP Top 10 for LLM (2025)

The industry's risk catalog is the **OWASP Top 10 for LLM Applications** (2025 edition). It earns its keep as a systematic checklist of attack surfaces; most of its entries the book has already touched on in passing.

Code	Risk	Where in the book
LLM01	Prompt Injection	§3.2, §12.5, §15.7, this chapter
LLM02	Sensitive Information Disclosure	§18.6, §19.6
LLM03	Supply Chain	§15.7 (untrusted MCP servers)
LLM04	Data and Model Poisoning	§6 (corpus quality), §8
LLM05	Insecure Output Handling	§5 (schema), §19.7
LLM06	Excessive Agency	§10.6, §12.5, §19.4
LLM07	System Prompt Leakage	§19.6
LLM08	Vector and Embedding Weaknesses	§8 (RAG as an attack surface)
LLM09	Misinformation	Part III (grounding), Chapter 17
LLM10	Unbounded Consumption	§10.5 (budgets), §20

Prompt injection holds the top position because it exploits the model's own design (§19.1). For autonomous agents OWASP released a separate companion—the Top 10 for Agentic Applications (2026 edition, published in December 2025)—reflecting that agency *amplifies* the risks: once a model can not only generate text but also reach for tools,

execute code, and call APIs, the blast radius of a single injection or of excessive agency widens sharply. The companion's categories (ASI01–ASI10) and its cross-cutting Least-Agency principle are worked through where the corresponding material lives: §12.5 (tools, privileges) and §14.4–14.5 (memory, inter-agent communication).

19.4 Deep Dive: the lethal trifecta and excessive agency

The most operational frame for exfiltration risk is the **lethal trifecta** (§12.5, Willison 2025): the danger arises when access to private data, untrusted content, and an external communication channel coincide. Indirect injection (§19.2) is the mechanism; the trifecta is the condition under which an injection grows into data theft. The answer is the **rule of two** (Meta 2025): an autonomous agent holds at most two of the three properties, otherwise human confirmation is required—or, in Meta's original formulation, a start from a clean context: a fresh session with no untrusted content carried over also breaks the chain, and where the task tolerates a reset it is the architecturally cheaper alternative to a gate; and **taint-tracking**: untrusted content marks the execution as tainted, blocking actions with exfiltration potential (§12.5).

Excessive agency (OWASP LLM06) is the second axis, substantially expanded in the 2025 edition, with three roots:

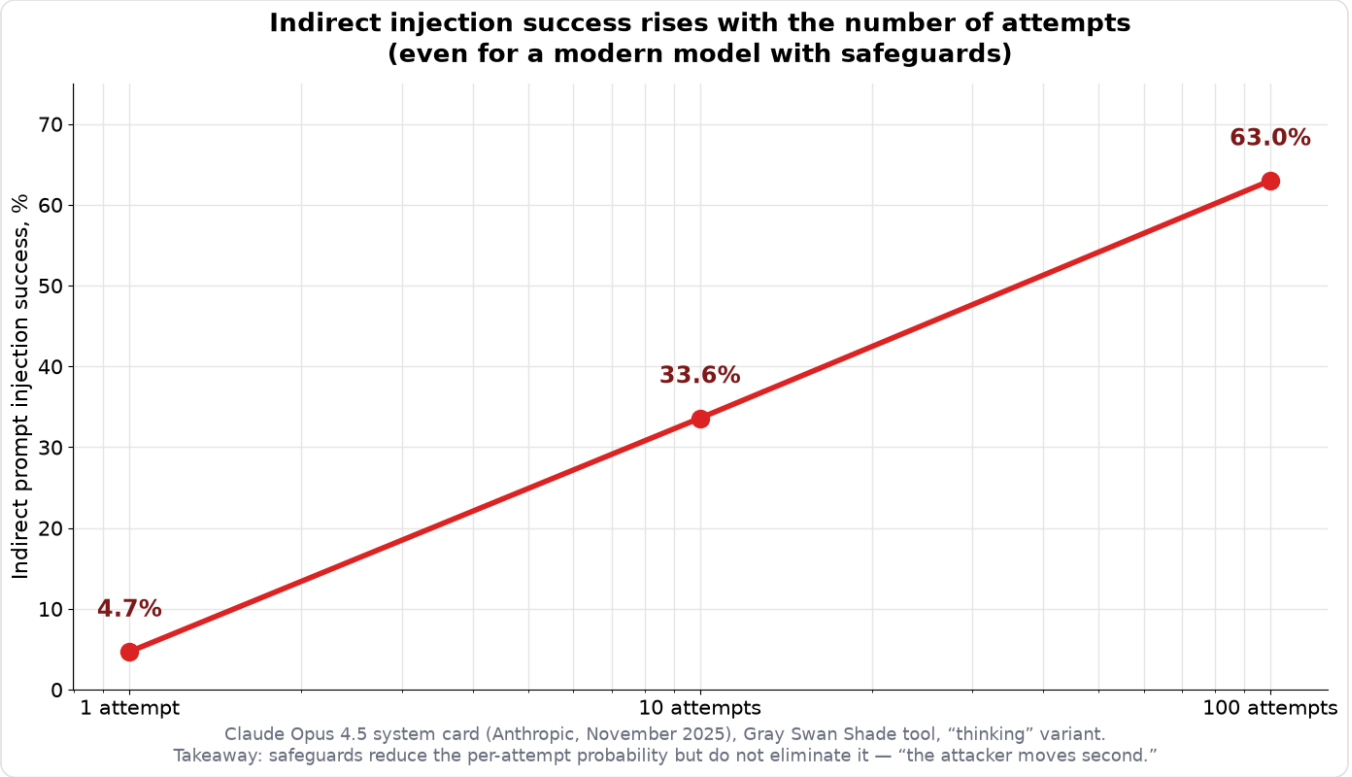
- *Excessive functionality*—more tools than the task needs (§12.3, curating the set).
- *Excessive permissions*—rights broader than required (§10.6, least privilege; authorization with the user's own rights, §12.5).
- *Excessive autonomy*—acting without human confirmation where the action is irreversible (§10.6, human-gate).

Injection and excessive agency amplify each other: the injection knocks the model off course, and excessive agency turns that into a destructive action. Constraining agency narrows the consequences of a successful injection, even when the injection itself could not be prevented.

19.5 Deep Dive: guardrails and their limits

Guardrails are filters on the input and the output: detectors for injection and malicious input (classifiers), checks of the output for leaks and unsafe content, programmable "rails" for permissible behavior. They are useful and they lower the rate of successful attacks, but they have a fundamental limit: they are *probabilistic*, and an adaptive attacker gets around them.

The empirical record is unambiguous. Systematic trials of published defenses under adaptive attacks showed that defenses which initially reported near-zero vulnerability yield over 90% attack success under adaptive search, and manual red teaming[°] in those same trials drove the success rate to practically total (work on adaptive attacks against published defenses, 2025). Even in modern models with built-in defenses, the probability of a successful indirect injection—small on a single attempt—climbs quickly with the number of attempts:

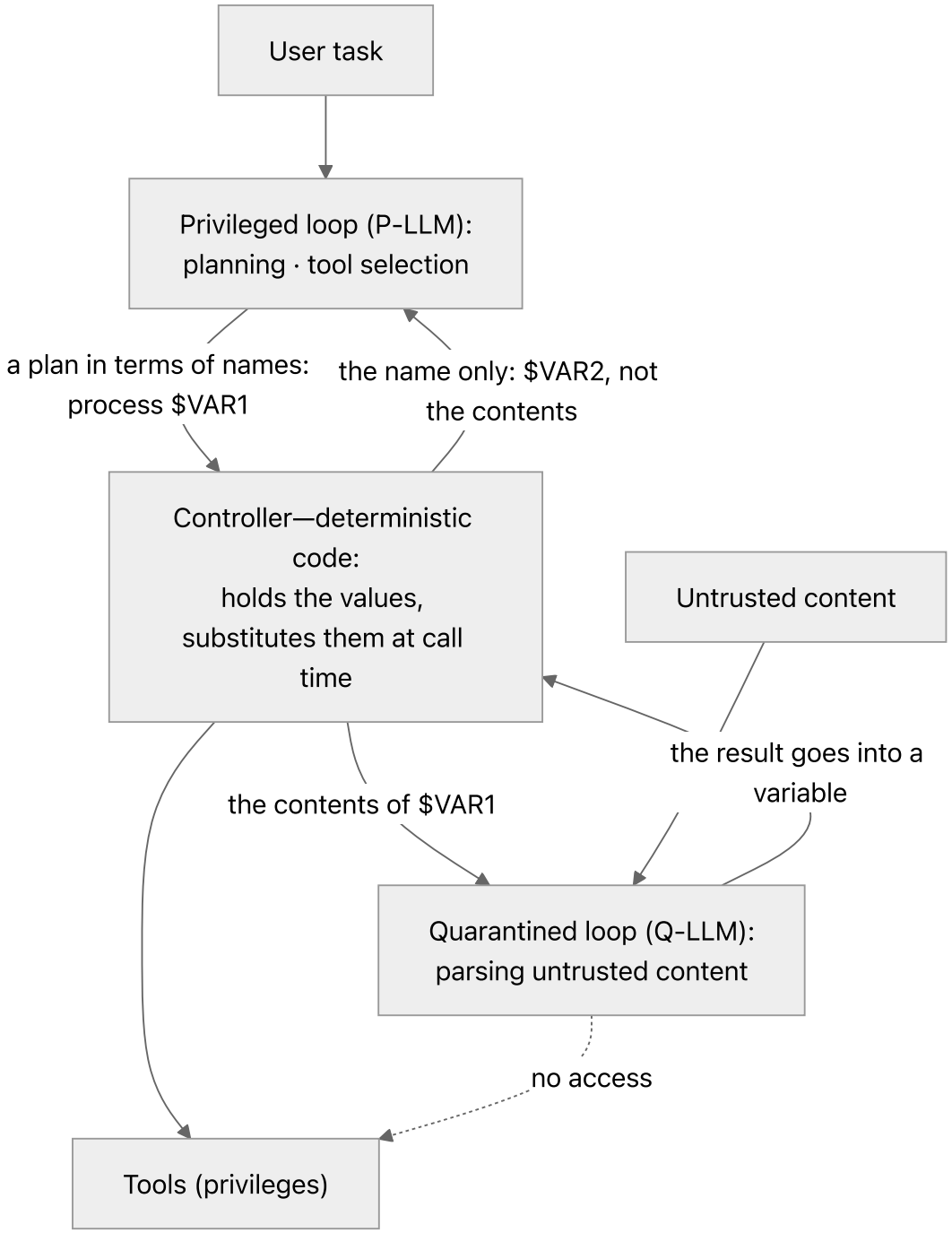


The conclusion for architecture: guardrails are a *probability-reduction layer*, not a guarantee. The analogy that has taken hold in the industry — prompt injection as the new SQL injection — carries exactly this far and no further: the injection is structural, and filters alone do not close it. Relying on filters as the sole defense is a mistake; they belong in defense in depth, but they do not replace architectural constraints (§19.6).

19.6 Deep Dive: architectural defense

Since injection cannot be eliminated at the model level, the defense is built at the level of the *system*—with deterministic constraints around the probabilistic component. The key architectural patterns:

- **Privilege separation and quarantine of the untrusted (dual-LLM).** The proposed design patterns for securing agents (Beurer-Kellner et al., 2025; the CaMeL approach, Google DeepMind) separate a *privileged* part, which plans and calls tools, from a *quarantined* part, which processes untrusted content but has no access to tools or privileges. Untrusted data never reaches the privileged control loop directly. What guarantees this is not good intentions but a specific mechanism—symbolic variables and a controller between the loops; it is worked through below, after the list.



- **Isolating data from instructions**—untrusted content is fed in a clearly demarcated region of the context and is not mixed with instructions (§4.6); the output format is pinned by a schema (§5), so an injection cannot substitute its own structure.
- **Least privilege and the rule of two**—constraining the trifecta and agency (§19.4), taint-gating (§12.5).
- **Execution sandbox**—for agents that execute code and commands: isolation at the OS/container/virtualization level (seccomp profiles and gVisor at the container level, Firecracker-class microVMs at the virtualization level), an ephemeral environment per task, and an **egress allowlist**—the list of permitted outbound destinations, which cuts off exfiltration even when an injection succeeds (the trifecta's third "leg" is physically removed). Declarative protocol boundaries (MCP roots, §15.3) are not enforcement without this layer.

- **A human-gate on the irreversible**—human confirmation before an action with consequences (§10.6); showing the user what is being sent out (§15.7).
- **Observability for detection**—tracing (Chapter 18) captures anomalous calls and exfiltration attempts, closing the defense with monitoring.

The dual-LLM mechanism: symbolic variables. The first item on the list needs unpacking: you cannot assemble a working system out of the phrase "the quarantined loop processes the untrusted," and the book prescribes this pattern in its checklist (§19.7). The key detail is the **controller**: deterministic code, not a model, standing between the loops. It calls the quarantined model, puts its answer into a variable, and returns to the privileged model **the name only**: `$VAR1`, `$VAR2`. The privileged model plans in terms of names—"summarize `$VAR1`," "send it to `$VAR2`"—without seeing the contents of either; the controller substitutes the values at the moment the tool is called. Willison (2023), where the pattern was proposed, states this without any give: the privileged model sees only variable names and sees neither the untrusted content nor the tainted summary that came back from quarantine. Hence the requirement on the diagram: what travels along the arrow from quarantine into the privileged loop is **the variable's name, not the result**. A "structured result with no instructions" is not a defense—it still brings attacker-controlled content into the loop that makes the decisions, which is exactly what the pattern must not allow.

What symbolic variables do not cover. Quarantine protects the *control flow*, not the *values*—the CaMeL paper works this through itself (Debenedetti et al., 2025). Take the plan: "find the right document and the client's address in the meeting notes, send the document to that address." The privileged model planned correctly and saw no content. But the notes are poisoned, and the quarantined model extracts the attacker's address from them: the controller substitutes `$VAR2` = `attacker@evil.com`, and the flawlessly executed plan sends a confidential document out. The plan was not hijacked—the argument was. A variable's name protects the decision about *what to do*, and says nothing about *what it is done with*.

CaMeL: capabilities on values. Hence the strengthening. The privileged model emits **code** in a stripped-down subset of Python before the system has read a single byte of untrusted data: the control flow is fixed before the data appears, and is therefore out of the attacker's reach by construction. The quarantined model is called from that code as a function with a mandatory response schema, and it has no tools. What executes the code is not Python but a **custom interpreter**: it maintains a dependency graph (which value was derived from which) and attaches to every value a **capability**°—its provenance and the set of permitted readers. Before every call to a tool with a side effect, the policy engine checks the arguments' capabilities. This is the same taint-tracking (§12.5), but not at the level of "the whole run is tainted"—at the level of the individual value, which is why it takes away less functionality.

Checking the policy. A policy in CaMeL is an ordinary Python function returning "allowed/denied" with a reason. Its substantive core, for a sending tool:

$$\text{allow}(f, \{v_1, \ldots, v_n\}) \iff \text{trusted}(R_f) \ \vee \ \forall i \in [1..n] : \ R_f \subseteq \text{readers}(v_i)$$

Legend (live values from the document example above):

- f—the tool being called, one with a side effect: `send_email`, `create_calendar_event`.
- R_f—who actually receives the data on this call: for `send_email`, the recipient addresses. Live value: `{attacker@evil.com}`.
- v_i—the values that go into the arguments: the message body, the attachment, the subject.
- readers(v)—from the value's capability: the set of permitted readers, or `Public` ("anyone may read it"). Live value for a document out of the corporate store: `{user@corp, bob@corp}`—the people it is shared with.
- trusted(R)—the provenance of R: whether it arrived as a literal in code written by the privileged model (that is, from the user's trusted request) rather than out of the data.

Live numbers. The document: readers(v_{doc}) = `{user@corp, bob@corp}`. The address the quarantined model extracted from the poisoned notes: R_f = `{attacker@evil.com}`; its provenance is a tool, not a literal, so trusted(R_f) = false and the fast path did not fire. That leaves the inclusion: `{attacker@evil.com}` ⊆ `{user@corp, bob@corp}` → **false**. The outcome is `Denied`: the send is blocked and goes to the user for confirmation.

In plain terms: the tool fires only if everyone who will receive the data was already permitted to read *every* value in the arguments—or if the recipient list came from the user themselves rather than out of the data. The check compares sets; it does not judge text: its outcome does not depend on how persuasively the injection is written—and that is what separates it from guardrails (§19.5), which judge precisely that text and are therefore evaded.

The honest price. The pattern is not free, and its failure modes are named in the paper itself:

- *Lost functionality*. CaMeL solves 77% of AgentDojo° tasks with a provable guarantee, against 84% for the unprotected system (Debenedetti et al., 2025)—roughly seven percentage points handed over for the guarantee (an early revision of the paper reported 67%; the figure here is from the current one, June 2025).
- *Side channels*. The authors give them a section of their own, with a direct admission: CaMeL is vulnerable to side-channel attacks—an attacker infers sensitive information by observing the system's behavior. The guarantee here is about isolating data flows, not about making a leak impossible.
- *Cost*. 2.82× the input tokens and 2.73× the output tokens on the median AgentDojo task versus native tool calling—the most expensive of the defenses the authors compared against one another. What a multiplier like that costs on the bill: Chapter 20.
- *Labor*. Policies have to be written and maintained: an unspecified policy is a hole; an over-strict one means work refused and confirmation fatigue (§10.6).

The deployment rule follows directly: dual-LLM with capabilities goes where the price of compromise exceeds the price of lost functionality and a tripled token bill—in high-risk loops (§19.7), not by default.

The general principle: **constrain the system rather than trust the model**. The model is assumed to be knockable off course by untrusted content, so authorization, isolation, and confirmation are provided by the deterministic components around it.

19.7 In practice: defense in depth

A summary checklist, drawing the measures from across the book into a layered defense:

- **Isolate untrusted content** from instructions (§4.6); a fixed output schema (§5).
- **Least privilege** and a minimal tool set (§10.6, §12.3); the **rule of two** and constraining the trifecta (§12.5, §19.4).
- **Input validation and output filtering** (guardrails)—as a probability-reduction layer, not as a guarantee (§19.5).
- **Authorization at the tool with the user's own rights** (§12.5), which prevents the confused deputy.
- **A human-gate on irreversible and dangerous actions** (§10.6).
- **Quarantine of the untrusted loop** from the privileged one (dual-LLM, §19.6) for high-risk systems.
- **A sandbox for code execution**—isolation at the container or microVM level, an ephemeral environment, an egress allowlist (§19.6).
- **Treating third-party MCP servers as untrusted**—integrity of definitions, OAuth (§15.7).

- **Observability and evaluation** for detection and security regression control (Chapters 17–18).

No single measure is self-sufficient; reliability comes from all of them together—because the probabilistic component can be knocked off course, and the system has to stay safe even once that has happened.

19.8 Chapter summary

Prompt injection is a structural defect: the model does not separate instructions and data in a single channel (§19.1), and so an injection cannot be eliminated at the model level. It is distinguished from jailbreaking, and by source—direct from indirect, the more dangerous (§19.2). The systematic map of surfaces comes from the OWASP Top 10 for LLM 2025, with prompt injection in first place and a companion for agents (§19.3). The operational axes of risk are the lethal trifecta and excessive agency (§19.4). Guardrails lower the probability but do not remove the attack: under adaptive pressure, and with the number of attempts, success grows (§19.5). The defense is therefore architectural: privilege separation and quarantine of the untrusted (dual-LLM), isolation, least privilege, human-gate, observability (§19.6), assembled into defense in depth (§19.7). The principle: constrain the system rather than trust the model. Next, Chapter 20: cost and latency.

Sources. OWASP, “Top 10 for LLM Applications” (2025) and “Top 10 for Agentic Applications” (2026 edition, published in December 2025)—© the OWASP Foundation, licensed under CC BY-SA 4.0 (https://creativecommons.org/licenses/by-sa/4.0/); the risk categories in §19.3 are reproduced as published, the mapping to this book’s sections is added; S. Willison—prompt injection, the distinction from jailbreaking, the lethal trifecta (2022–2025); Meta, “Agents Rule of Two: A Practical Approach to AI Agent Security” (2025)—the rule of two; K. Beurer-Kellner et al., “Design Patterns for Securing LLM Agents against Prompt Injections” (arXiv:2506.08837, 2025); S. Willison, “The Dual LLM pattern for building AI assistants that can resist prompt injection” (2023)—the controller and symbolic variables; E. Debenedetti et al., “Defeating Prompt Injections by Design” (CaMeL; Google / Google DeepMind / ETH Zurich, arXiv:2503.18813v2, June 2025)—capabilities, the interpreter, 77% against 84% on AgentDojo, side channels, cost in tokens; E. Debenedetti et al., “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents” (arXiv:2406.13352, 2024)—the benchmark itself, on which these numbers were obtained; work on adaptive attacks against injection defenses (2025); the Claude Opus 4.5 system card (Anthropic, 2025)—empirical data on indirect injection success; F. Perez, I. Ribeiro, “Ignore Previous Prompt” (2022); NeMo Guardrails (Rebedea et al., 2023).

In the Margins: Chapter 19

> ↩ **Red teaming**—adversarial testing: a specially designated party sets out deliberately to break the system’s defenses before a real adversary does; for LLMs, searching for injections and jailbreaks, by people or by automation.
>
> ↩ **Capability**—in systems security: a label bound to the object itself, describing what may be done with it—as opposed to checking “who is asking” against an external list of rights. In CaMeL a capability is attached to every *value* and holds two things: provenance (from the user, from a transformation inside the execution, or from a specific tool) and the set of permitted readers. The idea is borrowed from the classic systems—KeyKOS/EROS, Capsicum, CHERI.
>
> ↩ **AgentDojo** (Debenedetti et al., 2024)—an environment for evaluating prompt-injection attacks and the defenses against them: 97 user tasks with tools across four domains (email with calendar, a corporate messenger, a bank, travel booking) and 27 malicious goals planted in the data the agent reads; their combinations yield 629 security checks. It measures two things at once—the share of tasks solved (utility, hence the 77% against 84%) and the share of attacks that succeed.

Chapter 20. Cost and Latency

§2.2–2.3 introduced the economics of a call and the components of latency. This chapter is the operational digest: how to hold cost and latency inside budget in production. The thesis: **cost is a product constraint**, not a technical detail; it drives margin, pricing, and scalability, and an agent's autonomy *multiplies* both cost and latency (every step of the loop is a model call, §14.3). Optimizing here is not premature micro-optimization but a condition of the system's viability under load.

20.1 Cost as a product constraint

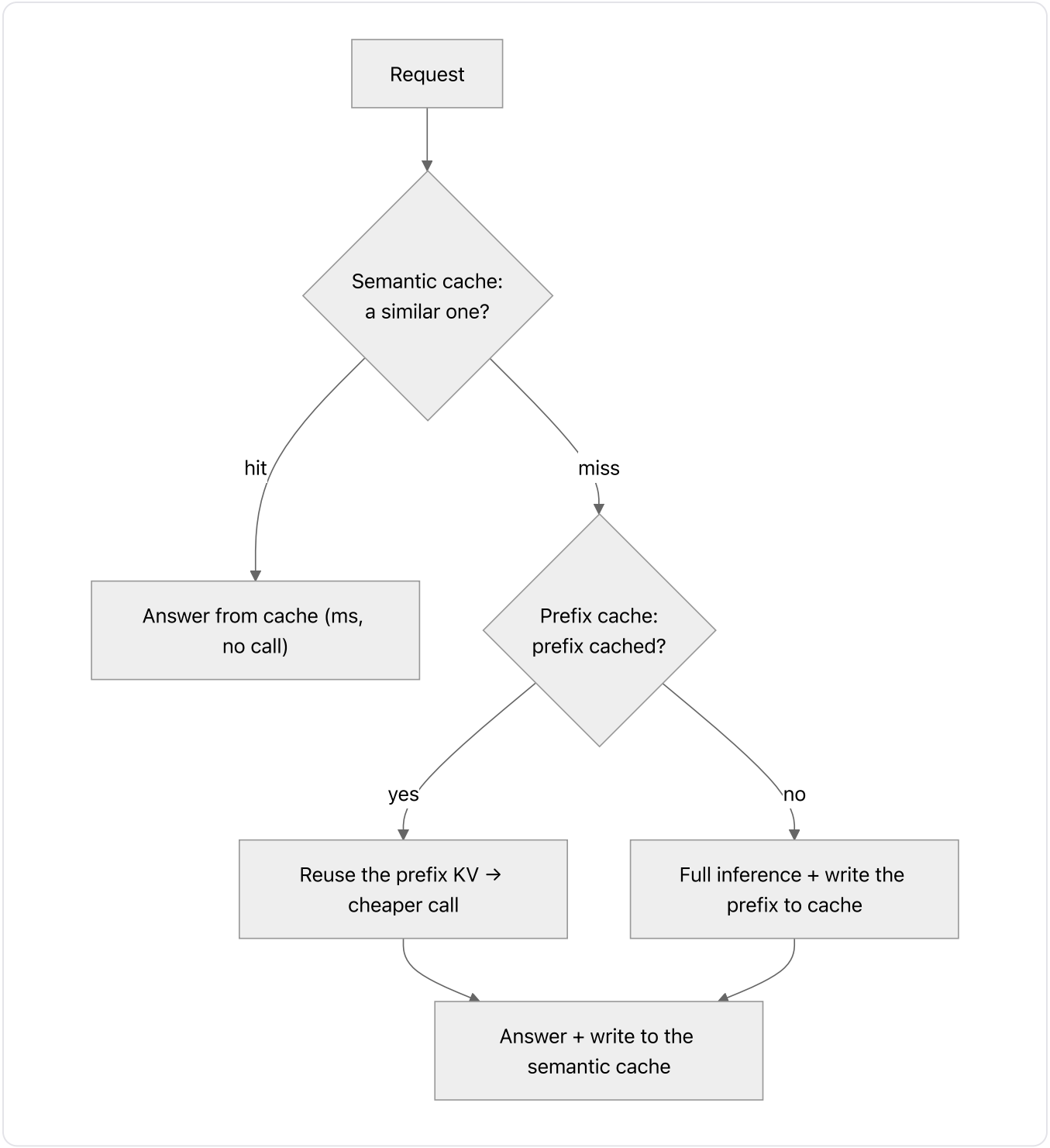
An LLM system's costs are linear in usage and variable: an agent making dozens of calls per session on a strong model runs up a noticeable per-session cost before anyone even checks the bill, and in unoptimized systems actual spend overshoots the plan by a multiple. The three axes of optimization—cost, latency, and quality—are related in a nontrivial way: some techniques improve one axis *by trading away* another (routing to a cheap model or compacting the context lowers cost but puts quality at risk), while others improve several axes *at once* with no loss of quality (caching lowers cost and latency both). So optimization is not one blanket "pick two of three" trade-off but a set of levers with different profiles (§20.7). None of it is manageable without cost attribution (§18.5): spend is broken out by feature, tenant, and user so you can see *what* is actually consuming the budget.

20.2 Deep Dive: the economics of caching

Caching is one of the highest-yield levers, and it works at two levels.

Prefix/prompt caching caches the processed KV state (§1.6) of static prompt prefixes, letting the computation be reused instead of redone from scratch. The economics are asymmetric, and their parameters are vendor-specific. In Anthropic's pricing model a cache write costs more than an ordinary input token (a premium on the order of 25%), while a read is far cheaper (a discount on the order of 90%, that is, about 10% of the base price); hence the break-even point is roughly two hits or more per cached prefix. Other providers have a different cost structure (OpenAI—automatic caching, a read discount and no write premium; Google Gemini—an explicit cache with hourly storage billing), so a break-even calculation does not carry across vendors without being redone. Prefix caching is the first optimization in any system whose system prompt changes less often than its user requests—which is to say nearly everywhere; it improves TTFT (§2.3) along the way. A caveat: badly placed cache boundaries can instead *increase* latency, so a caching strategy gets verified by measurement.

Semantic caching works at the level of *semantically similar* requests rather than exact matches: the request is embedded, the cache is searched for a near neighbor in embedding space (Chapter 8), and if similarity clears the threshold the cached answer comes back; otherwise the answer is generated and stored. On a hit the model call disappears entirely (zero cost, latency in milliseconds). This catches the paraphrases that exact caching misses ("what is the return policy?" and "how do I return an item?" differ textually but match semantically). A material limitation: semantic caching assumes a *stateless* "request → answer" mapping and therefore does not apply to tool calls that change state.



In practice the levels stack into a multi-level scheme: the semantic cache first (it removes the call outright), then the prefix cache (it makes the unavoidable call cheaper), then inference. Their combined effect on cost and latency is substantial.

20.3 Deep Dive: model routing and cascades

Model routing (§2.2) works like this in practice: a classifier scores the difficulty of a request and sends easy tasks to a cheap model, reserving the strong one for the genuinely hard ones. Because a significant share of an agent's requests are routine, the weighted average cost drops sharply, and multi-model routers can hold their own against

the best single model on quality at a lower average cost. Classification itself costs a little (a small call, added latency on the order of hundreds of milliseconds), which pays for itself at volume; what the classifier is made of and how it fails silently—Deep Dive §2.2.

A related form is the **cascade**: the request is handled first by the cheap model and escalates to the strong one only on insufficient confidence or a failed check (kin to the "don't add complexity before you need it" ladder, §13.7). Routing and the cascade trade cost against quality: the savings come from the fact that part of the traffic does not require a frontier model, and the risk is a classification error that sends a hard task to a weak model—which is why the difficulty decision is itself evaluated (Chapter 17). The latency asymmetry: direct routing pays the latency of one model, while with a cascade the escalated share of traffic pays the *sum* of the delays (cheap call + check + strong call)—for P99-sensitive systems that argues for routing, even if the cascade is cheaper on average.

20.4 Cutting tokens

Cutting the token count outright works on both axes.

- **Minimizing the output.** Output tokens cost more than input tokens (§2.2), so a `max_tokens` limit, stop sequences, and a strict output schema (Chapter 5) shrink the expensive part; `max_tokens` is a lever on more than the bill—it also cuts the room the output takes up in the window it shares with the input (§1.8), and when you serve the model yourself it further bounds how long a request holds KV-cache blocks, taking them away from neighboring sessions (§2.3); for reasoning models you manage the reasoning budget instead of paying for excess "thinking" on easy tasks (§13.8, over-thinking).
- **Compacting the context.** Removing what is redundant from history and tool results before the call cuts input tokens (§4.4); compaction by deletion differs from summarization in that what remains stays verbatim.
- **Batch API.** Where deferral is acceptable, batch processing gives a discount on the order of 50% (an SLA of up to a day) and stacks with caching: a batch with a cached prefix combines both discounts.

20.5 Deep Dive: latency

Latency is managed separately from cost, though some of the levers are shared (§2.3).

- **TTFT versus total time.** Perceived latency comes down to the time to the first token; streaming emits the answer as it is generated, sharply improving perceived responsiveness at the same total duration (§2.3).
- **Parallel tool calls.** Independent actions execute in parallel, not in sequence (§12.2): a sequential chain adds the delays up, a parallel one takes the maximum.
- **Caching.** Prefix caching improves TTFT; semantic caching, on a hit, brings latency down to milliseconds (§20.2).
- **Model choice and length.** A smaller model and a shorter output reduce generation time; at the serving level, decode latency drops further with speculative decoding (Chapter 22).

An agent's latency accumulates step by step (§2.3, §14.3), so cutting the number of steps (Plan-and-Execute versus a long ReAct, §13.4) and parallelism deliver the biggest effect on multi-step tasks.

20.6 Budgets, the gateway, and attribution

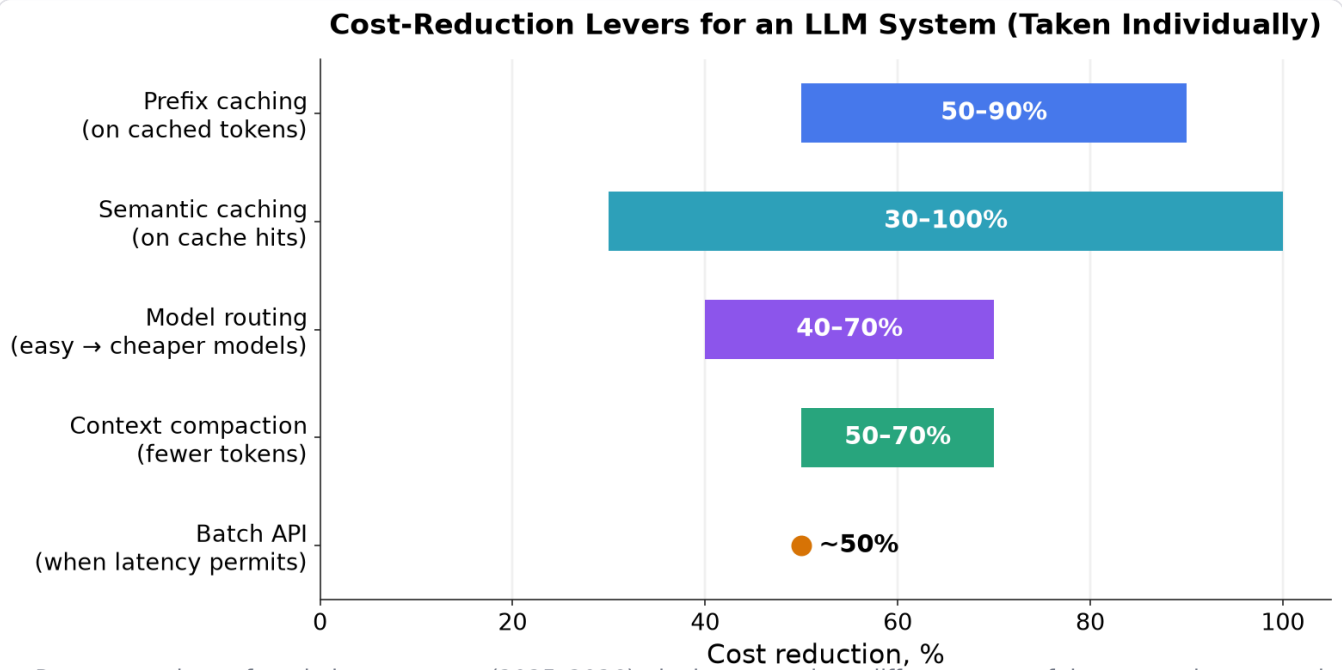
Operational control rests on three elements.

- **Budgets and limits.** Caps on step count and tokens per task and per session (§10.5) bound both cost and the "unbounded consumption" failure class (OWASP LLM10, §19.3); rate limiting is the standard production barrier at scale.
- **The LLM gateway.** A layer between the application and the providers that centralizes routing, semantic caching, fault tolerance (fallback, circuit breaker, §2.5), and cost accounting. Moving these concerns into infrastructure takes them off application code and gives a single point of control and of portability across providers (§16.5).
- **Attribution and SLOs.** Token, cost, and latency metrics (§18.5) are broken out by feature and tenant and tied to latency and availability SLOs; cost anomalies raise alerts the same way latency anomalies do.

Taken together, these practices—cost attribution, budgets, unit economics as a scaling gate (§25.6)—have consolidated in the industry under the name **FinOps for AI** (an extension of FinOps, the discipline of managing cloud spend as an engineering practice, to LLM workloads). The book lays the material out across its own chapters, but in an interview and in a job posting it will go by that name.

20.7 The combined effect and the order of application

The levers apply to different parts of the spend and they stack.



Ranges are drawn from industry reports (2025–2026); the levers apply to different parts of the cost and compound. Applied together, they deliver roughly a 70–85% reduction with no loss of output quality.

Individually each one delivers a noticeable reduction; applied together—on the order of a 70–85% cost reduction with no degradation of the output. The arithmetic is multiplicative: cache −40% × routing −50% × compaction −30% ⇒ 0.6·0.5·0.7 ≈ 0.21 of the original cost, that is, −79% (the multipliers are illustrative and depend on the workload profile—hence the spread of the range). The practical order of adoption runs from least effort to most: prefix caching first (often automatic, or needing nothing but markup), then model routing, then context

compaction for agentic and long-context workloads, then semantic caching and batch where they apply. The same caveat applies here as in §13.7: do not optimize before measurement (Chapters 17–18) shows *what* is actually consuming the budget and the latency; premature optimization complicates the system with no proven gain.

20.8 Summary of the chapter and of Part VI

Cost is a product constraint that grows with autonomy; the three axes (cost, latency, quality) are related in a nontrivial way (§20.1). Caching works at two levels—prefix (asymmetric economics, break-even from two hits on) and semantic (the call is removed on similar requests, but only for stateless ones)—and lowers both axes (§20.2). Routing and cascades move the routine work onto cheap models at the price of classification risk (§20.3); cutting tokens means minimizing the output, compaction, batch (§20.4); latency is managed through TTFT, streaming, parallelism, and fewer steps (§20.5). Control rests on budgets, the LLM gateway, and attribution (§20.6); the levers stack up to a 70–85% reduction, but they are applied in proportion to need and after measurement (§20.7).

Part VI in sum. The system is now operable: its quality is measured (Chapter 17), its behavior is observable (Chapter 18), it is defended against manipulation (Chapter 19), and its cost and latency are held inside budget (Chapter 20). Next comes **Part VII: your own models**—when and how to fine-tune, deploy, and train models instead of calling an API.

Sources. Industry reports and practices for optimizing the cost and latency of LLM systems (2025–2026): prefix/prompt caching and its economics (the Anthropic and OpenAI documentation), semantic caching, model routing (RouterBench), context compaction, batch APIs; C. Huyen, *AI Engineering* (2025)—the economics and latency of inference; §2.2–2.3 and §18.5 of the book.

Part VII. Models of Your Own

Parts I–VI treated the model as a given—something called through an API. Part VII takes up the case where that is not enough and an organization moves to a model of its own: adapting an existing one (Chapter 21, fine-tuning), deploying an open model on its own infrastructure (Chapter 22), or training from scratch (Chapter 23). The shift in Part VII is from "using a model" to "owning a model," with the corresponding rise in both control and cost. The mathematical substrate of these operations is carried out into Appendix A, which the chapters reference.

Chapter 21. Fine-Tuning: Adapting the Model

Fine-tuning continues training a finished model on target data, changing its behavior. This chapter answers two questions: *when* to fine-tune instead of prompting and RAG, and *how* to do it efficiently—with methods that train a small fraction of the parameters, and with techniques for aligning to preferences.

21.1 When to fine-tune: the decision frame

Fine-tuning is neither the first tool nor a universal one. A ladder of rising cost and invasiveness applies: prompting (Part II) → RAG (Part III) → fine-tuning. The fundamental difference is *what* each level changes:

- **RAG adds knowledge**—factual information placed into the context at run time (Part III).
- **Fine-tuning changes behavior**—the form, the style, the answer format, adherence to domain conventions, tool-use patterns.

Hence the common mistake: fine-tuning for the sake of *knowledge*. Putting facts into the weights is expensive, they go stale, and every update demands retraining, whereas RAG refreshes the corpus without retraining (§6). Fine-tuning is justified when what needs to change is *behavior*: a stable format or style unreachable by prompting; specialization for a narrow domain with its own terminology; reliable tool-calling patterns; and lower cost and latency, because a fine-tuned *small* model replaces a large one on a narrow task (§20.3). As in §13.7, the rule is not to climb the ladder before measurement (Chapter 17) shows the previous level falling short.

21.2 Full fine-tuning versus PEFT

Full fine-tuning updates *all* of the model's weights. For large models that demands training-grade memory and compute, storing a full copy of the weights for every task, and it carries the risk of catastrophic forgetting (§21.8). Hence **parameter-efficient fine-tuning (PEFT)**: only a small fraction of the parameters is trained while the bulk of the model stays frozen. The key empirical observation underlying PEFT: the *update* to the weights for a specific task has low intrinsic rank—it can be approximated with a small number of parameters at no loss of quality. The intuition: a pretrained model already "knows how" to do almost everything required; adaptation does not rewrite knowledge, it nudges already-learned representations slightly—and that correction lives in a low-dimensional subspace of the full weight space.

21.3 Deep Dive: LoRA

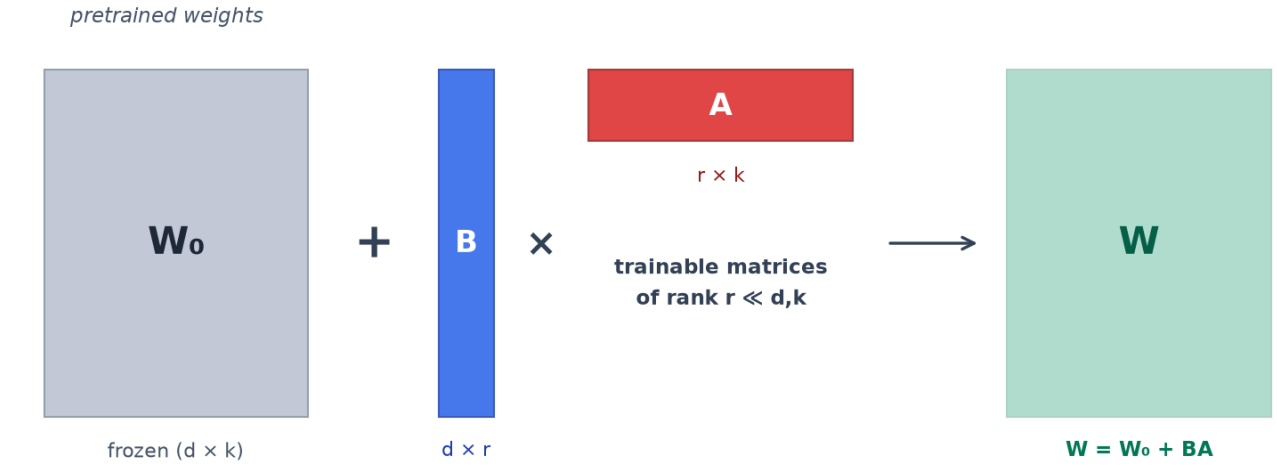
LoRA (Low-Rank Adaptation) (Hu et al., 2022) freezes the pretrained weights W_0 and adds a trainable low-rank update that enters the weight with a fixed multiplier:

$$W = W_0 + (\alpha/r) \cdot BA$$

Where:

- **W**—the effective (final) weight: the matrix the model actually computes with once the adapter is applied. The adapter does not change the shape of the layer—the dimensions are the same $d \times k$ as W_0 (4096 × 4096).
- **W_0**—the pretrained matrix of size $d \times k$, frozen; for the attention matrices of a LLaMA-like model, typically 4096 × 4096.
- **B** ($d \times r$) and **A** ($r \times k$)—the only trainable matrices. The product BA matches W_0 in size, but its rank is at most r—that is the low-rank assumption.
- **r**—the rank of the update, dimensionless; $r \ll \min(d, k)$, typically 8–64 (§21.8). The width of the "bottleneck" the correction is pushed through.
- **α**—the scaling constant, dimensionless; set in the config next to r, typically 16–128 (a direct consequence of the $\alpha \approx 2r$ default at r = 8–64, §21.8).
- **α/r**—the effective scale: the factor by which the trained increment enters the final weight. At r = 8 and α = 16 that is 16/8 = 2— BA is folded in at double weight.

It reads like this: the model computes on the frozen weights plus a correction squeezed through a rank-r bottleneck and amplified α/r times. Only A and B are trained; the number of trainable parameters drops from d·k to r·(d + k).



Two properties make LoRA the practical standard. First, the sharp cut in trainable parameters (for a 4096×4096 matrix at r = 8, roughly 256-fold) lowers memory and makes it possible to keep many lightweight adapters for different tasks on top of one base model—up to serving many adapters simultaneously from a single server (multi-LoRA serving: S-LoRA, LoRAX; supported in vLLM—a bridge to Chapter 22, where the base is loaded once and adapters are attached per request). Second, the $(\alpha/r)\cdot BA$ update can be *merged* into W_0 after training, yielding

an ordinary weight matrix—which is why LoRA adds no latency at inference. The method's limitation is the low-rank assumption itself: the rank of the update is capped at r, which narrows expressiveness on tasks whose update has complex structure (§21.5).

Initialization: starting exactly at the pretrained model. Matrix `A` is filled with small random numbers (Gaussian in the paper; Kaiming-uniform in the reference implementation and by default in PEFT), and `B` with **exact zeros**. Hence at step 0: `BA = 0`, so `(α/r)·BA = 0` for any α, and `W = W₀` exactly. An untrained adapter is the identity transform, a no-op: training starts precisely at the pretrained model, and the very first step cannot damage the base. The practical consequence—attaching a fresh adapter to a production model is safe by construction (which is what serving many adapters over one base rests on). The asymmetry here is mandatory: you cannot zero *both* matrices—the gradient with respect to `B` is proportional to `A`, the gradient with respect to `A` is proportional to `B`, and at `A = B = 0` both would vanish, leaving the adapter dead forever.

Why the α/r multiplier. The reason the authors give is operational, not mathematical: tuning α is roughly equivalent to tuning the learning rate, so α is fixed once and r is swept afterward without re-picking the learning rate. That also explains the `α ≈ 2r` default from §21.8: there is no magic two here, it is just the condition `α/r = 2` written out—"an effective scale of 2 regardless of rank." Which is why the default is expressed as a multiple of r rather than an absolute number: it holds the scale constant while r is swept. The flip side—α and r are part of the adapter's contract, not free knobs at merge time: what gets merged is `W₀ + (α/r)·BA` with *its own* multiplier, so an adapter trained at r = 8, α = 16 (scale 2) but merged at α = 32 (scale 4) produces a weight with the correction amplified twofold—silently, with no error and no warning. The adapter's config (`adapter_config.json`) stores both `r` and `lora_alpha`; tooling is obliged to take them from there and not from the current run's settings.

The r divisor is a heuristic, not a derivation. Why divide by r specifically, rather than by, say, √r, the paper does not derive—it confines itself to citing Yang & Hu (2021) and asserting that this scaling reduces the need to retune hyperparameters as r is swept. Kalajdzievski (2023) showed that the multiplier which keeps an adapter stable as rank grows is `α/√r`, whereas the classic `α/r` decays too fast as r rises and, in that work's own phrasing, slows learning and limits quality on high-rank adapters. The arithmetic of the divergence at a fixed α = 16: raising the rank from 8 to 64 drops `α/r` from 16/8 = 2 to 16/64 = 0.25—an eightfold fall, while `α/√r` goes from 16/2.83 ≈ 5.7 to 16/8 = 2, a fall of only √8 ≈ 2.83. The practical takeaway, worth keeping next to §21.8: if r has been pushed into the 32–64 range and quality did not follow the rank, suspect the multiplier first and the ceiling of the low-rank assumption (§21.5) only after; in PEFT this is exactly one flag, `use_rslora=True`, which switches the divisor to `√r`. A caveat, so three different regimes do not get conflated: at a fixed α the multiplier decays as 1/r; rsLoRA makes it decay as 1/√r; the α = 2r convention removes the rank dependence altogether, pinning the multiplier hard at 2.

21.4 Deep Dive: QLoRA and memory

QLoRA (Dettmers et al., 2023) removes the main barrier—the memory taken by the base model. Gradients propagate through a *frozen, 4-bit-quantized* base model into the LoRA adapters. Three techniques make this work without a material loss of quality: the 4-bit NormalFloat format (NF4), matched to the distribution of the weights; double quantization (the quantization constants are themselves quantized); and paged optimizers, which page out optimizer state during memory spikes. The result—fine-tuning a large model on a single GPU: the base is stored in 4 bits and is not trained, only the adapters are. Through 2024–2026 QLoRA is the standard choice for fine-tuning large models under constrained resources.

21.5 Deep Dive: beyond low rank—DoRA and the Kronecker product

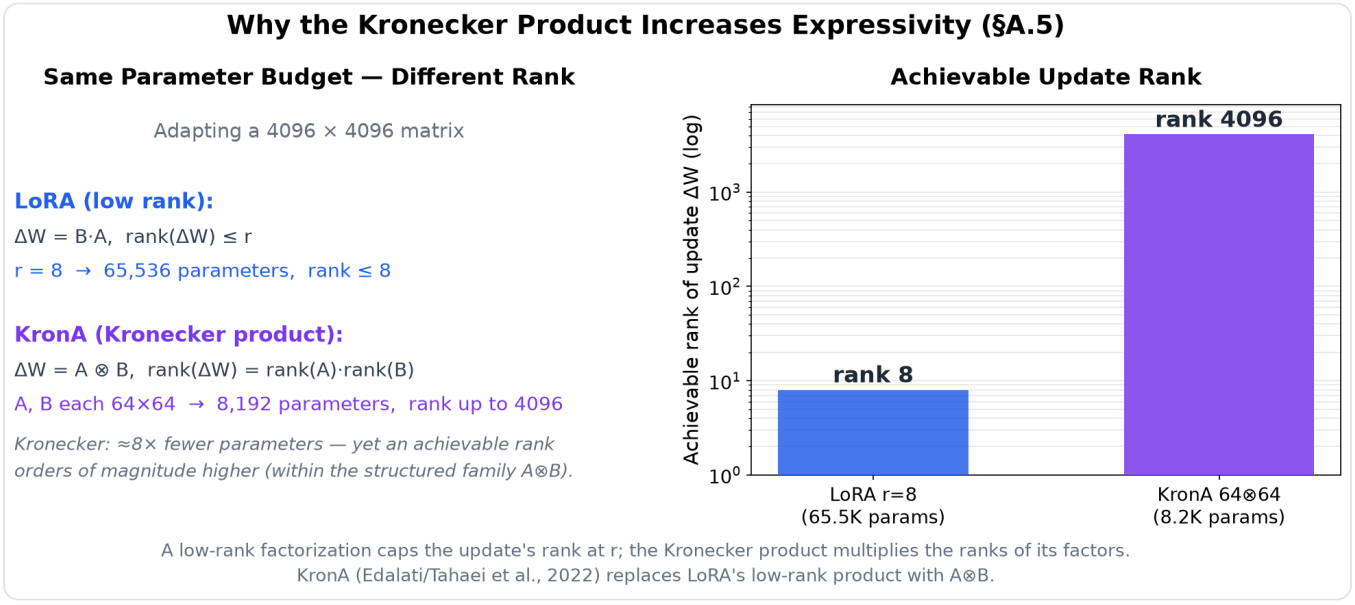
LoRA's low-rank assumption is at once the source of its efficiency and the limit on its expressiveness. Development runs in two directions.

DoRA (Weight-Decomposed Low-Rank Adaptation) (Liu et al., 2024) decomposes the pretrained weight into a *magnitude* and a *direction* and applies the low-rank update only to the direction component. The formula, in the LoRA parameters from §21.3:

$$W = m \cdot (W_0 + BA) \,/\, \|W_0 + BA\|_c$$

It reads like this: $W_0 + BA$ is the ordinary LoRA update (the α/r scale from §21.3 is dropped here, following the original paper: it does not affect how the decomposition works, so it is kept inside BA); dividing by $\|\cdot\|_c$ (the norm of each *column* taken separately) leaves only the direction—every column becomes unit length; the vector m (one entry per column, initialized with the norms of the original W_0) gives each column its length back and is trained *separately* from BA. The point of the decomposition: the analysis by Liu et al. showed that full fine-tuning changes the magnitude and the direction of the weights in a *loosely coupled* way, whereas LoRA is forced to change them jointly—through a single low-rank increment. By separating the components, DoRA brings the training dynamics closer to full fine-tuning at the same parameter budget (plus one vector m per layer—negligible). At inference everything merges back into a dense matrix, so there is no added latency—the same merge property LoRA has.

The Kronecker product (KronA). A deeper answer to the rank limit is to replace the low-rank product with a structure that yields *high* rank from few parameters. **KronA** (Edalati/Tahaei et al., 2022) uses the Kronecker product A ⊗ B in place of BA. The key algebraic property (Appendix A, §A.5): the rank of a Kronecker product is *multiplicative*—rank(A ⊗ B) = rank(A) · rank(B)—whereas the rank of an ordinary product (a contraction) is capped by the smaller of the dimensions. So at the *same* parameter budget the Kronecker product reaches an update rank orders of magnitude higher.



A low-rank factorization caps the update's rank at r; the Kronecker product multiplies the ranks of its factors. KronA (Edalati/Tahaei et al., 2022) replaces LoRA's low-rank product with A⊗B.

The distinction is fundamental and traces back to §A.5: a *contraction* (ordinary matrix multiplication) *lowers* or caps the rank, while the *Kronecker product raises* it. KronA exploits exactly this, getting a more expressive update out of fewer parameters—with one caveat: the reachable updates form a *structured* family A ⊗ B, not arbitrary full-rank matrices. The idea develops into LoKr (a low-rank increment on the Kronecker component), AdaKron, MoKA

(a mixture of Kronecker factors), and Kron-LoRA (a hybrid). Compressing adapters and the weights themselves further rests on tensor decompositions (Tensor-Train, Tucker, CP; §A.5)—the generalization of matrix factorization to multidimensional arrays. The algebra of all these constructions is worked through in Appendix A.

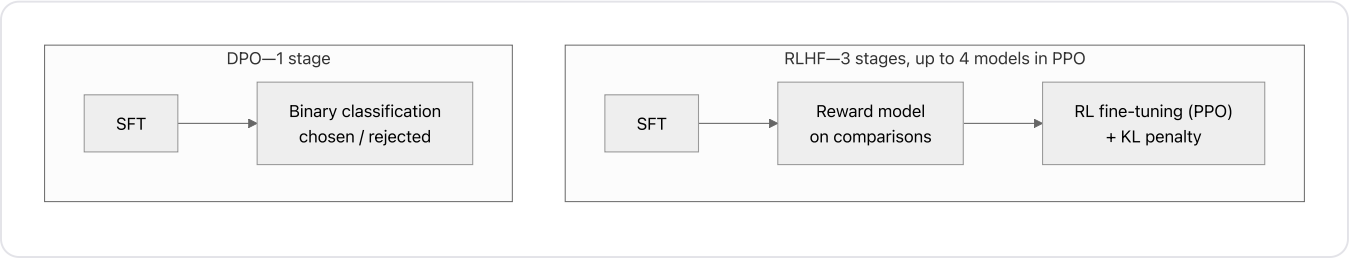
21.6 Deep Dive: preference alignment—RLHF and DPO

Fine-tuning for *behavior* covers more than imitation—it includes aligning to human preferences. Post-training usually runs through several stages: **SFT (supervised fine-tuning)**—imitation on "query → reference answer" pairs; **preference alignment**—tuning toward which answers people *prefer*; and, for verifiable reasoning tasks, **RLVR** (§13.8). The two main preference-alignment techniques differ radically.

RLHF (reinforcement learning from human feedback) (Ouyang et al., 2022, InstructGPT)—three stages: SFT; training a *reward model* on human comparisons of answers (rankings rather than absolute scores, to cut the noise from annotator calibration); fine-tuning the policy° through RL (PPO), maximizing reward under a penalty (KL°) for drifting from the original model. The procedure is powerful but complex and unstable, and the PPO phase holds *four* models in memory at once (the policy, the reference, the reward, and the critic).

GRPO (Group Relative Policy Optimization) (Shao et al., 2024, DeepSeekMath)—a lightened PPO that became the standard of the RLVR line (§13.8): instead of a separate value model (the critic), the advantage is estimated against a *group baseline*—the mean reward of several answers sampled for the same prompt. Three models stay in memory instead of four, and the algorithm sits naturally on tasks with a verifiable reward, where sampling a group of solutions for one prompt is cheap.

DPO (Direct Preference Optimization) (Rafailov et al., 2023) eliminates both the reward model and the RL. Alignment is recast as plain binary classification over chosen/rejected pairs: the model learns to raise the probability of the preferred answer and lower that of the rejected one. DPO implicitly posits the same reward model (Bradley–Terry°) but extracts the optimal policy *in closed form*, with no sampling from the model during training. The result is substantially simpler and more stable than RLHF, less prone to reward hacking°, at comparable or better quality on style control, summarization, and dialogue.

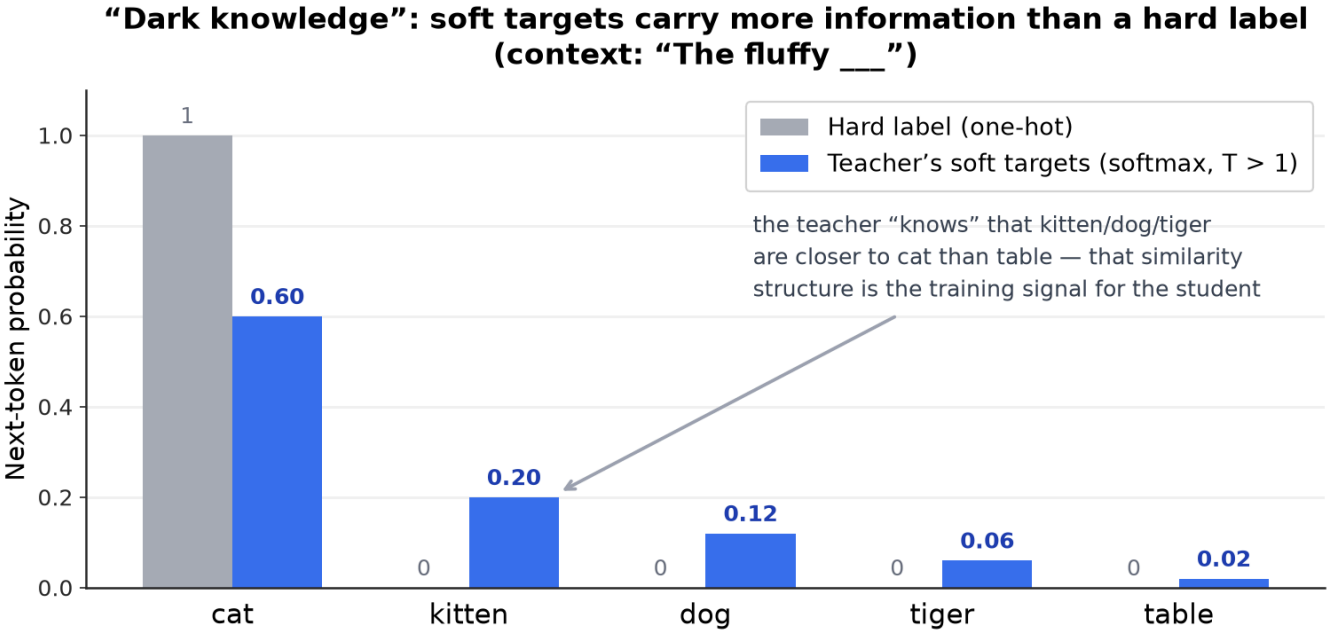


An important caveat: DPO is strong on preference and dialogue tasks, but on *mathematical reasoning* it yields only a marginal gain or even hurts—this is precisely where RLVR with a verifiable reward applies (§13.8). Related approaches swap human preferences for machine critics (Constitutional AI, RLAIF) and simplify DPO further (ORPO, SimPO°—no reference model; KTO—on *unpaired* data: "good/bad" labels suffice instead of chosen/rejected pairs, which matters when there is nowhere to get pairwise annotation from).

21.7 Deep Dive: knowledge distillation (teacher→student)

Fine-tuning and PEFT adapt *one* model; knowledge distillation solves an adjacent problem—transferring the abilities of a large model into a *smaller* one to cut cost, latency, and memory consumption (§21.1). Introduced by Hinton, Vinyals, and Dean (2015), it trains a compact **student** model to reproduce the behavior of a large **teacher** model.

Soft targets and "dark knowledge." The key idea—the student learns not from *hard* labels (one correct token) but from *soft targets*: the teacher's full probability distribution over the vocabulary. That distribution carries what Hinton called **dark knowledge**: the relative probabilities the teacher assigns to the *wrong* options encode the similarity structure it has learned and give the student more information per example than a single label does. The distribution is "softened" with **temperature** (§1.7): a softmax at T > 1 exposes that structure and turns it into a training signal.



A hard label conveys only "the correct answer is cat." The teacher's soft distribution (Hinton et al., 2015) adds the relative probabilities of the other options — more information per example, which is exactly what transfers to the student.

Response-, feature-, and relation-based. By *what* the student takes over, the taxonomy runs (Gou et al., 2021): **response-based** (logit distillation)—matching the final output distribution, Hinton's classic form; **feature-based** (hint-based, Romero et al., 2015)—aligning intermediate representations and hidden states; **relation-based**—matching the relations between representations (pairwise distances, angles). The response-based loss minimizes the divergence between distributions—the **forward KL divergence** (Hinton; DistilBERT), while for generative models the **reverse KL**° (MiniLLM, Gu et al., 2023) and generalized schemes (GKD°, Agarwal et al., 2023) are also used, as they better reflect the specifics of autoregression.

Hinton's objective. Formally, response-based distillation trains the student on a weighted sum of two terms: a *soft* one—agreement with the teacher's distribution—and a *hard* one—agreement with the true label:

$$L = \alpha \cdot T^2 \cdot KL\left(\sigma(z_t/T) \parallel \sigma(z_s/T)\right) + (1-\alpha) \cdot CE\left(y, \sigma(z_s)\right)$$

Symbol	What it is	Live value
L	The student's loss—what training minimizes	—
z_t, z_s	The teacher's and the student's logits—raw scores before softmax, one number per vocabulary token (§1.7)	a vector as long as the vocabulary: 30,000–130,000 in an LLM (§1.2)
σ	softmax; $\sigma(z/T)$ is the temperature-softened distribution	—
T	Temperature (§1.7), dimensionless. In the soft term the same T is applied to both the teacher and the student; at inference it returns to 1	in Hinton: 2 (speech), 20 (MNIST)
α	The weight of the soft term, from 0 to 1	Hinton: a "considerably lower weight" on the hard term—that is, α closer to 1
KL	The forward KL divergence—a measure of how far the student's distribution departs from the teacher's	—
CE	Cross-entropy against the hard label y; computed at $T = 1$	—
y	The true label—the correct next token from the training data	one-hot: 1 on the correct token, 0 on the other 30,000+

In plain terms: the student reaches at once toward the teacher's full "opinion" about *all* the options (the soft term) and toward the single correct answer (the hard one); α sets which of the two dominates the mix, and T^2 is the correction without which α does not mean what it says. A caveat for anyone checking against the original: Hinton writes the soft term as cross-entropy against the soft targets rather than through KL; the two differ by the teacher's entropy—a term that does not depend on the student's weights and therefore contributes zero gradient, so for optimization the forms are equivalent.

Why the T^2 multiplier is not cosmetic. This is the one non-obvious detail in the formula, and it is purely an engineering one. The gradient of the soft term with respect to a student logit equals $\frac{1}{T}(q_i - p_i)$, where q and p are the softened distributions of the student and the teacher. The factor $1/T$ enters here *twice*: explicitly, from differentiating z/T with respect to z, and implicitly, because softening shrinks the difference $(q_i - p_i)$ itself in proportion to $1/T$. Hinton writes out the limit: for T large relative to the logits, and logits centered at zero, the gradient $\approx \frac{1}{NT^2}(z_{s,i} - z_{t,i})$, where N is the vocabulary size (the number of classes the softmax normalizes over); in the original the same logits are written z for the student and v for the teacher. The upshot: the strength of the soft signal falls as $1/T^2$.

Hence what is at stake. Let $\alpha = 0.9$—nominally the soft term outweighs the hard one 9 : 1. Without the T^2 multiplier the actual ratio equals $\frac{\alpha/T^2}{1-\alpha}$:

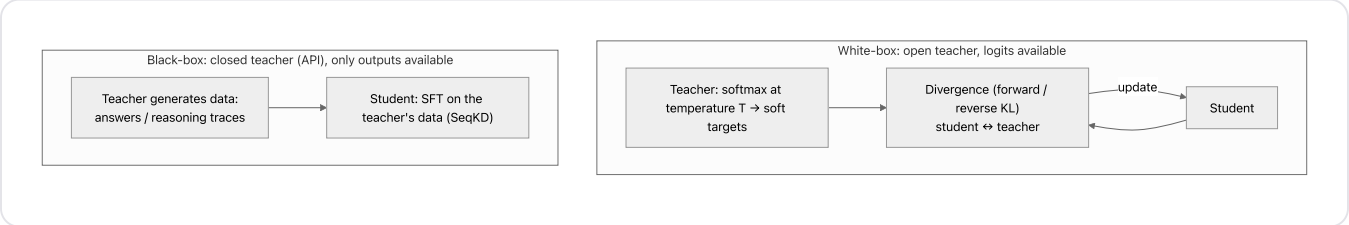
- at $T = 2$: $0.9/4 = 0.225$ against 0.1—a ratio of 2.25 : 1, the soft term still leads;
- at $T = 4$: $0.9/16 = 0.056$ against 0.1—a ratio of 0.56 : 1, meaning **the hard term now dominates by nearly twofold**.

A single doubling of the temperature flipped the mix, though nobody touched α. With the T^2 multiplier the product $T^2 \cdot \frac{1}{T^2} = 1$, and the ratio equals $\alpha : (1 - \alpha) = 9 : 1$ at any T.

The invariant: T^2 makes T and α *independent* knobs. Without it, raising the temperature silently devalues the soft term relative to the hard one, and a sweep over T measures something other than what the experimenter assumes: the balance of the two terms rides along with T. A separate trap is the convention for α: different implementations use it to mean the weight of the soft term in one place and the weight of the hard term in another. You establish which from the code, not from the parameter's name.

Temperature and the student's capacity. There is no universal T: the working range is tied to the student's capacity, and Hinton measured this directly. On MNIST a student with 300 or more units per layer is nearly insensitive to temperature—every value above 8 gave a close result (the published run used $T = 20$). But for a radically trimmed student, 30 units per layer, the best values of T turned out to be in the 2.5–4 range—markedly better than both higher and lower ones. On speech recognition, out of the sweep [1, 2, 5, 10] the best was $T = 2$ at a relative weight of 0.5 on the hard term. The mechanism is simple: a high T lifts the tail of the teacher's tiny probabilities; to a high-capacity student that tail is signal, to a cramped one it is noise—noise it cannot reproduce and that crowds out what matters. Hence the rule: T is picked together with the size of the student, not carried over from someone else's config; for heavily compressed students a sensible starting point is $T \approx 2$–5, and from there go by evaluation (Chapter 17)—the same discipline as with LoRA's rank (§21.8).

White-box versus black-box—the main division for LLMs. By access to the teacher, LLM distillation splits into two classes:



- **White-box (open teacher)**—logits and internal representations are available. The student trains on soft targets (token probabilities), getting a rich signal; it requires an open teacher model and is computationally heavier.
- **Black-box (closed teacher)**—only the teacher's textual *outputs* are available (through a proprietary model's API, for instance). The standard technique is **sequence-level knowledge distillation (SeqKD)** (Kim & Rush, 2016): the teacher generates a set of answers and the student is fine-tuned (SFT) on them. Simpler, but it loses the teacher's internal distribution. The modern extension is SFT on the teacher's *reasoning traces* to transfer the ability to reason (the DeepSeek-R1-Distill line, §13.8; earlier, Alpaca/Vicuna for transferring instruction following).

On-policy distillation. Naive black-box distillation suffers from a distribution mismatch: the student trains on the teacher's data but at inference walks its *own* trajectories. On-policy distillation removes this by training the student on *its own* generations, scored or corrected by the teacher—an important modern direction.

Inherited positional bias. The distribution mismatch is not the only defect introduced not by the loss function but by the *data-collection protocol*. The teacher that produces the data reads its own prompt—and is itself subject to **positional bias** (lost in the middle: the middle of a long input is used less well than the edges, §1.8, §4.3). The difference in consequences is fundamental. At inference this is a *transient* error: move the fact from the middle to an edge and the answer is right again. In distillation there are no transient errors: nobody rejects the teacher's answer—it **becomes the training target**. If the data was generated on long prompts and the teacher answered while ignoring their middle, the student gets a gradient step that encodes "ignore the middle" as the *correct* behavior. A bias that lived in the teacher for a fraction of a second in a single run settles into the student's weights forever.

Why the loss function does not see this. In black-box mode the student is fine-tuned (SFT) on the teacher's outputs—it maximizes the likelihood of the sequences the teacher generated (SeqKD, above). The loss is cross-entropy averaged over the answer's tokens:

$$L = -\frac{1}{|y|}\sum_{t=1}^{|y|}\log p_{student}(y_t^{teacher} \mid y_{<t}^{teacher}, x)$$

Where:

- L—the loss itself (not to be confused with the L of Hinton's objective earlier in this section: there it is a dimensionless weighted sum of a soft and a hard term, here it is a single cross-entropy over the teacher's outputs; different distillation regimes); the unit is **nats per token**, the logarithm natural (in $\log_{10}$ the same number would come out as 0.046 instead of 0.105—the base must be pinned down, or the scale lies by a factor of 2.3). The student's confidence sets the scale: $p_{student} = 0.5$ gives $L \approx 0.69$, $p_{student} = 0.9$ gives $L \approx 0.105$, $p_{student} \to 1$ gives $L \to 0$. This is the very number the training curve plots.
- t—the position index in the answer, from 1 to $|y|$: the sum runs over every token of the answer, and dividing by $|y|$ makes L the *average* loss per token rather than a quantity that grows with the answer's length.
- x—the prompt the teacher was asked to generate data on (from tens of tokens to hundreds of thousands). Its length is the risk variable in question: on a short input the teacher reads all of it, on a long one it no longer does (how long is a matter of measurement, not of a lookup table, §4.3).
- $y_t^{teacher}$—the t-th token of the teacher's answer; for the student this is the "correct answer" at position t. $|y|$ is the answer's length in tokens (hundreds for a typical answer).
- $y_{<t}^{teacher}$—all the **teacher's** preceding tokens: the student predicts the next token while looking at the teacher's prefix, not at its own (**teacher forcing**—training on the reference prefix). Hence the main point: the student has nowhere to diverge from a defective target—at every step it is put back on the teacher's rails. This is exactly the dependency on-policy distillation (above) breaks, by feeding the student its own prefix.
- $p_{student}$—the probability (dimensionless, 0…1) the student assigns to the teacher's next token.

Live numbers. Suppose the student has converged and assigns $p_{student} = 0.9$ to every token of the teacher, and the answer's length is $|y| = 300$ tokens. The loss per token equals $-\ln 0.9 \approx 0.105$ nats, the sum over the answer is $300 \times 0.105 \approx 31.6$ nats, and dividing by those same 300 gives back $L \approx 0.105$ nats per token. The training curve will show exactly the same figure in the case where $y^{teacher}$ was obtained by ignoring the middle of the prompt. The reason: **the formula contains no term accountable for the teacher being right**—the input x enters only as a condition, and nothing checks whether the teacher read x in full. In plain terms: the loss function measures exactly one thing—how precisely the student repeats the teacher; a perfect zero is reached on a defective target just as on a sound one. The metric the engineer watches is blind to this defect by construction.

The defect is in the generation protocol, not in the method. Distillation does not *create* positional bias—the data-collection recipe introduces it. That positional behavior is distillable at all is not a guess, but it rests on two supports, and they are about different things. Wang et al. (2025) demonstrate transferability *within a single model*: their Pos2Distill carries behavior over from "winning" input positions to "losing" ones, where teacher and student are the same model, differing only in where the fact sits in the input (of the regimes listed below, this is closest to *self-distillation*—distilling a model into itself). It follows that positional behavior is a quantity that can be distilled in principle; it says nothing about transfer *between* models. Transfer from teacher to student is shown by Huber et al. (2026): they trace how positional perturbations from the query and key vectors (Q/K, §1.4) reach the teacher's output distribution "and, further, the distillation signal received by the student." The caveat matters: their channel is logits (white-box), whereas what is at issue here is black-box SFT on textual outputs; in black-box the transfer is more direct and needs no citation—the defective answer literally becomes the target. The general formulation is in Zhang et al. (2024): distillation passes on "both correct and incorrect knowledge," and the teacher's bias misleads the student.

Hence the rule: **generate the distillation data on short, dense fragments**, not on a long document whole. The common reference point is 1024–2048 tokens per prompt; this is an engineering heuristic (the original source could not be traced), a starting point for a sweep rather than a constant, and it rests on an argument rather than on a measurement: at that length the teacher's attention is not smeared, the whole fragment lies in the zone where it works confidently, and the student's gradient is gathered from answers that genuinely rest on the entire input. The quality of the training signal is a function of how loaded the teacher's context is.

The objection—"won't the student then lose long context?"—is defused by separating two lengths that are easy to conflate: the short one belongs to the *fragment* at generation time, the long one to the *training sequence*. Huber et al. (2026) transfer long-context retrieval into the student by training it exclusively on short examples **packed into a long window** (packed short-context samples): the fragments themselves are short, but the training sequence is full length, so the RoPE positions across the whole window stay exercised (§1.4). Packing here is not a formatting detail but a condition for the thing to work at all: a student trained on short *sequences* is precisely the one that will not get a long window—which Huber et al. do not claim. The same caveat holds: their channel is logits, so carrying their conclusion over to black-box SeqKD remains an argument from mechanism, not a citation. The short-fragment rule does not need that prop anyway: it rests on the teacher being obliged to read its input in full.

Practical reference points. The standard scheme is *offline* (the teacher is frozen); there are also *online* (teacher and student train together) and *self-distillation* (teacher and student are the same network). Empirically the principle "pretrain the student first, then distill" (Turc et al.) works: a strong base plus the teacher's outputs compound. The classic result at scale is **DistilBERT** (Sanh et al., 2019): the student is initialized from part of the teacher's weights and trained on soft targets, retaining about 97% of BERT's quality at 40% smaller size.

Where it sits on the compression map. Distillation must be told apart from two other techniques for getting a compact model: **quantization** (§22.6) lowers the *precision* of the weights of the *same* model; **pruning** removes weights or structures from the *same* model; **distillation** trains a *fundamentally new, smaller* model—possibly of a different architecture—to reproduce the teacher. The techniques are complementary and are often combined (distill, then quantize). Caveats: the student rarely catches the teacher fully, and the gap widens on hard and reasoning tasks (partially closed by distilling traces); white-box requires an open teacher, while black-box out of a proprietary API may violate its terms of use—that gets checked before you apply it.

21.8 Practice: data, forgetting, evaluation

Three operational factors determine whether fine-tuning succeeds.

- **Data.** The quality and representativeness of the target set matter more than its size; fine-tuning amplifies the patterns in the data, defects included. Alignment needs *preference* pairs, SFT needs reference answers. One special case deserves its own control: if a teacher produced the set (§21.7), the teacher's own defects enter the data as ground truth—so what you check about the generation is not only the answers but the **length of the prompts** they were obtained on.
- **Catastrophic forgetting.** Fine-tuning for a narrow task can degrade the model's general abilities. PEFT (a frozen base) lowers the risk compared with full fine-tuning but does not remove it; fine-tuning for several tasks in sequence demands particular care. The practical mitigation is to mix a share of general data into the target set (replay), which holds the model back from drifting.
- **LoRA starting heuristics.** Rank r = 8–16 for style/format tasks, 32–64 when the task is structurally harder (and that is a signal to re-check whether a different method is needed at all); alpha ≈ 2r as the common default —an effective scale of $\alpha/r = 2$ regardless of rank (§21.3); if raising r yields no gain in quality, check the multiplier

(rsLoRA, §21.3) before blaming the rank ceiling; target modules—all the attention matrices (Q/K/V/O), and if quality falls short, the FFN projections as well. These are starting points for a sweep, not constants: they are confirmed by evaluation (Chapter 17).

- **Evaluation.** A fine-tuned model is evaluated with the same discipline as any other (Chapter 17): on a *held-out* set and against a baseline, checking not only the target task but also that general abilities have not regressed. A gain on the target paid for by degradation everywhere else is a common outcome, and one that goes unnoticed.

21.9 Chapter summary

Fine-tuning changes the model's *behavior*, whereas RAG adds *knowledge*; fine-tuning for the sake of knowledge is the typical mistake (§21.1). Full fine-tuning is expensive, so PEFT is used instead, leaning on the low intrinsic rank of the update (§21.2). LoRA trains a low-rank increment $W_0 + (\alpha/r)\cdot BA$ that merges in at inference with no latency (§21.3); QLoRA removes the memory barrier with a 4-bit-quantized base (§21.4). The low-rank limit is overcome by DoRA and, more deeply, by the Kronecker product (KronA), which raises the reachable rank through the multiplicativity of Kronecker rank—a direct consequence of §A.5 (§21.5). Preference alignment goes through RLHF (three stages, a reward model, PPO) or the radically simpler DPO (binary chosen/rejected classification), with the caveat about DPO's weakness on reasoning, where RLVR is what you need (§21.6). Knowledge distillation transfers the teacher's abilities into a smaller model through soft targets ("dark knowledge"), in white-box (via logits) or black-box (SFT on the teacher's outputs, SeqKD) mode, and differs from quantization and pruning in that it yields a *new* compact model; along with the abilities, the teacher's defects transfer too—including positional bias, if the data was generated on long prompts (§21.7). Success is determined by the data, by keeping forgetting in check, and by honest evaluation against a baseline (§21.8). Next up is Chapter 22: deploying open models on your own infrastructure.

Sources. E. Hu et al., "LoRA" (arXiv:2106.09685, 2022); D. Kalajdzievski, "A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA" (arXiv:2312.03732, 2023)—rsLoRA, the $\alpha/\sqrt{r}$ multiplier; T. Dettmers et al., "QLoRA" (arXiv:2305.14314, 2023); S. Liu et al., "DoRA" (arXiv:2402.09353, 2024); A. Edalati / M. Tahaei et al., "KronA: Parameter Efficient Tuning with Kronecker Adapter" (arXiv:2212.10650, 2022) and its development (LoKr, AdaKron, MoKA, Kron-LoRA, 2024–2025); R. Rafailov et al., "Direct Preference Optimization" (arXiv:2305.18290, 2023); L. Ouyang et al., "InstructGPT / RLHF" (2022); N. Lambert, *RLHF Book* (2025); Z. Shao et al., "DeepSeekMath" (2024)—GRPO; K. Ethayarajh et al., "KTO: Model Alignment as Prospect Theoretic Optimization" (2024); on distillation: G. Hinton, O. Vinyals, J. Dean, "Distilling the Knowledge in a Neural Network" (2015); J. Gou et al., "Knowledge Distillation: A Survey" (2021); V. Sanh et al., "DistilBERT" (2019); Y. Kim, A. Rush, "Sequence-Level Knowledge Distillation" (2016); Y. Gu et al., "MiniLLM" (2023) and R. Agarwal et al., "GKD" (2023); Y. Wang et al., "Position Bias Mitigates Position Bias: Mitigate Position Bias Through Inter-Position Knowledge Distillation" (arXiv:2508.15709, 2025)—transfer between positions within one model; J. Zhang et al., "Can Students Beyond The Teacher? Distilling Knowledge from Teacher's Bias" (arXiv:2412.09874, 2024); P. Huber et al., "Short Data, Long Context: Distilling Positional Knowledge in Transformers" (arXiv:2604.06070, 2026)—teacher→student transfer via logits; surveys of LLM distillation and model compression (2023–2025); §13.8 (RLVR) and Appendix A (§A.5) of this book.

In the Margins: Chapter 21

↩ **Policy** in the RL sense—the strategy for choosing an action given a state; for an LLM the policy is the model itself (the next-token distribution), and "training the policy" means updating its weights. ≠ Homonym: in this book "policy" is also a governance rule of an organization or a gateway (Chapters 20, 25), an RLS access policy in Postgres (Chapter 8), and a memory forgetting policy (§4.5). Four senses from four different disciplines; in Chapter 21 "policy" is always the RL term.

↩ **KL divergence**—a measure of the discrepancy between two distributions. The KL penalty holds the policy being trained close to the original model: without it the policy "hacks" the reward and pays for it with degraded language. It is asymmetric: KL(P‖Q) ≠ KL(Q‖P), hence the forward and reverse forms (§21.7).

↩ **Bradley–Terry** (1952)—the classic model of pairwise comparisons: the probability that "a beats b" is expressed through the latent "strengths" of the options; this is exactly what DPO implicitly estimates from chosen/rejected pairs.

↩ **Reward hacking**—exploiting flaws in the reward model: the policy finds answers that score high without real quality (longer, more self-assured, more formulaic).

↩ **ORPO, SimPO**—simplifications of DPO with no separate reference model: ORPO merges alignment with SFT through an odds ratio, SimPO normalizes the implicit reward by the answer's length.

↩ Forward KL (mode-covering) forces the student to "smear" probability across all of the teacher's modes; reverse KL (mode-seeking) makes it concentrate on the dominant ones. For generation the reverse is often the more honest: better to reproduce the strong modes confidently than all of them badly.

↩ **GKD**—Generalized Knowledge Distillation (Agarwal et al., 2023): distillation on the student's *own* generations (on-policy), closing the gap between "trained on the teacher's texts" and "generates its own."

Chapter 22. Serving Open Models

Chapter 21 adapted the model; this chapter deploys it—an open-weight model—on infrastructure you own. The question is twofold: *when* to serve it yourself instead of calling an API, and *how* efficient serving works—the techniques that make GPU inference economical. The chapter consolidates the KV-cache (§1.6) and the economics of inference (Chapter 2) at the infrastructure level.

22.1 Why deploy an open model

Serving it yourself is justified where an API does not fit, for one of these reasons:

- **Control and privacy.** Data never leaves the perimeter—which matters for regulated domains and for data residency.
- **Cost at scale.** At a large, sustained volume, serving it yourself can come out cheaper than paying an API per call (§2.2), though it requires amortizing the infrastructure.
- **No vendor lock-in.** The model and its version stay under the organization's control; nothing depends on an API or a price list changing.
- **Customization.** Deploying *your own* fine-tuned model (Chapter 21), and full control over inference parameters, quantization, and version.

The price: infrastructure (GPUs), operations, and the engineering complexity of serving—which the rest of the chapter is about. The "API or self-host" decision is the same trade-off of control against cost that runs through the rest of Part VII.

22.2 The serving stack

GPU inference is handled by specialized servers, not by a naive loop on a training framework. The main ones:

- **vLLM** (Kwon et al., 2023)—introduced PagedAttention and popularized continuous batching (Orca, 2022; §22.4); a common production choice at tens to hundreds of concurrent users, with iteration that needs no recompilation and support across a range of GPUs.
- **SGLang** (Zheng et al., 2024)—RadixAttention for prefix caching, and efficient constrained decoding (§5.4).
- **TensorRT-LLM** (NVIDIA)—optimized kernels through NVIDIA's compilation stack; maximum performance on NVIDIA hardware at the price of a compilation step.
- **TGI** (Text Generation Inference) and others—the remaining inference servers.

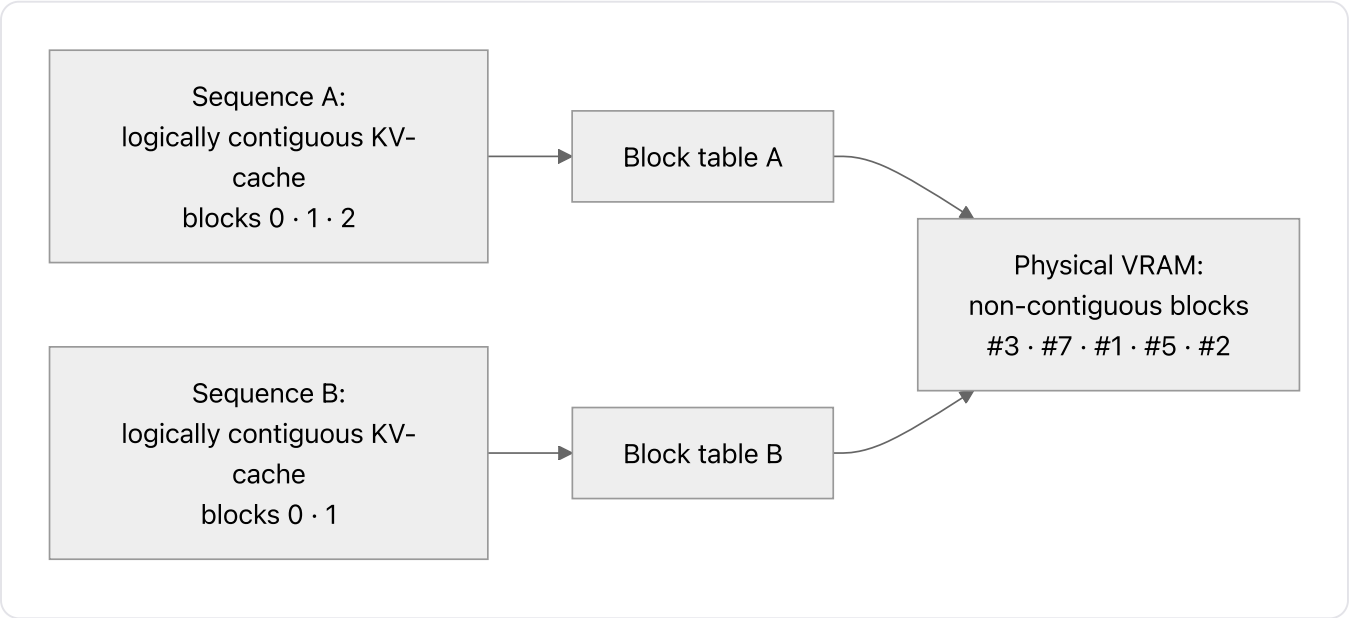
How to choose: **vLLM**—the general-purpose default (broad model and GPU support, multi-LoRA, fast iteration without compilation); **SGLang**—when the load profile is built on heavy prefix reuse (agent trees, many-shot) or on heavy constrained decoding; **TensorRT-LLM**—to squeeze the most out of an NVIDIA fleet, at the price of a compilation cycle per model and configuration; **TGI**—simplicity inside the Hugging Face ecosystem. On typical workloads the performance differences among the first three are smaller than the effect of tuning any one of them properly.

A naive PyTorch inference loop falls short of these systems by a multiple; what follows are the techniques that account for the gap.

22.3 Deep Dive: PagedAttention

The main resource in serving is GPU memory for the KV-cache (§1.6), which grows linearly with sequence length and with the number of concurrent requests. Naive management demands a *contiguous* block of VRAM per request, sized to the maximum length—which leads to over-allocation and fragmentation.

PagedAttention (Kwon et al., SOSP 2023) solves this by mapping a sequence's KV-cache through a *logical block table* onto *non-contiguous physical blocks* of GPU memory. By the direct analogy the authors themselves draw, this is the same principle of virtual memory by which an OS manages RAM: logical pages map onto physical frames, and contiguity is not required.



The consequences are substantial: VRAM is not allocated up front to the maximum, fragmentation goes away, and the memory freed up holds more concurrent requests. PagedAttention is the foundation vLLM's high throughput is built on; along the way, it makes sharing common prefixes between requests possible (§22.7).

Exactly how much memory sits idle. The authors measured it on their own workload (OPT models, the ShareGPT and Alpaca datasets): in existing systems only **20.4–38.2%** of KV-cache memory goes to *actual* token states (Fig. 2 of the paper)—the other 61.8–79.6% is eaten by the reserve against maximum length and by fragmentation. For vLLM the same figure is **96.3%**, meaning residual waste under 4%. Hence the headline result: **2–4× the throughput at the same latency** against FasterTransformer and Orca. This is a measurement on one specific traffic profile, not a constant of nature, but the order of the gap tells: a naive memory manager puts a fifth of VRAM to work.

The top of the range—38.2%—is the telling one. It belongs to the Orca (Oracle) variant, which knows the actual answer length *in advance*. Even perfect knowledge of the length does not save it: as long as the request is alive, its entire contiguous chunk of memory stays reserved for it, and other, shorter requests cannot move into the idle tail. What you pay for, then, is not ignorance of the length but the requirement of contiguity itself—which is why the cure is sought in indirection rather than in predicting lengths more accurately.

Where the idle memory comes from. Take the derivation in §1.6—320 KB per token for a 70B-class configuration (80 layers, 8 KV heads, `d_head` = 128, fp16). A request with a maximum of 8192 tokens that actually generated 500: the naive manager reserves 8192 × 320 KB ≈ 2.5 GB, of which 500 × 320 KB ≈ 156 MB does real work. Idle: ≈2.3 GB—94% of what was reserved, and idle for the request's entire lifetime. This is an extreme point, not a recomputation of the 20.4–38.2% measured above: here one request carries a ratio of "declared maximum : actual" = 16:1, whereas the bottom of the band, 20.4%, came from the variant that reserves against the model's own ceiling (2048 tokens for OPT), averaged over traffic—there the ratio is about 5:1. The mechanics of the idling are the same; the multiplier differs. These are exactly the gigabytes that paging returns to the shared pool.

What block allocation buys. The allocation unit in vLLM is a block of B tokens (by default $B = 16$; the value was picked experimentally: larger and internal fragmentation grows, smaller and the GPU is utilized worse). Blocks fill left to right and a new one is allocated only when the previous is full, so only the last block can be underused—and only if L is not a multiple of B. The waste per request:

$$W = \lceil L/B \rceil \cdot B - L \le B - 1$$

Where:

- ***L***—the actual sequence length, in tokens (500 in the example).
- ***B***—the block size, in tokens (16 by default in vLLM).
- ***W***—the unoccupied slots in the last block; equally, the whole of the request's wasted memory.

Live numbers: $\lceil 500/16 \rceil = 32$ blocks, taking 32 × 16 × 320 KB = 160 MB. Of that, 156 MB does real work, and $W = 32 \times 16 - 500 = 12$ slots ≈ 3.8 MB. The worst case is $B - 1 = 15$ slots ≈ 4.7 MB.

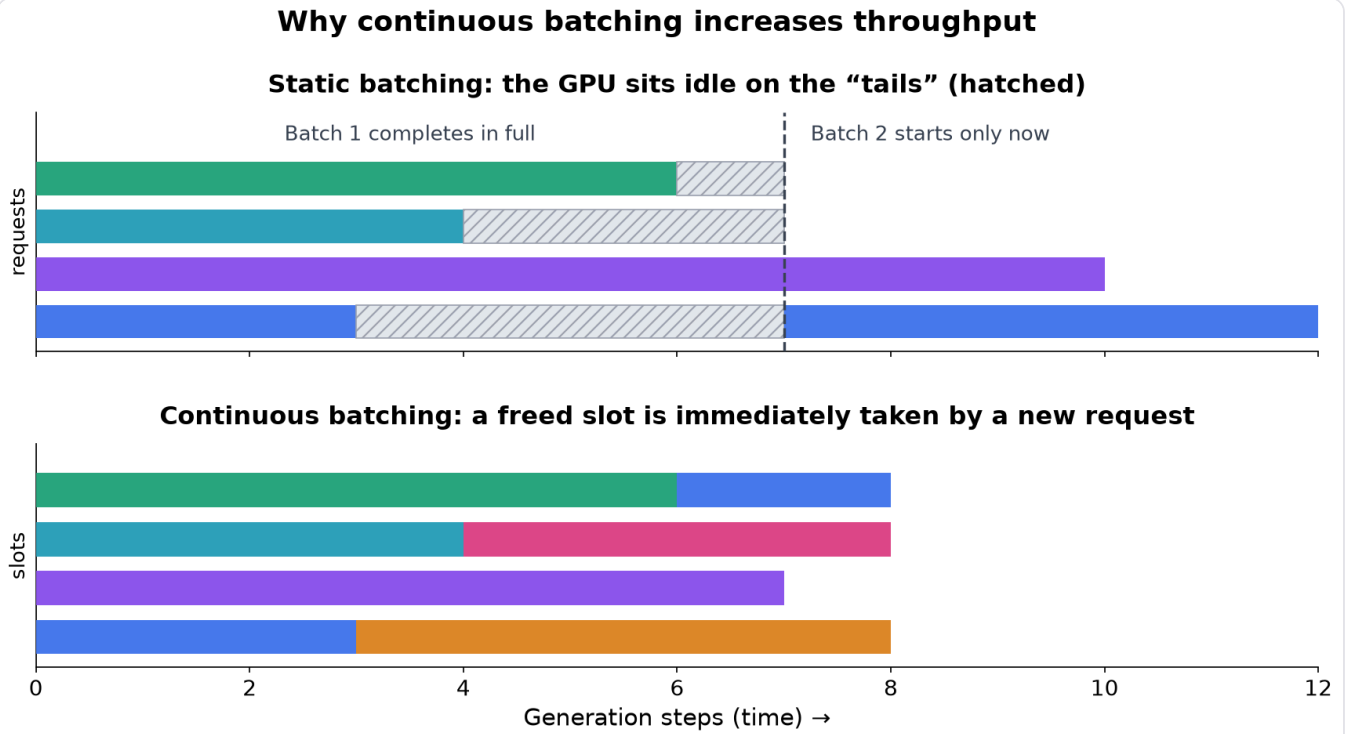
In plain terms: with paged allocation only the tail of the last block is lost, so the waste is a constant of no more than 15 tokens, whereas with a naive reservation it grows along with the *declared* maximum. In the example that is ≈2.3 GB against ≈4.7 MB per request, and the request's footprint in memory is 2.5 GB against 160 MB—exactly 16× smaller.

The invariant for capacity planning. This is what makes "more requests" countable. Naively, the number of concurrent requests is the VRAM budget for the cache divided by L_{max} × 320 KB—that is, it follows the *declared* maximum; under paged allocation it follows the *actual* distribution of lengths in the traffic. The sixteen-fold memory headroom from the example, however, does not carry over into throughput in full: the 2–4× the authors measured is noticeably lower, and that is not a contradiction. Decode runs up against memory *bandwidth*, not memory capacity (§22.5), so past a certain batch size the bottleneck moves from capacity to the HBM bus and to compute. PagedAttention lifts the capacity limit; how much of what it frees turns into tokens per second is decided by the length distribution and the bandwidth ceiling. The practical consequence: capacity is planned from a histogram of *actual* answer lengths, not from `max_tokens`, and checked against a measurement on your own traffic (§22.8).

22.4 Deep Dive: continuous batching

The second foundation is how the batch gets formed. *Static* batching groups requests together and holds the GPU until the longest of them finishes; because answer lengths vary widely, short requests sit idle waiting and the next batch does not start—the GPU is underloaded.

Continuous batching (Orca, Yu et al., 2022; later vLLM, SGLang) removes this: on *every* forward pass the scheduler adds newly arrived requests to the running batch and retires the finished ones, filling a freed slot with a new request immediately.



Static: the batch holds the GPU until its longest request finishes — short requests sit idle.
Continuous (Orca, 2022 → vLLM): the scheduler adds and removes requests on every pass, keeping the GPU busy.

This keeps GPU utilization high across heterogeneous lengths and yields a multiple-fold throughput gain under high concurrency—one of the key things separating a modern server from a naive loop.

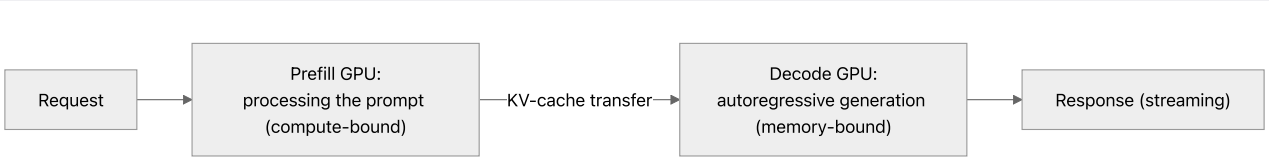
22.5 Deep Dive: prefill vs. decode and disaggregation

Inference consists of two phases with *different* computational profiles:

- **Prefill**—processing the input prompt, parallel across tokens; *compute-bound*.
- **Decode**—autoregressive generation one token at a time; *memory-bound*, since every step reads the entire KV-cache.

Running both phases on one device produces *interference*: adding a new prefill to the active batch crowds out the decode in flight, delaying token generation and causing spikes in inter-token latency (ITL). Two answers:

- **Chunked prefill** (Sarathi, Agrawal et al., 2023)—splits a long prefill into chunks and interleaves it with decode, capping peak memory; a small rise in p50 TTFT buys a markedly better p95.
- **Disaggregated serving** (DistServe, Zhong et al., 2024; Splitwise, Patel et al., 2024)—physically separates prefill and decode onto *different* GPUs, removing the interference and letting you match hardware to each phase’s profile (compute-bound against memory-bound). It has become a common technique, supported by vLLM and SGLang.



22.6 Quantization for inference

Quantization for *inference* differs from QLoRA (§21.4): QLoRA introduces adapters on top of a quantized base *for training*, whereas the goal here is to compress the *deployed* weights, cutting memory and speeding up reads (decode is memory-bound, §22.5). The formats:

- **AWQ and GPTQ**[°]—methods of 4-bit post-training weight quantization; AWQ accounts for the salience of activations. 4-bit quantization brings a large model down to a single GPU: Llama-3-70B, for instance, goes from 4×A100 (FP16) to one GPU with AWQ.
- **FP8**—the natural format for the H100/H200 with their native FP8 tensor cores; it holds quality with minimal calibration.
- **FP4 (NVFP4/MXFP4)**—micro-scaled 4-bit floating-point formats with native hardware support in NVIDIA's Blackwell generation (B100/B200/GB200, 2024–2025); the next step in density after FP8, with the same condition attached—confirm the degradation by evaluating on your own tasks.
- **INT8**—integer quantization of the weights.
- **KV-cache quantization**—separate from the weights; in practice available in FP8, which further shrinks the main resource in serving (§22.3).

The trade-off is memory and throughput against accuracy: the more aggressive the quantization, the greater the saving and the higher the risk of degradation, so the choice of format is checked against the hardware and the quality tolerance (and confirmed by evaluation, Chapter 17).

22.7 Speculative decoding and prefix caching

Two techniques cut latency and repeated work in serving.

- **Speculative decoding** (§1.6, §2.3)—a small *draft* model proposes several tokens, and the large *target* model verifies them in a single pass, accepting the ones that match. It speeds up generation without changing the output distribution; it pays off most under *low* concurrency, where per-request latency matters rather than aggregate throughput. Model-free variants (Medusa, EAGLE) do without a separate draft model: the draft tokens come from lightweight extra heads on top of the target model itself, which removes the problem of finding a well-matched draft-target pair.

Deep Dive: why speculation speeds up generation without distorting it

The technique exploits the asymmetry from §22.5: decode is memory-bound—for every token the model reads *all* the weights to take one step. But γ already-drafted tokens can be verified in **one** forward pass: feed them as input and get the target model's distributions for all γ positions at once—the same regime as prefill, with the weights read once instead of γ times. The draft model (smaller by a multiple) generates γ tokens fast; the target model decides on each of them in a single pass—accept or reject; at the first rejection the chain is truncated, and the target model emits the next token itself.

Two questions decide everything. **The first is correctness.** Acceptance works as rejection sampling (Leviathan et al., 2023): a draft token `x` is accepted with probability `min(1, p(x)/q(x))`, where `q` is the draft's distribution and `p` the target model's; on a rejection the token is resampled from the corrected residual distribution `norm(max(0, p − q))`. The scheme's provable property: the resulting sequence is distributed *exactly* as it would be under ordinary decoding by the target model—the speedup is mathematically free in quality terms, and the only loss is the time spent on unsuccessful drafts.

The second is the size of the speedup. Let α be the average probability of accepting a single draft token (a measure of how well the draft-target pair agrees on the domain at hand). The expected number of tokens per cycle of "γ drafts + one verification":

$$E = (1 - \alpha^{\gamma+1}) / (1 - \alpha)$$

Reading the formula: this is the sum of a geometric series $1 + \alpha + \alpha^2 + \ldots + \alpha^{\gamma}$—the probability that the first i tokens are accepted in a row is α^i, and each such outcome adds a token; the $+1$ in the exponent is the token the target model emits itself even under a total rejection (which is why $E \geq 1$ always: in number of steps you cannot end up worse than ordinary decoding). At $\alpha = 0.8$ and $\gamma = 4$: $E = (1 - 0.8^5)/(0.2) \approx 3.4$ tokens per cycle—roughly a third as many passes of the target model; the real speedup is lower (the draft is not free), typically 2–3×. The same view shows when the technique *does not* work: under weak agreement ($\alpha \approx 0.5$ on a niche domain) $E \approx 1.9$ and the gain is eaten by overhead, while raising γ at a low α adds almost nothing (the series converges). The draft model is chosen or fine-tuned for the target domain, and γ is kept small (2–5).

- **Prefix caching in serving** (SGLang RadixAttention)—the KV state of a common prefix (the system prompt, shared context) is reused across requests instead of recomputed. This is the server-side counterpart of prompt caching (§20.2), resting on non-contiguous block storage of the KV (§22.3).

22.8 Throughput vs. latency and scaling

Serving is tuned to a target load profile, and the key trade-off is *throughput against latency*. A larger batch raises throughput (the weight load is amortized) but increases an individual request's latency; speculative decoding, conversely, cuts latency at the cost of throughput. For models that do not fit on a single GPU there is *tensor parallelism* (a layer's weights are cut across GPUs) and *pipeline parallelism* (layers on different GPUs); for MoE models *expert parallelism* joins them: the experts are laid out across devices, and tokens are routed to their own experts through an all-to-all exchange. For MoE this is not an exotic mode but the baseline one—most of the top open weights of 2025–2026 (DeepSeek-V3/R1, Qwen3-MoE, Llama 4) are exactly that: memory for the *total* parameters is large while active compute is small (§1.5), so the sharding profile and the bottleneck (interconnect bandwidth on the all-to-all) differ from the dense case.

The orders of magnitude are illustrative: on an H100 the combination of continuous batching, PagedAttention, and chunked prefill gives Llama-3.3-70B in FP8 on the order of a few thousand tokens per second under high concurrency—a multiple above a naive PyTorch loop. The figures depend on the load, though: when answer lengths vary widely, continuous batching contributes more and chunked prefill less, so the configuration is checked against the *actual* distribution of traffic rather than against a synthetic one.

22.9 Chapter summary

Deploying an open model is justified by control, privacy, cost at scale, freedom from lock-in, and customization—at the price of infrastructure and complexity (§22.1). Serving is run by specialized servers (vLLM, SGLang, TensorRT-LLM) that beat a naive loop by a multiple (§22.2) on two foundations: PagedAttention, which manages the KV-cache through non-contiguous blocks on the principle of virtual memory (§22.3), and continuous batching, which keeps the GPU loaded across heterogeneous lengths (§22.4). The two phases of inference—compute-bound prefill and memory-bound decode—are separated by chunked prefill and by disaggregation, removing the interference (§22.5). Quantization for inference (AWQ/GPTQ/FP8/INT8, and the KV-cache in FP8) compresses the deployed

weights at the price of accuracy (§22.6); speculative decoding and prefix caching cut latency and repeated work (§22.7). Tuning is a trade-off between throughput and latency, with tensor, pipeline, and—for MoE—expert parallelism for large models (§22.8). Next is Chapter 23: pre-training, for when even fine-tuning is not enough.

Sources. W. Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention" (vLLM, SOSP 2023); G.-I. Yu et al., "Orca" (2022)—continuous batching; A. Agrawal et al., "Sarathi" (2023)—chunked prefill; P. Zhong et al., "DistServe" (2024) and P. Patel et al., "Splitwise" (2024)—prefill/decode disaggregation; L. Zheng et al., "SGLang / RadixAttention" (2024); Y. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding" (2023)—rejection sampling and the speedup formula (§22.7); the vLLM documentation (AWQ/GPTQ/FP8/INT8 quantization, tensor parallelism, H100 benchmarks, 2025); §1.6 (KV-cache, speculative decoding) and Chapter 2 of this book.

In the Margins: Chapter 22

> ↩ **GPTQ** (Frantar et al., 2022)—layer-by-layer quantization that minimizes reconstruction error through a Hessian approximation; **AWQ**—Activation-aware Weight Quantization (Lin et al., 2023)—protects from coarsening the small fraction of weights that the activation statistics mark as critical.

Chapter 23. Pre-Training

Chapter 21 adapted the model, Chapter 22 deployed it; this chapter is the top of Part VII's ladder: training a model *from scratch*. For the overwhelming majority of organizations this is the wrong choice, and the chapter explains what pre-training entails and what laws govern it—so that an architect can reason about the decision on solid ground, even while making it extremely rarely.

23.1 What pre-training is and when it is justified

Pre-training trains a model from scratch on an enormous corpus in a self-supervised regime: the model predicts the next token (§1.2), and out of that single task, knowledge and capabilities emerge. It is the top rung of the ladder "prompting → RAG → fine-tuning → pre-training"—maximum control at the price of maximum cost.

Pre-training is justified only in narrow cases: a fundamentally new architecture or modality that existing models do not cover; a language or a domain genuinely unrepresented in open models; sovereignty or regulatory requirements that rule out any external weights. For applied tasks the earlier rungs are almost always enough: fine-tuning (Chapter 21) changes behavior more cheaply, and RAG (Part III) adds knowledge without training at all. What makes pre-training almost never justified for a typical organization: cost (orders of magnitude above fine-tuning), the volume and quality of the data required (§23.6), and specialized expertise. The honest conclusion: absent one of the narrow reasons above, do not pre-train.

23.2 The pre-training pipeline

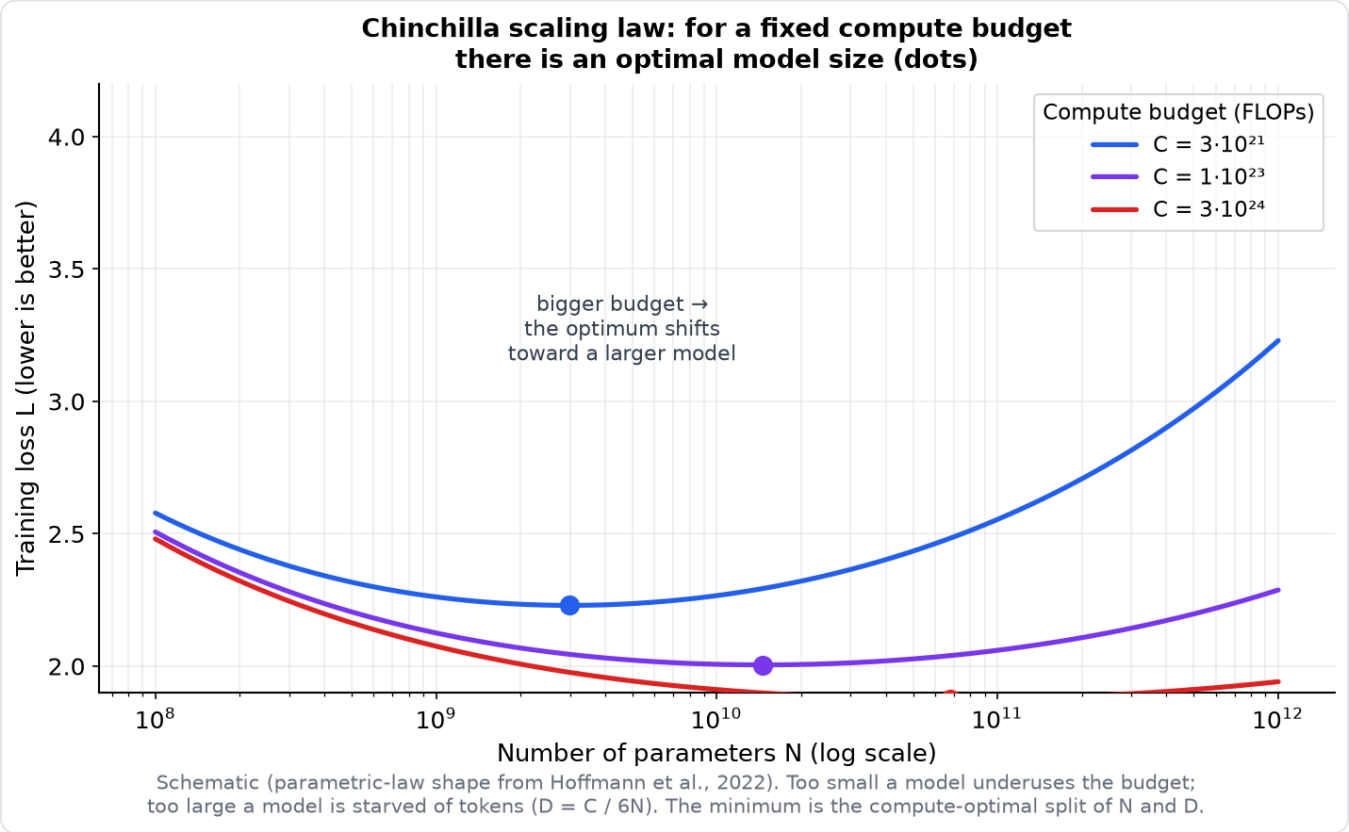
Pre-training is a pipeline of several stages:

- **Data collection**—a corpus on the scale of trillions of tokens from the web, books, code, and specialized sources.
- **Cleaning and deduplication**—filtering out junk, removing duplicates and near-duplicates, screening out harmful and low-quality material; the quality and cleanliness of the data drive the model's quality more strongly than its volume does (§23.6).
- **Tokenization** (§1.3)—training the tokenizer (BPE and its relatives) and encoding the corpus into token sequences.
- **Training**—optimizing next-token prediction over the corpus; this is where the scaling laws (§23.3) and the compute budget (§23.4) apply.
- **Post-training**—SFT and preference alignment (Chapter 21), which turn the "raw" pre-trained model into one that follows instructions; pre-training supplies only the base.

23.3 Deep Dive: scaling laws

How to split a fixed compute budget between *model size* and *token count* is the central question of pre-training, and the scaling laws are what answer it.

The early laws (Kaplan et al., 2020) described quality as a power law in scale, but leaned toward putting most of the budget into model *size*. **Chinchilla** (Hoffmann et al., 2022) revised them: across more than 400 trained models (from 70M to 16B parameters at controlled budgets) it showed that for *compute-optimal* training the model size N and the token count D must grow *in equal measure*—each as ~C^0.5 (doubling the budget C multiplies each by √2; doubling both takes four times the budget, since C = 6ND). Hence the rule of thumb of **≈20 tokens per parameter**. The Chinchilla model (70B, trained on 1.4T tokens) beat Gopher (280B) at the same budget with four times fewer parameters and nearly five times more tokens—direct proof that the preceding models were *undertrained* (GPT-3 at ~1.7 tokens per parameter).



The optimum is located by three complementary methods: varying the token count at a fixed size; varying the size at a fixed budget (IsoFLOP); and fitting a parametric loss surface L(N, D), then deriving the optimal split from it. As the chart shows, at a fixed budget too small a model underuses it, too large a model comes up short on tokens, and the minimum marks the compute-optimal split.

23.4 Deep Dive: the compute budget C ≈ 6ND

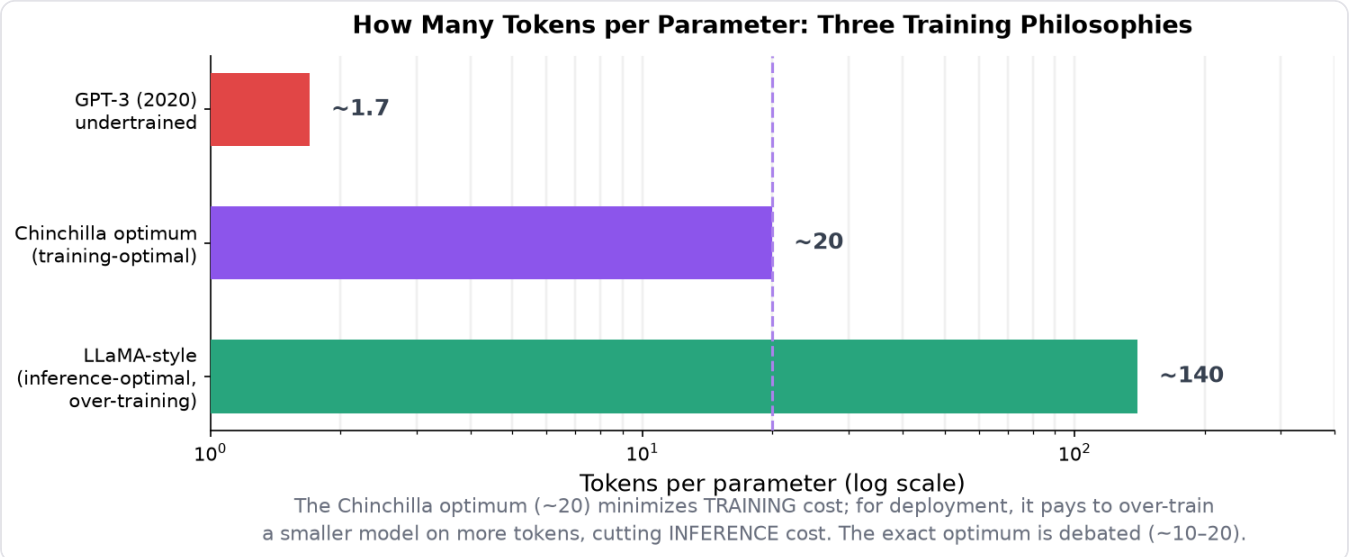
The compute cost of pre-training is estimated by a simple and practically important relation: **C ≈ 6ND**, where C is compute in FLOPs, N is the number of parameters, and D is the number of (training) tokens. Let's take the factor of 6 apart term by term—it is not empirical but follows from how the forward pass is built (§1.4, Appendix A).

Where the 2 FLOPs come from. The forward pass is matrix multiplication, and in it every parameter (an element of a weight matrix) takes part in processing every token exactly once: one multiply (weight × input number) and one add (add the product to a running sum). Multiply + add = 2 operations per "parameter × token" pair, giving **2ND** for the whole forward pass over all the data.

Where the ×3 comes from. The backward pass costs roughly *two* forward passes, because the gradient is computed twice: with respect to the *activations* (to push the error deeper into the network—without which the earlier layers cannot be trained) and with respect to the *weights* themselves (to update them). Each of the two operations is the same matrix multiplication at the same sizes as the forward. In total: 2ND × (1 forward + 2 forward-equivalents for the backward) = **6ND**. The relation makes the budget tangible: training a model in the tens of billions of parameters on trillions of tokens takes on the order of 10^{23}–10^{24} FLOPs—from hundreds of thousands to millions of GPU-hours on an expensive cluster (for scale, per Meta's model cards: Llama 2 70B—about 1.7M A100 GPU-hours; Llama 3 405B—about 31M H100 GPU-hours; in money at market rental prices, single-digit to tens of millions of dollars per run, and for the largest models at list prices, noticeably more), which is exactly why pre-training is out of reach as a routine operation. C ≈ 6ND plus the 20-tokens-per-parameter rule also yields the data estimate: a 70B-parameter model needs about 1.4T tokens, a 175B model about 3.5T (on the order of several terabytes of text).

23.5 Deep Dive: compute-optimal vs. inference-optimal

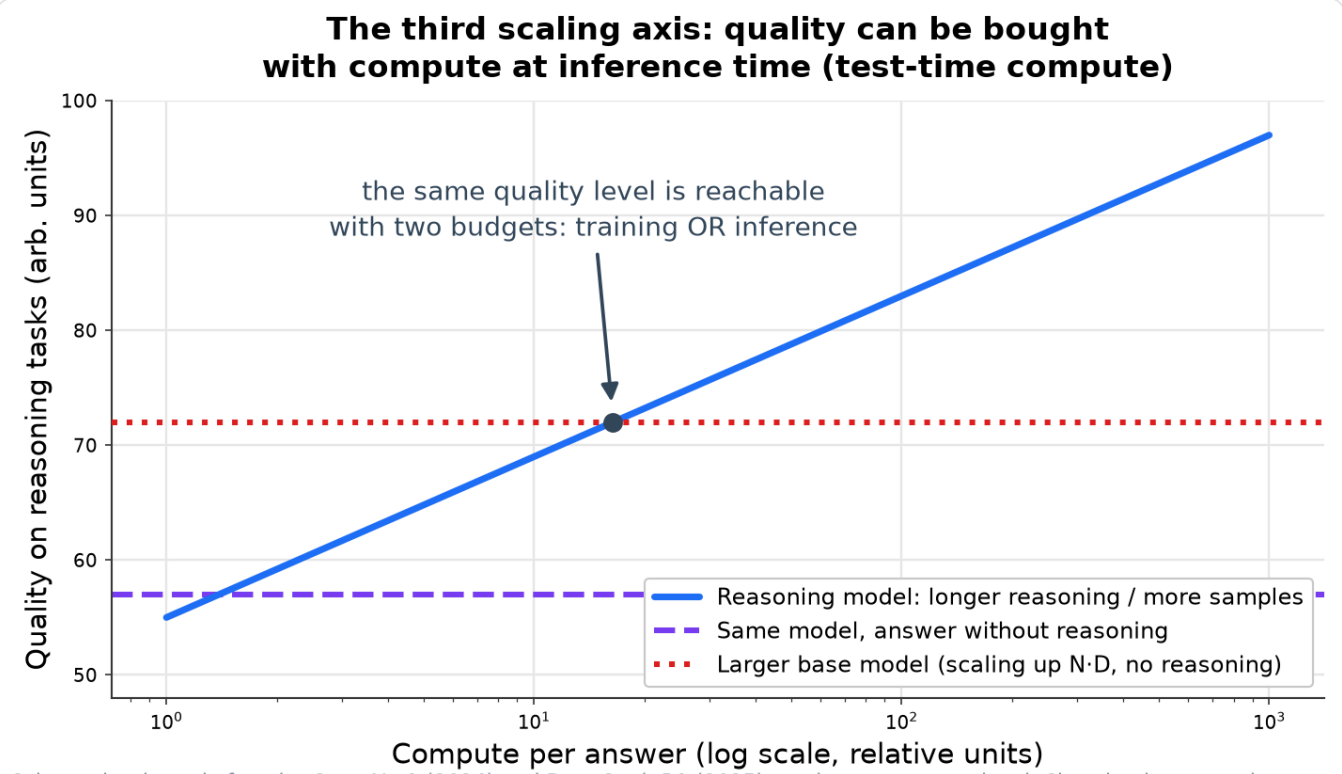
A substantial and frequently missed subtlety: the Chinchilla optimum minimizes the cost of *training*, not of *operating*. A model is trained once and inferenced many times over; so for a model destined for wide deployment it pays to train a *smaller* model on *more* tokens than the 20:1 rule prescribes—over-training° raises the training cost but lowers the inference cost across the entire service life.



This is exactly how models meant for mass deployment are trained (the LLaMA family, for one, was trained well past the Chinchilla optimum). Hence three philosophies: undertraining (GPT-3, ~1.7 tokens per parameter), training-optimal (Chinchilla, ~20), and inference-optimal over-training (~100 and up). The caveats matter: the exact value of the optimal ratio is open to debate (a number of later works find ~10–12), and the Chinchilla law itself was derived for a decoder-only transformer with next-token prediction—for mixture-of-experts, encoder-decoder, or other objectives, the 6ND estimate and the exponents may not carry over directly.

Deep Dive: the third axis—test-time compute

The classical scaling laws split the *training* budget between N and D. With the arrival of reasoning models (o1, 2024; DeepSeek-R1, 2025), scaling gained a third axis—**test-time compute**: on reasoning tasks quality grows with the amount of compute spent on the answer itself (the length of the reasoning chain, the number of samples), and that growth is bought without retraining (§13.8). For allocating the budget this changes the arithmetic: part of the quality that once came only from raising N or D is now reachable through post-training (RLVR) plus inference spend—and conversely, for cheap inference at scale a large base model without long reasoning may be the better buy. The full calculation becomes three-dimensional—pre-training × post-training × inference budget; no settled Chinchilla analogue exists for that space yet, and the practical conclusion for an architect is a single one: count the cost per *task* (reasoning tokens included, §2.2), not per model.



Schematic, shaped after the OpenAI o1 (2024) and DeepSeek-R1 (2025) results; axes are notional. Choosing between the axes is a matter of deployment economics: high-volume cheap inference versus rare, expensive answers (§13.8, §23.5).

23.6 Data and infrastructure

Two practical constraints decide whether pre-training is feasible.

- **Data.** Trillions of tokens are required, but their *quality* and *cleanliness* matter more than their volume: deduplication, filtering out low-quality and harmful content, and the selection of sources affect the result more strongly than the raw size of the corpus does (§23.2). At the upper scales the worry of a “data wall” arises—

running out of available quality text (the quantitative frame is data-constrained scaling laws, Muennighoff et al., 2023: repeated epochs over quality data do work, but with sharply diminishing returns).

- **Infrastructure.** The training run fits on no single node and demands distributed training on a cluster of thousands of accelerators: *data parallelism* (replicas of the model on different data), *tensor parallelism* (a layer's weights sliced across devices), and *pipeline parallelism* (layers spread across devices)—the same forms of parallelism as in serving (§22.8), but at training scale. The tool names to search under: FSDP (PyTorch), DeepSpeed ZeRO (stages 1–3 of sharding the optimizer state, the gradients, and the weights), Megatron-LM (tensor/pipeline), and their combination, "3D parallelism." Hence the cost and the expertise that put pre-training beyond the reach of most.

23.7 Summary of the chapter and of Part VII

Pre-training trains a model from scratch by next-token prediction and stands at the top of the ladder; for a typical organization it is almost never justified—RAG and fine-tuning are enough (§23.1). The pipeline covers data collection, cleaning and deduplication, tokenization, training, and post-training (§23.2). The scaling laws set how the budget splits between size and data: Chinchilla established equal scaling of N and D and the ~20-tokens-per-parameter rule, showing that earlier models were undertrained (§23.3); the cost is estimated as $C \approx 6ND$ (§23.4). But the Chinchilla optimum minimizes training, not inference, which is why widely deployed models are over-trained—smaller ones on more tokens (§23.5). Feasibility comes down to data quality and the infrastructure for distributed training (§23.6).

Part VII in sum. A model of your own is a spectrum of rising control and rising cost: adapt a ready-made one (Chapter 21), deploy an open one (Chapter 22), or train one from scratch (Chapter 23). The decisive principle of the whole part: do not climb this ladder before the rung below it has demonstrably failed, because each rung costs several times the one before. Next comes **Part VIII: synthesis**, where all the parts come together into a reference architecture, an organizational frame for adoption, and a worked end-to-end example with a package of architectural decisions.

Sources. J. Kaplan et al., "Scaling Laws for Neural Language Models" (2020); J. Hoffmann et al., "Training Compute-Optimal Large Language Models" (Chinchilla, NeurIPS 2022)—the ~20-tokens-per-parameter rule, $C \approx 6ND$, equal scaling of N and D; H. Touvron et al., "LLaMA" (2023)—inference-optimal over-training; the discussion of the exact ratio and of reconciling Kaplan/Chinchilla (2024); material on curating pre-training data and on distributed training; §1.2–1.3 and Chapter 22 of the book.

In the Margins: Chapter 23

> ⮌ ≠ A false friend: **over-training** here means a deliberate inference-optimal regime (a smaller model on more tokens than the compute optimum prescribes), not classic overfitting—degradation from memorizing the training set. It improves operating economics without "spoiling" anything.

Part VIII. Synthesis

Parts I–VII built the components; Part VIII assembles them into a whole. Chapter 24 draws all the material together into a single reference architecture for a production AI-agent system, Chapter 25 adds the organizational scaffolding of adoption—ownership, privacy, regulation, economics, and process—and Chapter 26 works an end-to-end example from requirements to a package of architectural decisions. Part VIII shifts the question from "how each part is built" to "how it all comes together into one working, observable, secure, and economical system."

Chapter 24. Reference Architecture

This chapter is a map of the whole book: how the components of Parts I–VII fit together into one reference architecture. Not a new framework but a systematization—which layers make up a production AI-agent system, how they interact, where the boundaries run, and which invariants hold across the entire system. Industry practice has converged on a similar layered split, and the architecture laid out here agrees with it.

24.1 The layers of the reference architecture

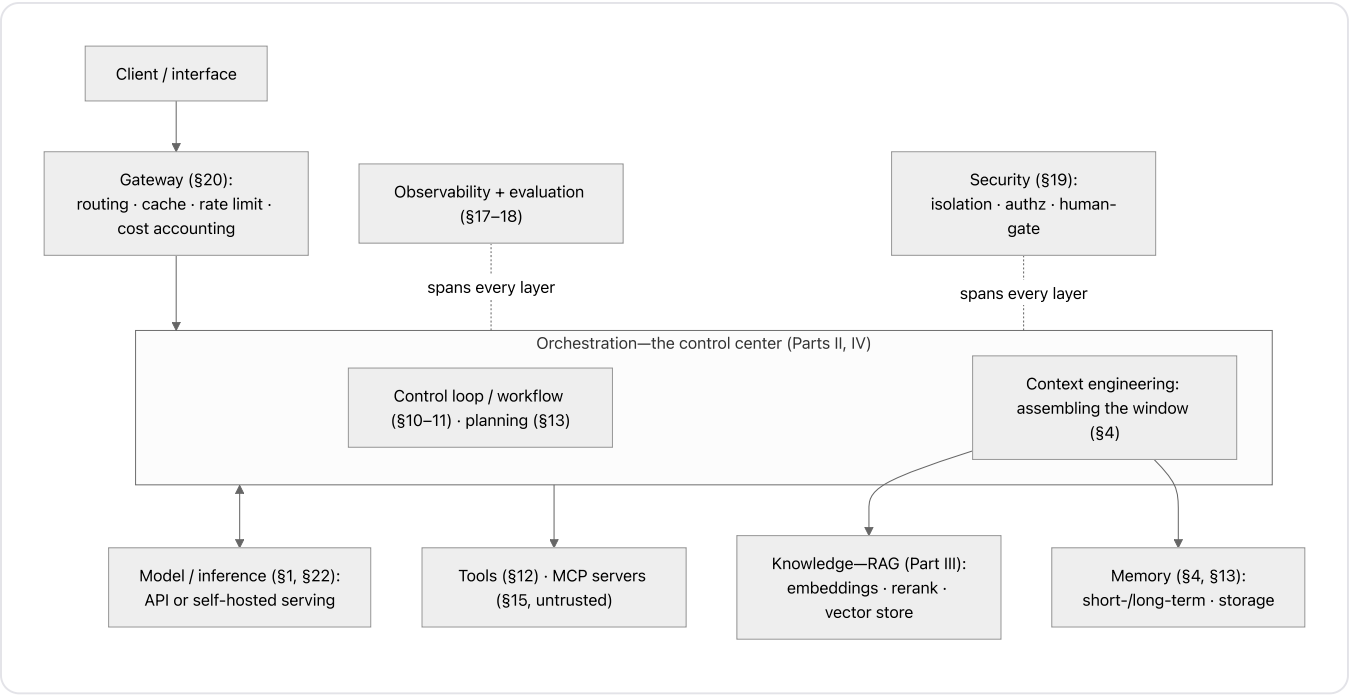
A production AI-agent system decomposes into layers with separate responsibilities:

- **Interface and gateway**—the entry point; the gateway centralizes model routing, caching, rate limiting, and cost accounting (§20.6).
- **Orchestration**—the control center: the control loop or workflow (Chapters 10–11), planning and reasoning (Chapter 13), context assembly (Chapter 4). This is where decisions get made, where the other layers are called, and where the discipline of reliability is held (Chapter 14).
- **Model and inference**—the LLM itself, called through an API or served yourself (Chapters 1, 22).
- **Knowledge (RAG)**—grounding in data: embeddings, retrieval, reranking, the vector store (Part III).
- **Memory**—state across steps and sessions: short- and long-term, with persistent storage (§4, §13).
- **Tools and integration**—acting on the outside world: tool calling (Chapter 12) and MCP servers (Chapter 15).

Above these layers sit three **cross-cutting concerns**—not layers themselves, but running through all of them: observability and evaluation (Chapters 17–18), security (Chapter 19), and cost and latency (Chapter 20). The organizing principle, borne out by industry practice: **orchestration controls everything, and the model is a constrained component**. In particular, the model never reaches the vector store or the tools directly—retrieval and calls are mediated by orchestration, which turns them from the model's improvisation into a predictable, controlled mechanism (the same point underpins security, §19.6, and the MCP separation, §15.2).

24.2 The full component diagram

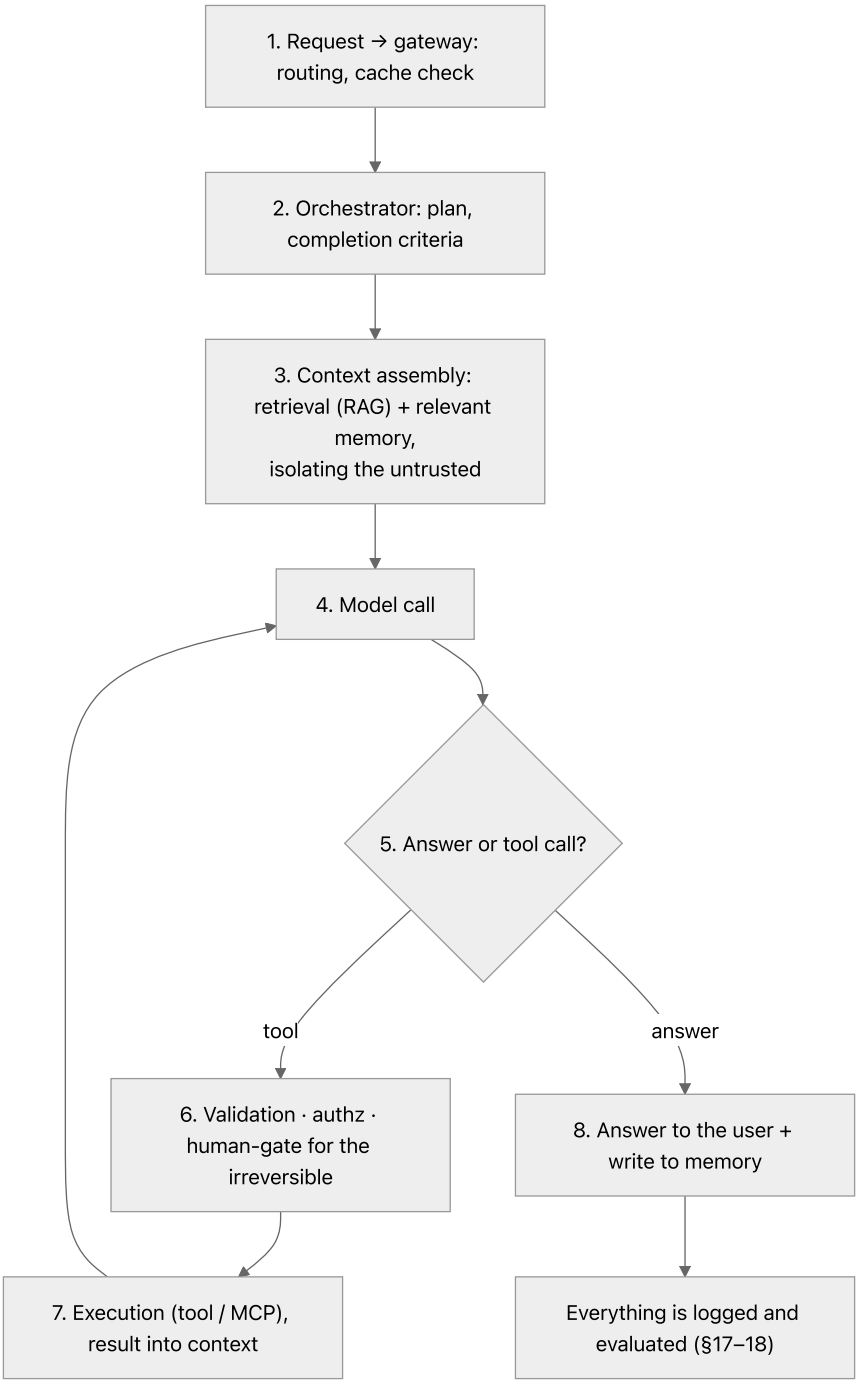
The layers and how they connect:



Read the diagram downward from the entry point: a request passes the gateway and lands in orchestration as the control center, which calls the model, assembles context out of knowledge and memory, and invokes tools; observability and security span every layer. What matters is the topology: orchestration at the center, the other layers subordinate to it, and the stores (vector and memory) and external tools (MCP) sitting beyond a boundary, out of the model's direct reach.

24.3 Deep Dive: the flow of a request through the architecture

One request, from entry to answer, sets the layers in motion:



- The request passes the gateway: routing to a suitable model (§20.3) and a cache check (§20.2).
- The orchestrator settles the plan and the completion criteria (§10.5, §13.4).
- Context gets assembled: retrieving the relevant documents (Part III) and the relevant memory (§4.5), with untrusted content isolated (§4.6, §19.6).
- The model is called and decides: answer, or call a tool (§12.2).
- On a tool call—argument validation (§12.4), authorization under the user's own rights (§12.5) and, for irreversible actions, human confirmation (§10.6); then execution (tool or MCP, §15) and the result returns into the context—the loop repeats.
- When it is ready—the answer goes to the user and gets written to memory.
- The whole path is captured in traces and evaluated (Chapters 17–18), closing the improvement loop (§17.8).

24.4 Deep Dive: boundaries and invariants

The architecture's reliability and security rest on a handful of invariants that hold in every layer:

- **The model has no direct access to the stores or the tools.** Retrieval and calls are mediated by orchestration; this keeps behavior predictable and denies an injection any direct path to data and actions (§19.6, §24.1).
- **Untrusted content is isolated.** External data (documents, tool results, MCP) is fed in a clearly separated region of the context and never mixed with instructions (§4.6, §19.1); for high-risk systems—a quarantined loop (dual-LLM, §19.6).
- **The irreversible goes through a human.** Actions with consequences require confirmation; compensation is only for the reversible (§10.6).
- **The trust boundary sits at orchestration.** Orchestration is trusted; the model is a manipulable component; servers and external sources are untrusted (§15.2). Authorization and isolation are enforced by deterministic components, not by trusting the model.
- **Every step is bounded by a budget.** Limits on iterations and on tokens per step and per task (§10.5) hold cost down and prevent unbounded consumption (§19.3, §20.6).

These invariants are not properties of any one layer but rules that cut across all of them; break one anywhere and the whole system is undermined.

Deep Dive: physical topology and scaling

The logical diagram of §24.2 unfolds into a physical one by the usual rules of distributed systems—with one decisive choice: **orchestration is designed stateless**, and all session state (history, plan, memory) lives in an external store (externalized state, §4.4). Orchestration replicas then scale horizontally behind a load balancer without sticky sessions, a pod restart does not lose the conversation, and a pause at a human-gate (§10.6)—state that lives for hours—survives a deploy, because it sits in a database rather than in process memory. Long-running agent tasks are shaped asynchronously: a task queue plus workers, not an HTTP request holding a connection open for minutes. Multi-agent (Chapter 14) does not change the external topology: supervisor and worker agents are logical roles inside the orchestration layer, executed by the same replicas. The GPU pool for self-hosted inference (Chapter 22) scales on a track of its own—it has its own scaling units (model replicas) and its own utilization economics (§2.6).

24.5 Cross-cutting concerns

Three concerns run through every layer and are designed in from the start rather than bolted on after the fact.

- **Observability and evaluation** (Chapters 17–18) span every layer: traces record a request's path through all the components, metrics aggregate cost and latency, and evaluation judges quality on those same traces. Without this you can neither debug the system nor improve it.
- **Security** (Chapter 19) is built into the architecture from day one: isolation, least privilege, human-gate, and quarantine are architectural decisions, not after-the-fact filters.
- **Cost and latency** (Chapter 20) are managed at the level of the gateway and orchestration: routing, caching, budgets, and attribution.

What they share: none of them localizes to a single layer, which is why they are treated as cross-cutting dimensions of the whole architecture.

24.6 Decisions by layer: a composite map

Each layer demands explicit architectural decisions; the composite map ties them to the chapters:

Layer / cross-cutting concern	Key decision	Where in the book
Gateway	Routing, cache, budgets	§20
Orchestration	Workflow versus agent; level of autonomy	§10.7, §11.8
Context	Strategy for assembling and isolating the window	§4
Model	Model tier; API versus self-host	§2, Chapter 22
Knowledge	Retrieval pattern; index; reranking	Part III
Memory	Short-/long-term; forgetting strategy	§4.5, §13.1
Tools	The set and the contracts; MCP	§12.3, Chapter 15
Implementation	Framework (Spring AI / LangChain4j)	Chapter 16
Evaluation	Offline gate + online monitoring	Chapter 17
Observability	Traces, metrics, attribution	Chapter 18
Security	Isolation, privileges, human-gate	Chapter 19
Your own model	Fine-tuning / serving / pre-training	Part VII

The map does not prescribe specific values; it lists the *decisions* an architect makes deliberately. What gets chosen in each row follows from the requirements (Chapter 2, Chapters 25–26).

24.7 Chapter summary

A production AI-agent system decomposes into layers: gateway, orchestration (the control center), model, knowledge (RAG), memory, tools—with observability/evaluation, security, and cost/latency as cross-cutting concerns (§24.1). The full diagram (§24.2) puts orchestration at the center and the stores and external tools beyond a boundary the model cannot cross directly; the request flow (§24.3) sets the layers in motion from gateway to answer, with an evaluation loop closing it. Invariants hold the architecture together: the model never touches the stores or the tools directly, the untrusted is isolated, the irreversible goes through a human, the trust boundary sits at orchestration, and every step is bounded by a budget (§24.4). The cross-cutting concerns are designed in from day one (§24.5), and the composite map ties each layer's decisions back to the material of the book (§24.6). Next, Chapter 25: the organizational scaffolding for adopting this architecture; the end-to-end example that drives it from concrete requirements through to a package of decisions is Chapter 26.

Sources. Industry reference architectures for agentic systems and the "agentic stack" (2025–2026)—the layered split, the principle that "orchestration controls retrieval, the model has no direct access to the store," observability as a cross-cutting concern, security from day one; C. Richardson, *Microservices Patterns* (2018)—the principles of layers and boundaries; this book (Parts I–VII) as the source of the layers' content.

Chapter 25. Adoption in the Organization

Chapter 24 assembled the technical architecture. But the platform the book's subtitle promises—*automating the organization*—runs up against more than technology: who owns the prompts, who pays for the tokens, who answers to the regulator, and why employees will start using it (or refuse to). This chapter is the platform's organizational scaffolding: the lifecycle of AI artifacts, the point where policy gets applied, the containment loop around personal data, regulatory readiness, the economic case, and the adoption process. It is an overview with bridges to the technical chapters, not legal or management advice.

25.1 Why architecture is not enough

A probabilistic component breaks the organizational contracts everyone is used to. Quality is a distribution, not a boolean property: "works / doesn't work" gives way to the share of acceptable answers against a threshold (Chapter 17), and that reshapes the SLA, the definition of an incident ("systematically bad answers" is an incident with no stack trace), and the postmortem process. Artifacts appear that did not exist before: prompts, schemas, golden sets, indexes, judge versions—with no lifecycle deliberately built for them, they live in chats and notebooks and degrade in silence. Finally, an agent's actions (Chapter 10) raise a question of accountability that deterministic code settles through testing and that here demands autonomy levels and auditing (§10.6–10.7). Three questions adoption has to answer beyond the architecture: **who owns, who pays, who answers**.

25.2 Ownership and the lifecycle of AI artifacts

A prompt, an output schema, a golden set, and a retrieval configuration are versioned artifacts with a release cycle (§3.6): review, an offline gate against the golden set (§17.8), a canary rollout, a rollback. Organizationally this takes the shape of two registries.

Pattern: Prompt Registry

Context. Everyone edits the prompts a little: product edits the wording, engineering the format, security the constraints. A change in one place quietly changes behavior in another; nobody can answer "which prompt is in production right now, and why." **Problem.** How do you give prompts the lifecycle of code—versions, review, rollback, attribution of regressions—without turning every edit into an application deploy? **Forces.** A prompt needs the iteration speed of text and the discipline of code at the same time; an edit is cheap to make and expensive to detect—a regression surfaces in the quality of the answers, not in the compiler. **Solution.** A central registry: the prompt is stored and versioned separately from the code, each one has an owner, an attached golden set, and a change log; the application fetches a prompt by version or alias. **Mechanics.** A prompt change is a pull request with an eval run as the CI gate (§17.8); rollout is a canary on a fraction of traffic with online metrics (§17.6); every telemetry record is tagged with the prompt version (§18.5), so a regression is attributed to a specific change; rollback is a version switch, with no rebuild. **Resulting Context.** Reproducibility and controlled evolution; prompt engineering stops being folklore. The price is infrastructure and discipline, plus the risk of bureaucratizing small edits (mitigated by automating the gate). **When to Use.** More than one prompt in production, or more than one person changing them. **When Not To.** A single team's prototype—prompts in git next to the code are enough (§3.6). **Related Patterns.** Prompts as code (§3.6); the eval gate (§17.8); observability with version attributes (§18.5). **Sources.** LLMOps practice for versioning and rolling out prompts (2024–2026); §3.6 and §17.8 of this book.

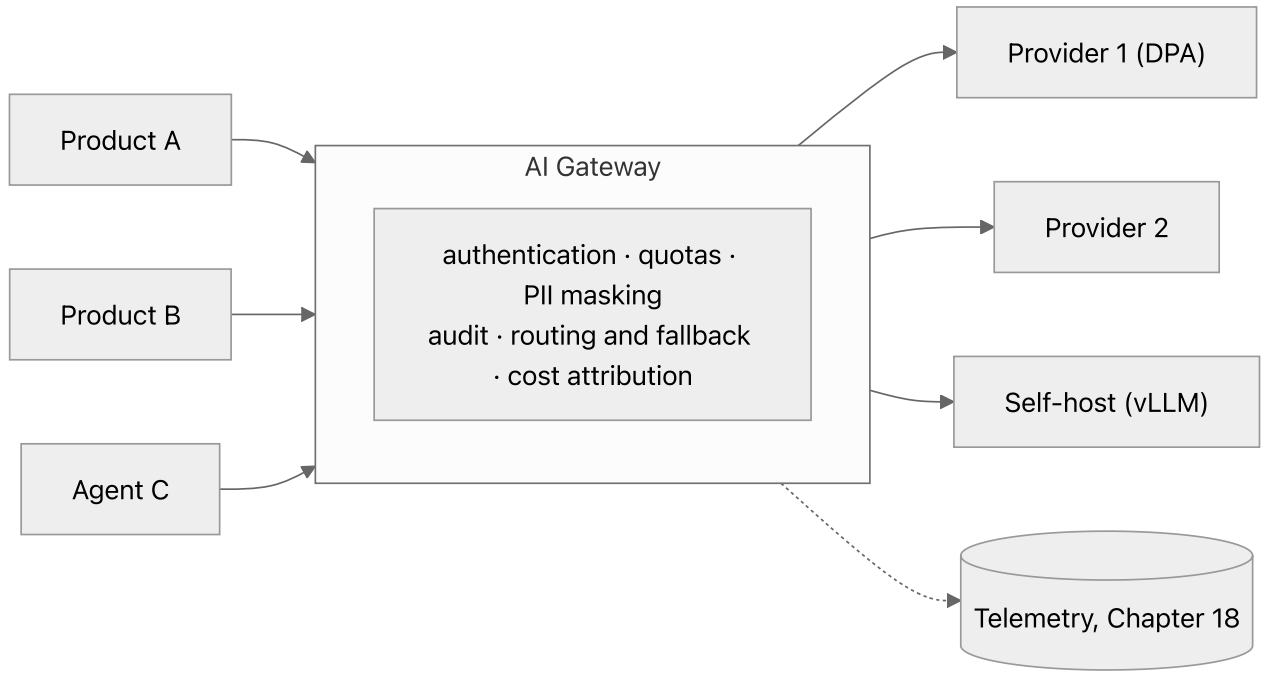
The second registry is the **organization's registry of AI systems**: which systems exist, which models and providers they run on, what risk class they carry, and who owns them (for agents, including their machine accounts —NHI, §15.7). It looks like bureaucracy right up until the regulator's first question, the first security incident, or the provider's first price change—and it is also the entry point for risk classification (§25.5).

Shadow AI. The registry describes what the organization *knows* about itself. **Shadow AI** is unsanctioned LLM use outside the registry: a personal API key in a script, a browser extension with access to work email, "gray" automation on a no-code platform, an employee pasting customer data into a public chatbot. It is a direct descendant of shadow IT, sharpened by two properties of LLMs: the barrier to entry is negligible (a key and five lines of code), and the leak channel is built into the function itself—the data goes to an external provider *by definition*. The detection vectors are engineering ones: analyzing outbound traffic to provider domains, the financial trail (API subscriptions in expense reports), a periodic amnesty ("legalize your script and you get budget and support"). But the main lever is not prohibition—it is economics: the sanctioned path through the gateway (§25.3), with self-service and fast onboarding, has to be *cheaper and more convenient* than the unsanctioned one, or the shadow path reproduces itself forever.

25.3 Pattern: AI Gateway as the governance point

Pattern: AI Gateway

Context. Teams integrate with providers directly: keys spread across team secret stores, spend goes unattributed, privacy policy is applied at will, and switching providers means edits in every service. **Problem.** How do you apply organizational policy—access, budgets, privacy, audit, choice of models—to *every* LLM call without building it into every application? **Forces.** Centralization buys control, visibility, and portability, but it puts a component in the path of every call; direct integrations are fast for an individual team but smear policy, spend, and audit across the whole organization. **Solution.** A single egress point to the models: every call goes through a gateway that authenticates the caller, applies policy, and routes to the providers. **Mechanics.** Authentication and cost attribution by team, feature, and user (§20.6); quotas and budgets that degrade rather than cut off (§10.5, §20.6); PII masking before anything goes to the provider (§25.4); routing only to approved destinations (DPA°, residency—§2.6); an audit log of requests; centralized timeouts, retries, and fallback models (§2.5); swapping a model with no edits in the applications (§20.3). The gateway is the natural place to collect telemetry (Chapter 18).

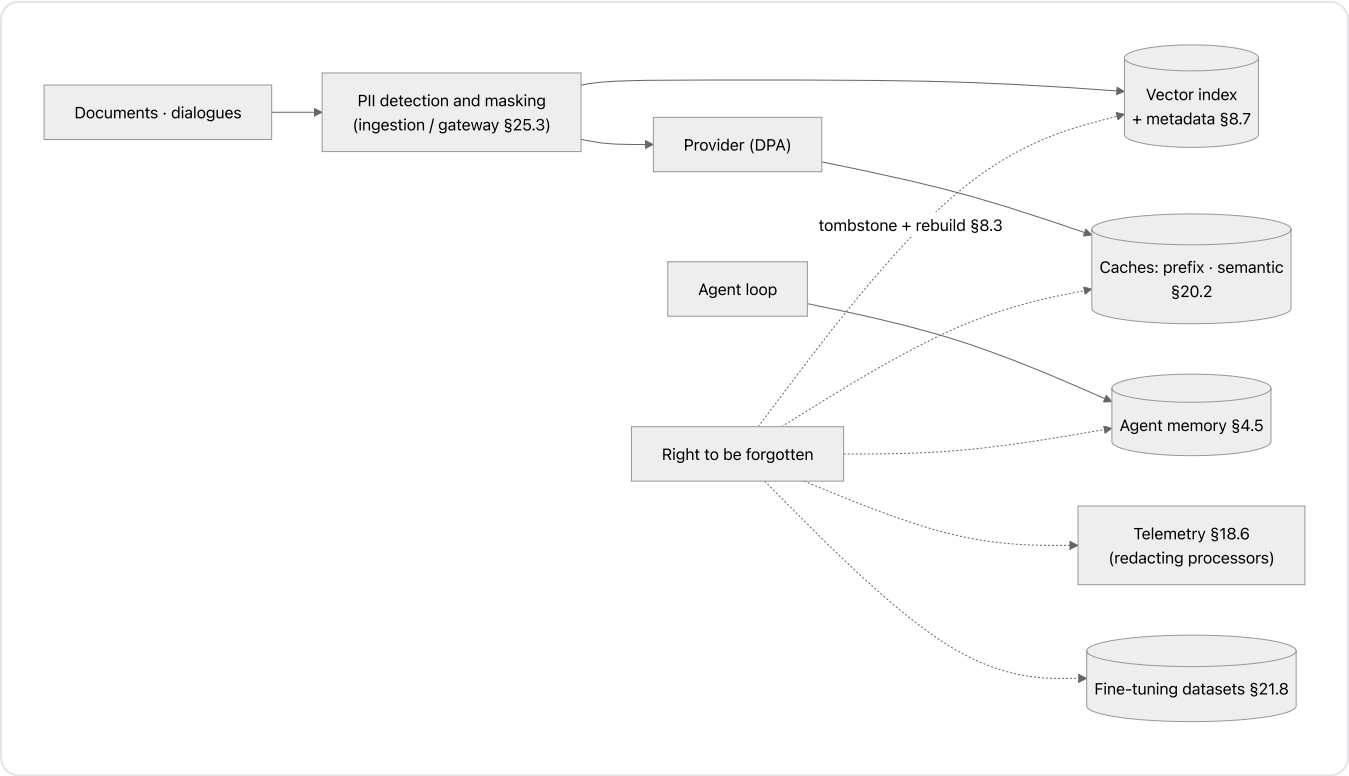


Resulting Context. Policy in one place; portability across vendors (§16.5, §20.6); a complete picture of spend. The price: a component in the path of every call—availability and latency requirements at the level of critical infrastructure; and the risk of turning the platform team into a bottleneck on everyone's process. **When to Use.** More than one team or system; any regulatory requirement at all. **When Not To.** One product from one team—a library carrying the same concerns is enough. **Related Patterns.** Model Routing / Fallback (§2.2, §2.5, §20.3); budgets and attribution (§20.6); the PII containment loop (§25.4); observability (Chapter 18). **Sources.** Chip Huyen, *AI Engineering* (2025)—the platform layer and gateways; LLM gateway practice (2024–2026); §20.6 of this book.

25.4 Data and privacy: the PII containment loop

Personal data settles in every layer of the system, and the loop is built around the places where it settles:

- **Ingestion into the index.** Detect and mask/pseudonymize PII *before* embedding and indexing; the clearance and the owner go into the chunk's metadata (§8.7). What never entered the index cannot leak through retrieval.
- **Telemetry.** Capturing prompts and answers runs through redacting processors (§18.6); debugging completeness against privacy is a managed trade-off.
- **The provider.** Routing through the gateway (§25.3) guarantees DPA-compatible destinations: what the provider stores, whether it trains on the data, where residency lands (§2.6, ADR-8 in §26.4).
- **The right to be forgotten.** The most inconvenient requirement of all: the subject's data has to be deleted from the vector index (in HNSW, a tombstone plus a periodic rebuild, §8.3), from the caches (including the semantic cache, §20.2), from the agent's memory (§4.5), and from the fine-tuning datasets (§21.8). The list of places where data settles follows directly from the systems registry (§25.2); without it, "delete everything" cannot be carried out.



How PII detection itself works. "Detection and masking" above is not one operation but a ladder of three techniques, in rising order of coverage and cost:

1. **Deterministic recognizers**—regular expressions, dictionaries, checksums (a card number validates against the Luhn algorithm, a Russian tax number against its own check digit). High precision on *formatted* data, zero on free text: "Ivan's wife from the third stairwell" will match no pattern at all.
2. **NER models** (named entity recognition, the classic NLP task)—a trained recognizer finds names, addresses, and organizations in free text. It needs language support: Russian needs Russian-language models, and an English-language default silently misses Cyrillic entities. The open reference point is **Microsoft Presidio**: it combines both classes of recognizer and extends with custom ones (internal identifiers, contract numbers).

3. **An LLM classifier**—the only one that understands *contextual* PII ("the client we talked about yesterday" is a disclosure too), and the most expensive. The recursive trap: the text is sent to an LLM in order to find the PII in it—so that call is itself obliged to go to a local or DPA-covered model, or the protective loop *is* the leak.

The failure modes are asymmetric, and that sets the calibration: a **false negative** means PII went out unmasked (a compliance incident with regulatory consequences); a **false positive** means too much got masked (quality degrades: "contact [PERSON] at [PHONE]" is useless as an answer). So the loop's target metric is recall (Chapter 9—the same logic as in retrieval) on paths leaving the perimeter, at the cost of precision; inside the perimeter the balance is softer. And keep the operations distinct: **masking** (an irreversible placeholder), **pseudonymization** (reversible through a mapping table—which itself becomes a critical secret), and format-preserving **tokenization** (the value is replaced, the format and the relationships survive—indexes and analytics keep working).

25.5 The regulatory landscape

An overview of the frameworks; check the details and the deadlines against the current state—regulation moves faster than books.

- **The EU AI Act** (Regulation (EU) 2024/1689)—a risk-based classification of systems: unacceptable risk (banned), high (hard obligations: risk management, data quality, logging, human oversight, documentation), limited (transparency), minimal. Obligations differ by role: **provider** and **deployer**; general-purpose models (GPAI) get a regime of their own. The dates of entry are staggered and have been revised: as of this writing (mid-2026), pushing the obligations for high-risk systems out to 2027–2028 has been approved by the European Parliament and the Council and awaits publication—check the timetable as of the date you are reading.
- **Personal-data regimes**—GDPR, Russia's 152-FZ, and their analogues: the legal basis for processing, cross-border transfer (calling a hosted API from another jurisdiction is a transfer, §2.6), the subject's rights (→ §25.4).
- **Sector-specific overlays**—finance, medicine, and the public sector add requirements of their own for explainability, localization, and audit.
- **Standards as a maturity frame**—NIST AI RMF (2023) and ISO/IEC 42001:2023 systematize the management of AI risk; useful as checklists even where they carry no legal force.

Liability for autonomous actions (not legal advice—a map of the questions to bring to a lawyer). The doctrine for agents is still taking shape, but two settled principles already apply: the principal's liability for the actions of the instrument it employs (in contract law, the actions of an automated system are attributed to the party deploying it—"the agent decided on its own" is not a defense), and precedents of the Moffatt v. Air Canada (2024) kind: the tribunal held the company to terms its chatbot had promised in error, rejecting the argument that the bot was a "separate entity." The engineering consequence: autonomy levels (§10.7), the human-gate (§10.6), and the audit trail (Chapter 18) are not only compliance but the evidentiary basis for due diligence in a dispute. Where the roles divide: *which* actions may be delegated to an agent in a given jurisdiction and industry is the lawyer's call; the architect's job is to make the system capable of technically enforcing those boundaries and demonstrably observing them.

Practical consequences for the architect: (1) a systems registry with a risk class recorded on intake (§25.2)—without it you cannot even determine which obligations apply; (2) human oversight is already built into the architecture—the human-gate (§10.6) and autonomy levels (§10.7); (3) tracing (Chapter 18) serves two purposes—debugging and evidence for the regulator; (4) a map of the places where data settles (§25.4).

25.6 The economic case: TCO and ROI

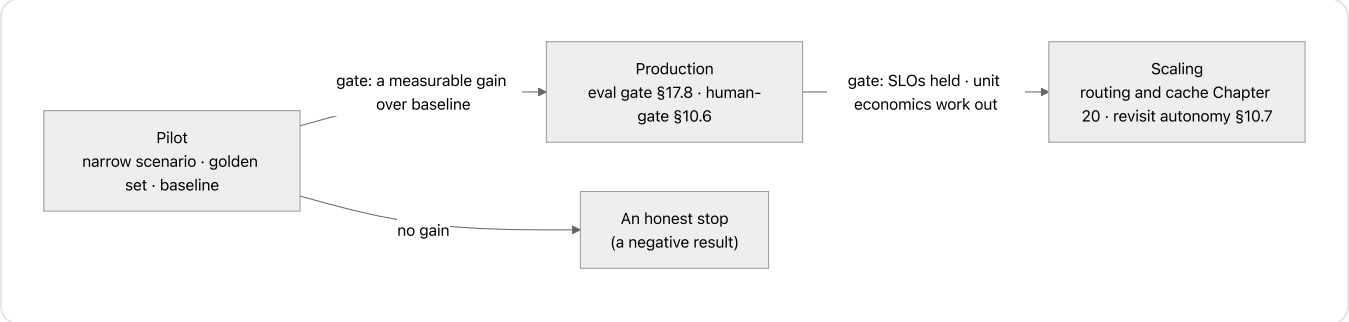
The cost of ownership is wider than the token bill: **TCO = inference** (Chapters 2, 20) **+ infrastructure** (retrieval Chapter 8, the eval system §17.8, observability Chapter 18) **+ people** (the platform team, HITL reviewers §10.6, maintaining the golden sets) **+ risk** (incidents, compliance). Of these terms, tokens are often not the largest—people cost more.

The ROI frame: value = time saved per task × rate × volume × **the share of people actually using it (adoption)**—or a direct effect on conversion and revenue; payback = value – TCO. Adoption is the main unknown multiplier, which is why you do not forecast it—you establish it with **a pilot carrying a measurable metric**: a baseline before rollout, a target gain, a stopping criterion (the methodology is Chapter 17; the mechanics of costing it out are the worked example in §2.2). The standard traps: counting tokens only; measuring usage (queries/day) instead of the outcome (tasks closed, cycle time); ignoring the cost of an error—which is exactly what sets the autonomy level and the volume of HITL work (§10.6–10.7).

25.7 Teams and the adoption process

Dividing responsibility. The configuration that works: a **platform team** (the gateway §25.3, eval infrastructure §17.8, observability Chapter 18, the registries §25.2—the "rails") and **product teams** (prompts, tools, answer quality, the product metric—the "trains"). The platform does not write prompts on the product's behalf: owning the quality of the answer is inseparable from owning the domain.

The stages. A pilot (a narrow scenario, a golden set, a baseline) → production (the eval gate §17.8, a human-gate on anything irreversible §10.6) → scaling (routing and caching Chapter 20, revisiting the autonomy level §10.7). The gate at every stage is metrics, not impressions: a pilot that "everybody liked" but that showed no gain over the baseline is a negative result.



People and trust. Users are taught to *calibrate* their trust: blind acceptance of answers (automation bias) and total distrust that routes around the system kill the value equally; transparency—source citations (§6.7) and an explanation of the actions taken—works in favor of calibration. HITL review is planned workload with metrics of its own (§10.6: the approval rate as a detector of rubber-stamping). The feedback channel closes onto the golden set (§17.8): a complaint that never became an eval case is lost.

25.8 Chapter summary

Adoption = architecture (Chapter 24) × organizational scaffolding: AI artifacts get a lifecycle and registries (§25.2), policy is applied at a single point—the gateway (§25.3), personal data is controlled across every place it settles (§25.4), regulatory readiness rests on the registry, human oversight, and tracing (§25.5), the economics are proven by a measurable pilot (§25.6), and the process is built on a platform/product split with staged gates on metrics (§25.7). The next chapter instantiates the technical and the organizational parts alike in a worked example.

Sources. EU, Regulation (EU) 2024/1689 (AI Act); NIST, "Artificial Intelligence Risk Management Framework 1.0" (2023); ISO/IEC 42001:2023—AI management systems; Microsoft Presidio—an open-source PII detection tool (§25.4); Moffatt v. Air Canada (Civil Resolution Tribunal, 2024)—liability for a chatbot (§25.5); Google SRE Book (Beyer et al., 2016)—operational discipline, error budgets; Chip Huyen, *AI Engineering* (2025)—platform concerns and the lifecycle of AI applications.

In the Margins: Chapter 25

> ↩ The AI Act's roles: **provider**—develops the system or model and places it on the market; **deployer**—uses it under its own responsibility. An organization that has wired a hosted model into its process is usually a deployer; the obligations of the roles differ.
>
> ↩ **GPAI**—general-purpose AI in the AI Act's terminology: general-purpose models (the base LLMs) carrying a separate set of provider obligations, regardless of the end application.
>
> ↩ **NIST AI RMF**—a voluntary framework for managing AI risk (the functions Govern / Map / Measure / Manage); a jurisdiction-neutral maturity checklist.
>
> ↩ **DPA**—a Data Processing Agreement: the contract with the provider about what it does with your data (the scope of processing, retention, use for training, subprocessors). On the §26.5 diagram it appears as the abbreviation.

Chapter 26. A Worked Example and a Package of Architectural Decisions

This closing chapter applies the whole book to one concrete case, turning principles into a sequence of **architecture decision records (ADRs)**—the standard format for recording a decision together with its rationale (M. Nygard). That makes the material operational: each decision rests on a specific chapter, and together they instantiate the reference architecture (Chapter 24) for a given set of requirements.

26.1 The scenario and functional requirements

Scenario. An internal support and operations assistant for an organization: it answers employees' questions from internal documentation (HR policies, IT runbooks, engineering docs), citing sources, and performs a limited set of actions in internal systems.

Functional requirements:

- Answer questions by *grounding* the answer in internal documents and citing sources.
- Search and create tickets in the support system (creation is an action with a side effect).
- Check the status of internal systems (a read-only action).
- Hold context within a conversation, and remember the employee's team and role across sessions.
- Escalate to a human when uncertain and before sensitive actions.

26.2 Non-functional requirements and budgets

The requirements calibrate the architecture (Chapter 2):

- **Latency**—interactive: TTFT within a couple of seconds, with streaming (§2.3); write actions may be slower.
- **Cost**—a bounded budget per conversation; an internal tool, cost-sensitive at scale (Chapter 20).
- **Accuracy**—high faithfulness (grounding and citations), refusal or escalation when confidence is insufficient (Part III, Chapter 17).
- **Security**—internal data must not leak; the content of documents and tickets is potentially untrusted (injection risk); irreversible actions stay under control (Chapter 19).
- **Privacy and residency**—processing internal data drives the choice between an API under a data processing agreement (DPA, §25.3) and self-hosted serving (Part VII).
- **Availability**—a business-hours SLO.

The budget in numbers (illustrative, in the discipline of §2.2). 2,000 requests per working day; the average conversation runs 3 turns at ~2,000 input tokens each (with RAG context) and ~300 output ⇒ ~6k input + 0.9k output per request ⇒ ~12M input + 1.8M output per day. At mid-tier pricing ($1/M in, $4/M out): 12 + 7.2 ≈ **$19 a day, ~$420 a month** before optimizations; caching the system prompt and routing (ADR-4, ADR-11) cut

that several-fold. The number settles the ADR-8 argument on the spot: at this volume, self-hosting (renting a GPU node runs thousands of dollars a month, §2.6) costs an order of magnitude more than the API—and it stays an option on residency grounds, not on economics.

26.3 The ADR format

An **ADR** records one architecture decision: its *context* (the forces and constraints), the *decision* itself, its *consequences*, and the *alternatives rejected*. The format earns its keep through the explicit rationale: the decision can be revisited later by someone who understands why it was made. What follows is a package of ADRs covering the key decisions across the layers of the reference architecture (§24.6).

26.4 The architectural decision package

ADR-1. Orchestration: a bounded agent, not full autonomy. *Context:* the task is partly structured (grounded Q&A, a limited set of actions) but needs flexibility. *Decision:* orchestrate as a bounded agent built on a state machine (§11.8, §16.4)—a deterministic "retrieve → answer" path with tool calls for actions and an iteration limit (§10.5); not open-ended autonomy. *Consequences:* predictability, debuggability, lower cost; novel, atypical requests get escalated. *Rejected:* a fully autonomous agent (a multiple of the cost, and unpredictable, §14.3) and a pure workflow (does not cover the variability of the requests).

ADR-2. Knowledge: RAG, not fine-tuning or a long context. *Context:* the corpus is large and changes often; answers must be current and carry citations. *Decision:* RAG over the internal documents (Part III). *Consequences:* current knowledge, updatable without retraining, with citations; retrieval infrastructure gets added. *Rejected:* fine-tuning (it changes behavior, not knowledge, and goes stale, §21.1) and loading every document into a long window (U-shaped degradation and cost, §4.3).

ADR-3. Retrieval: hybrid with reranking. *Context:* queries mix exact terms (error codes, policy identifiers) with semantics. *Decision:* hybrid retrieval (BM25 + dense) with RRF fusion, followed by cross-encoder reranking of the top candidates (Chapter 7). *Consequences:* robustness to both query types; reranking adds latency over a small candidate set. *Rejected:* dense-only retrieval (misses exact matches) and no reranking (precision suffers).

ADR-4. Model: a cascade by complexity. *Context:* most requests are routine, some require reasoning. *Decision:* route by complexity—a cheap model for routine Q&A, a strong one for complex multi-step requests, with classification ahead of routing (§20.3). *Consequences:* lower average cost; the classifier itself costs latency and money—keep it light (a heuristic or a cheap model; anatomy in Deep Dive §2.2) and account for it in the TTFT budget (§26.2); the risk of a classification error is *silent* (there is no feedback loop, §2.2) and is controlled only by evaluation (Chapter 17). *Rejected:* one strong model everywhere (cost) and one cheap model everywhere (quality).

ADR-5. Memory: a session window plus a user profile. *Context:* the system needs continuity within a conversation and stable employee context (team, role) across sessions. *Decision:* short-term memory is the session's message window; long-term memory is a user profile in a store, injected into the context (§4.5, §13.1). *Consequences:* continuity and personalization at a bounded volume; a forgetting policy is required. *Rejected:* no memory at all (repetition) and full memory of every past conversation (cost and privacy).

ADR-6. Tools: a minimal set, reads and writes kept separate, over MCP. *Context:* the assistant needs ticket search and status (reads) and ticket creation (a write). *Decision:* a curated minimal set (§12.3); low-risk reads, and separately the write, under control (ADR-7); integration over MCP wherever a standard server exists,

treating the servers as untrusted (§15.7). *Consequences:* a small attack surface, clear contracts. *Rejected:* broad tool access (excessive agency, §19.4).

ADR-7. Security: isolation, least privilege, a human-gate on writes. *Context:* private data, potentially untrusted content (documents, tickets), and a write tool coincide here—a configuration close to the lethal trifecta (§12.5, §19.4). *Decision:* isolate untrusted content from instructions (§4.6); authorize the tool with the requesting employee's own rights (§12.5); require human confirmation (a human-gate) on ticket creation and on any configuration-changing action (§10.6); apply the rule of two—do not combine private data, untrusted content, and an autonomous write without a gate. *Consequences:* an injection cannot silently create a ticket or walk off with the data; a little friction on writes. *Rejected:* relying on guardrails alone (an adaptive attack gets around them, §19.5).

ADR-8. Model deployment: an API under a data processing agreement, self-hosting as an option. *Context:* privacy of internal data, moderate scale, limited ML-infrastructure expertise. *Decision:* start on a managed API under a data processing agreement; self-hosted serving of an open model on vLLM (Chapter 22) is a documented option should residency requirements tighten or scale grow; no pre-training and no fine-tuning at the start (§21.1, §23.1). *Consequences:* a fast start with little infrastructure load; vendor dependence is softened by portability at the gateway layer (§16.5, §20.6). *Rejected:* self-hosting from day one (cost without proven need) and pre-training (§23.1).

ADR-9. Evaluation: an offline golden-set gate plus online monitoring. *Context:* nondeterminism, no tolerance for regression, the need to catch drift. *Decision:* a golden set of representative queries with faithfulness and correctness thresholds as a CI gate ahead of any change to the prompt, the model, or the tools; online reference-free monitoring (faithfulness, refusal rate, retrieval quality) on a sample of traffic; an LLM judge validated against human labels and pinned to a fixed version (§17.4). *Consequences:* protection from regressions and detection of drift; the cost of eval infrastructure and calibration. *Rejected:* manual checks only (they do not scale and do not catch regressions, §17.1).

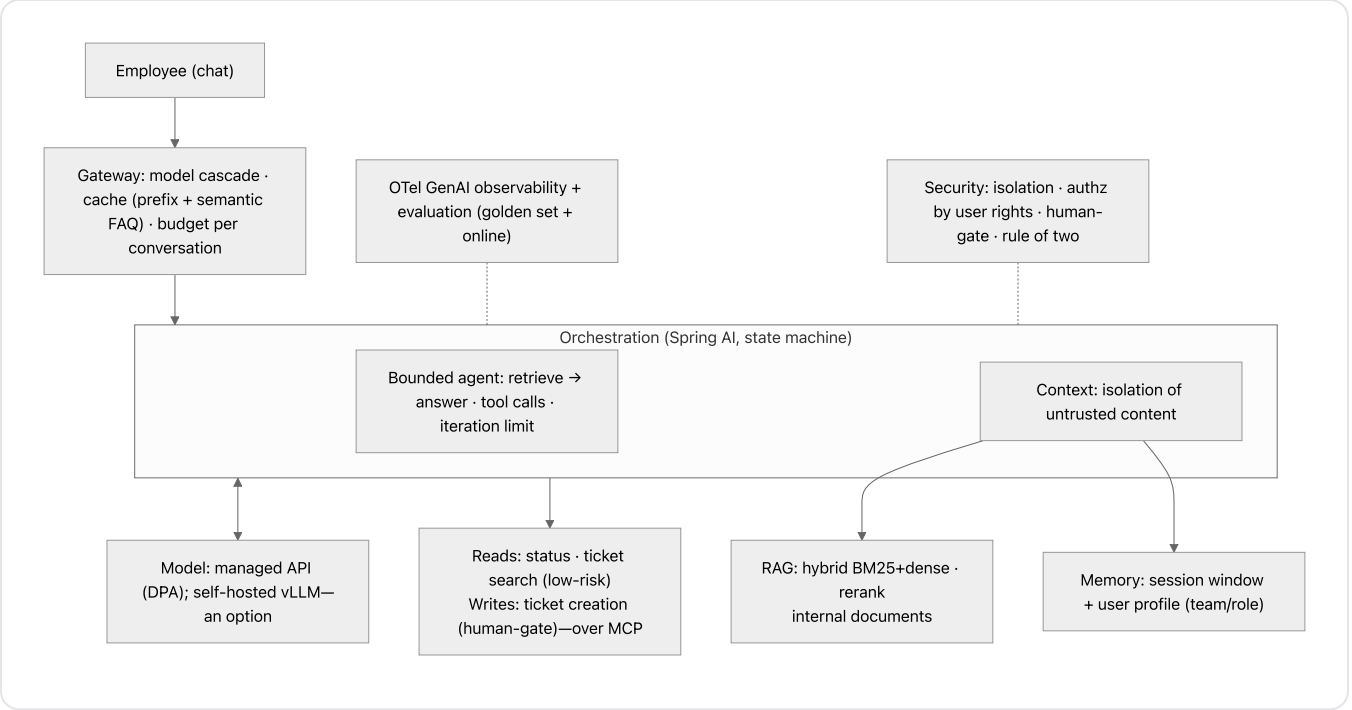
ADR-10. Observability: OpenTelemetry GenAI traces and cost attribution. *Context:* many steps and nondeterminism demand debugging and cost allocation. *Decision:* instrument against the OpenTelemetry semantic conventions for GenAI (§18.4); a span tree per request; token, cost, and latency metrics attributed by team and user (§18.5); content capture is optional and redacted, given privacy (§18.6). *Consequences:* debuggability and cost allocation; the discipline of telemetry retention and redaction.

ADR-11. Cost and latency: a gateway with caching and budgets. *Context:* cost sensitivity at scale and interactive latency. *Decision:* an LLM gateway that centralizes routing (ADR-4), prefix caching of the stable system prompt and prefix, semantic caching of frequent, routine answers (stateless), per-conversation token budgets, and streaming for TTFT (Chapter 20). *Consequences:* a substantial cost reduction (§20.7) and responsiveness; the discipline of invalidating the cache when documents and the FAQ are updated.

ADR-12. Implementation: Spring AI, given an existing Spring landscape. *Context:* the organization runs Spring Boot and wants ecosystem fit and unified observability. *Decision:* Spring AI—ChatClient and advisors (RAG, memory, safeguard), MCP client and server, observability through Micrometer, a node-based state-machine approach for control (§16.4). *Consequences:* ecosystem fit and minimal integration work; managing the framework's release cadence. *Rejected:* LangChain4j (viable; rejected here purely on fit with the Spring landscape, §16.6).

26.5 The instantiated architecture

The ADR package instantiates the reference architecture (§24.2) for the scenario:



Every component in the diagram is the trace of a specific ADR; together they form a complete, justified architecture rather than a set of disconnected choices.

26.6 How the system evolves

The architecture is not static, and the same ladder that ran through the whole book governs how it grows. The system starts at the least sufficient complexity—grounded Q&A with a limited set of actions—and grows more complex only as need is proven and confirmed by evaluation (Chapter 17): move to multi-agent only when a single agent demonstrably cannot cope (§14.1); to fine-tuning when prompting and RAG are exhausted (§21.1); to self-hosted serving when the API stops meeting the requirements (§22.1); to pre-training essentially never (§23.1). Each such move is written up as a new ADR that builds on the earlier ones and preserves the rationale for the evolution.

26.7 Summary of the chapter and of the book

One concrete scenario, carried from functional and non-functional requirements (§26.1–26.2) through a package of twelve architecture decisions (§26.4), instantiates the reference architecture under the given constraints (§26.5) and sets the trajectory of its evolution by the principle of least sufficient complexity (§26.6). Each decision rests on a specific chapter—which was the point of the book: not a catalog of tricks, but a coherent design discipline.

The book in sum. The path ran from the model as a component (Part I) through taming its interface (Part II) and grounding knowledge (Part III) to organizing action with agent patterns (Part IV), integration into the architecture (Part V), operation in production (Part VI), and owning your own models (Part VII), assembled into a single reference architecture, an organizational adoption framework, and a justified decision package (Part VIII). The through-line of the entire book: **a probabilistic component embedded in a deterministic discipline**. The model is nondeterministic and manipulable, so reliability, security, and economy are delivered by the architecture around it—constraints, isolation, evaluation, and observability—not by trusting the model itself. The mathematical substrate of every mechanism covered here is set out in Appendix A.

Sources. M. Nygard, “Documenting Architecture Decisions” (2011)—the ADR format; this book (Parts I–VII) as the foundation for every decision in the package; S. Willison, “The Lethal Trifecta” (2025) and Meta, “Agents Rule of Two” (2025)—the security frame for ADR-7 (introduced in §12.5, §19.4); C. Richardson, *Microservices Patterns* (2018)—the discipline of architecture decisions and boundaries.

Appendices

Appendix A. The Linear-Algebra Substrate

A.1 Why this appendix exists

Part I's chapters give the architecture and its consequences; this appendix is the *computational substrate* underneath it. To be an architect who understands the consequences (cost, memory, what the hardware is doing), you have to grasp the basic operations the forward pass is assembled from. This is not a derivation of the transformer from scratch (for that, see CS336/Raschka) but a **vocabulary of operations**, and how the model reduces to them.

A.2 A tensor is a multidimensional array

Let's clear up a terminological muddle right away. A **tensor** in ML (PyTorch/TensorFlow) is simply a **multidimensional array** of numbers, nothing more. It is *not* the tensor of physics/differential geometry and *not* a tensor product (§A.5). Its characteristics:

- **shape**—the sizes along each axis, e.g. `(batch, seq, d_model)`.
- **ndim** (the order; ML frameworks sometimes call this same thing "rank")—the number of axes (0 is a scalar, 1 a vector, 2 a matrix, 3+ a tensor-array). Not to be confused with the **rank of a matrix** from linear algebra (the dimension of the image, a number from 0 to min(rows, columns)): it is this second sense of "rank" that is at work in LoRA (§21.3) and in rank(A⊗B) (§21.5). Any matrix has ndim = 2 regardless of its linear-algebraic rank; from here on this appendix uses the word "order" for the number of axes.
- **dtype**—the type: float32; float16/**bf16** (bf16 has float32's exponent with a smaller mantissa—the same range, less precision, which is why it holds up better in training); int8 and below is quantization (Chapter 22).

Typical LLM tensors: embeddings `(batch, seq, d_model)`—3-dimensional; a weight matrix `(d_in, d_out)`—2-dimensional; attention scores `(batch, heads, seq, seq)`—4-dimensional.

A.3 The core operations (a vocabulary)

- **Element-wise:** `+`, `*`, the activation functions (ReLU, GELU, SiLU). Applied to every element; the operands share a shape (or get there by broadcasting).
- **Broadcasting:** the rule that automatically "stretches" axes of size 1 to make shapes compatible (e.g., adding a vector `(d,)` to a matrix `(seq, d)`). Understanding broadcasting is critical to reading LLM code.
- **Matrix multiplication (matmul):** `(m×k)·(k×n) → (m×n)`. Summing over the shared axis `k` is a **contraction**. Cost `O(m·k·n)`. The model's principal operation. Besides FLOPs there is a second characteristic—**arithmetic intensity**: how many operations (FLOPs) there are for each byte read from memory. Numbers

to make it tangible (H100, FP16): peak throughput ~989 TFLOPS against memory bandwidth ~3.35 TB/s; their ratio is ≈ **300 FLOP/byte**—the card's "balance point": to keep the compute units busy, every byte read has to carry ~300 operations. Now take decode at batch 1: it is a matrix-vector multiply, and every weight (2 bytes in FP16) is read for the sake of a single multiply-add pair—2 FLOPs / 2 bytes = **1 FLOP/byte**. One against three hundred: the compute units idle ~99.7% of the time, and a decode step is bounded by how fast the weights can be read, not by the arithmetic. Prefill, on the other hand, processes all the prompt's tokens in one batched matmul—a weight read once is used once per input token, intensity grows with prompt length, and the card finally computes. That is how the entire contrast between memory-bound decode and compute-bound prefill (§22.5) follows "from below," out of two numbers on the accelerator's spec sheet.

- **Batched matmul (bmm):** matmul applied along the batch axis: `(b×m×k)·(b×k×n) → (b×m×n)`. This is how the model handles *batches* and *multi-head* attention (each head/example gets its own matmul, in parallel). **LLM operations are batched matmuls, not single ones**—which matters for the intuition about GPU utilization (§1.6).
- **einsum (Einstein summation / contraction):** the general notation for tensor contractions in terms of indices. For example, QK^T across heads is written `einsum("bhqd,bhkd->bhqk", Q, K)`: sum over the shared index `d` (the head dimension), keep `b,h,q,k`. Reading it letter by letter: `b` and `h` are in both operands and in the result—batch axes (independent repeats); `d` is in both but absent from the result—that is the axis being summed over; `q` and `k` are the operands' own axes, and they form the result's axes. A convenient way to write any matmul/contraction. **A contraction lowers or preserves a tensor's order** (it sums over an index)—remember this for §A.5.
- **softmax:** turns a vector of numbers into a probability distribution (exponentiate + normalize) along a given axis. Used in attention (the weights) and at the output (§1.7).
- **Normalization (LayerNorm / RMSNorm):** normalizes values along the feature axis (stabilizes training, §1.4). RMSNorm is cheaper (scale only, no subtraction of the mean).
- **reshape / view / transpose:** reorganizing axes without changing the data—the code's "plumbing" (e.g., slicing `d_model` into `(h, d_head)` for multi-head attention and assembling it back).
- **Concatenation (`torch.cat`):** gluing several tensors end to end along a chosen axis: `[576, 4096]` and `[20, 4096]` with `dim=0` give `[596, 4096]`. Unlike the element-wise `+` above, it does not require the sizes to match along the axis being glued—only along all the others; and it does not blend values, it keeps each operand in its own positions. This is how a multimodal sequence is assembled out of text and visual patches (§1.2).

≠ A false friend: «свёртка». In Russian technical literature this one word denotes three *different* operations; when reading translated texts you have to keep them apart:

- **Contraction (tensor contraction; Russian «свёртка тензоров», «эйнштейнова свёртка»)**—summing products over a shared index, as in the matmul and einsum above; it lowers or preserves a tensor's order. This is the operation of the LLM forward pass, and it is exactly why the book's primary term is "contraction" rather than «свёртка».
- **Convolution (the «свёртка» of CNNs and signal processing)**—dragging a kernel-filter across the input in a sliding window and summing the overlaps. Mathematically a different operation: the basis of convolutional networks (CNNs); there is none of it in a transformer block.
- **Fold / reduce (the «свёртка» of a list from functional programming; `Stream.reduce` in Java)**—sequentially aggregating a collection with a binary operation. It has nothing to do with linear algebra, but in engineering prose it turns up under the same Russian word.

In the rest of this appendix, «свёртка» always means the first—contraction.

A.4 How a transformer block reduces to these operations

(The full trace with dimensions is in the Deep Dive of §1.4.) In brief, a block = a sequence:

- **A linear layer** = matmul (the Q/K/V projections, the output one, two in the FFN).
- **Attention scores** = batched matmul ($QK^\top$) → scale → **softmax** → batched matmul ($\cdot V$).
- **FFN** = matmul → activation (element-wise) → matmul.
- **Residual** = element-wise addition; **norm** = normalization.
- **reshape/transpose**—slicing/assembling the heads between steps.

The upshot: the whole forward pass is **batched matrix multiplications (contractions)**, interleaved with softmax, normalization, and element-wise nonlinearities. There are no tensor or Kronecker products (§A.5) in it at all. The backward pass is the same matmuls with transposed operands; its roughly double cost relative to the forward is what gives the factor of 3 (one forward + two equivalents for the backward) in the formula $C \approx 6ND$ (§23.4).

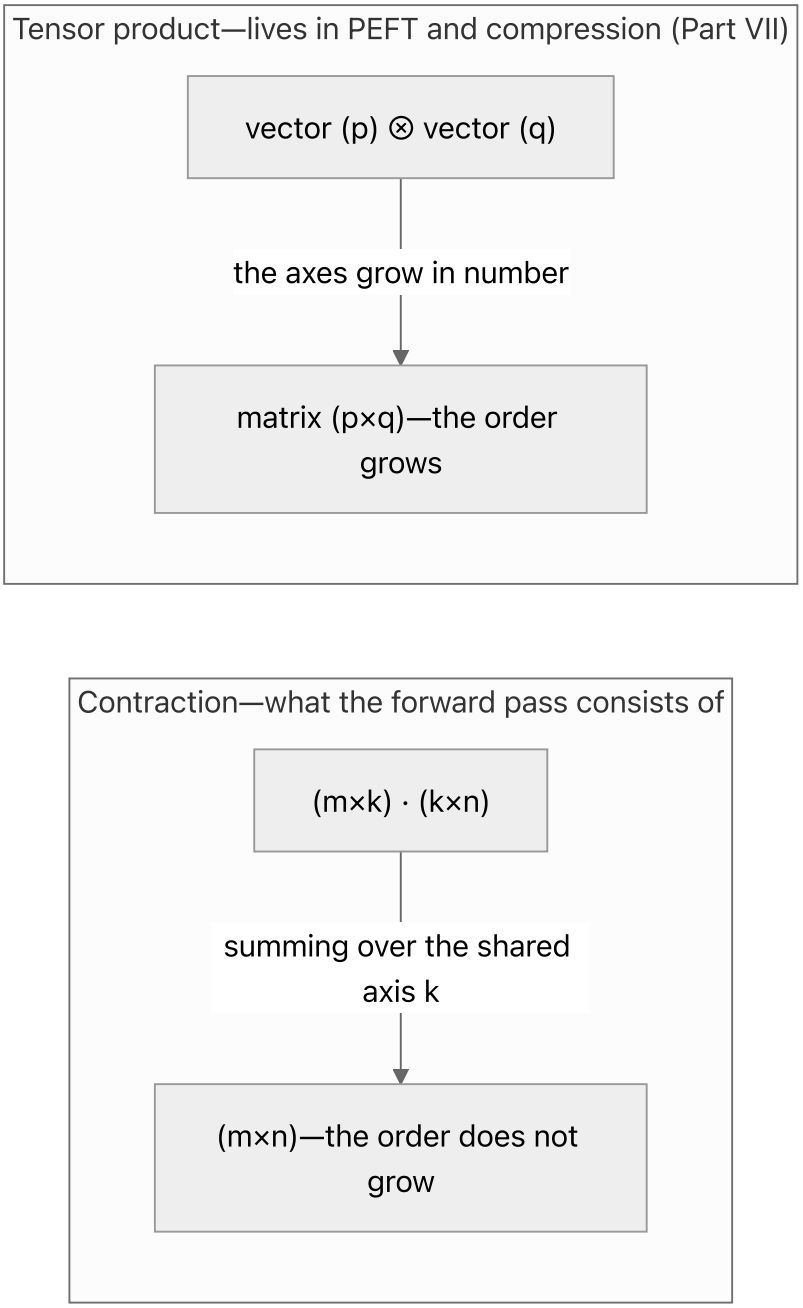
A.5 Kronecker and tensor products: where they really are (and where they are NOT)

A section of its own, because the words “tensor” and “tensor product” sound as though they were precisely what the transformer rests on—and that expectation is worth taking apart explicitly.

Definition. A **tensor product** *raises the order*: the product of objects of orders p and q has order $p + q$ (two vectors, for example, give a matrix). Its matrix form is the **Kronecker product** $\otimes$: the result’s order stays at 2, but the sizes multiply, $(m\times n) \otimes (p\times q) \to (mp\times nq)$. In both cases a *larger* object *is built* out of smaller ones. A numeric example: A of size 2×2 ⊗ B of size 3×3 → a 6×6 matrix—each element a_{ij} is replaced by an entire block $a_{ij}\cdot B$ (four 3×3 blocks in a 2×2 grid). The same example shows the multiplicativity of rank: rank(A) = 2, rank(B) = 3 → rank(A⊗B) = 2·3 = 6—full rank out of 4 + 9 = 13 stored numbers, whereas for an ordinary product of two matrices the rank does not exceed the smaller of the factors’ ranks (§21.5 exploits exactly this difference).

The key contrast with matmul. The transformer’s forward pass is built on **contraction** (matmul/einsum), which *lowers or preserves* the order—it sums over a shared index. The tensor product is the *reverse* in spirit: it *raises* the order. Hence:

> **There is no tensor product and no Kronecker product in an LLM's forward pass.** Attention, the linear layers, the FFN—these are contractions, not products. "Tensor" here = array; the operations on it = contractions. This is not some foundation of the architecture that got left out—they are simply not there.



Where the Kronecker/tensor product *really does* show up in ML (adjacent topics, all in Part VII):

- **PEFT: Kronecker adapters** (KronA, for one)—an alternative to LoRA in which the weight update is represented as a Kronecker product of small matrices → **Part VII (fine-tuning)**.
- **Model compression: tensor decompositions** (Tensor-Train, Tucker, CP)—representing a weight/embedding matrix as a product of smaller tensors for the sake of compression → **Part VII (compression/serving)**.

- **Second-order optimization: K-FAC** (Kronecker-Factored Approximate Curvature)—approximating the curvature with Kronecker factors (a training topic, not a forward-pass one).

The upshot: these operations live in *fine-tuning and compression* (Part VII), not in the core of the forward pass—and they are worked through where they do their work.

A.6 The practical bridge: why an architect needs this

The point of the appendix is not math for math's sake; it is that everything the architect governs follows from these operations:

- **Cost/FLOPs** are dominated by matmul → accelerators (GPU/TPU) are in essence matrix multipliers, and utilization is measured in matmul throughput (§1.6).
- `seq²` in the attention matmul is the root of the quadratic price of long context (§1.4, §1.8).
- **The sizes of the weight matrices** = the parameter count = the memory for the weights (§1.6, Chapter 21).
- **The batched nature** of the operations explains why batching raises GPU utilization (§1.6, continuous batching).

Understanding the model as "a stack of batched matmuls wrapped in attention and nonlinearities" gives you that base level from which the consequences (cost, latency, memory, hardware choice) are *derived* rather than memorized.

Sources. S. Raschka, *Build a LLM from Scratch* (2024)—the implementation of these operations; Stanford CS336—resource accounting and architectures; the PyTorch documentation (matmul, einsum, broadcasting). For Part VII: the work on KronA, Tensor-Train/Tucker decompositions, K-FAC.

Appendix B. Pattern Index

Every pattern in the book in a single list, for reference. The full card (Context → Problem → Forces → Solution → Mechanics → Resulting Context → When → Related → Sources) is in the section listed.

Resilience and call economics

Pattern	Problem it solves	Where
Model Routing / Cascade	Pay frontier prices only for frontier tasks	§2.2, §20.3
Fallback Model	Survive a provider outage or degradation	§2.5
AI Gateway	Apply policies (access, budgets, PII, audit) to every call at once	§25.3

Managing context and output

Pattern	Problem	Where
Context Compaction	A long session won't fit the window; truncate and you lose the decisions	§4.4
Sliding Window · Externalized State · Context Caching	Mini-patterns for the window budget	§4.4 (catalog)
Retrieval-based Memory (episodic/semantic/procedural)	State beyond the window, retrieving what's relevant	§4.5
Validate-then-Retry (Reask)	Invalid output with no access to the logits	§5.5
Two-step generate-then-format	A rigid schema strangles reasoning	§5.5

Retrieval (RAG)

Pattern	Problem	Where
Naive RAG	The baseline: ground the answer in a corpus	§7.2
Hybrid Search	Semantic search is blind to exact terms, lexical search to meaning	§7.3
Reranking	The right document came back, just not in the top-k	§7.4
Query transformation (rewriting, HyDE, decomposition)	A user's question ≠ a good search query	§7.5
GraphRAG	Questions about relationships, not about similarity	§7.7
Agentic RAG	One search pass isn't enough	§7.8
Adaptive RAG	Not every query needs the expensive pipeline	§7.9

Agents and orchestration

Pattern	Problem	Where
Prompt Chaining	A fixed sequence of steps	§11.2
Routing	Heterogeneous input with distinguishable categories	§11.3
Parallelization (Sectioning / Voting)	Independent parts; reliability through redundancy	§11.4
Orchestrator-Workers	The decomposition is known only at runtime	§11.5 (demo—Preface)
Evaluator-Optimizer	Iterative quality when a criterion exists	§11.6
Human-in-the-Loop Gate	An irreversible action from a nondeterministic component	§10.6

Reasoning strategies (loop mechanisms, not cards): ReAct—§13.3; Plan-and-Execute—§13.4; Reflection/Reflexion—§13.5; Tree-of-Thought—§13.6. Coordinating multi-agent systems (supervisor / handoff / network / hierarchical)—§14.2.

Organizational

Pattern	Problem	Where
Prompt Registry	Prompts need the lifecycle of code	§25.2
AI Gateway (governance role)	See above	§25.3

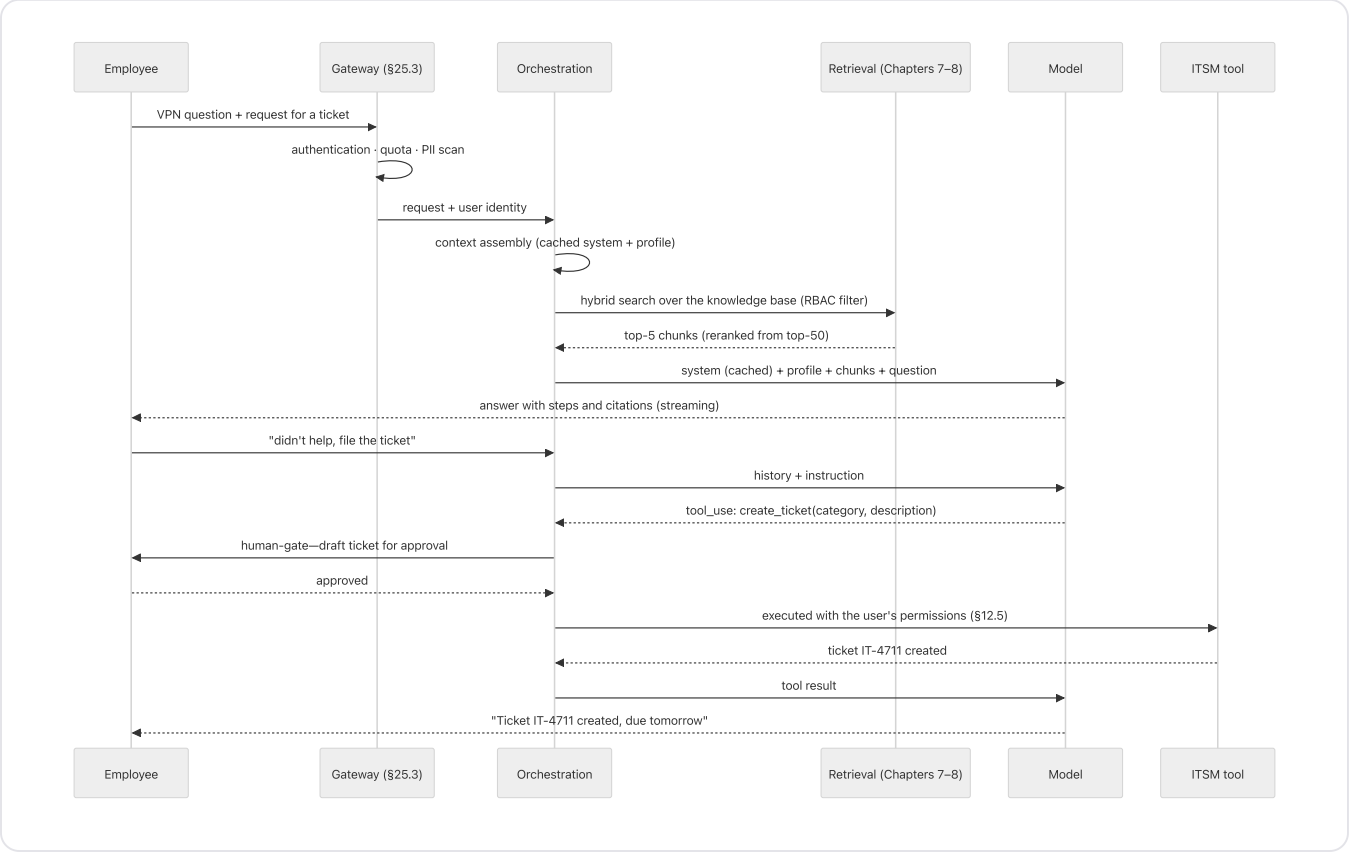
Cross-cutting defenses (not cards, but named practices): isolating untrusted content—§4.6; the rule of two and taint-tracking—§12.5; the dual-LLM/quarantine loop and the execution sandbox—§19.6; autonomy levels L0–L3—§10.7.

Appendix C. Anatomy of a Single Trajectory

The book's chapters take the system apart layer by layer; this appendix puts the layers back together—it traces **one concrete request** through the system of Chapter 26 (the internal assistant, 12 ADRs, the architecture of §26.5) from the press of Enter to the entry in the log. Every abstraction in the book—context assembly, retrieval, ACI, the human-gate, the trace, the budget—comes together here in one tangible example. The numbers line up with the budget of §26.2 and are illustrative.

The request. An employee writes to the assistant: *"After the update my laptop won't connect to the VPN. What do I do? If I need a new access token—file a ticket."* The conversation runs two turns: an answer from the knowledge base, and creating a ticket through the gate.

C.1 The request flow



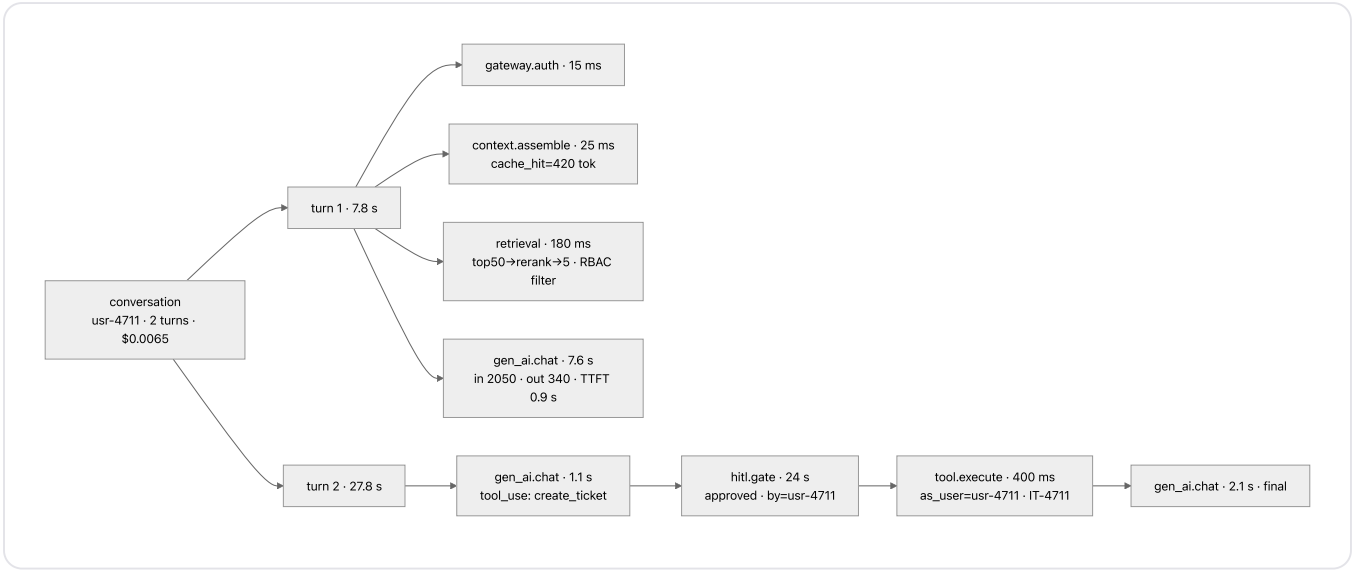
C.2 The budget, step by step

Step	Tokens (in / out)	Latency	Cost*
Gateway: authentication, quota, PII scan	—	15 ms	—
Context assembly: system 420 (cache hit, §20.2) + profile 130 + instructions 110	—	25 ms	—
Retrieval: query embedding → dense+BM25 → RRF → rerank top-50→5	—	180 ms	~$0.0001
Turn 1, generation: in 2,050 / out 340, TTFT 0.9 s, streaming	2,050 / 340	7.6 s (perceived ~0.9 s)	$0.0030
Turn 2, the tool decision: in 2,600 (420 of it from cache) / tool_use 90	2,600 / 90	1.1 s	$0.0026
Human-gate: draft ticket → approval	—	24 s (waiting on a human)	—
Tool: create the ticket with the employee's permissions	—	400 ms	—
Final answer: in 160 (the tool result) / out 160	160 / 160	2.1 s	$0.0008
Conversation total	**~4.8k / 0.6k**	**~11 s machine + 24 s human**	**~$0.0065**

* A mid-tier rate ($1/M in, $4/M out—an illustrative tier, not Claude's pricing: there an output token costs exactly five times an input one, §2.5), the system prefix coming from cache at ~10% of the price. Against the §26.2 budget (≈$0.0096 per interaction), it comes in with room to spare; the human wait at the gate does not count toward system latency, but it does count toward the process SLA (§10.6).

C.3 What observability recorded

The span tree for this trace (semantics—§18.3–18.4):



This one trace shows everything Chapters 10–20 taught: the citations in the answer make faithfulness checkable (§9.2); the action ran with the user's permissions rather than the agent's—no confused deputy (§12.5); the gate recorded *who* approved (§10.6); cost is attributed to a user and a feature (§18.5, §20.6).

C.4 Closing the loops

A request's life does not end with the answer. This trace: (1) landed in the **online sample**—a reference-free judge scored the faithfulness of the turn-1 answer (§17.6); (2) is a candidate for the **golden set**—the VPN-after-an-update question turned out to be a new class, and an SME will label the reference answer (§9.3, §17.8); (3) fed the **metrics**—token spend, cache-hit rate, time in the gate queue (§25.7: the approval rate as a detector of rubber-stamping). One request, three feedback loops; a system with those loops closed improves off its own traffic.

Appendix D. The Architect's Preflight Checklist

One page to read before an LLM system goes to production—the book distilled into a set of checks. Every item points to the section that justifies it; an unchecked box is a risk you took on deliberately, not one you forgot.

Contracts

- [ ] The feature's NFR specification is filled in: quality, latency, cost, degradation (§2.7)
- [ ] The output schema is declared, the post-hoc validator is on, reask is configured (§5.4–5.5)
- [ ] A timeout on every network call; retries with jitter; a fallback model or degradation (§2.5)
- [ ] The agent's actions are idempotent on an operation key (§2.5, §10.4)

Context and data

- [ ] Untrusted content is structurally isolated and marked as data (§4.6)
- [ ] The window budget is allocated explicitly and the reserve for the output is accounted for (§1.8); compaction and memory kick in at a threshold (§4.2, §4.4–4.5)
- [ ] PII is masked before indexing and before it reaches the provider; telemetry is redacted (§25.4, §18.6)
- [ ] The right to erasure can actually be honored everywhere the data settles (§25.4)

Actions and security

- [ ] The tool set is curated; risk metadata is filled in (§12.3, §12.5)
- [ ] Authorization runs with the user's permissions, at the point of execution (§12.5)
- [ ] The tools *as a set* have been checked against the lethal trifecta; the rule of two is satisfied or a gate is in place (§12.5, §19.4)
- [ ] Third-party MCP servers are untrusted: OAuth, pinning of approved tool descriptions (§15.7)
- [ ] A human gate on the irreversible; a gate timeout means a safe rejection (§10.6)
- [ ] Iteration and token limits per step and per task (§10.5)
- [ ] The defense-in-depth checklist has been worked through in full (§19.7), including the dual-LLM quarantine for high-risk loops and the sandbox for code execution (§19.6)

Retrieval (if you have RAG)

- [ ] Access control is applied at retrieval; recall has been checked with the filters on (§8.6–8.7)
- [ ] A re-indexing plan for when the embedder changes; a strategy for deleting from the index (§8.8, §8.3)

Evaluation and observability

- [ ] The golden set exists, is versioned, and includes "unanswerable questions" (§9.3)
- [ ] The judge is validated against humans (kappa) and its version is pinned (§17.4, §9.4)
- [ ] An eval gate sits in CI on changes to the prompt/model/tools (§17.8)
- [ ] Traces are written to semconv; cost is attributed by feature and tenant (§18.3–18.5, §20.6)
- [ ] The sample size behind release decisions is computed, not guessed (§17.7)

Organization

- [ ] The system is entered in the registry with a risk class and an owner (§25.2, §25.5)
- [ ] Prompts are in the registry, each with an owner and a link to the golden set (§25.2)
- [ ] Routes to providers are DPA-approved only, and go through the gateway (§25.3)
- [ ] HITL review is planned for as a workload; the rubber-stamping metric is on (§25.7, §10.6)
- [ ] The pilot has a baseline, a target lift, and a stopping criterion (§25.6)

Appendix E. Consolidated Bibliography

The per-section *Sources* lists remain the primary place for attribution (they record *what exactly* is drawn from each work); this list consolidates them for navigation. Grouping: books → papers and preprints → industry material, documentation, and standards.

Books and courses

- Alammar J., Grootendorst M. *Hands-On Large Language Models* (2024)—Chapter 1.
- Beyer B. et al. *Site Reliability Engineering* (Google SRE Book, 2016)—Chapters 2, 17, 25.
- Dibia V. *Designing Multi-Agent Systems* (2025)—Chapters 10–11, 14.
- Hohpe G., Woolf B. *Enterprise Integration Patterns* (2003)—Chapter 11.
- Huyen C. *AI Engineering* (2025)—the source of the engineering frame running throughout.
- Lambert N. *RLHF Book* (2025)—Chapters 13, 21.
- Nygard M. *Release It!* (2nd ed., 2018)—Chapter 2; "Documenting Architecture Decisions" (2011)—Chapter 26.
- Raschka S. *Build a Large Language Model (From Scratch)* (2024)—Chapter 1, Appendix A; "The Big LLM Architecture Comparison" (2025)—Chapter 1.
- Richardson C. *Microservices Patterns* (2018)—Preface, Chapters 10–11, 14, 24, 26.
- Stanford CS336 "Language Modeling from Scratch"—Chapters 1, 23, Appendix A.

Papers and preprints

- Agarwal R. et al. "On-Policy Distillation… (GKD)" (2023)—§21.7.
- Agrawal A. et al. "Sarathi" (2023)—§22.5.
- Asai A. et al. "Self-RAG" (2023)—§7.8.
- Aumüller M., Ceccarello M. "The Role of Local Intrinsic Dimensionality in Benchmarking Nearest Neighbor Search" (SISAP, 2019)—§8.2.
- Besta M. et al. "Graph of Thoughts" (2023)—§13.6.
- Beurer-Kellner K. et al. "Design Patterns for Securing LLM Agents against Prompt Injections" (2025)—§19.6.
- Beyer K., Goldstein J., Ramakrishnan R., Shaft U. "When Is 'Nearest Neighbor' Meaningful?" (ICDT, 1999)—§8.2.
- Bhatt M. et al. "ETDI" (2025)—§15.7.
- Cemri M. et al. "Why Do Multi-Agent LLM Systems Fail?" (MAST, 2025)—§14.4.
- Chen Z. et al. "Exploring the Meaningfulness of Nearest Neighbor Search in High-Dimensional Space" (2024)—§8.2.
- Cormack G. et al. "Reciprocal Rank Fusion Outperforms Condorcet…" (SIGIR 2009)—§7.3.
- Dao T. et al. "FlashAttention" (2022)—§1.4.
- DeepSeek. "DeepSeek-V3 Technical Report" (2024)—§1.5; "DeepSeek-R1" (2025)—§13.8.
- Dettmers T. et al. "QLoRA" (2023)—§21.4.
- Edalati A. et al. "KronA" (2022)—§21.5.
- Es S. et al. "RAGAS" (2023)—Chapter 9.
- Ethayarajh K. et al. "KTO" (2024)—§21.6.

- Faysse M. et al. “ColPali” (2024)—§7.6.
- Fedus W. et al. “Switch Transformers” (2021)—§1.5.
- Frantar E. et al. “GPTQ: Accurate Post-Training Quantization” (2022)—§22.6.
- Gao Y. et al. “Retrieval-Augmented Generation for LLMs: A Survey” (2023)—Chapters 6–7, 9.
- Gou J. et al. “Knowledge Distillation: A Survey” (2021)—§21.7.
- Gu Y. et al. “MiniLLM” (2023)—§21.7.
- Hardy N. “The Confused Deputy” (1988)—§12.5.
- Hinton G., Vinyals O., Dean J. “Distilling the Knowledge in a Neural Network” (2015)—§21.7.
- Hoffmann J. et al. “Training Compute-Optimal LLMs” (Chinchilla, 2022)—§23.3–23.5.
- Holtzman A. et al. “The Curious Case of Neural Text Degeneration” (2019)—§1.7.
- Hong K., Troynikov A., Huber J. “Context Rot” (Chroma Research, 2025)—§1.8, §4.3.
- Hou X. et al. “MCP: Landscape, Security Threats, and Future Research Directions” (2025)—Chapter 15.
- Hu E. et al. “LoRA” (2022)—§21.3.
- Huang J. et al. “LLMs Cannot Self-Correct Reasoning Yet” (2024)—§13.5.
- Jégou H. et al. “Product Quantization for Nearest Neighbor Search” (2011)—§8.3.
- Jeong S. et al. “Adaptive-RAG” (2024)—§2.2, §7.9.
- Johnson J. et al. “Billion-scale Similarity Search with GPUs” (FAISS, 2017)—§8.3.
- Kamradt G. “Needle In A Haystack” (2023)—§4.3.
- Kaplan J. et al. “Scaling Laws for Neural Language Models” (2020)—§23.3.
- Khattab O., Zaharia M. “ColBERT” (2020)—§7.4.
- Kim S. et al. “An LLM Compiler for Parallel Function Calling” (2023)—§13.4.
- Kim Y., Rush A. “Sequence-Level Knowledge Distillation” (2016)—§21.7.
- Jimenez C. et al. “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” (2024)—§17.5.
- Kojima T. et al. “LLMs are Zero-Shot Reasoners” (2022)—§3.4.
- Kudo T. “Subword Regularization” / SentencePiece (2018)—§1.2.
- Kusupati A. et al. “Matryoshka Representation Learning” (2022)—§1.3.
- Kwon W. et al. “Efficient Memory Management for LLM Serving with PagedAttention” (2023)—§1.6, Chapter 22.
- Leviathan Y. et al. “Fast Inference from Transformers via Speculative Decoding” (2023)—§1.6, §22.7.
- Lewis P. et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” (2020)—Chapter 6.
- Liu N. et al. “Lost in the Middle” (2023)—§1.8, §4.3.
- Liu S. et al. “DoRA” (2024)—§21.5.
- Liu X. et al. “AgentBench: Evaluating LLMs as Agents” (2023)—§17.5.
- Longpre S. et al. “Entity-Based Knowledge Conflicts in QA” (2021)—§6.7.
- Malkov Y., Yashunin D. “Efficient and Robust ANN Search Using HNSW” (2016/2018)—§8.3.
- Mialon G. et al. “GAIA: A Benchmark for General AI Assistants” (2023)—§17.5.
- Mikolov T. et al. “word2vec” (2013)—§1.3.
- Muennighoff N. et al. “MTEB” (2022)—§1.3, §7.6; “Scaling Data-Constrained Language Models” (2023)—§23.6.
- Nogueira R., Cho K. “Passage Re-ranking with BERT” (2019)—§7.4.
- Ouyang L. et al. “InstructGPT” (RLHF, 2022)—§21.6.
- Packer C. et al. “MemGPT” (2023)—§4.5.
- Panickssery A. et al. “LLM Evaluators Recognize and Favor Their Own Generations” (2024)—§17.4.

- Parasuraman R., Riley V. “Humans and Automation: Use, Misuse, Disuse, Abuse” (1997)—§10.6.
- Park J. et al. “Generative Agents” (2023)—§4.5.
- Patel P. et al. “Splitwise” (2024)—§22.5.
- Peng B. et al. “YaRN” (2023)—§1.4.
- Perez F., Ribeiro I. “Ignore Previous Prompt” (2022)—Chapter 19.
- Rafailov R. et al. “Direct Preference Optimization” (2023)—§21.6.
- Rebedea T. et al. “NeMo Guardrails” (2023)—§19.5.
- Robertson S., Zaragoza H. “The Probabilistic Relevance Framework: BM25 and Beyond” (2009)—§7.3.
- Reimers N., Gurevych I. “Sentence-BERT” (2019)—§1.3, §7.4.
- Rumbelow J., Watkins M. “SolidGoldMagikarp” (2023)—§1.2.
- Saito S. et al.—LLM-judge verbosity bias (2023)—§17.4.
- Sanh V. et al. “DistilBERT” (2019)—§21.7.
- Santhanam K. et al. “PLAID” (2022)—§7.4.
- Schulman J. et al. “Proximal Policy Optimization” (2017)—§13.8, §21.6.
- Sennrich R., Haddow B., Birch A. “Neural Machine Translation of Rare Words with Subword Units” (BPE, 2016)—§1.2.
- Shao Z. et al. “DeepSeekMath” (GRPO, 2024)—§13.8, §21.6.
- Shazeer N. et al. “Outrageously Large Neural Networks” (MoE, 2017)—§1.5.
- Shi L. et al. “Judging the Judges: Position Bias in LLM-as-a-Judge” (2024/2025)—§17.4.
- Shinn N. et al. “Reflexion” (2023)—§13.5.
- Sprague Z. et al. “To CoT or not to CoT?” (2024)—§3.4.
- Su J. et al. “RoFormer” (RoPE, 2021)—§1.4.
- Subramanya S. et al. “DiskANN” (2019)—§8.3.
- Sumers T. et al. “Cognitive Architectures for Language Agents” (CoALA, 2023)—§4.5.
- Tam Z. et al. “Let Me Speak Freely?” (2024)—§5.3.
- Thakur N. et al. “BEIR” (2021)—§9.3.
- Touvron H. et al. “LLaMA” (2023)—§23.5.
- Tyen G. et al. “LLMs Cannot Find Reasoning Errors, but Can Correct Them” (2024)—§13.5.
- Traag V., Waltman L., van Eck N. “From Louvain to Leiden: Guaranteeing Well-Connected Communities” (2019)—§7.7.
- Vaswani A. et al. “Attention Is All You Need” (2017)—§1.4.
- Veličković P. et al. “Softmax is not Enough (for Sharp Size Generalisation)” (arXiv 2024; ICML 2025)—§4.3.
- Wallace E. et al. “The Instruction Hierarchy” (2024)—§3.2.
- Wang L. et al. “Plan-and-Solve Prompting” (2023)—§13.4.
- Wang X. et al. “Executable Code Actions Elicit Better LLM Agents” (CodeAct, 2024)—§12.6.
- Wang X. et al. “Self-Consistency Improves Chain of Thought Reasoning” (2022)—§3.4, §11.4.
- Wataoka K. et al. “Self-Preference Bias in LLM-as-a-Judge” (2024)—§17.4.
- Weber R., Schek H.-J., Blott S. “A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces” (VLDB, 1998)—§8.2.
- Wei J. et al. “Chain-of-Thought Prompting Elicits Reasoning in LLMs” (2022)—§3.4.
- Willard B., Louf R. “Efficient Guided Generation for LLMs” (Outlines, 2023)—§5.3.
- Xiao G. et al. “Efficient Streaming Language Models with Attention Sinks” (StreamingLLM, 2023)—§1.2, §4.3.

- Xu P. et al. “Retrieval Meets Long Context LLMs” (2023)—§6.7.
- Yao S. et al. “ReAct” (2022)—§13.3; “Tree of Thoughts” (2023)—§13.6.
- Yu G.-I. et al. “Orca” (2022)—§22.4.
- Yue Y. et al. “Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model?” (2025)—§13.8.
- Zhang B., Sennrich R. “RMSNorm” (2019)—§1.4, Appendix A.
- Zheng L. et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena” (2023)—Chapters 9, 17.
- Zheng L. et al. “SGLang: Efficient Execution of Structured Language Model Programs” (RadixAttention, 2024)—Chapter 22.
- Zhong Y. et al. “DistServe” (2024)—§22.5.
- Zhou S. et al. “WebArena: A Realistic Web Environment for Building Autonomous Agents” (2023)—§17.5.

Industry material, documentation, standards

- Anthropic: “Building Effective Agents” (2024)—Chapters 10–12; the engineering material on context engineering (2024–2025)—Chapter 4; “Contextual Retrieval” (2024)—§7.6; the report on the multi-agent research system (2025)—§14.3; the context management API (2025)—§4.4; “Code execution with MCP” (2025) and “Agent Skills” (2025)—§12.6; the Claude Opus 4.5 system card (2025)—§19.5; the Model Context Protocol specification, revisions 2025-06-18 and 2025-11-25—Chapter 15.
- Google: the Agent2Agent announcement (2025)—§15.6; DeepMind, CaMeL (2025)—§19.6; Microsoft Research, GraphRAG and LazyGraphRAG (2024)—§7.7; Microsoft Presidio (an open-source PII-detection tool)—§25.4.
- Letta: sleep-time compute—consolidating an agent’s memory in the background (2025)—§13.1.
- Linux Foundation: Agentic AI Foundation (2025), A2A Project, MCP Registry—Chapter 15.
- Meta: “Agents Rule of Two” (2025)—§12.5, §19.4; “Llama 4” (2025)—§1.5; the Llama model cards—§23.4.
- OpenAI: “Learning to Reason with LLMs” (o1, 2024)—§13.8; the documentation on prompting, structured outputs, and function calling—Chapters 3, 5, 12.
- OWASP: “Top 10 for LLM Applications” (2025), “Top 10 for Agentic Applications” (2026 revision)—Chapters 12, 14, 19.
- OpenTelemetry: Semantic Conventions for Generative AI (GenAI SIG, since 2024)—Chapter 18.
- Invariant Labs: analyses of tool poisoning and rug pull (2025)—§15.7.
- Karpathy A. “Let’s Build the GPT Tokenizer” (2024)—§1.2.
- Willison S.—the blog: prompt injection and how it differs from jailbreaking, lethal trifecta, dual-LLM (2022–2025)—Chapters 12, 19.
- Eval and observability platforms (Langfuse, LangSmith, Arize Phoenix, Braintrust, DeepEval)—Chapters 17–18.

Colophon

Typeset with KaTeX (https://katex.org): its stylesheet and web fonts are embedded in this file. They remain under their own license and are not relicensed.

```
KaTeX — https://katex.org
Bundled into the built HTML/PDF artifacts: stylesheet (katex.min.css) and web fonts.
Version pinned in build/package.json.

The MIT License (MIT)

Copyright (c) 2013–2020 Khan Academy and other contributors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

This work itself — prose, figures, and diagrams — is licensed under CC BY-SA 4.0; its code samples under the MIT License. See the NOTICE file at https://github.com/Drobiazkin/ai-agent-architecture.